

STUDY GUIDE · FROM ZERO TO THE CODE

AI Kavach CRS

How an autonomous system finds security bugs in C programs, gets AI to fix them, and then proves, and attacks, every fix before trusting it.

Written for readers with little or no technical background.
Every code excerpt is copied from the repository and labelled with its file and line numbers.

CONTENTS

What's inside

Chapter 1 The Background You Need

1. Programs: from words to a machine
2. Memory: a very long street of numbered boxes
3. Why C is fast but dangerous
4. The bug zoo
5. Naming bugs: CWE and CVE
6. Proof of vulnerability (PoV)
7. Sanitizers: making silent bugs loud
8. Fuzzing: throwing millions of odd inputs at a program
9. Static analysis: reading code without running it
10. Large language models (LLMs)
11. Formal verification: proof instead of testing
12. Python basics you will meet later
13. How it all fits together

Chapter 2 The Big Picture: What AI Kavach Does

1. The problem, and the one idea that shapes everything
2. The pipeline at a glance
3. A map of the repository
4. Three front doors: which program to run
5. The life of one bug
6. The data objects that flow between stages

Chapter 3 Finding and Locating Bugs

3.1 Static scanning with Semgrep
3.2 Fuzz harnesses: the plug between fuzzer and target
3.3 The fuzzer manager: building and running a campaign
3.4 Crash triage: from a pile of crash files to named bugs
3.5 The optional CASR path
3.6 Driller and angr: the "solver" idea (not wired in)
3.7 The fuzz pipeline: from clusters to patch jobs
3.8 The context engine: showing the AI the right code

Chapter 4 The AI Team: Writing the Fix

4.1 Where the settings come from: environment variables
4.2 The three agents: personalities made of words
4.3 LLMClient: the single telephone line
4.4 Two ways to hand in a patch
4.5 The case file and the job description
4.6 Running the team: parallel or one at a time
4.7 Picking the winner, and counting the bill
4.8 The Critic, Agent Delta
4.9 EscalationPolicy: the rulebook with no AI in it
4.10 Where the documents and the code disagree

Chapter 5 Proving the Fix, Then Attacking It

5.1 Gates, and the three answers a gate can give
5.2 The DRV loop from start to finish
5.3 The workspace: a fresh scratch copy for every attempt
5.4 The five gates inside _verify
5.5 Pre-flight: test the smoke alarm before trusting it
5.6 The judge scripts: how "the attack worked" is decided
5.7 Hardening: trying to break a patch that passed
5.8 Formal verification with CBMC: a proof, within limits
5.9 The cheat detector: three patches, one honest
5.10 Honest weaknesses

Chapter 6 Memory, Benchmarks, Reports and Running It

6.1 Agent memory: the four notebooks

6.2 Reports: SKIPPED never means PASS
6.3 The everyday front door: src/main.py
6.4 The measuring harness: benchmark.py
6.5 The benchmark suites: what each one measures
6.6 The unit tests: guarding the honesty rules
6.7 The reported results, and how to read them
6.8 How to run it yourself

Chapter 7 Honest Assessment, FAQ and Glossary

1. What is genuinely strong (and why it matters)
2. Known bugs that can inflate results
3. Places where the docs disagree with each other or with the code
4. Features that exist but aren't wired in
5. Operational risks
6. If I had one day: a prioritized improvement list
7. Frequently asked questions
8. Glossary

BEFORE YOU START

How to use this guide

This guide is meant to be read in order. Each chapter builds on the one before it. Chapter 1 teaches the background ideas from scratch. Chapters 2 to 6 walk through the project one stage at a time, with real code. Chapter 7 gives an honest assessment, a FAQ and a glossary you can come back to at any time.

Look out for these boxes:

ANALOGY

An everyday comparison that makes a technical idea easier to picture.

KEY IDEA

The one thing to remember from a section.

WATCH OUT

A weakness, a gap or a common misunderstanding.

TRY IT YOURSELF

A command you can run in the project folder to see it in action.

Code appears in shaded panels. The dark strip above each panel names the file and line numbers, so you can open the same spot in the real project. A green "Line by line" box after a panel explains what each part does. Don't worry about understanding every symbol in the code. The explanations matter more than the syntax.

Each chapter ends with a **recap** and a few **check yourself** questions. If you can answer them in your own words, you're ready for the next chapter.

CHAPTER 1

The Background You Need

This guide is about a project called **AI Kavach CRS**. In one sentence: it is a program that reads other programs written in the C language, hunts for a particular family of dangerous mistakes in them, asks an AI to write repairs, and then refuses to believe any repair until a set of strict, mechanical tests has confirmed it. Before we can follow how it does that, we need a shared vocabulary. This chapter builds that vocabulary from nothing.

You only need to know one thing to start: a program takes some *input*, does something with it, and produces some *output*. Everything else is explained here, usually with an everyday comparison first and then with real code from the project. Take it slowly. The rest of the guide leans on almost every idea in this chapter.

KEY IDEA

The whole project rests on one sentence that the authors repeat often: "**the LLM proposes, deterministic tools decide.**" The AI (the "LLM") is allowed to suggest fixes, but only predictable, repeatable checking tools get to say whether a fix is accepted. By the end of this chapter you will know what every word in that sentence means and why the rule matters.

1. Programs: from words to a machine

1.1 Source code and the compiled program

A computer's processor (the chip that does the actual work) understands only very simple instructions, written as long patterns of numbers: "load this number", "add these two", "jump to that place if the result was zero". Writing programs directly in that form is miserable, so people write in a **programming language** instead. C is one such language. A program written in a language that humans can read is called **source code**. It is just text in a file, for example `tests/demo_vulns/06_off_by_one_loop.c`. The `.c` at the end tells you it is C source code.

The processor cannot run that text file directly. Something has to translate it into the processor's own number language. That translator is called a **compiler**, and the translated result is the **compiled program** (also called the **binary** or the **executable**). The compiler this project uses is called **clang**, which is part of a larger toolkit called **LLVM**.

ANALOGY

Source code is a recipe written in English. The compiled program is the same recipe translated into a language only the kitchen robot understands. The robot cannot read English, and you cannot easily read the robot's language, so a translator (the compiler) sits in between. If the English recipe says "add salt until it tastes right", the translator has to turn that into exact robot steps, and any mistake in the recipe gets faithfully translated into a mistake in the robot's steps.

Turning source code into a runnable program is called **building** (or "compiling"). The instruction you type to do it is a **build command**. Here is the build command the project uses for most of its small test programs:

tests/benchmarks/targets.py · lines 39-46

```
# Sanitizers are made fatal (-fno-sanitize-recover=all) so that a UBSan
# report is an abort, not a printed warning that leaves the exit code at 0.
SAN_CFLAGS = (
    "-fsanitize=address,undefined -fno-sanitize-recover=all "
    "-fno-omit-frame-pointer -g -O0"
)

SINGLE_FILE_BUILD = f'clang {SAN_CFLAGS} "{{src}}" -o "{{bin}}"'
```

LINE BY LINE

1. The lines starting with `#` are **comments**: notes for humans that the computer ignores. This one explains why the build is set up so that any problem found is "fatal" (it stops the program).
2. `SAN_CFLAGS = (...)` stores a piece of text under the name `SAN_CFLAGS`. The text is a list of **flags**: options handed to the compiler, each starting with a dash.
3. `-fsanitize=address,undefined` asks clang to build in two "sanitizers", extra safety alarms we explain in section 6.

4. `-fno-sanitize-recover=all` means "when an alarm goes off, stop the program immediately; don't just print a warning and carry on."
5. `-fno-omit-frame-pointer` and `-g` keep extra bookkeeping in the binary so that, when something goes wrong, the error report can name the exact file, function and line.
6. `-O0` means "optimisation level zero": do not rearrange the code to make it faster. That keeps the compiled program close to the source, so errors point at the line you expect.
7. `SINGLE_FILE_BUILD` glues it all into one command: `clang`, then the flags, then the source file (`{src}`), then `-o` ("output to") the binary (`{bin}`). The `{src}` and `{bin}` are placeholders that get filled in later with real file names.

So when you later read that a patch must pass the "build gate", it simply means: after the AI's change, this command must still succeed. If the AI wrote something that is not valid C, the compiler refuses to translate it, and the patch fails right there.

1.2 Running a program with arguments

All twenty small test programs in `tests/demo_vulns/` take their input from the **command line**: the words you type after the program's name when you start it. If you run `./program hello`, the word `hello` is an **argument**. Inside a C program the arguments arrive in a list called `argv` ("argument vector"), and `argc` ("argument count") says how many there are. `argv[0]` is the program's own name and `argv[1]` is the first real argument. You will see `argv[1]` constantly: it is the attacker-controlled input in almost every example below.

When a program finishes it hands back a small number called its **exit code**. By convention, `0` means "all fine" and anything else means "something went wrong". This project leans heavily on exit codes because they are a simple, mechanical yes/no signal that a machine can check.

2. Memory: a very long street of numbered boxes

2.1 Bytes and addresses

While a program runs, it keeps its data in the computer's **memory** (RAM). Picture memory as an enormous row of small boxes, each holding one **byte**. A byte is a tiny number between 0 and 255; one byte is enough to store one letter such as `A` (which is stored as the number 65). Every box has a house number, called its **address**. Addresses are usually written in **hexadecimal** (base 16, using digits 0–9 and letters a–f), which is why you will see things like `0x602000000014`; the `0x` just means "this is hexadecimal".

> **ANALOGY**
>
> Memory is a street with billions of tiny houses, all in a single row, each with one number painted on the door. Each house can hold one small item. A program does not ask for "the house with the name in it"; it asks for "house number 4,102,337". If it gives the wrong number, it gets whatever happens to be in that house, and nobody stops it.

A **pointer** is a value that holds an address: "the data you want starts at house number such-and-such". In C you will see pointers written with a star, as in `char* dest`, which reads as "`dest` holds the address of some characters". The arrow `->` means "follow this pointer and look at a field of what it points to".

2.2 Buffers, arrays and strings

A **buffer** is a run of neighbouring boxes set aside to hold some data, for example 128 boxes for a user's name. An **array** is the same idea with a name and a fixed number of slots: `char log_msg[256]` means "256 boxes in a row, called `log_msg`".

Two facts about arrays cause a huge share of the bugs in this guide:

- **Counting starts at zero.** An array with 4 slots has slots numbered 0, 1, 2 and 3. There is no slot 4. Writing to "slot 4" means writing into whatever box comes next on the street, which belongs to something else.
- **C strings end with an invisible zero.** A piece of text, called a **string**, is stored as its letters followed by one extra byte with the value 0, called the **NUL terminator** and written `'\0'`. It marks "the text stops here". So the four-letter word `AAAA` actually needs *five* boxes: A, A, A, A, and the 0. Forgetting that extra box is one of the most common mistakes in C.

2.3 The stack and the heap

A running program uses two main regions of memory for its data. You do not need the deep details, just the difference:

	The stack	The heap
What goes there	Local variables of a function: things declared inside it, such as `char cmd[256];`	Memory the program explicitly asks for while running, with `malloc`

	The stack	The heap
Who cleans up	Automatic: when the function returns, its stack space is reclaimed	The programmer must hand it back with `free`
Size	Fixed when you write the code	Can be chosen at run time, e.g. "as many bytes as the user typed"
Everyday picture	A stack of plates in a cafeteria: each function call puts a plate on top, and it is removed when the call ends	A warehouse that rents out storage units on request

The stack also stores some housekeeping, most importantly the **return address**: the note that tells the computer where to go back to when the current function ends. That matters for security. If an attacker can overwrite that note, they can send the program somewhere it was never meant to go.

2.4 `malloc` and `free`

On the heap, C uses two functions:

- `malloc(n)` ("memory allocate") asks for `n` bytes and returns a pointer to the first one. If memory cannot be provided, it returns a special "nowhere" pointer called **NULL** (address zero).
- `free(p)` hands those bytes back so they can be reused for something else later.

ANALOGY

`malloc` is renting a storage unit: the warehouse gives you a unit number and a key. `free` is returning the key. After you return it, the unit may be rented to someone else. If you keep using your old unit number anyway, you are now rummaging through a stranger's belongings, or worse, leaving your things in their unit. And if you "return the key" twice, the warehouse's records get confused about who owns what.

3. Why C is fast but dangerous

C was designed in the early 1970s to be close to the machine. When you write `dest[i] = 'A'`, C translates it into, roughly, "compute the address of `dest` plus `i`, and put 65 there". It does **not** add a check that `i` is inside the buffer. It does not remember how big each buffer is. It does not notice when you use memory you already gave back. Skipping all those checks is a big reason C programs are fast and small, which is why C still runs operating systems, routers, medical devices, military equipment and countless other systems where speed and control matter.

The price is that mistakes are **silent**. Newer languages such as Python, Java or Rust either check every array access or make many of these mistakes impossible to write. In C, a program with a memory mistake often keeps running and prints the right answer, until one day the input is slightly different, or an attacker supplies it deliberately.

The C rules even have a name for "you broke the rules, and anything may now happen": **undefined behaviour**. Once a program has undefined behaviour, the language makes no promises at all about what happens next. It might crash, it might quietly produce garbage, or it might do exactly what an attacker wants.

KEY IDEA

A **memory-safety bug** is a mistake where a program reads or writes memory it should not touch: past the end of a buffer, after it was freed, through a garbage pointer, and so on. These bugs are the main target of this project. They are dangerous for two reasons: they are often *invisible* in normal testing, and they can let an attacker read secrets or take control of the program.

4. The bug zoo

The project includes twenty small, deliberately broken C programs in `tests/demo_vulns/`, numbered `01` to `20`. Each one demonstrates one kind of mistake. The project calls them the "synthetic suite" (synthetic meaning "made on purpose for testing"), and gives each one an ID such as `SYN-06-OFF-BY-ONE`. The file `tests/benchmarks/targets.py` records, for each program, which line is buggy, what kind of bug it is, and how to prove the bug exists.

For every bug type we give: what it means in plain words, what an attacker gains from it, and a real snippet from the repo where one exists. Five of them get a full line-by-line walkthrough.

Small reading aids for the C snippets:

- `#include <stdio.h>` at the top of a file pulls in ready-made tools (here "standard input/output", which provides `printf`, the function that prints text).

- `void name(...) { ... }` defines a **function**: a named block of steps. The things in brackets are its inputs. `void` means it returns no result.
- `if (condition) { ... }` runs the block only when the condition is true. `for (...)` repeats a block (a **loop**).
- `==` means "is equal to"; a single `=` means "store this value". `!` means "not", `||` means "or", `&&` means "and".
- `size_t`, `uint32_t` and `int` are kinds of whole numbers, which we explain properly in 4.5.

4.1 Buffer overflow (out-of-bounds write)

Plain meaning: the program writes more data into a buffer than the buffer can hold, so the extra spills into the neighbouring boxes and overwrites whatever lived there.

> **ANALOGY**
>
> Pouring two litres of water into a one-litre jug on a crowded table. The jug fills, and then the water flows onto everything next to it: papers, a laptop, someone's lunch. The jug does not stop you; it just overflows.

What an attacker gains: the ability to overwrite nearby data. On the stack, that can include the return address, which can redirect the program to code of the attacker's choosing. That is the classic route to taking over a program entirely. On the heap, it can corrupt other objects, such as flags that say "this user is an administrator".

Example: `tests/demo_vulns/02_deep_typedef_confusion.c` (labelled CWE-787 in the manifest, "Stack buffer overflow via three-level typedef chain"):

tests/demo_vulns/02_deep_typedef_confusion.c · lines 20-26

```
void initialize_user_crypto(UserObject* user, const char* provided_key, size_t key_len) {
    if (!user || !user->ctx || !provided_key) {
        return;
    }
    memcpy(user->ctx->crypto.key, provided_key, key_len);
    printf("User crypto initialized for session %u\n", user->ctx->session_id);
}
```

`memcpy(destination, source, count)` copies `count` bytes. The destination `crypto.key` is 64 bytes (declared as `u8 key[64]` on line 9), but `key_len` is the length of whatever the user typed. The proof the project uses is 200 letter A's. That copies 200 bytes into a 64-byte space, and 136 bytes of A's spill over the neighbouring stack memory. Notice that the function carefully checks for missing pointers (line 21) and still forgets the one check that matters: is `key_len` at most 64?

4.2 Off-by-one

Plain meaning: a special, extremely common kind of overflow where the program goes exactly *one* step too far, usually because of counting from zero or forgetting the string's terminating 0.

> **ANALOGY**
>
> A fence 10 metres long with a post every metre needs *eleven* posts, not ten. People get this wrong all the time. Programmers make the same mistake with loops: "go from 0 up to and including 4" visits five slots, not four.

What an attacker gains: one byte seems harmless, but it is often enough. Overwriting one byte of a neighbouring heap record can corrupt the bookkeeping the memory system uses, and real attacks have been built from a single stray byte.

This is the project's favourite teaching example, so we go through all of it:

tests/demo_vulns/06_off_by_one_loop.c · lines 1-23

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
void copy_and_null_terminate(const char* src, size_t size) {
    if (size == 0 || size > 1024) return;
    char* dest = (char*)malloc(size);
    if (!dest) return;
    for (size_t i = 0; i <= size; i++) {
        if (i == size) {
```

```
            dest[i] = '\0';
        } else {
            dest[i] = src[i];
        }
    }
    printf("Copied string: %s\n", dest);
    free(dest);
}
int main(int argc, char* argv[]) {
    if (argc > 1) {
        copy_and_null_terminate(argv[1], strlen(argv[1]));
    }
    return 0;
}
```

LINE BY LINE

1. Lines 1–3 bring in standard tools: printing, memory allocation, and string helpers.
2. Line 4 starts a function that should copy the text `src`, which is `size` letters long, and put a terminating 0 on the end of the copy.
3. Line 5 is a sanity check: give up if the text is empty or longer than 1024 letters.
4. Line 6 rents `size` bytes on the heap. **This is where the plan goes wrong:** a copy of a `size`-letter string needs `size + 1` bytes, one extra for the terminating 0. For `AAAA` (size 4) we get 4 boxes, numbered 0–3.
5. Line 7: if the rental failed (NULL), give up.
6. Line 8 is the loop. `i <= size` means "keep going while `i` is less than *or equal to* `size`", so with size 4 it visits 0, 1, 2, 3 *and 4*. That is five steps for four boxes.
7. Lines 9–10: on the last step (`i == size`, i.e. 4) it writes the terminating 0 to `dest[4]`, the box just past the end. That is the out-of-bounds write. The manifest records the bug at line 8, the loop, because the loop's limit is what is wrong.
8. Lines 11–12: on every other step it copies one letter.
9. Line 15 prints the copy, and line 16 returns the memory.
10. Lines 18–22: `main` is where every C program starts. If the user gave an argument, it calls the function with that text and its length (`strlen` counts the letters, not including the terminating 0).

The scary part: if you compile this normally and run it with `AAAA`, it will very likely print `Copied string: AAAA` and exit happily. The stray byte lands in a box that nobody happens to be checking. The bug is real, but silent. In section 6 we will see how a "sanitizer" turns this silent bug into a loud crash. The right fix is small: allocate `size + 1` bytes.

4.3 Out-of-bounds read

Plain meaning: the same mistake as an overflow, but the program *reads* beyond the buffer instead of writing.

What an attacker gains: information leaks. Reading past the end of one buffer can reveal whatever sits next to it: passwords, encryption keys, or addresses that help defeat other defences. The famous "Heartbleed" bug of 2014 was an out-of-bounds read that leaked secrets from web servers worldwide.

tests/demo_vulns/11_oob_multidim_array.c · lines 3–9

```
int matrix[5][5];
int read_matrix(int row, int col) {
    if (row < 0 || row >= 5) {
        return -1;
    }
    return matrix[row][col];
}
```

`matrix` is a 5 × 5 grid of numbers. The function checks that `row` is between 0 and 4, and then forgets to check `col` at all. The project's proof input asks for row 0, column 1,000,000, which reads memory far beyond the grid. Half a check is no check.

4.4 Use-after-free

Plain meaning: the program gives memory back with `free` and then keeps using the old pointer. A pointer that still points at freed memory is called a **dangling pointer**.

What an attacker gains: after a `free`, the memory system may hand those same boxes to a different part of the program. If the attacker can arrange for their own data to land there, the old code, still using its dangling pointer, will treat the attacker's data as its own object. This is one of the most exploited bug classes in web browsers.

The repo's example, `tests/demo_vulns/19_ast_macro_expansion.c`, hides the free inside a **macro** (a shorthand that the compiler expands into longer code before translating):

tests/demo_vulns/19_ast_macro_expansion.c · lines 3–16

```
#define ALLOC_NODE(type) (type*)malloc(sizeof(type))
#define SAFE_FREE(ptr) if(ptr) { free(ptr); ptr = NULL; }
typedef struct Node {
    int data;
    struct Node* next;
} Node;
void process_node(Node* head) {
    if (!head) return;
    Node* current = head;
    SAFE_FREE(current);
    if (current && current->data == 0) {
        printf("Data is zero\n");
    }
}
```

`SAFE_FREE` frees the memory and sets the *local* copy `current` to NULL. But the caller still holds its own copy of the address (`head`, and `n` back in `main`), which now dangles. The project is honest about this one: its manifest says there is "no live use-after-free at runtime", because the check on line 13 stops the local pointer being used, and the caller never touches its copy again. It is a *latent* defect: a trap waiting for future code to step in. We include it because it shows that a fix that looks "safe" on one copy of a pointer does nothing for the other copies.

4.5 Numbers that wrap: integer overflow, underflow and truncation

First, how computers store whole numbers. A number type has a fixed number of bytes, which fixes the largest value it can hold:

Type	Size	Range (roughly)
`uint32_t`	4 bytes	0 to 4,294,967,295 (that is 2^32 − 1)
`size_t` (on the 64-bit machines used here)	8 bytes	0 to about 18.4 quintillion (2^64 − 1)
`int`	4 bytes, signed	about −2.1 billion to +2.1 billion

"Unsigned" types (the `u` in `uint32_t`, and `size_t`) cannot be negative. When an unsigned calculation goes past the top, it **wraps around** to the bottom, like a car's odometer rolling from 999,999 back to 000,000. Going below zero wraps to the very top. This wrap is perfectly legal C, so nothing complains.

> **ANALOGY**
>
> A 12-hour clock. If it is 11 o'clock and you add 3 hours, the clock says 2, not 14. The clock is not broken; it simply has no room for 14. Computers do the same thing with numbers, and a program that calculates "how many bytes do I need?" can end up with a tiny answer when the true answer was enormous.

What an attacker gains: almost always, a buffer overflow in disguise. The wrapped number is used to decide how much memory to rent, so the buffer is far too small, and then the real (large) amount of data is copied into it.

Integer overflow (multiplication that wraps)

tests/demo_vulns/03_silent_integer_overflow.c · lines 5–21

```
typedef struct {
    uint32_t id;
```

```
    uint32_t type;
    char data[24];
} EventRecord;
void process_events(uint32_t num_events, const char* event_data) {
    uint32_t total_size = num_events * sizeof(EventRecord);
    EventRecord* records = (EventRecord*)malloc(total_size);
    if (!records) {
        return;
    }
    for (uint32_t i = 0; i < num_events; i++) {
        memcpy(&records[i], event_data + (i * sizeof(EventRecord)), sizeof(EventRecord));
        printf("Processed event %u\n", records[i].id);
    }
    free(records);
}
```

LINE BY LINE

1. Lines 5–9 describe a **struct**: a bundle of fields kept together. An `EventRecord` holds an `id` (4 bytes), a `type` (4 bytes) and 24 bytes of `data`, so `sizeof(EventRecord)`, "the size of one record", is 32 bytes.
2. Line 10: the function receives how many events there are (`num_events`) and where their raw bytes are.
3. Line 11 computes the total bytes needed: count × 32, stored in a `uint32_t`, which tops out just under 4.3 billion. **This is the bug line** (the manifest says line 11, CWE-190).
4. The proof input is `134217729`. Multiply by 32 and you get 4,294,967,328, which is just 32 more than 2^32. The odometer rolls over, and `total_size` becomes **32**.
5. Line 12 rents 32 bytes, room for exactly one record.
6. Line 13–15: the rental succeeded, so no early exit. Nothing here notices that 32 is ridiculous.
7. Line 16 loops `num_events` times, that is 134 million times, still using the true, un-wrapped count.
8. Line 17 copies one 32-byte record into slot `i`. On the second pass (`i = 1`) it is already writing past the 32-byte rental: a heap overflow.
9. Line 20 returns the memory (if the program ever gets that far).

The calculation and the loop disagree about the number of records: one uses the wrapped value, the other the real one. That disagreement is the whole bug. A correct fix checks, *before* multiplying, whether the multiplication would overflow.

Integer truncation

Truncation means squeezing a big number type into a smaller one, which chops off the high part. `tests/demo_vulns/04_extreme_integer_truncation.c` (CWE-197):

tests/demo_vulns/04_extreme_integer_truncation.c · lines 15–21

```
    header->object_size = (uint32_t)incoming_size;
    header->data = (char*)malloc(header->object_size);
    if (!header->data) {
        free(header);
        return;
    }
    memcpy(header->data, incoming_data, incoming_size);
```

`incoming_size` is an 8-byte `size_t`; `(uint32_t)` forces it into 4 bytes. The proof input is 4,294,967,396, which is 2^32 + 100. Chop off everything above 2^32 and only **100** is left. So line 16 rents 100 bytes, and line 21 copies the full four-billion-plus bytes into them.

Integer underflow

tests/demo_vulns/12_integer_underflow.c · lines 4–12

```
void process_chunk(size_t chunk_end, size_t chunk_start, const char* data) {
    size_t length = chunk_end - chunk_start;
    char* dest = (char*)malloc(length + 1);
    if (!dest) return;
```

```
    memcpy(dest, data, length);
    dest[length] = '\0';
    printf("Processed chunk.\n");
    free(dest);
}
```

The function assumes the end comes after the start. The proof input gives end = 5 and start = 10. In normal arithmetic $5 - 10 = -5$, but `size_t` cannot be negative, so it wraps to the very top: about 18.4 quintillion. Line 6 then asks for that absurd amount of memory. The manifest labels this CWE-191 ("Integer Underflow"). The project's crash-classification table (see section 5) maps the AddressSanitizer message "requested allocation size", which is what an absurd request like this produces, to the integer-overflow family.

4.6 Null pointer dereference

Plain meaning: using a pointer that is NULL (address zero, "points nowhere") as if it pointed at real data. "Dereference" just means "follow the pointer to the data".

What an attacker gains: usually a crash, which is a **denial of service** (DoS): the attacker can knock the program over at will. In some special situations, mostly inside operating-system kernels, it can be worse.

tests/demo_vulns/10_null_deref_complex.c · lines 8–13

```
void evaluate_context(int type) {
    ContextObj* ctx = (ContextObj*)malloc(sizeof(ContextObj) * 100000);
    switch(type) {
        case 1:
            ctx[0].status = 1;
            printf("Context status set to %d\n", ctx[0].status);
```

`malloc` can return NULL when memory runs out, but this code uses `ctx` on line 12 without checking. The project's notes admit the bug cannot be triggered on demand: an 800 KB request does not fail on a normal machine. So this one can only be found by *reading* the code (static analysis, section 8), not by running it.

4.7 Double free

Plain meaning: calling `free` twice on the same memory.

What an attacker gains: the memory system keeps an internal list of free boxes. Freeing the same block twice can put it on that list twice, so two later `malloc` calls may hand out the *same* memory to two different users. That is a powerful building block for corrupting memory and, in skilled hands, for taking control.

tests/demo_vulns/05_double_free_conditional.c · lines 4–17

```
void process_data(const char* input) {
    char* buffer = (char*)malloc(128);
    if (!buffer) return;
    if (input) {
        strncpy(buffer, input, 127);
        buffer[127] = '\0';
        if (strcmp(buffer, "ERROR") == 0) {
            printf("Error condition hit.\n");
            free(buffer);
        }
    }
    printf("Processing complete.\n");
    free(buffer);
}
```

> **LINE BY LINE**
>
> 1. Line 5 rents 128 bytes. Line 6 gives up if that failed.
> 2. Line 7: only if there is some input…
> 3. Line 8 copies at most 127 characters of it (`strncpy` is a "copy up to N" function), and line 9 makes sure the last box is a terminating 0. This part is done carefully, so there is no overflow here.

4. Line 10: `strcmp` compares two strings and gives 0 when they are equal. So this asks "did the user type exactly `ERROR`?"
5. Lines 11–12: if so, print a message and **free the buffer**. The programmer probably meant to stop here, but there is no `return`.
6. Line 15: execution carries on regardless and prints "Processing complete."
7. Line 16 frees the buffer **again**. On the `ERROR` path, that is the second free of the same memory.

With any normal input the program is fine; only the exact word `ERROR` triggers the bug. That is typical: bugs hide in rarely-taken paths, especially error-handling paths, which are tested least. The proof input is, unsurprisingly, `ERROR`.

4.8 Format string vulnerability

Plain meaning: C's `printf` takes a "format string", a template with placeholders, followed by values to fill in. `%s` means "insert a string here", `%d` "insert a number", and so on. If a program lets the *user's* text become the template, the user can type placeholders, and `printf` will obey them, fetching "values" that were never supplied.

ANALOGY

A receptionist reads out forms using a template like "Please welcome ____". If a visitor is allowed to write the template itself, they could write "Please read out the contents of the manager's desk drawer", and a receptionist who follows templates literally would do it.

What an attacker gains: `%s` and friends make `printf` read whatever happens to be lying around in memory (an information leak). The special placeholder `%n` is worse: it *writes* a number into memory at an address taken from those same lying-around values. Reading and writing arbitrary memory is often enough to take over the program.

tests/demo_vulns/07_format_string_vuln.c · lines 4–9

```
void log_user_activity(const char* username) {
    char log_msg[256];
    snprintf(log_msg, sizeof(log_msg), "User logged in: %s", username);
    printf(log_msg);
    printf("\n");
}
```

LINE BY LINE

1. Line 5 sets aside a 256-byte buffer on the stack.
2. Line 6 builds the message safely: `snprintf` fills `log_msg` with "User logged in: " followed by the username, and it never writes more than `sizeof(log_msg)` (256) bytes. So far, so good.
3. Line 7 is the bug. It passes the finished message to `printf` **as the template**. Any `%` codes that were in the username are now live instructions.
4. The project's proof input is `%s%s%s%s%s%s%s%s%n`. The eight `%s` make `printf` treat leftover stack values as addresses of strings and read from them; the final `%n` tries to write through one of them. The result is a crash or memory corruption.
5. The fix is tiny: `printf("%s", log_msg);`. Now the template is fixed text and the user's message is only ever *data*.

4.9 Command injection

Plain meaning: the program builds a command for the computer's **shell** (the command-line interpreter that runs programs, like the Terminal on a Mac) by pasting user input into it. The shell treats some characters specially; for example `;` means "and then run this next command". If the user includes those characters, they can tack on commands of their own.

What an attacker gains: the ability to run any command with the program's permissions: read files, install malware, delete data. This is one of the most direct and dangerous bug classes. Note that it is not a memory bug at all: no buffer is overrun. It is a "mixing data with instructions" bug, the same family as the format string bug.

tests/demo_vulns/09_command_injection_shell.c · lines 4–9

```
void ping_host(const char* ip_address) {
    char cmd[256];
    snprintf(cmd, sizeof(cmd), "ping -c 1 %s", ip_address);
    printf("Executing: %s\n", cmd);
```

```
    system(cmd);
}
```

LINE BY LINE

1. The function's job is to "ping" a computer: send it one small network message to see if it answers.
2. Line 6 builds the text `ping -c 1` followed by whatever the user gave. With `127.0.0.1`, that is a harmless command.
3. Line 7 prints the command being run.
4. Line 8 hands the whole text to `system`, which asks the shell to run it.
5. Now suppose the input is `127.0.0.1; touch /tmp/marker`. The shell sees two commands: the ping, and then `touch /tmp/marker`, which creates a file. Replace `touch` with anything nastier and you see the danger.

The project proves this bug with exactly that trick, using a "marker file" as evidence. Its proof script, `tests/benchmarks/harness/pov_injection.sh`, runs the program with `"127.0.0.1; touch $marker"` and then checks whether the marker file appeared. If it did, the injected command ran, and the bug is real.

4.10 Path traversal

Plain meaning: the program builds a file name from user input, expecting to stay inside one folder, but the user includes `..` ("go up one folder") to climb out of it.

tests/demo_vulns/08_path_traversal.c · lines 5–8

```
void read_user_file(const char* filename) {
    char filepath[512];
    snprintf(filepath, sizeof(filepath), "/var/www/html/users/%s", filename);
    FILE* f = fopen(filepath, "r");
```

The program means to open files only inside `/var/www/html/users/`. Input such as `../../../../etc/passwd` produces `/var/www/html/users/../../../../etc/passwd`, which climbs back up to the top of the file system and into a system file. **What an attacker gains:** reading (or, in other programs, writing) files they should never reach, such as configuration files with passwords. The project notes honestly that it cannot demonstrate this one on its test machine, because that base folder does not exist there.

4.11 TOCTOU race

Plain meaning: TOCTOU stands for "**time of check to time of use**". The program checks something ("am I allowed to read this file?"), then, a moment later, acts on it ("open the file"). In the gap between the two, an attacker swaps the thing being checked. A **race condition** is any bug whose outcome depends on which of two things happens first.

ANALOGY

A bouncer checks your ticket at the door, then turns to open the rope. While he turns, you swap your valid ticket-holder friend for someone else. The check was correct when it happened; it just no longer describes reality when the door opens.

tests/demo_vulns/13_toctou_race.c · lines 4–6

```
void read_secure_file(const char* filepath) {
    if (access(filepath, R_OK) == 0) {
        FILE* f = fopen(filepath, "r");
```

`access` asks "may the real user read this path?" and `fopen` then opens it. Between those two lines, an attacker could replace the file with a **symbolic link** (a signpost file that points somewhere else) to a file they are not allowed to read. **What an attacker gains:** access to files the check was meant to protect, especially in programs that run with higher privileges than the user. The project states that this race cannot be reproduced reliably in a single test run, so it is detected only by reading the code.

4.12 Uninitialised memory

Plain meaning: using a variable before giving it a value. In C, a fresh local variable does not start at zero; it contains whatever bytes were left in that memory by earlier code. That leftover content is often called "garbage".

tests/demo_vulns/14_uninitialized_memory.c · lines 7–14

```
char* secrets[] = {"Secret1", "Secret2", "Secret3"};
void handle_request(int req_id) {
    UserRequest req;
    req.id = req_id;
    if (req.index >= 0 && req.index < 3) {
        printf("Secret: %s\n", secrets[req.index]);
    }
}
```

Line 9 creates `req` without filling it in. Line 10 sets `id` but never `index`. Line 11 then makes a security decision based on `index`, a garbage value. **What an attacker gains:** if they can influence what was previously in that memory (often possible), they control the decision, and here, which secret is printed. Uninitialised memory can also leak old data, such as a password a previous function left behind. The project notes that catching this at run time needs a tool called MemorySanitizer, which does not work on the Mac it was developed on, so this one too is found statically.

4.13 Hardcoded secret

Plain meaning: writing a password or encryption key directly into the source code.

tests/demo_vulns/15_hardcoded_key.c · lines 4–7

```
void encrypt_data(const char* data) {
    const char* secret_key = "SUPER_SECRET_AES_KEY_12345";
    printf("Encrypting data '%s' with key '%s'\n", data, secret_key);
}
```

(This "key" is a fake one written for the demo.) Text written into source code is copied as-is into the compiled program. Anyone with the binary can pull it out with a standard tool called `strings`, which lists every readable piece of text inside a file. **What an attacker gains:** the key itself, and with it the ability to decrypt data or impersonate the system, on every copy of the program ever shipped. The project's proof for this bug is not a crash at all: `tests/benchmarks/harness/secret_scan.sh` runs `strings` on the built program and fails if the key is found.

4.14 Type confusion

Plain meaning: the program stores a value as one kind of thing and then reads it back as a different kind of thing.

tests/demo_vulns/16_type_confusion.c · lines 3–11

```
typedef union {
    int id;
    char* name;
} UserIdentifier;
void render_user(const char* raw) {
    UserIdentifier uid;
    uid.id = atoi(raw);
    printf("User name: %s\n", uid.name);
}
```

A **union** is a set of boxes that can hold *either* one field *or* another, sharing the same memory. Here it holds either a number (`id`) or a pointer to text (`name`). Line 9 converts the user's input to a number (`atoi` means "ASCII to integer") and stores it as `id`. Line 10 immediately reads the same bytes as `name`, a pointer, and asks `printf` to print the text "at" that address. So a plain number is treated as a memory address. The default input `1094795585` is not random: in hexadecimal it is `0x41414141`, which is the bytes of "AAAA", a value security people use to show "I control this address". **What an attacker gains:** the power to make the program read (or in other cases write or jump to) an address of their choosing.

4.15 Memory leak

Plain meaning: renting memory with `malloc` and never returning it with `free`.

tests/demo_vulns/17_memory_leak_error_path.c · lines 4–15

```
int process_transaction(int amount) {
    char* log_msg = (char*)malloc(256);
```

```
    if (!log_msg) return -1;
    snprintf(log_msg, 256, "Processing %d", amount);
    if (amount < 0) {
        printf("Error: Negative amount.\n");
        return -1;
    }
    printf("%s\n", log_msg);
    free(log_msg);
    return 0;
}
```

On the normal path, line 13 frees the 256 bytes. But for a negative amount, line 10 returns early, skipping the `free`. Those 256 bytes are lost until the program exits. Once is harmless; in a server handling millions of requests, the memory slowly fills up until the machine runs out. **What an attacker gains:** a slow denial of service: send enough bad requests and the program eventually dies. Again, the bug lives on the error path.

4.16 The whole zoo at a glance

Bug class	Demo file	CWE in manifest	Typical attacker gain
Buffer overflow	`02_deep_typedef_confusion.c`, `18_ast_deep_struct.c`	CWE-787	Overwrite data or the return address; take control
Off-by-one	`06_off_by_one_loop.c`	CWE-193	One-byte corruption, sometimes enough for control
Out-of-bounds read	`11_oob_multidim_array.c`	CWE-125	Leak secrets
Use-after-free	`19_ast_macro_expansion.c`	CWE-416	Attacker data treated as a live object
Integer overflow	`01_complex_magic_bytes.c`, `03_silent_integer_overflow.c`	CWE-190	Undersized buffer, then overflow
Integer truncation	`04_extreme_integer_truncation.c`	CWE-197	Undersized buffer, then overflow
Integer underflow	`12_integer_underflow.c`	CWE-191	Huge length, memory errors
Null pointer dereference	`10_null_deref_complex.c`	CWE-476	Crash (denial of service)
Double free	`05_double_free_conditional.c`	CWE-415	Memory manager corruption
Format string	`07_format_string_vuln.c`	CWE-134	Read and write memory
Command injection	`09_command_injection_shell.c`	CWE-78	Run any command
Path traversal	`08_path_traversal.c`	CWE-22	Read forbidden files
TOCTOU race	`13_toctou_race.c`	CWE-367	Bypass a permission check
Uninitialised memory	`14_uninitialized_memory.c`, `20_ast_opaque_pointer.c`	CWE-457	Leak old data, steer decisions
Hardcoded secret	`15_hardcoded_key.c`	CWE-321	Steal the key from the binary
Type confusion	`16_type_confusion.c`	CWE-843	Use a number as an address
Memory leak	`17_memory_leak_error_path.c`	CWE-401	Slow denial of service

WATCH OUT

Not every demo bug can actually be *triggered* by running the program. The manifest in `tests/benchmarks/targets.py` says so explicitly for several of them (01, 08, 10, 13, 14, 18, 19, 20) and gives a written reason each time; for example, file 01's overflow is unreachable because the required header contains a zero byte, which a command-line argument cannot contain. This honesty matters later: those bugs can be found by reading code, but they cannot have a runtime "proof", so the project does not pretend they do.

5. Naming bugs: CWE and CVE

Security people need shared names, just as doctors need shared names for diseases. Two catalogues are used everywhere:

- **CWE (Common Weakness Enumeration)** is a catalogue of *kinds* of mistake. Each kind has a number: CWE-787 is "out-of-bounds write", CWE-416 is "use after free", CWE-190 is "integer overflow", CWE-78 is "OS command injection", and so on. A CWE is like a disease name, such as "influenza".
- **CVE (Common Vulnerabilities and Exposures)** is a catalogue of *specific* bugs found in specific real products, each with an ID like `CVE-2019-11835` (year, then a number). A CVE is like one recorded outbreak: "the influenza outbreak in this town in 2019". This project's "real CVE" test suite uses two real CVEs in a small, widely-used C library called cJSON, which reads JSON data (JSON is explained in section 11).

When the project catches a crash, it reads the crash report's description and translates it into a CWE number using a lookup table:

src/analysis_engine/crash_triage.py · lines 38–58 (trimmed)

```
# ASan/UBSan crash class -> (CWE, severity). Used to label the report the agents see.
CRASH_CLASS_TO_CWE = {
    "heap-buffer-overflow": ("CWE-122", "critical"),
    "stack-buffer-overflow": ("CWE-121", "critical"),
    "global-buffer-overflow": ("CWE-787", "critical"),
    "heap-use-after-free": ("CWE-416", "critical"),
    "double-free": ("CWE-415", "critical"),
    ...
    "SEGV": ("CWE-476", "critical"),
    ...
    "requested allocation size": ("CWE-190", "critical"),
    "signed-integer-overflow": ("CWE-190", "high"),
    ...
    "index-out-of-bounds": ("CWE-125", "critical"),
    ...
    "null-pointer-dereference": ("CWE-476", "critical"),
}
```

Read each line as "if the crash report mentions *this phrase*, call it *this CWE*, with *this severity*". For instance a report containing "double-free" becomes CWE-415, critical. `SEGV` is short for "segmentation violation", the operating system's name for "you touched memory you have no right to", typically from a NULL or garbage pointer. This label is later handed to the AI so it knows what kind of problem it is fixing.

6. Proof of vulnerability (PoV)

Claiming "this code has a bug" is cheap. Showing it is better. A **PoV (proof of vulnerability)** is a concrete input, plus a way of running the program on it, that demonstrates the bug actually happens. You have already met several: `AAAA` for the off-by-one, `ERROR` for the double free, `134217729` for the integer overflow.

The project's rule for every PoV is a deliberate, slightly back-to-front convention: **the PoV command exits with 0 ("success") only when the bug is absent**. So on the original, broken program it must *fail*; on a correctly repaired program it must *pass*. Before any repair is attempted, the project runs the PoV on the original program to confirm it fails (this is called **pre-flight**). If a PoV does not fail on the broken program, it proves nothing, and the project refuses to count it.

Here is the core of the general-purpose PoV runner:

tests/benchmarks/harness/pov_run.sh · lines 14–39 (trimmed)

```
ASAN_OPTIONS=detect_leaks=0
MallocNanoZone=0
export ASAN_OPTIONS MallocNanoZone

output=$("$@" 2>&1)
rc=$?
    ...
case "$output" in
    *"AddressSanitizer"*|*"UndefinedBehaviorSanitizer"*|*"runtime error:"*|*"LeakSanitizer"*)
        echo "$output"
```

```
        exit 1
        ;;
esac

# 128+N indicates death by signal N (SIGSEGV=139, SIGABRT=134, SIGBUS=138).
if [ "$rc" -ge 128 ]; then
    echo "process died on signal (exit $rc); output follows:"
    echo "$output"
    exit 1
fi

exit 0
```

LINE BY LINE

1. This is a **shell script**: a file of commands for the shell, rather than a C or Python program.
2. The first three lines set two **environment variables** (settings that a program can read when it starts). `detect_leaks=0` switches off leak reports for this runner, so only the bug under test can make it fail. `MallocNanoZone=0` is a macOS-specific setting that keeps the Mac's own memory system from interfering with the sanitizer.
3. `output=$("$@" 2>&1)` runs the program with all the arguments the script received (`"$@"`), and captures everything it prints, including error messages (`2>&1` means "send the error channel to the same place as normal output").
4. `rc=$?` saves the program's exit code.
5. The `case` block checks whether the captured text contains the name of any sanitizer, or the phrase `runtime error:` . If so, it prints the report (so the AI can read it later) and exits with 1: "bug still present".
6. The `if` block catches a different kind of failure: an exit code of 128 or more means the operating system killed the program with a **signal** (an emergency stop), such as 139 for a segmentation fault. That also counts as "bug still present".
7. Otherwise, exit 0: the program ran on the dangerous input without any alarm, so the bug is considered gone.

A PoV does not have to be about a crash. As we saw, the command injection PoV looks for a marker file, and the hardcoded key PoV looks for the key's text inside the binary. What all PoVs share is the contract: *exit 0 only when the weakness is absent.*

WATCH OUT

A PoV checks *one* input. A sloppy repair could block that one input and leave the underlying bug in place (for example, "if the input is exactly `AAAA` , stop"). This is called **overfitting**. The project has extra machinery, "hardening", to catch cheats like this, which a later chapter covers. Keep in mind: passing the PoV is necessary, not sufficient.

7. Sanitizers: making silent bugs loud

Remember that the off-by-one program will usually run "successfully" even though it writes past its buffer. If bugs are silent, how can a machine ever catch them? The answer is a **sanitizer**: an extra layer of checking that the compiler weaves into the program when you build it with a `-fsanitize=...` flag. The program then checks itself as it runs and stops with a detailed report the instant it misbehaves.

ANALOGY

Imagine painting a bright red strip on the floor around every storage unit in the warehouse, and installing a camera that sounds an alarm whenever anyone steps on red. Normally nobody notices someone standing slightly outside their unit. With the red strips, the alarm goes off at the exact moment, with a photo of who did it and where. That is roughly how AddressSanitizer works: it surrounds every buffer with "poisoned" boxes, called **red zones**, and checks every memory access against a hidden map of which boxes are poisoned.

The three sanitizers in this project:

Sanitizer	Flag	Catches
AddressSanitizer (ASan)	`-fsanitize=address`	Reading or writing out of bounds (heap, stack and global variables), use-after-free, double free
UndefinedBehaviorSanitizer (UBSan)	`-fsanitize=undefined`	Many "undefined behaviour" cases: signed integers overflowing, invalid shifts, misaligned memory accesses, and more

Sanitizer	Flag	Catches
LeakSanitizer (LSan)	bundled with ASan; enabled with `detect_leaks=1`	Memory that was never freed when the program ends

The cost is speed: a sanitized program typically runs noticeably slower and uses more memory. That is fine for testing, which is the only place this project uses them.

What does a report look like? Below is a **simplified illustration** of what AddressSanitizer prints for the off-by-one program run with `AAAA` (real reports have more lines, and the addresses differ from run to run):

```
==12345==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x602000000014
WRITE of size 1 at 0x602000000014 thread T0
    #0 in copy_and_null_terminate 06_off_by_one_loop.c:10
    #1 in main 06_off_by_one_loop.c:20

0x602000000014 is located 0 bytes after 4-byte region
allocated by thread T0 here:
    #0 in malloc
    #1 in copy_and_null_terminate 06_off_by_one_loop.c:6

SUMMARY: AddressSanitizer: heap-buffer-overflow 06_off_by_one_loop.c:10
```

How to read it:

- **The headline** names the crash class (`heap-buffer-overflow`), which is exactly the phrase the CWE table in section 5 looks for.
- "**WRITE of size 1**": the program wrote one byte (our terminating 0).
- **The stack trace** (lines starting `#0`, `#1`): the chain of function calls at the moment of the error, most recent first. `#0` is where it happened: line 10. `#1` is who called that function: `main`, line 20. This is why the build kept debugging information with `-g`.
- "**0 bytes after 4-byte region**": the write landed immediately after a 4-byte buffer. That is the textbook signature of an off-by-one.
- "**allocated by … here**": where that buffer was rented, line 6. That points straight at the real fix.

This text is gold for automation. The project's triage code reads it to find the file, line and crash type, and the text is passed to the AI as the "crash trace" so it can see exactly what went wrong.

WATCH OUT

Sanitizers are powerful but not all-seeing, and the project documents the gaps. ASan tracks the edges of whole allocations, not of fields inside a struct, so in `18_ast_deep_struct.c`, where a copy overruns a 64-byte field but stays inside the larger 100-byte object, ASan sees nothing. Uninitialised reads need a different tool, **MemorySanitizer**, which is Linux-only. And macOS's built-in compiler ships without LeakSanitizer, so the leak demo needs the separately installed (Homebrew) LLVM.

8. Fuzzing: throwing millions of odd inputs at a program

Sanitizers only shout when a bad input actually reaches the bug. Someone still has to *find* such an input. Guessing by hand is slow. **Fuzzing** automates it: a **fuzzer** generates enormous numbers of inputs, feeds each to the program, and watches for crashes or sanitizer alarms.

ANALOGY

A toddler let loose on a new TV remote, pressing random buttons thousands of times a second, will eventually find the strange combination that freezes the TV. A *smart* toddler would notice which presses made new things happen on screen and focus on those. Modern fuzzers are the smart toddler.

Key terms:

- **Coverage-guided fuzzing**: the fuzzer measures which parts of the code each input reached (its **coverage**). Inputs that reach new code are kept and mutated further (bits flipped, bytes inserted, pieces spliced together). Over time the fuzzer burrows deeper into the program.
- **Seeds**: starting inputs you give the fuzzer so it does not start from nothing. The project keeps a few in `tests/fuzz_harnesses/seeds/` (`num.bin`, `pair.bin`, `short.bin`; the last one is just the two bytes `AB`).
- **Corpus**: the growing collection of "interesting" inputs the fuzzer has kept.
- **Harness**: a small piece of C code that connects the fuzzer to the function under test. The fuzzer produces raw bytes; the harness turns them into a sensible call.

The project uses two well-known fuzzers:

- **libFuzzer**, part of LLVM. It runs inside the program being tested and calls the harness function over and over, extremely fast.
- **AFL++** ("American Fuzzy Lop plus plus", named after a breed of rabbit). It starts a fresh copy of the program for each input. The project includes a small adapter, `tests/fuzz_harnesses/afl_driver.c`, so the same harness works with both.

Here is the real harness for the off-by-one program:

tests/fuzz_harnesses/SYN-06-OFF-BY-ONE.c · lines 1–17

```
/* Fuzz harness for 06_off_by_one_loop.c
 * The bug: loop writes dest[size] on a malloc(size) buffer. */
#include <stdint.h>
#include <stddef.h>
#include <stdlib.h>
#include <string.h>

void copy_and_null_terminate(const char* src, size_t size);

int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    if (size == 0 || size > 1024) return 0;
    char *s = (char *)malloc(size + 1);
    memcpy(s, data, size); s[size] = '\0';
    copy_and_null_terminate(s, size);
    free(s);
    return 0;
}
```

LINE BY LINE

1. Lines 1–2 are a comment (`/* ... */` is C's other comment style) describing the bug being hunted.
2. Line 8 announces that a function called `copy_and_null_terminate` exists elsewhere (in the demo file); the harness is going to call it.
3. Line 10: `LLVMFuzzerTestOneInput` is the special name libFuzzer looks for. libFuzzer calls it once per generated input, passing the bytes (`data`) and how many there are (`size`).
4. Line 11 respects the function's own rules: it refuses empty or over-long inputs. (The project's harness README stresses that a harness must not feed a function inputs its contract forbids, otherwise no fix could ever satisfy it.)
5. Lines 12–13 make a properly terminated copy of the fuzz bytes. Note that the *harness* correctly uses `size + 1`.
6. Line 14 calls the buggy function. With ASan built in, the very first input reaching the bad write triggers a report.
7. Lines 15–16 tidy up and tell libFuzzer "done, give me the next input".

One practical detail: a C program may have only one `main`, and the fuzzer brings its own. So when the project builds a demo program for fuzzing, it renames the program's `main` out of the way with a compiler flag (`-Dmain=kavach_disabled_main_<id>`), as noted in the project overview.

When the fuzzer finds a crash, it saves the guilty input as a file. That file becomes a ready-made PoV: replay it and see whether the alarm still fires.

9. Static analysis: reading code without running it

Fuzzing is **dynamic** analysis: it learns by running the program. **Static analysis** learns by reading the source code, the way a proofreader reads a manuscript without acting out the play. It can spot bugs on paths the fuzzer never reached, or bugs that never crash (like the hardcoded key, or the TOCTOU race). The trade-off: it often raises **false positives** (warnings about things that are actually fine), because it guesses about behaviour rather than observing it.

	Static analysis	Fuzzing + sanitizers
Runs the program?	No, reads the code	Yes, millions of times
Strength	Sees every line, including rare paths and non-crashing bugs	When it reports a bug, the bug is real and comes with a crashing input
Weakness	False alarms; no proof the bug is reachable	Only finds what it manages to reach; some bugs never crash

The project's static analyser is **Semgrep**. Semgrep works with **rules**: patterns that describe suspicious code, such as "a call to `printf` whose first argument is not fixed text" or "a call to `system`". The project uses a public rule pack called `p/security-audit`. Here is how it runs Semgrep:

src/analysis_engine/semgrep_runner.py · lines 102–119

```
        try:
            cmd = [
                "semgrep", "scan",
                "--config", self.rules,
                "--sarif",
                "--output", sarif_path,
                "--quiet",
                "--no-git-ignore",
                str(target),
            ]

            logger.info(f"Running: {' '.join(cmd)}")
            result = subprocess.run(
                cmd,
                capture_output=True,
                text=True,
                timeout=timeout,
            )
```

LINE BY LINE

1. This is Python, not C (section 11 explains Python basics). `cmd = [ ... ]` builds a **list** of words: the same words you would type in a terminal to run Semgrep yourself.
2. `"--config", self.rules` chooses the rule pack (by default `p/security-audit`).
3. `"--sarif"` plus `"--output", sarif_path`: write the results in SARIF format to a file (explained just below).
4. `"--quiet"` reduces chatter; `"--no-git-ignore"` tells Semgrep to scan files even if the project's version-control settings say to ignore them.
5. `str(target)` is the file or folder to scan.
6. `logger.info(...)` writes a note to the program's log, showing the full command.
7. `subprocess.run(...)` actually runs Semgrep as a separate program, captures what it prints, and gives up if it takes longer than `timeout` seconds.

SARIF (Static Analysis Results Interchange Format) is a standard file layout for analyser results, so that any tool can read any other tool's findings. Each finding says which rule fired, a message, and the file and line. It is written in JSON (section 11). The project converts each finding into its own internal record and, if a patch is being verified, it can run Semgrep again afterwards to make sure the patch did not introduce new warnings.

WATCH OUT

The `p/security-audit` rule pack is downloaded from the internet when Semgrep runs. The project overview flags this as a problem for "air-gapped" use (a computer deliberately cut off from the internet, common in military settings).

10. Large language models (LLMs)

10.1 What they are

A **large language model (LLM)** is a program trained on a vast amount of text, including a lot of source code, to do one thing: given some text, predict what text should come next. Ask it "fix the bug in this function" alongside the code, and it continues with something that looks like a fix, because it has seen countless examples of bugs and fixes. ChatGPT is built on LLMs of this kind. This project talks to OpenAI's models through the internet (by default a small, cheap one called `gpt-4o-mini`), or to any compatible server you run yourself.

The project uses several LLM "agents". An **agent** here is just the same model given different standing instructions (a **system prompt**) so that it behaves in a particular role, such as a careful analyst, a minimalist who writes the smallest possible fix, or a critic who reviews failed attempts. Later chapters cover them.

10.2 Tokens and cost

LLMs do not read letters or whole words; they read **tokens**, chunks of text that are often a word or part of a word. As a rough rule of thumb, one token is about four characters of English. Providers charge per token, separately for the text you send in (**input**) and the text the model writes back (**output**). The project keeps a price list:

src/agent_orchestrator/llm_client.py · lines 317-323

```
    # USD per 1M tokens, (input, output). Keyed by model id prefix.
    PRICING = {
        "gpt-4o-mini": (0.15, 0.60),
        "gpt-4o": (2.50, 10.00),
        "gpt-4.1-mini": (0.40, 1.60),
        "gpt-4.1": (2.00, 8.00),
    }
```

So `gpt-4o-mini` costs 15 US cents per million input tokens and 60 cents per million output tokens. That is very cheap: the project's own benchmark document reports about $0.07 in total for 161 model calls across 44 test programs. The low cost is a deliberate design choice, made possible because the model does not have to be trusted (see 10.4).

10.3 Temperature

At every step the model has a list of possible next tokens, each with a probability. **Temperature** controls how adventurous its choice is. At high temperature it sometimes picks less likely tokens (more creative, more varied). At temperature 0 it always picks the most likely one (called **greedy decoding**), which should make answers as repeatable as possible. The project sets it to zero:

config/__init__.py · lines 37-43

```
        # Greedy decoding. Note this does NOT make the API deterministic:
        # batch-dependent floating-point reduction order and tie-breaking between
        # equally-probable tokens still vary. `seed` below plus the recorded
        # system_fingerprint make a deviation *detectable* even when it cannot
        # be eliminated.
        "temperature": 0.0,
        "seed": int(os.getenv("KAVACH_SEED", "1337")),
```

The comment is admirably honest. Even at temperature 0, the same question can occasionally get a different answer from a hosted model, because of tiny numerical differences in how the provider's computers do the arithmetic. A **seed** (here 1337 by default, overridable with the `KAVACH_SEED` setting) is a starting number for the randomness; supplying the same seed asks the service to make the same choices, and recording a "fingerprint" of the provider's system lets you at least *detect* when results drifted.

10.4 Why LLMs can be confidently wrong

An LLM produces text that is *likely*, not text that is *checked*. It has no built-in step where it compiles the code or runs a test. So it can produce a patch that looks perfectly professional, explains itself fluently, and is wrong: it may not compile, may fix the wrong line, may block only the one example input, or may "fix" the bug by deleting the feature. The model sounds equally confident either way. Making things up that sound plausible is often called **hallucination**.

> **KEY IDEA**
>
> This is exactly why the project's motto is "the LLM proposes, deterministic tools decide". A **deterministic** tool is one that gives the same answer every time for the same input: a compiler, a sanitizer, a PoV script, a test. The AI's patch is treated as a *suggestion*. It is accepted only if it applies cleanly, builds, makes the PoV pass, keeps normal behaviour working (the regression test), and so on. The AI never gets to grade its own homework.

11. Formal verification: proof instead of testing

Testing, and even fuzzing, can only show that the inputs you *tried* are fine. **Formal verification** tries to show something stronger: that *no* input can cause a problem, by reasoning mathematically about the code instead of running it.

ANALOGY

Testing a bridge means driving a lot of different trucks over it and seeing that none fall through. Formal verification means doing the engineering calculations that show *every* truck under 40 tonnes is safe, including ones you never drove.

The project uses **CBMC**, the "C Bounded Model Checker". You give it the C code plus a small **proof harness** that describes which inputs are possible, and it explores *all* of them at once (using clever logic rather than trying them one by one). If it finds any input that breaks a safety rule, it reports a **counterexample**: the exact input that does it.

The catch is the word **bounded**. Loops could, in principle, run forever, so CBMC only "unrolls" each loop up to a chosen number of times, the **unwind bound**. The proof says "safe for every input, as long as no loop runs more than N times". The flag `--unwinding-assertions` makes CBMC *tell you* if some loop could run longer than N, so a too-small bound is reported rather than silently producing a false "proven". Here are the safety checks the project asks CBMC to prove:

src/patch_validator/formal.py · lines 55–65

```
# Safety properties CBMC is asked to prove. Each maps to a class of defect the
# corpus actually contains.
DEFAULT_CHECKS = (
    "--bounds-check",              # array index out of range
    "--pointer-check",             # invalid / null / out-of-object dereference
    "--conversion-check",          # lossy integer conversion (truncation)
    "--signed-overflow-check",     # signed arithmetic overflow
    "--unsigned-overflow-check",   # unsigned wrap-around
    "--div-by-zero-check",
    "--unwinding-assertions",      # tell us when the bound was insufficient
)
```

Notice how the list lines up with our bug zoo: out-of-bounds access, bad pointers, truncation, overflow and wrap-around. And here is the proof harness for the off-by-one program:

tests/benchmarks/cbmc_harnesses/SYN-06-OFF-BY-ONE.c · lines 1–15

```
/* Proof: copy_and_null_terminate must not write outside its allocation for any
 * size it accepts. The caller passes strlen(argv[1]), so any size is possible;
 * the function's own guard rejects 0 and >1024. */
#include <stddef.h>
void copy_and_null_terminate(const char *src, size_t size);
unsigned long nondet_ulong(void);

#define SRC 64
static char source[SRC];

void harness(void) {
    size_t size = nondet_ulong();
    __CPROVER_assume(size <= SRC);      /* src really holds `size` bytes */
    copy_and_null_terminate(source, size);
}
```

LINE BY LINE

1. Lines 1–3 state the goal: the function must never write outside the memory it rented, for any size it accepts.
2. Line 6 declares `nondet_ulong`. "Nondet" means **nondeterministic**: to CBMC this function returns "any possible number at all". There is no code for it; CBMC treats it as "consider every value".
3. Lines 8–9 create a 64-byte source buffer.
4. Line 12 picks a size: every possible size simultaneously, as far as CBMC is concerned.
5. Line 13, `__CPROVER_assume`, narrows that down: "only consider sizes up to 64", because the source buffer really only holds 64 bytes. Without this, CBMC would blame the function for a mistake the *caller* made.
6. Line 14 calls the function. CBMC then checks every write inside it against the safety rules. On the buggy original it finds the counterexample (any accepted size writes one byte too far). On a correctly fixed version it reports `VERIFICATION SUCCESSFUL`.

The manifest sets this target's unwind bound to 70 (the comment says the loop runs up to the 64-byte harness buffer plus one), which comfortably covers every loop run the harness allows. Only three targets in the project have CBMC harnesses; the rest are not formally verified, and the reports say so rather than pretending otherwise.

12. Python basics you will meet later

The demo programs are in C, but AI Kavach itself is written in **Python**, a language that is much easier to read. You do not need to write Python to follow this guide, but you will see a lot of it. Here are the building blocks.

12.1 Modules and files

Each `.py` file is a **module**: a self-contained unit of code. Folders of modules are **packages**. When you see `src/analysis_engine/crash_triage.py`, that is the module `crash_triage` inside the package `analysis_engine`. One module uses another with an `import` line, for example `from dataclasses import dataclass` ("from the standard module `dataclasses`, bring in the tool called `dataclass`"). In Python, `#` starts a comment, and **indentation matters**: lines indented under a heading line belong to it.

12.2 Functions: `def`

A function is defined with `def name(inputs):` followed by an indented body. `return` hands back a result. Python functions can return several values at once, as a **tuple** (a fixed bundle of values in brackets). Here is a real one that asks "is CBMC installed?":

src/patch_validator/formal.py · lines 110–119

```
    def available() -> tuple[bool, str]:
        try:
            r = subprocess.run(["cbmc", "--version"], capture_output=True, text=True, timeout=30)
            if r.returncode == 0:
                return True, f"cbmc {r.stdout.strip()}"
            return False, "cbmc present but did not report a version"
        except FileNotFoundError:
            return False, "cbmc not installed (brew install cbmc)"
        except Exception as e:
            return False, f"cbmc probe failed: {e}"
```

LINE BY LINE

1. `def available() -> tuple[bool, str]:` defines a function with no inputs. The `-> tuple[bool, str]` part is a **type hint**, a label for human readers: "this returns a pair of a yes/no value and some text". `bool` means a yes/no value, `True` or `False`; `str` means a string of text.
2. `try:` begins a block that might fail; the `except` blocks below say what to do if it does. This is how Python handles errors gracefully instead of crashing.
3. The `subprocess.run(...)` line runs the separate program `cbmc --version`, captures what it prints, and waits at most 30 seconds. `r` stores the outcome.
4. `r.returncode` is that program's exit code. 0 means success, so return `True` plus a message. `f"..."` is an **f-string**: text where anything inside `{ }` is filled in, here CBMC's version text with spaces trimmed (`.strip()`).
5. If the exit code was not 0, return `False` with an explanation.
6. `except FileNotFoundError:` runs if there is no program called `cbmc` at all, and suggests how to install it.
7. `except Exception as e:` catches any other error and reports it.

12.3 Running other programs: `subprocess`

You have now seen `subprocess.run` twice. `subprocess` is Python's standard module for starting *other* programs (clang, Semgrep, CBMC, the PoV scripts, fuzzers) and collecting their exit codes and output. This is the glue of the whole project. AI Kavach does very little of the heavy lifting itself; mostly it *orchestrates* trusted external tools and reads their verdicts. That fits the motto: the deterministic tools decide, and Python just asks them and listens.

12.4 Classes and dataclasses

A **class** is a blueprint for a kind of object that bundles data with the functions (called **methods**) that work on it. `class FormalVerifier:` describes a "formal verifier" thing; each actual one created from it is an **instance** or **object**. Inside methods, `self` means "this particular object",

so `self.rules` is "this Semgrep runner's chosen rule pack".

A **dataclass** is a shortcut for the very common case of a class that mainly holds named fields, like a form with labelled boxes. You list the fields and their types, and Python writes the boring setup code for you. The project's description of one test target is a dataclass:

tests/benchmarks/targets.py · lines 75-89 (trimmed)

```
@dataclass
class Target:
    """One benchmark target and its complete verification contract."""

    id: str
    suite: str
    file_path: str
    line_number: int
    cwe_id: str
    description: str
    complexity: str                      # easy | medium | hard | ast-dependent
    build_command: str
    pov_command: Optional[str] = None
    pov_na_reason: str = ""
    regression_command: Optional[str] = None
    ...
```

LINE BY LINE

1. `@dataclass` is a **decorator**: a tag placed above a definition that modifies it. Here it says "turn the following class into a dataclass".
2. The text in triple quotes is a **docstring**, a built-in description of the class.
3. Each following line is one field: a name, a colon, and its type. `int` is a whole number; `str` is text.
4. `Optional[str] = None` means "this may be text, or may be missing", and if you do not supply it, it defaults to `None`, Python's word for "nothing".

And this is how the off-by-one target is filled in:

tests/benchmarks/targets.py · lines 192-200

```
    Target(
        id="SYN-06-OFF-BY-ONE", suite="Synthetic", complexity="easy",
        file_path=f"{D}/06_off_by_one_loop.c", line_number=8,
        cwe_id="CWE-193", description="Off-by-one heap write in NUL-termination loop",
        build_command=SINGLE_FILE_BUILD,
        pov_command=_pov("AAAA"),
        regression_command=_regress("Copied string: hi", "hi"),
        cbmc_unwind=70,          # loop runs up to the 64-byte harness buffer + 1
    ),
```

Everything you learned in this chapter is in these few lines: the source file, the buggy line (8), the CWE (193), the sanitizer build command, the PoV input (`AAAA`), a **regression test** (run with the normal input `hi` and confirm it still prints `Copied string: hi`, so a "fix" that breaks normal use is caught), and the CBMC unwind bound.

Dataclasses can also run a check right after being filled in, in a method called `__post_init__`. The project uses this to enforce honesty:

tests/benchmarks/targets.py · lines 112-118

```
    def __post_init__(self):
        if self.pov_command is None and not self.pov_na_reason:
            raise ValueError(f"{self.id}: pov_command=None requires pov_na_reason")
        if self.regression_command is None and not self.regression_na_reason:
            raise ValueError(
                f"{self.id}: regression_command=None requires regression_na_reason"
            )
```

In words: "if this target has no PoV, it must have a written reason why not; otherwise stop with an error (`raise ValueError`)." That is why every untriggerable demo bug in section 4 came with an explanation. The code literally refuses to load a target that skips the proof without saying why.

12.5 JSON

JSON (JavaScript Object Notation) is a simple, very widely used text format for structured data. It has named fields in curly braces, lists in square brackets, text in quotes, plus numbers and `true` / `false` / `null`. For example, the project's critic agent is required to answer in JSON with the fields `verdict`, `root_cause`, `what_the_patch_missed` and `instruction`. An answer shaped like that might look like this (an invented illustration, not real output):

```
{
  "verdict": "reject",
  "root_cause": "buffer allocated with size bytes but loop writes size+1",
  "what_the_patch_missed": "the allocation on line 6",
  "instruction": "allocate size + 1 bytes instead of changing the loop"
}
```

Because JSON is so regular, a program can read it reliably, which is exactly why the project demands it from the AI when the answer has to be processed by code rather than read by a person. SARIF files and the project's memory files (for example `benchmark_workspace/memory/semantic.json`) are JSON too.

TRY IT YOURSELF

From the repository folder, you can look at any of these files in a terminal:

- `cat tests/demo_vulns/06_off_by_one_loop.c` shows the off-by-one program.
- `cat -n tests/demo_vulns/05_double_free_conditional.c` shows it with line numbers, so you can match the walkthrough above.
- If clang works on your machine, build the off-by-one demo with sanitizers and run it: `clang -fsanitize=address,undefined -g -O0 tests/demo_vulns/06_off_by_one_loop.c -o /tmp/off1` then `/tmp/off1 AAAA`. Compare with a build that leaves out `-fsanitize=...`: the plain build will probably print the copy happily, while the sanitized one stops with a report.
- Then try the proof script: `sh tests/benchmarks/harness/pov_run.sh /tmp/off1 AAAA; echo $?`. It should print the report and the number `1` ("bug still present").

13. How it all fits together

You now have every piece. Here is how the project combines them, in the vocabulary of this chapter:

```
C source code (with a bug)
      |
      +--> Semgrep reads it (static) ---------------+
      |                                             |
      +--> build with clang + sanitizers            |
           fuzz with libFuzzer / AFL++ (dynamic)    |
           crash report -> crash class -> CWE       |
                                                    v
                     bug report: file, line, CWE, crash trace
                                    |
                                    v
                           LLM agents PROPOSE a patch
                                    |
                                    v
           deterministic tools DECIDE:
             applies? -> builds? -> PoV passes? -> regression passes?
                                    |
                     fail: feed the reason back, try again
                     pass: try to break it (hardening),
                           optionally prove it (CBMC)
```

The following chapters take each box in turn and walk through the real code that implements it.

CHAPTER RECAP

- **Source code** is human-readable text; a **compiler** (here clang) translates it into a runnable **binary**. Doing so is **building**, and flags such as `-fsanitize=address,undefined` change how it is built.
- **Memory** is a long row of numbered byte-boxes. **Buffers**/arrays are runs of boxes, counted from 0; C strings end with a hidden 0 byte. The **stack** holds function-local data automatically; the **heap** is rented with `malloc` and returned with `free`.
- **C does not check** memory accesses, which makes it fast but lets bugs be **silent**, with **undefined behaviour** as the result.
- The bug zoo: overflow, off-by-one, out-of-bounds read, use-after-free, integer overflow/truncation/underflow, null dereference, double free, format string, command injection, path traversal, TOCTOU, uninitialised memory, hardcoded secret, type confusion and memory leak. Each lets an attacker crash, leak data, or take control.
- A **CWE** names a kind of weakness; a **CVE** names one specific real-world bug.
- A **PoV** demonstrates a bug; in this project it exits 0 only when the bug is *absent*, and must fail on the original before it counts.
- **Sanitizers** (ASan, UBSan, LSan) make silent bugs stop the program with a precise report, but have blind spots.
- **Fuzzing** automatically generates inputs, guided by coverage, starting from **seeds**, keeping a **corpus**, through a **harness**.
- **Static analysis** (Semgrep, results in **SARIF**) reads code without running it: wider reach, more false alarms.
- **LLMs** predict likely text, are billed per **token**, can be made more repeatable with **temperature** 0, and can be confidently wrong, so their patches must be checked by deterministic tools.
- **CBMC** gives a **bounded proof** covering every input, up to a loop-unwinding limit.
- The project is written in **Python**: modules, `def` functions, classes and **dataclasses**, `subprocess` for running tools, and **JSON** for structured data.

CHECK YOURSELF

1. In `06_off_by_one_loop.c`, the program calls `malloc(size)` and then loops with `i <= size`. With input `AAAA`, which box gets written that should not, and why does the program usually still appear to work when built without sanitizers?
2. In `03_silent_integer_overflow.c`, why does the input `134217729` lead to only 32 bytes being allocated, and why does that turn into a heap overflow a moment later?
3. The PoV for the command injection demo never crashes the program. How does it prove the bug, and what does an exit code of 0 from a PoV mean in this project?
4. Name one bug in the demo set that AddressSanitizer cannot detect at run time, and say which other technique the project relies on to find it.
5. Why is it safe for the project to use a cheap LLM that is sometimes confidently wrong? Answer using the phrase "the LLM proposes, deterministic tools decide".

CHAPTER 2

The Big Picture: What AI Kavach Does

Chapter 1 gave you the vocabulary: memory, buffers, the kinds of memory bugs, fuzzers, sanitizers, large language models (LLMs) and the CBMC proof tool. This chapter uses that vocabulary to give you a map of the whole project before later chapters go into each part. By the end you should be able to say in your own words what AI Kavach does, name every folder in the repository, pick the right program to run, and follow one real bug from "found" to "proven fixed."

The chapter has six parts:

1. The problem and the philosophy behind the solution.
2. The pipeline: the stages a bug passes through, one paragraph each.
3. A folder map of the repository.
4. The three "front doors" (entry points) and when to use each.
5. **The life of one bug**: a detailed story that follows a single off-by-one bug through the whole system.
6. The data objects that carry information from one stage to the next.

1. The problem, and the one idea that shapes everything

The problem

A lot of the world's important software is written in C, including operating systems, routers, embedded controllers in vehicles and weapons, and network services. C gives the programmer direct control over memory and does almost nothing to stop mistakes. One wrong `<=` where a `<` belonged can let a program write one byte past the end of a buffer. An attacker who finds that byte may be able to crash the program or take it over.

Finding such bugs is slow, expert work, and fixing them is slower still. AI Kavach (*kavach* means "armour" in Hindi) is an attempt to automate both jobs. The README introduces it as "an autonomous Cyber Reasoning System for the Indian Army Terrier Cyber Quest 2026." A **Cyber Reasoning System (CRS)** is the general name for a program that finds and fixes security bugs with no human in the loop. The term comes from DARPA's Cyber Grand Challenge and its later AI Cyber Challenge.

In one sentence: you give AI Kavach some C code; it **finds** memory-safety bugs, **locates** the exact line, asks several LLM "agents" to **write a patch**, and then **proves** the patch works by rebuilding the program and running the original attack against it. It then goes a step further and **tries to break its own patch**. Here is how the README puts it:

> **KEY IDEA (IN THE PROJECT'S OWN WORDS)**
>
> "If a patch cannot be shown to work, it is not reported as working." (README.md)

The philosophy: "the LLM proposes, deterministic tools decide"

LLMs are good at writing plausible-looking code, but they are not reliable. A model can say "I fixed the overflow" while the overflow is still there. A model can also "fix" a bug in a lazy way, for example by making the function return immediately, so nothing useful happens at all. If you let the LLM grade its own work, you will get confident wrong answers.

AI Kavach therefore splits the work into two roles that never mix:

- **The LLM proposes.** Its only job is to suggest a patch. It never decides whether the patch is good.
- **Deterministic tools decide.** A tool is *deterministic* if it gives the same answer every time for the same input, with no opinions involved. The compiler (does the patched code build?), the sanitizer (does the attack input still trip a memory error?), a regression test (does normal input still produce normal output?), a fuzzer and CBMC all work this way. Only these tools can accept or reject a patch.

> **ANALOGY: THE STUDENT AND THE EXAM BOARD**
>
> Think of the LLM as a bright but sometimes careless student, and the deterministic tools as an exam board with an answer key. The student writes an answer, but the student never marks it. The exam board marks it, and if the answer is wrong it hands the paper back with the exact reason ("line 8 still writes past the buffer"). The student tries again. The exam board can't be talked round, so even a student who bluffs can't get a wrong answer accepted.

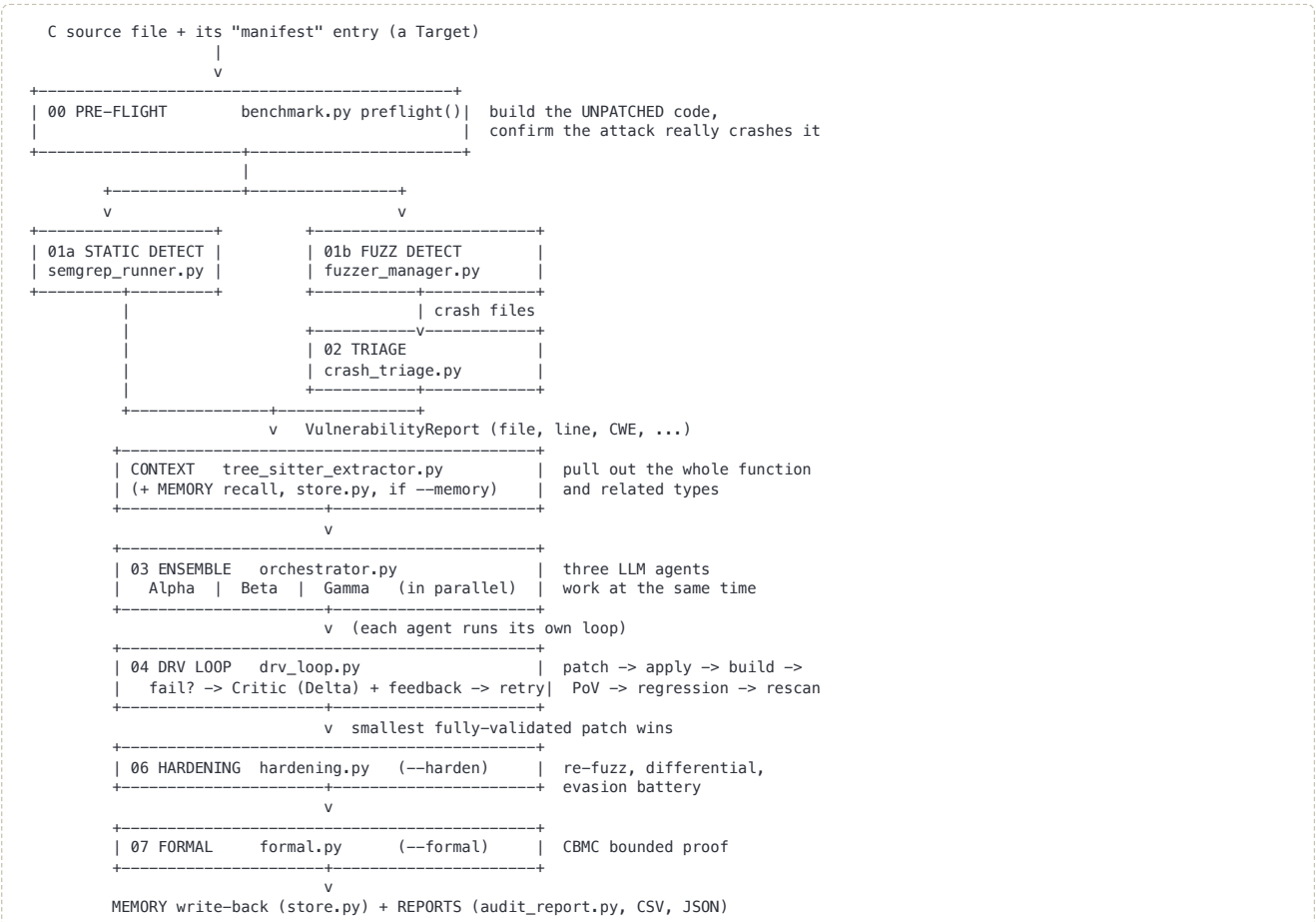
This split has a useful side-effect. Because a tool checks every answer, the LLM does not need to be brilliant, only good enough to get there eventually. The project uses a small, cheap model (`gpt-4o-mini` by default, set in `config/__init__.py`). The README reports a total LLM bill of

$0.0739 for a run over 44 targets.

> **KEY IDEA: THREE INCREASINGLY STRONG LEVELS OF TRUST**
>
> 1. **Gates** (in the "DRV loop"): the patch applies, builds, defeats the original attack input, and keeps normal behaviour. Passing these means the status is `patched`.
> 2. **Hardening**: the system *attacks* the accepted patch with a fresh fuzzer and with side-by-side comparison against the original program. A patch that survives this is "hardened."
> 3. **Formal proof**: for a few targets, CBMC proves mathematically that no input (within a stated bound) can make the patched function unsafe.
>
> Each level is optional and is reported separately, so you always know how much a given "fixed" claim is worth.

2. The pipeline at a glance

A **pipeline** is a series of stages where the output of one stage becomes the input of the next, like an assembly line. Here is AI Kavach's pipeline. Each box names the file that does the work.



The numbers (00, 01a, 01b…) follow the project's own phase numbering. Phase 05, "Driller" (a technique that mixes fuzzing with a symbolic-execution tool called angr), exists in `src/analysis_engine/driller_monitor.py`, but only its self-test is ever called. It is not part of the live pipeline, which is why it does not appear in the diagram.

Stage by stage

00 Pre-flight (`benchmark.py`, function `preflight`, lines 116–160). Before any AI is involved, the system compiles the original, unpatched program and runs the "proof of vulnerability" (**PoV**) against it. A PoV is a command that runs the program on the attack input and exits with 0 only if no memory error happened. On the unpatched program the PoV *must fail*. If it doesn't, the attack was never shown to work, and the system refuses to use it as evidence later. Without this rule a patch could get credit for "fixing" a bug that never fired in the first place.

01a Static detection (`src/analysis_engine/semgrep_runner.py`). Semgrep is a pattern-matching scanner. It reads the source code without running it and flags risky patterns, such as `strcpy` into a fixed-size buffer. It writes its findings in SARIF, a standard JSON format for scanner

results, and the runner turns each one into a `SarifFinding` object. This is the only detection method the command-line tool `src/main.py` uses.

01b Fuzz detection (`src/analysis_engine/fuzzer_manager.py`). The fuzzer manager builds the target together with a small "harness" file. The harness passes the fuzzer's random bytes into the vulnerable function. The manager then runs libFuzzer or AFL++ (two well-known fuzzers) with AddressSanitizer and UndefinedBehaviorSanitizer switched on. Any input that trips a sanitizer is saved as a "crash file."

02 Triage (`src/analysis_engine/crash_triage.py`). Fuzzers often produce hundreds of crash files for the same underlying bug. Triage replays each one, reads the sanitizer's report (what kind of error, which file, which line), groups crashes with the same signature into one "cluster," and labels each cluster with a CWE number. The CWE (Common Weakness Enumeration) is the standard catalogue of bug types; CWE-193, for example, is "off-by-one error." Each cluster becomes one `VulnerabilityReport`.

Context (`src/context_engine/tree_sitter_extractor.py`). An LLM can't fix code it can't see, but pasting a whole large project into it is wasteful and confusing. tree-sitter is a parser: it turns source code into a tree of its grammatical parts (functions, statements, type definitions). The extractor uses that tree to find the *whole function* that contains the buggy line, plus any type definitions that function uses and the file's `#include` lines. If parsing fails, it falls back to grabbing nearby lines. If the `--memory` flag is on, `src/memory/store.py` also looks up fixes that worked for similar bugs in earlier runs and adds up to two of them to the prompt.

03 Ensemble (`src/agent_orchestrator/orchestrator.py`). An *ensemble* is a team whose members attempt the same job independently. Here there are three LLM "agents," each with a different personality written into its system prompt in `config/__init__.py`. **Alpha,** "**The Analyst,**" reasons step by step before patching. **Beta,** "**The Minimalist,**" adds the smallest possible check, one to five lines. **Gamma,** "**The SARIF Patcher,**" applies the textbook fix for the CWE. All three receive the same description of the bug and work at the same time in separate threads. The agents are not separate programs; each one is just a different system prompt sent to the same model.

04 DRV loop (`src/patch_validator/drv_loop.py`). DRV stands for Detect–Repair–Verify. Each agent runs its own loop: ask the LLM for a patch, copy the source into a fresh scratch folder, apply the patch, then put it through a series of **gates** (pass/fail checks). The gates are apply, build with sanitizers, replay the PoV, run a regression test, and re-scan with Semgrep when a scanner is configured. If any gate fails, the exact error text goes back to the agent as feedback, and a fourth agent, **Delta,** "**the Critic**" (`src/agent_orchestrator/critic.py`), may be asked to diagnose what went wrong. A plain-Python rulebook called `EscalationPolicy` decides when to call the Critic, when to demand a different patch format, and when to give up. Of the agents that pass every gate, the one whose patch changes the fewest lines wins.

06 Hardening (`src/patch_validator/hardening.py`, enabled with `--harden`). Gates only check a few specific inputs, so a patch can pass them by cheating. It might block only the exact attack input (called "overfitting"), or it might switch the feature off entirely (called "gutting"). Hardening attacks the accepted patch. It re-fuzzes the patched program for 20 seconds, compares the original and patched programs side by side on 25 harmless inputs (*differential testing*), and for input-validation fixes runs an "evasion battery" of bypass tricks. If any of these breaks the patch, the result is downgraded to `unresolved_hardening`.

07 Formal (`src/patch_validator/formal.py`, enabled with `--formal`). For the three targets that have a proof harness (SYN-06, SYN-11 and SYN-12), CBMC checks every possible input up to a loop-count bound and reports whether any of them can cause an out-of-bounds access, a pointer error or an arithmetic overflow. A counterexample downgrades the result to `unresolved_formal`.

Memory and reports (`src/memory/store.py`, `src/reporting/audit_report.py`, and `generate_reports` in `benchmark.py`). With `--memory`, successful fixes are saved as reusable "fix patterns" for future runs. Every run writes CSV tables, a JSON and Markdown summary, and an audit report that records every gate's real outcome. A gate that didn't run is printed as SKIPPED, never as PASS.

3. A map of the repository

The command `git ls-files` lists every file the project tracks in version control. Here is that list, grouped by folder, with what each part is for.

TRY IT YOURSELF

From the repository root, run:

```
git ls-files                      # every tracked file
git ls-files | sed 's|/[^/]*$||' | sort | uniq -c    # files per folder
```

```
ai_kavach_crs/
|-- README.md  architecture.md  benchmark.md  INSTALLED.md   (documentation)
|-- requirements.txt  ".env copy"  .gitignore                (setup)
|-- benchmark.py                (main measurement harness, entry point #1)
|-- submission.html             (competition submission deck)
|-- config/__init__.py          (API key, model names, agent prompts)
|-- src/                        (the system itself)
|   |-- main.py                 (CLI, entry point #2)
|   |-- fuzz_pipeline.py        (fuzz -> triage -> patch, entry point #3)
|   |-- analysis_engine/        (finding bugs)
|   |-- context_engine/         (reading code)
|   |-- agent_orchestrator/     (talking to LLMs)
|   |-- patch_validator/        (judging patches)
|   |-- memory/                 (learning across runs)
|   `-- reporting/              (writing audit reports)
|-- tests/                      (benchmarks, vulnerable samples, unit tests)
|-- benchmark_workspace/memory/ (saved memory files)
|-- reports/                    (three standalone suite reports)
|-- design/ui-concept/          (a mock-up of a dashboard)
`-- presentation/               (pitch deck; not yet committed)
```

Top-level files

File	What it is
`README.md`	The front page. It explains the idea, the latest measured results (44/44 targets patched, 95% confidence interval [92.0, 100.0], $0.0739), quick-start commands, the verification model, a project layout, platform notes and how to run without internet ("air-gapped").
`architecture.md`	A longer design document. It walks through the seven phases (detection, triage and context, the multi-agent ensemble, the DRV loop, Driller, hardening, formal proof), the four-tier memory system, swapping model providers, the technology stack and the pre-flight rules.
`benchmark.md`	The results write-up: headline numbers, the hardening outcome ("32 attacked, 0 falsified, 5 not attacked"), suite-by-suite detail, cost, time and memory use, and a candid "Not Measured" section.
`INSTALLED.md`	A record of everything installed for the project (a conda environment, pip packages, Homebrew tools such as LLVM, AFL++ and CBMC), plus clean-up instructions. **Watch out:** its cleanup section contains `rm -rf` commands with another developer's absolute paths. Never run them as written.
`requirements.txt`	The Python libraries the code needs: `openai`, `tree-sitter` with its C and C++ grammars, `python-dotenv`, `pydantic`, `rich` (coloured terminal output) and `pytest`. LangGraph is listed but commented out as "NOT used." Semgrep has to be installed separately, as the file itself notes.
`.env copy`	A template for the secrets file (the name really does contain a space). It holds a single setting, `OPENAI_API_KEY`. You copy it to `.env` and put your own key in. The real `.env` is excluded from git by `.gitignore`. `config/__init__.py` also reads optional variables if you set them: `OPENAI_BASE_URL` (point at a local, offline model server), `MODEL_ALPHA` / `MODEL_BETA` / `MODEL_GAMMA` and `KAVACH_SEED`.
`.gitignore`	Tells git what not to track. This includes `.env`, caches, fuzzing output, the cloned cJSON project and, importantly, `reports/benchmark_runs/`. The raw output of benchmark runs is therefore not in the repository.
`benchmark.py`	The main program you run (888 lines). It handles pre-flight, runs suites, applies the optional hardening, formal and memory steps, computes statistics and writes reports. Section 4 covers it.
`submission.html`	A self-contained HTML slide deck submitted for the competition.

`config/`

A single file, `config/__init__.py`. It loads `.env` from the project root, and `get_llm_config()` returns the API key, base URL, model per agent, maximum output length per agent (Alpha 4096 tokens, Beta and Gamma 2048), `temperature` 0.0 and a `seed` (default 1337). If the key is missing it raises an error straight away. The file also holds `AGENT_PROMPTS`, the three long system prompts that turn the same model into Alpha, Beta or Gamma.

`src/`: the system itself

Path	Role
`src/main.py`	Command-line front door for scanning your own code (Semgrep only).
`src/fuzz_pipeline.py`	The fuzz → triage → locate → patch pipeline, class `FuzzDiscoveryPipeline`.
`src/analysis_engine/`	**Finding bugs.** `fuzzer_manager.py` builds and runs libFuzzer or AFL++. `crash_triage.py` replays, groups and labels crashes. `semgrep_runner.py` runs the static scanner. `driller_monitor.py` is the angr-based concolic engine; it is present but only its self-test runs. `casr_triage.py` is an outdated alias for the triage class.

Path	Role
`src/context_engine/`	**Reading code.** `tree_sitter_extractor.py` finds the enclosing function and related types. It can also *replace* a whole function by name, which is how most patches actually get applied.
`src/agent_orchestrator/`	**Talking to LLMs.** `orchestrator.py` runs the ensemble and picks the winner. `llm_client.py` is the single OpenAI-compatible client, with retries, token counting and cost estimates. `critic.py` holds Agent Delta (the Critic) and the deterministic `EscalationPolicy`.
`src/patch_validator/`	**Judging patches.** `drv_loop.py` has the gates and the retry loop. `hardening.py` runs re-fuzzing, differential testing and the evasion battery. `formal.py` runs CBMC.
`src/memory/`	**Learning across runs.** `store.py` has four kinds of memory: working (attempts within one loop), episodic (one log line per bug), semantic (reusable fix patterns) and procedural (per-CWE statistics, recorded but not used to make decisions).
`src/reporting/`	**Writing the audit trail.** `audit_report.py` produces a Markdown report per run listing each gate's real outcome.

Every subpackage also contains an `__init__.py`. This is the file that tells Python "this folder is a package you can import from." Most are one line. `src/memory/__init__.py` re-exports the memory classes so other code can write `from src.memory import MemorySystem`.

`tests/`: test material, benchmarks and unit tests

Path	What is inside
`tests/demo_vulns/`	20 small, deliberately vulnerable C programs, `01_complex_magic_bytes.c` to `20_ast_opaque_pointer.c`. Each shows one kind of bug: integer overflow, double free, off-by-one, format string, command injection, race condition, memory leak, type confusion, hard-coded key and so on. Together they form the "Synthetic" suite (IDs SYN-01 to SYN-20).
`tests/benchmarks/targets.py`	The **manifest**: a list of `Target` records saying, for each benchmark bug, where it is, what kind it is, and exactly how to build, attack and regression-test it. You will see the SYN-06 entry below.
`tests/benchmarks/harness/`	Small shell scripts the gates call. `pov_run.sh` fails if a sanitizer fires. `regress_run.sh` checks normal output. `pov_leak.sh` checks for memory leaks. `pov_injection.sh` checks for command injection. `secret_scan.sh` checks for a hard-coded key.
`tests/benchmarks/cbmc_harnesses/`	Proof harnesses for CBMC for three targets (SYN-06, SYN-11, SYN-12), plus a README explaining why a harness is needed instead of proving `main()`.
`tests/benchmarks/juliet_subset/`	Two test cases from NIST's Juliet suite, a public, government-made collection of bug examples: a stack buffer overflow (CWE-121) and a use-after-free (CWE-416).
`tests/benchmarks/juliet_runner.py`, `cve_reproduction.py`, `run_real_cjson.py`, `tests/run_ultimate_benchmark.py`	Retired scripts marked "SUPERSEDED." They are kept only to record why they were dropped; earlier versions printed PASS for gates that never ran.
`tests/fuzz_harnesses/`	Nine libFuzzer harnesses, each named after a target ID (for example `SYN-06-OFF-BY-ONE.c`). There is also `afl_driver.c`, which lets AFL++ reuse the same harnesses, a `seeds/` folder of three starter inputs, and a README explaining that a harness must respect the function's preconditions.
`tests/real_cve_suite/`	Three published bugs in the real cJSON library (CVE-2019-11834, CVE-2019-11835 and a GitHub issue), tested on the unmodified source at the commit just before each fix. `setup.sh` fetches those versions, `manifest.py` describes them, `drivers/` holds small C programs that feed input into cJSON, and `run_real_cve.py` runs the suite standalone.
`tests/vanguard_nightmare/`	Four hand-written hard cases (a bitfield overflow, a use-after-free through an aliased global, a macro size underflow, an `rtos://` path traversal). Each has its own manifest, helper scripts (including `evasion_battery.sh`) and a runner, `run_vanguard.py`.
`tests/public_benchmarks_extension/`	Two synthetic programs modelled on known public bug types (a TIFF image crop out-of-bounds and a stack overflow in a service), with their own manifest, harness and runner.
`tests/cheat_detector/`	A demonstration that needs no LLM. It feeds hardening one honest fix and two cheating fixes (one "overfitted," one "gutted") and checks that both cheats are caught.
`tests/standalone_support.py`	Shared code that gives the standalone suites the same pre-flight and honest hardening labels as `benchmark.py`.
`tests/samples/test_vulnerable.c`	A small C file with several bugs, used by the unit tests for the context extractor.

Path	What is inside
`tests/unit/`	Eight `pytest` files (`test_analysis_engine.py`, `test_audit_report.py`, `test_core.py`, `test_formal.py`, `test_hardening.py`, `test_memory.py`, `test_orchestrator.py`, `test_repair_loop.py`). They make no API calls and need no network.

The remaining folders

- **`benchmark_workspace/memory/`** holds the three saved memory files: `episodes.jsonl` (one line per bug-run, 91 lines), `semantic.json` (28 learned fix patterns) and `procedural.json` (per-CWE counters). They are committed to git, so a fresh checkout starts with this "experience" already loaded. Everything else under `benchmark_workspace/` (fuzzing output, the cloned cJSON) is generated and ignored by git.
- **`reports/`** holds three committed Markdown reports from the standalone suites: `public_benchmarks_report.md`, `real_cve_report.md` and `vanguard_nightmare_report.md`. Output from `benchmark.py` goes to `reports/benchmark_runs/<timestamp>/`, which git ignores.
- **`design/ui-concept/index.html`** is a visual mock-up of a dashboard. Its own title says "illustrative data, not a live system." No code depends on it.
- **`presentation/`** contains `ai_kavach_pitch.html`, a pitch deck. At the time of writing it is *untracked* (git status shows `?? presentation/`), meaning it exists on disk but has not been committed.

4. Three front doors: which program to run

An **entry point** is a file you run directly, as opposed to one that other code imports. AI Kavach has three. All of them end up calling the same `Orchestrator` and the same DRV gates. They differ in where the bug comes from and how much measurement surrounds the run.

Entry point	How bugs are found	Use it when...	Example
`benchmark.py`	Declared in the manifest (`targets.py`), or discovered by fuzzing with `--fuzz`	You want *measured* results: pre-flight, optional hardening, formal proof and memory, confidence intervals, CSV and JSON reports. This is the main tool and what the published numbers come from.	`python benchmark.py --suite synthetic --only SYN-06-OFF-BY-ONE --harden --formal`
`src/main.py`	Semgrep static scan only (no fuzzing)	You want to point the system at *your own* C file and supply your own build, PoV and test commands.	`python -m src.main -t file.c --build-cmd '...' --pov-cmd '...'`
`src/fuzz_pipeline.py`	Real fuzzing campaign, then triage	You want the "nobody told us where the bug is" path. In practice you reach it through `benchmark.py --fuzz` or `--suite fuzz`, which creates a `FuzzDiscoveryPipeline`. The file has no `main()` of its own.	`python benchmark.py --suite fuzz --fuzz-seconds 30`

WATCH OUT

- Always run from the repository root. Many paths, such as `tests/fuzz_harnesses` and `tests/benchmarks/cbmc_harnesses`, are relative to the folder you are in.
- With `src/main.py`, if you give no build, PoV or test command, a patch that merely *applies* counts as validated. The gates you leave out are SKIPPED, and "no gate failed" is technically true. Always supply at least a build and a PoV command.
- The `--language` option of `src/main.py` is parsed but never used.

5. The life of one bug

This is the centrepiece of the chapter. We follow one real bug, **SYN-06**, from source code to a proven fix, citing the actual functions that handle it at each step. The command is:

```
python benchmark.py --suite synthetic --only SYN-06-OFF-BY-ONE --harden --formal
```

In plain words: "run the Synthetic suite, but only the target called SYN-06-OFF-BY-ONE; after it is patched, try to break the patch (`--harden`) and try to prove it with CBMC (`--formal`)." Memory is *not* turned on, because there is no `--memory`.

Step 0: Meet the bug

tests/demo_vulns/06_off_by_one_loop.c · lines 1–23

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
void copy_and_null_terminate(const char* src, size_t size) {
    if (size == 0 || size > 1024) return;
    char* dest = (char*)malloc(size);
    if (!dest) return;
    for (size_t i = 0; i <= size; i++) {
        if (i == size) {
            dest[i] = '\0';
        } else {
            dest[i] = src[i];
        }
    }
    printf("Copied string: %s\n", dest);
    free(dest);
}
int main(int argc, char* argv[]) {
    if (argc > 1) {
        copy_and_null_terminate(argv[1], strlen(argv[1]));
    }
    return 0;
}
```

LINE BY LINE

1. Line 4: the function receives a piece of text (`src`) and its length (`size`).
2. Line 5: it refuses empty text and anything over 1024 characters.
3. Line 6: it reserves `size` bytes of memory for the copy. That is room for the characters, but **not** for the extra end-of-text marker `'\0'` that C strings need.
4. Line 8: the loop runs from `i = 0` up to **and including** `size` (that is what `<=` means). This is `size + 1` steps for a buffer of `size` bytes.
5. Lines 9–10: on the last step (`i == size`) it writes `'\0'` to `dest[size]` , one byte past the end of the buffer. This is the bug: a heap overflow of exactly one byte, the classic "off-by-one" (CWE-193).
6. Line 15: it prints `Copied string: …` , then frees the memory.
7. Lines 18–22: `main` passes the first command-line argument in. Because every non-empty argument reaches the buggy loop, **every** input triggers the overflow, harmless or not.

ANALOGY

You book a row of 4 theatre seats for 4 friends plus a coat, and then put the coat on seat number 5, which belongs to a stranger. Nobody may notice tonight, but you have used a seat you didn't pay for. AddressSanitizer is the usher who notices immediately.

Step 1: The command line is read

`main()` in `benchmark.py` (line 817 onwards) reads the flags with Python's `argparse` . It then creates the harness:

benchmark.py · lines 163–176

```
class BenchmarkHarness:
    def __init__(self, output_dir="reports/benchmark_runs", parallel=True, harden=False,
                 memory=False, formal=False):
        self.output_dir = Path(output_dir)
        self.output_dir.mkdir(parents=True, exist_ok=True)
        self.parallel = parallel
        self.harden = harden
```

```
        self.hardener = PatchHardening() if harden else None
        self.hardening_verdicts: dict[str, HardeningVerdict] = {}
        self.formal_results: dict[str, VerificationResult] = {}
        self.formal = FormalVerifier() if formal else None
        self.memory = MemorySystem() if memory else None
        self.llm_client = LLMClient()
        self.extractor = ContextExtractor()
```

LINE BY LINE

1. The output folder `reports/benchmark_runs` is created if it doesn't exist.
2. `parallel=True` because we didn't pass `--sequential`, so the three agents will work at the same time.
3. Because we passed `--harden`, a `PatchHardening` object is made. Because we passed `--formal`, a `FormalVerifier` is made. With no `--memory`, `self.memory` stays `None` (Python's "nothing").
4. `LLMClient()` reads the configuration. If `OPENAI_API_KEY` is missing, the program stops here with an error.
5. `ContextExtractor()` prepares the tree-sitter parsers for C and C++.

Next, `main()` calls `get_suite("synthetic")` (`tests/benchmarks/targets.py` line 508), which returns the list `SYNTHETIC`. The `--only` filter keeps just the target whose `id` is `SYN-06-OFF-BY-ONE`. That list of one is handed to `harness.run_suite("Synthetic", targets)`.

Step 2: The manifest entry (a `Target`)

Each benchmark bug is described by a `Target`, a Python **dataclass**. A dataclass is a record type: a labelled form with named fields, like a row in a spreadsheet. Here is its definition:

tests/benchmarks/targets.py · lines 75-118 (trimmed)

```
@dataclass
class Target:
    """One benchmark target and its complete verification contract."""

    id: str
    suite: str
    file_path: str
    line_number: int
    cwe_id: str
    description: str
    complexity: str                      # easy | medium | hard | ast-dependent
    build_command: str
    pov_command: Optional[str] = None
    pov_na_reason: str = ""
    regression_command: Optional[str] = None
    regression_na_reason: str = ""
    source_dir: Optional[str] = None     # copy this whole tree into workspace
    ...
    behaviour_contract: str = "preserve"
    ...
    evasion_command: Optional[str] = None
    ...
    cbmc_unwind: int = 16
    severity: str = "critical"
    source: str = "sanitizer_crash"      # sanitizer_crash | semgrep | side_effect

    def __post_init__(self):
        if self.pov_command is None and not self.pov_na_reason:
            raise ValueError(f"{self.id}: pov_command=None requires pov_na_reason")
```

LINE BY LINE

1. The first group of fields says *what* the bug is: its ID, which suite it belongs to, the file, the line, the CWE number, a description and a rough difficulty.
2. `build_command`, `pov_command` and `regression_command` say *how to check a fix*. Together they are the target's "verification contract."
3. `Optional[str] = None` means "a piece of text, or nothing." A target may have no PoV, but then `__post_init__` (run automatically after the record is created) demands a written reason in `pov_na_reason`. The same rule applies to the regression test. No gate can quietly go missing.
4. `behaviour_contract` tells hardening what kind of change the fix is allowed to make. `"preserve"` means normal behaviour must stay identical. (The alternatives, `"restrict"` and `"replace"`, are for input-validation fixes and hard-coded secrets.)
5. `cbmc_unwind` is how many times CBMC unrolls each loop; more on this in Step 11.

And here is SYN-06's actual entry:

tests/benchmarks/targets.py · lines 192-200

```
Target(
    id="SYN-06-OFF-BY-ONE", suite="Synthetic", complexity="easy",
    file_path=f"{D}/06_off_by_one_loop.c", line_number=8,
    cwe_id="CWE-193", description="Off-by-one heap write in NUL-termination loop",
    build_command=SINGLE_FILE_BUILD,
    pov_command=_pov("AAAA"),
    regression_command=_regress("Copied string: hi", "hi"),
    cbmc_unwind=70,          # loop runs up to the 64-byte harness buffer + 1
),
```

LINE BY LINE

1. `{D}` is short for `tests/demo_vulns` (line 135), so the file is `tests/demo_vulns/06_off_by_one_loop.c`. Line 8 is the `for` loop.
2. `SINGLE_FILE_BUILD` (line 46) expands to `clang -fsanitize=address,undefined -fno-sanitize-recover=all -fno-omit-frame-pointer -g -O0 "{src}" -o "{bin}"`. This compiles with both sanitizers on and makes every sanitizer finding fatal.
3. `_pov("AAAA")` (line 121) produces `sh "<repo>/tests/benchmarks/harness/pov_run.sh" "{bin}" AAAA`, which means "run the program with the argument `AAAA` and fail if a sanitizer complains."
4. `_regress("Copied string: hi", "hi")` produces a call to `regress_run.sh`: run the program with `hi`, and require exit code 0, no sanitizer report, and output containing `Copied string: hi`.
5. `cbmc_unwind=70`: the proof harness feeds up to 64 bytes, and the loop needs one more step than that, so 70 leaves a margin.
6. The words in braces (`{src}`, `{bin}`, `{srcdir}`, `{workspace}`) are placeholders. `DRVLoop.expand` (`drv_loop.py` line 587) fills them in later with real paths in a temporary folder.

Step 3: Pre-flight, proving the bug is real

`run_suite` (`benchmark.py` line 184) first calls `preflight(t)` for the target:

benchmark.py · lines 125-158 (trimmed)

```
with tempfile.TemporaryDirectory(prefix="kavach_pre_") as tmp:
    workspace = Path(tmp)
    try:
        src = DRVLoop._materialise(workspace, target.file_path, target.source_dir)
    except FileNotFoundError as e:
        res.build_error = str(e)
        return res

    binary = workspace / "kavach_target"
    ctx = {
        "src": str(src), "srcdir": str(src.parent),
        "bin": str(binary), "workspace": str(workspace),
    }

    ok, detail = DRVLoop._run(DRVLoop.expand(target.build_command, ctx), "Build", cwd=workspace)
    res.build_ok = ok
```

```
...
if target.pov_command:
    # PoV scripts exit non-zero when the vulnerability IS present.
    ok, detail = DRVLoop._run(DRVLoop.expand(target.pov_command, ctx), "PoV", cwd=workspace)
    res.pov_reproduces = not ok
    ...
if target.regression_command:
    ...
    res.regression_baseline_ok = ok
```

LINE BY LINE

1. A temporary folder named `kavach_pre_…` is created. It is deleted automatically when the `with` block ends.
2. `DRVLoop._materialise` (`drv_loop.py` line 416) copies the C file into that folder, keeping its relative path. The original file is never modified.
3. The placeholder table `ctx` maps `{bin}` to `<tmp>/kavach_target`, and so on.
4. The build command runs. For SYN-06 it succeeds.
5. The PoV runs with `AAAA`. The program writes one byte past its 4-byte buffer, ASan prints `heap-buffer-overflow`, and `pov_run.sh` exits 1. So `ok` is `False` and `pov_reproduces = not ok = True`: the bug is confirmed.
6. The regression test with `hi` *also* trips ASan on the unpatched program, because every input hits the bug, so `regression_baseline_ok` is `False`. This is expected. The docstring of `regression_usable` (lines 103–113) names "the off-by-one loop, which trips the sanitizer on every input" as the reason a failing baseline is recorded but does **not** disable the gate.

The result is a `PreflightResult`, and its property `pov_usable` (`build_ok and pov_reproduces`) is `True`. Had the PoV *not* failed on the original, the yellow warning "PoV did not reproduce — PoV gate disabled" would be printed, and the PoV gate would be switched off for this target.

Step 4: Building the vulnerability record and calling the orchestrator

benchmark.py · lines 217-242 (trimmed)

```
orchestrator = Orchestrator(
    self.llm_client,
    self.extractor,
    None,
    parallel=self.parallel,
    memory=self.memory,
)
...
    vuln = VulnerabilityReport(
        id=t.id, source=t.source, file_path=t.file_path,
        line_number=t.line_number, description=t.description,
        cwe_id=t.cwe_id, severity=t.severity,
    )
    result = orchestrator.process_vulnerability(
        vuln,
        build_command=t.build_command,
        # Only gate on a PoV we proved detects the real bug.
        pov_command=t.pov_command if pf.pov_usable else None,
        test_command=t.regression_command,
        source_dir=t.source_dir,
    )
```

LINE BY LINE

1. The orchestrator gets the shared LLM client and context extractor. The third argument, `None`, is the Semgrep runner, so **the post-scan gate is skipped in the benchmark**. Memory is also `None` in our run.
2. A `VulnerabilityReport` is filled in from the manifest: ID, source (`"sanitizer_crash"`), file, line 8, the description, CWE-193 and severity "critical."

3. `process_vulnerability` receives the three gate commands. The PoV is passed only if pre-flight proved it works (`pf.pov_usable`).

Step 5: Gathering context

Inside `Orchestrator.process_vulnerability` (`orchestrator.py` line 93), the first job is to enrich the report with code. It calls `self.context_extractor.extract_context(vuln.file_path, vuln.line_number)` (`tree_sitter_extractor.py` line 33). tree-sitter parses the file, finds the function whose span contains line 8 (`copy_and_null_terminate`), and collects the three `#include` lines. The combined text is stored in `vuln.code_context`, and `context_method` is recorded as `"tree-sitter"` (or `"fallback"` if parsing failed).

Because memory is off, nothing is recalled. With `--memory`, `self.memory.recall_for(...)` would add up to two earlier fix patterns to the prompt.

Step 6: Writing the prompt

`_build_vulnerability_prompt` (`orchestrator.py` lines 334–359) assembles plain text from the report:

src/agent_orchestrator/orchestrator.py · lines 338-358 (trimmed)

```
        parts.append(f"VULNERABILITY ID: {vuln.id}")
        parts.append(f"SOURCE: {vuln.source}")
        parts.append(f"FILE: {vuln.file_path}")
        parts.append(f"LINE: {vuln.line_number}")

        if vuln.cwe_id:
            parts.append(f"CWE: {vuln.cwe_id}")

        parts.append(f"SEVERITY: {vuln.severity}")
        parts.append(f"\nDESCRIPTION:\n{vuln.description}")

        if vuln.crash_trace:
            parts.append(f"\nCRASH TRACE:\n{vuln.crash_trace}")
        ...
        if vuln.code_context:
            parts.append(f"\nSOURCE CODE CONTEXT:\n{vuln.code_context}")

        return "\n".join(parts)
```

For SYN-06 there is no crash trace or SARIF finding (the bug came from the manifest, not from a fuzzer or scanner), so the prompt contains the ID, source, file, line, CWE, severity, description and the extracted function. All three agents get this identical text. Only their system prompts differ.

Step 7: The ensemble starts

Since `parallel` is on and there are three agents, `_run_parallel` is used:

src/agent_orchestrator/orchestrator.py · lines 247-271 (trimmed)

```
        with ThreadPoolExecutor(max_workers=len(agents)) as executor:
            futures = {}
            for agent in agents:
                future = executor.submit(
                    self.drv_loop.run,
                    agent_name=agent,
                    vulnerability_context=vuln_context,
                    target_file=vuln.file_path,
                    max_iterations=self.AGENT_MAX_ITERATIONS.get(agent, 5),
                    ...
                )
                futures[future] = agent

            for future in as_completed(futures):
                agent = futures[future]
```

```
            try:
                result.agent_reports[agent] = future.result()
            except Exception as e:
                logger.error(f"Agent {agent} failed with error: {e}")
                result.agent_reports[agent] = None
```

LINE BY LINE

1. A `ThreadPoolExecutor` is a small pool of workers that run jobs at the same time. Three workers, one per agent.
2. Each agent's job is "run `DRVLoop.run` for me." Its maximum number of tries comes from `AGENT_MAX_ITERATIONS` (lines 66–70): Alpha 5, Beta 8, Gamma 5.
3. `as_completed` collects results as each agent finishes. The program waits for **all** of them. (The file's header comment says "First validated patch wins," but in parallel mode the code collects every result and picks the smallest patch.)
4. If one agent crashes with an error, it is recorded as `None` and the others carry on.

Step 8: Inside one agent's DRV loop

`DRVLoop.run` (`drv_loop.py` lines 152–298) repeats up to the agent's budget. Each iteration goes like this.

(a) Ask for a patch. `self.llm_client.generate_patch(...)` (`llm_client.py` line 48) sends the system prompt plus the vulnerability text. From iteration 2 onwards it also sends a message starting "Your previous patch FAILED. Here is the feedback:". The reply is searched for one of two formats: a unified diff in a ```` ```diff ```` block, or a **whole replacement function** in a ```` ```c ```` block whose first line is `// FUNCTION: copy_and_null_terminate`. The system prompts recommend the whole-function format, because a diff is rejected if even one context line differs by a single character. If neither format is found, the iteration fails at the stage called `"generation"`.

(b) Verify. `_verify` (lines 302–413) builds a fresh workspace and runs the gates in order, stopping at the first failure:

src/patch_validator/drv_loop.py · lines 365–398 (trimmed)

```
            # Gate 2 — build
            if build_command:
                ok, detail = self._run(self.expand(build_command, ctx), "Build", cwd=workspace)
                result.syntax_ok = GATE_PASS if ok else GATE_FAIL
                if not ok:
                    result.failure_stage = "build"
                    result.error_feedback = f"BUILD FAILED:\n{detail}"
                    return False

            # Gate 3 — proof of vulnerability replayed on the patched binary
            if pov_command:
                ok, detail = self._run(self.expand(pov_command, ctx), "PoV", cwd=workspace)
                result.pov_ok = GATE_PASS if ok else GATE_FAIL
                if not ok:
                    result.failure_stage = "pov"
                    result.error_feedback = (
                        "PROOF-OF-VULNERABILITY STILL REPRODUCES. The patched binary "
                        ...
                    )
                    return False

            # Gate 4 — regression on benign input
            if test_command:
                ok, detail = self._run(self.expand(test_command, ctx), "Regression", cwd=workspace)
                result.regression_ok = GATE_PASS if ok else GATE_FAIL
                ...
```

LINE BY LINE

1. Before this excerpt, **Gate 1 (apply)** has already put the patch into the copied file. A whole-function replacement is spliced in by tree-sitter (`_apply_replacement`, line 501). Otherwise the diff is tried with the standard `patch` tool and, failing that, by matching its lines against the file's text (`_apply_patch`, line 445).

2. **Gate 2 (build):** compile the patched copy with sanitizers. A compiler error ends the attempt with the compiler's message as feedback.
3. **Gate 3 (PoV):** run the patched program on `AAAA`. For SYN-06, exit 0 means ASan stayed quiet, so the one-byte overflow is gone.
4. **Gate 4 (regression):** run the patched program on `hi`. It must exit 0, raise no sanitizer report, and print `Copied string: hi`. This catches a "fix" that just returns early, because then nothing is printed.
5. **Gate 5 (post-scan)** would follow, but only `if self.semgrep_runner`, which is `None` in the benchmark. It stays `GATE_SKIPPED`.
6. `_run` (line 603) runs each command with a 180-second time limit and returns `(True/False, output)`. If the command fails, the output is trimmed by `_salient` so that the important sanitizer lines survive.
7. If no gate failed, the code reaches line 412, `result.fully_validated = True`.

Each gate's state is one of three strings defined at the top of the file: `GATE_PASS = "pass"`, `GATE_FAIL = "fail"` and `GATE_SKIPPED = "skipped"`. A skipped gate is *recorded* as skipped and never counted as a pass.

(c) On failure: diagnose and retry. If `_verify` returns `False`, the loop (lines 247–287) does the following:

- tells the `EscalationPolicy` which stage failed (`record_failure`), together with a short "gist" of the attempted patch;
- writes the attempt into `WorkingMemory`, a ledger of everything already tried, so the agent stops going back and forth between two rejected fixes;
- asks `policy.should_invoke_critic(stage)`. This is *always* yes for a PoV failure, and otherwise yes once there have been at least two failures (`critic.py` lines 218–221). If yes, Agent Delta (`Critic.review`, `critic.py` line 143) returns a structured JSON verdict: what kind of failure it was, the root cause, what the patch missed, and an instruction. That verdict is added to the feedback;
- stops early if the last four attempt gists are identical, meaning the agent is stuck rather than making progress (`should_stop_early`).

And if the policy sees two apply failures or two generation failures, the next prompt gains a "MANDATORY FORMAT CHANGE" message demanding a whole function instead of a diff (lines 191–198).

WHAT ACTUALLY HAPPENED TO SYN-06 IN EARLIER RUNS

The committed memory file `benchmark_workspace/memory/episodes.jsonl` records three past runs of this target (they were run with `--memory` on). In each one the final status is `patched` and the winner is `alpha`, at a cost of about $0.0015. Two of those runs record a rejected attempt by **Beta** that changed the loop to `i < size` but was turned down by the **PoV** gate, with Delta's verdict recorded as `"insufficient"`. One run records a **Gamma** attempt rejected by the **regression** gate. So the gates really did turn away plausible-looking but wrong patches.

The winning fix that was stored as a pattern in `benchmark_workspace/memory/semantic.json` (recorded with `"hardened": true` and `"iterations_needed": 2`) looks like this:

benchmark_workspace/memory/semantic.json · SYN-06 pattern, "fix_strategy" field (shown unescaped)

```
// FUNCTION: copy_and_null_terminate
void copy_and_null_terminate(const char* src, size_t size) {
    if (size == 0 || size > 1024) return;
    char* dest = (char*)malloc(size + 1); // Allocate space for null terminator
    if (!dest) return;
    for (size_t i = 0; i < size; i++) { // Loop condition changed to < size
        dest[i] = src[i];
    }
    dest[size] = '\0'; // Null terminate at the end of the copied string
    printf("Copied string: %s\n", dest);
    free(dest);
}
```

This is the honest fix. It reserves `size + 1` bytes so the end marker has a seat of its own, copies exactly `size` characters, and then writes the marker into the extra byte. Note the whole-function format: the first line names the function, and tree-sitter swaps the old function body for this one.

(d) On success. The loop stores the applied patch in `report.winning_patch` and stops. The function returns a `DRVReport` listing every iteration (each a `PatchResult`), the escalations and how many times the Critic was consulted.

Step 9: Choosing the winner

src/agent_orchestrator/orchestrator.py · lines 306–328 (trimmed)

```
    @staticmethod
    def _select_winner(result: PipelineResult, agents: list[str]) -> None:
        ...
        winners = [
            (agent, report) for agent in agents
            if (report := result.agent_reports.get(agent)) is not None and report.success
        ]
        if not winners:
            return

        def patch_size(item):
            agent, report = item
            it = report.winning_iteration
            return (it.patch_line_count if it else 10**6, agents.index(agent))

        agent, report = min(winners, key=patch_size)
        result.status = "patched"
        result.winning_agent = agent
        result.winning_patch = report.winning_patch
```

LINE BY LINE

1. `winners` keeps only the agents whose loop ended in `success`.
2. If none succeeded, the status stays at its initial value, `"unresolved"`.
3. `patch_size` gives each winner a score: first the number of lines its patch actually changed, then its position in the list (alpha, beta, gamma) to break ties.
4. `min` picks the lowest score, so the smallest validated change wins. The status becomes `"patched"`.

Back in `process_vulnerability`, the elapsed time and this bug's LLM cost (the running total after minus the running total before) are filled in. With `--memory`, `_update_memory` would now write the outcome to disk.

Step 10: Hardening, trying to break the patch

Back in `run_suite` (`benchmark.py` line 246), because the status is `patched` and `--harden` was given, `_harden_patch` runs (lines 286–326). The DRV workspaces were temporary and are gone, so it re-applies the winning patch to a fresh copy (by function replacement, or by diff if necessary) and calls `PatchHardening.harden` (`hardening.py` line 350). For SYN-06 the contract is `preserve` and there is no evasion command, so two checks run:

1. **Differential testing** (`DifferentialTester`, lines 135–250). Both the original and the patched source are built. Then 25 harmless inputs are generated from a fixed random seed (1337): short words, small numbers and so on. `compare` skips any input on which the **original** already trips a sanitizer, because those are exploit cases the patch is meant to change. Remember that SYN-06's original overflows on *every* non-empty input, so we can expect all 25 to be skipped and `inputs_tested` to be 0. The `has_evidence` property (lines 77–92) exists for exactly this case: a differential test that compared zero inputs is **not** counted as evidence.
2. **Adversarial re-fuzzing** (`AdversarialRefuzz.check`, lines 266–340). A fuzz harness exists, `tests/fuzz_harnesses/SYN-06-OFF-BY-ONE.c`, which passes random bytes of length 1–1024 into `copy_and_null_terminate`. The patched source is compiled with libFuzzer, ASan and UBSan, and fuzzed for 20 seconds. Any crash means the patch only fixed one input, not the bug, and the verdict becomes `overfitted`. (The method can be seeded with the original crashing input, but `benchmark.py` does not pass one, so the fuzzer starts from an empty corpus.)

src/patch_validator/hardening.py · lines 72–104 (trimmed)

```
    @property
    def falsified(self) -> bool:
        """True when a check actually broke the patch. This is what downgrades a result."""
        return self.overfitted or self.diverged or self.evaded

    @property
    def has_evidence(self) -> bool:
```

```
        ...
        return (
            self.overfitting_checked
            or self.evasion_checked
            or (self.differential_checked and self.inputs_tested > 0)
        )

    @property
    def survived(self) -> bool:
        ...
        return self.has_evidence and not self.falsified
```

LINE BY LINE

1. `falsified`: any check broke the patch. This is what `run_suite` uses to downgrade the status to `unresolved_hardening` (`benchmark.py` lines 249–257).
2. `has_evidence`: at least one check genuinely ran. For SYN-06 this comes from re-fuzzing, not from the zero-input differential test.
3. `survived`: attacked *and* held. Note that this is deliberately not just "not falsified." A patch that nobody managed to attack earns no credit.

WATCH OUT

In `harden`, re-fuzzing is marked as "checked" whenever the result text does not contain "no fuzz harness" or "no clang" (line 445). A re-fuzz whose *build failed*, or which ran out of time, is therefore still counted as checked with no crash found. This can make a patch look hardened when nobody actually fuzzed it. Keep this in mind when reading "SURVIVED" labels; a later chapter returns to it.

Step 11: Formal proof with CBMC

Because `--formal` was given and the status is still `patched`, `_verify_formally` runs (`benchmark.py` lines 328–367). It looks for `tests/benchmarks/cbmc_harnesses/SYN-06-OFF-BY-ONE.c`, which exists:

tests/benchmarks/cbmc_harnesses/SYN-06-OFF-BY-ONE.c · lines 4-15

```
#include <stddef.h>
void copy_and_null_terminate(const char *src, size_t size);
unsigned long nondet_ulong(void);

#define SRC 64
static char source[SRC];

void harness(void) {
    size_t size = nondet_ulong();
    __CPROVER_assume(size <= SRC);      /* src really holds `size` bytes */
    copy_and_null_terminate(source, size);
}
```

LINE BY LINE

1. `nondet_ulong()` has no body. To CBMC that means "any value at all." CBMC reasons about every possible `size` at once instead of trying them one by one.
2. `__CPROVER_assume(size <= SRC)` limits the search to sizes the 64-byte `source` buffer can really supply. Without it, CBMC would report out-of-bounds reads that come from the test set-up rather than from the function.
3. The harness then calls the (patched) function, and CBMC checks every memory access inside it.

The patch is re-applied to a fresh copy, then `FormalVerifier.verify_patched` (`formal.py` line 270) removes `main` from it (`strip_main`, because proving `main` with arbitrary arguments produces false failures inside CBMC's model of the C library) and runs:

```
cbmc --bounds-check --pointer-check --conversion-check
     --signed-overflow-check --unsigned-overflow-check --div-by-zero-check
     --unwinding-assertions --unwind 70 --function harness
     SYN-06-OFF-BY-ONE.c  cbmc_06_off_by_one_loop.c
```

Four outcomes are possible (`formal.py` lines 71–99): **proven** (`VERIFICATION SUCCESSFUL`: no violation for any size up to the bound); **violated** (a real counterexample, which downgrades the status to `unresolved_formal`); **inconclusive** (the bound of 70 was too small, or there was a timeout); and **unavailable** (no `cbmc` installed or no harness). Only "proven" counts as a pass. With the honest fix shown above, no write ever goes past `dest[size]` in a `size + 1` buffer, so we would expect "proven." With the original code, the write to `dest[size]` in a `size`-byte buffer would be reported as a bounds or pointer violation.

Step 12: Recording and reporting

`_record` (`benchmark.py` line 369) writes one row of measured facts for the target: gate states, iterations, winning agent, cost, hardening and formal outcome. The console prints the result line from `PipelineResult.summary()`, which shows the status, winning agent, time and cost. Finally `generate_reports` (line 543) creates `reports/benchmark_runs/<timestamp>/` containing:

- `per_target_results.csv`: one row per target;
- `suite_summary.csv`: one row per suite, with Wilson confidence intervals (a way of putting honest error bars on a success rate; see `wilson_interval`, lines 58–76);
- `preflight_pov_validation.csv`: the pre-flight evidence;
- `benchmark_summary.json` and `benchmark_summary.md`: totals, cost, memory use, component availability and a "Not Measured" list;
- an audit report from `AuditReportGenerator` (`src/reporting/audit_report.py`), where a gate that didn't run is printed as SKIPPED.

THE WHOLE JOURNEY IN ONE PARAGRAPH

A file goes into a sealed scratch room and is shown to be broken (pre-flight). Three repair people each get the same case file (the prompt) and work in parallel rooms. Each repair is checked by machines: does it fit, does it compile, does the attack still work, does normal use still work? Failed repairs come back with the machine's exact complaint and, sometimes, a note from a reviewer (Delta). The smallest repair that passed every machine wins. Then an attacker (hardening) gets 20 seconds to break it, and a mathematician (CBMC) checks it for every input size up to 64. Everything is written into the logbook.

TRY IT YOURSELF

From the repository root, with the conda environment active and a key in `.env`:

```
python benchmark.py --suite synthetic --only SYN-06-OFF-BY-ONE --harden --formal
ls reports/benchmark_runs/          # newest folder holds this run's reports
```

Without an API key you can still watch the bug itself:

```
clang -fsanitize=address,undefined -g tests/demo_vulns/06_off_by_one_loop.c -o /tmp/syn06
sh tests/benchmarks/harness/pov_run.sh /tmp/syn06 AAAA; echo "exit=$?"   # expect exit=1
```

6. The data objects that flow between stages

Each stage passes its results on in a dataclass, a labelled record. Knowing these records makes the rest of the code much easier to read, because almost every function either fills one in or reads one.

```
Target --preflight()--> PreflightResult
   |
   +--> VulnerabilityReport --process_vulnerability()--> PipelineResult
                                                            |
                                    agent_reports: {alpha: DRVReport, beta: ..., gamma: ...}
                                                            |
                                            DRVReport.iterations: [PatchResult, ...]
                                                            |
                        winning PatchResult --harden()--> HardeningVerdict
                                            --verify_patched()--> VerificationResult
```

Object (file:line)	Created by	Key fields	Meaning
`Target` (`tests/benchmarks/targets.py:75`)	Written by hand in the manifest	`id`, `suite`, `file_path`, `line_number`, `cwe_id`, `description`, `complexity`, `build_command`, `pov_command` / `pov_na_reason`, `regression_command` / `regression_na_reason`, `source_dir`, `behaviour_contract`, `evasion_command`, `cbmc_unwind`, `severity`, `source`	One benchmark bug and the exact contract for checking a fix.
`PreflightResult` (`benchmark.py:87`)	`preflight()`	`build_ok`, `build_error`, `pov_reproduces`, `pov_detail`, `regression_baseline_ok`; properties `pov_usable`, `regression_usable`	Whether the unpatched original builds and really shows the bug. (A plain class, not a dataclass.)
`VulnerabilityReport` (`orchestrator.py:20`)	`benchmark.py` from a Target; `crash_triage.py` from a fuzz crash; `src/main.py` from a Semgrep finding	`id`, `source`, `file_path`, `line_number`, `description`, `cwe_id`, `severity`, `crash_trace`, `sarif_context`, `code_context`	The common "case file" every detector produces and every agent reads. `code_context` is filled in by tree-sitter.
`PatchResult` (`drv_loop.py:46`)	One per DRV iteration	`iteration`, `agent_name`, `patch_diff`, `replacement`, `applied_via`, gate states `applied`, `syntax_ok` (build), `pov_ok`, `regression_ok`, `post_scan_ok`, plus `fully_validated`, `error_feedback`, `analysis`, `patch_line_count`, `failure_stage`, `critic_verdict`; property `executed_gates`	One attempt and the verdict of every gate on it. Each gate is `"pass"`, `"fail"` or `"skipped"`.
`DRVReport` (`drv_loop.py:80`)	`DRVLoop.run`, one per agent	`agent_name`, `vulnerability_id`, `total_iterations`, `max_iterations`, `success`, `winning_patch`, `iterations` (list of PatchResult), `escalations`, `critic_invocations`, `attempt_ledger`; properties `winning_iteration`, `last_failure`, `last_failure_stage`	The full story of one agent's attempts on one bug.
`PipelineResult` (`orchestrator.py:35`)	`_run_parallel` / `_run_sequential`	`vulnerability`, `status`, `winning_agent`, `winning_patch`, `agent_reports` (agent → DRVReport), `elapsed_seconds`, `total_cost_usd`, `context_extracted`, `context_method`, `recalled_patterns`	The overall outcome for one bug. `status` is `patched` or `unresolved`, and `benchmark.py` may change it to `unresolved_hardening` or `unresolved_formal`.
`HardeningVerdict` (`hardening.py:50`)	`PatchHardening.harden`	`overfitting_checked` / `overfitted`, `evasion_checked` / `evaded`, `differential_checked` / `diverged`, `inputs_tested`, `skipped_reason` and detail texts; properties `falsified`, `has_evidence`, `survived`	What happened when the system tried to break the patch.
`VerificationResult` (`formal.py:71`)	`FormalVerifier.verify`	`status` (`proven` / `violated` / `inconclusive` / `unavailable`), `unwind`, `properties_checked`, `violations`, `bound_exhausted`, `seconds`, `detail`	The CBMC verdict on the patched function.

On the other paths, two more objects feed into `VulnerabilityReport`: `SarifFinding` (`semgrep_runner.py:18`), one Semgrep result, and `CrashCluster` (`crash_triage.py:112`), a group of fuzz crashes with the same signature. With `--memory`, successful fixes are saved as `FixPattern` records (`src/memory/store.py:183`), like the SYN-06 pattern shown above.

CHAPTER RECAP

- AI Kavach is a Cyber Reasoning System: it finds memory-safety bugs in C, has LLM agents write patches, and only accepts patches that deterministic tools confirm. In short, "the LLM proposes, deterministic tools decide."
- The pipeline is: pre-flight → static and/or fuzz detection → triage → context (tree-sitter) → three-agent ensemble → DRV gate loop with Critic feedback → optional hardening → optional CBMC proof → memory and reports.
- `src/` holds the system (analysis, context, agents, validation, memory, reporting); `tests/` holds the benchmark corpus, manifests, harness scripts and unit tests; `config/` holds model settings and agent prompts.
- `benchmark.py` is the main, measured entry point; `src/main.py` is a Semgrep-only CLI for your own code; `src/fuzz_pipeline.py` is the fuzz-discovery path, reached through `benchmark.py --fuzz`.
- For SYN-06, the off-by-one write to `dest[size]` is shown real in pre-flight. It is fixed by an agent (Alpha, in the recorded runs) with `malloc(size + 1)` and `i < size`, checked by the build, PoV and regression gates, then attacked by re-fuzzing and proved by CBMC.
- The key records are `Target` → `VulnerabilityReport` → `PatchResult` / `DRVReport` → `PipelineResult` → `HardeningVerdict` and `VerificationResult`.

CHECK YOURSELF

1. Why does pre-flight insist that the PoV *fails* on the unpatched program before it may be used as a gate? What could go wrong without this rule?
2. In the SYN-06 run, which DRV gate is always SKIPPED, and why? (Hint: look at the third argument passed to `Orchestrator` in `benchmark.py`.)
3. The regression test for SYN-06 fails on the *unpatched* program. Why is that, and why does the system still use the regression gate?
4. For SYN-06, differential testing is expected to compare zero inputs. Which property of `HardeningVerdict` stops that from counting as evidence, and which check provides the evidence instead?
5. Two agents both produce fully validated patches: Beta changes 3 lines and Alpha changes 5. Which one wins, and which function decides?

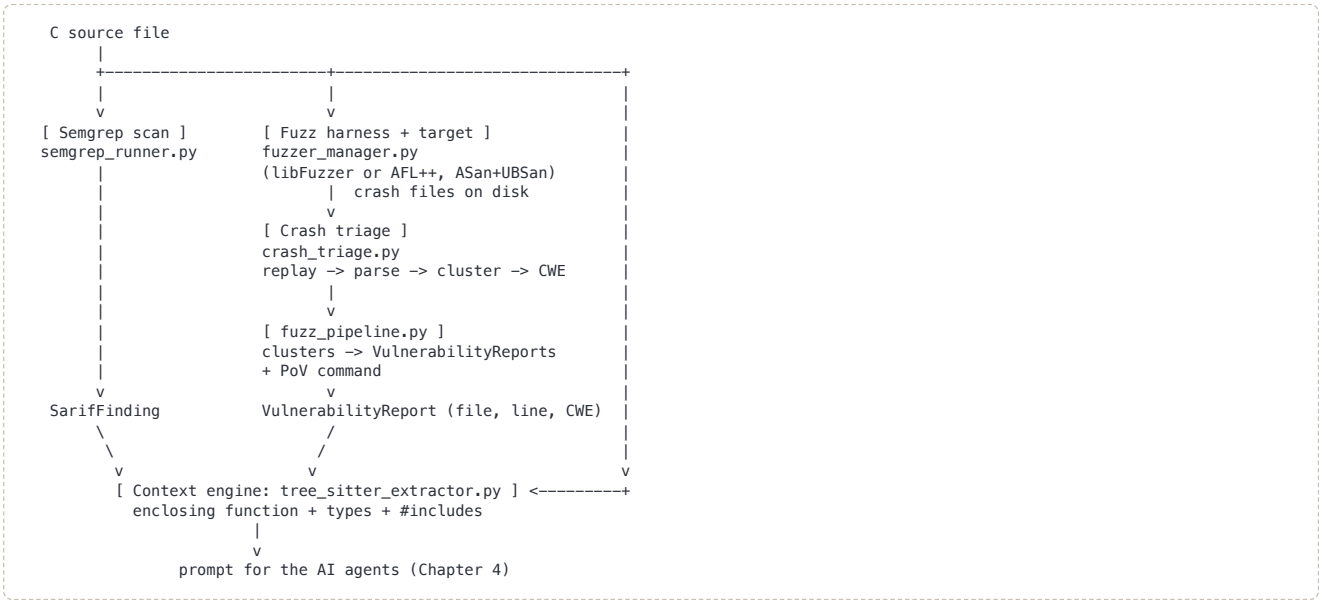
CHAPTER 3

Finding and Locating Bugs

Before AI Kavach can fix anything, it has to answer two questions: "**Is there a bug?**" and "**Exactly where is it?**" This chapter follows that detective work from start to finish. We will meet two very different detectives:

- **Semgrep**, the "reader", which studies the source code without running it and points at lines that look dangerous (this is called *static analysis*).
- **The fuzzer**, the "crash tester", which runs the program millions of times with random-ish inputs until something breaks (this is *dynamic analysis*).

Then we follow what happens after the fuzzer breaks something: **crash triage** (sorting a pile of crashes into a few distinct bugs and naming each one), the **fuzz pipeline** that turns those bugs into work orders for the AI agents, and finally the **context engine**, which cuts out exactly the right slice of source code to show the AI.



A quick reminder from Chapter 1: a **sanitizer** (ASan for memory errors, UBSan for "undefined behaviour" such as integer overflow) is extra checking code the compiler weaves into a program, so that a silent memory mistake turns into a loud, clearly described crash. Everything in this chapter leans on that loud description.

3.1 Static scanning with Semgrep

What Semgrep does

Semgrep is a pattern-matching tool for source code. It ships with libraries of **rules**, each describing a code shape that is often a bug. For example, "a call to `strcpy` whose destination is a fixed-size array" or "a `printf` whose format string comes from a variable". Semgrep reads the code, finds every place that matches a rule, and reports it. It never runs the program.

> **ANALOGY**
>
> Semgrep is like a proof-reader with a list of common grammar mistakes. It can flag "their/there" in seconds across a whole book, but it cannot tell you whether the story makes sense. It is fast and wide but shallow, and it sometimes flags things that are actually fine (false alarms).

In AI Kavach, Semgrep is used in two places: as the **only** bug-finder in the command-line tool `src/main.py`, and again later as a "re-scan" gate that checks a patch did not introduce new problems (covered in Chapter 5). The code lives in `src/analysis_engine/semgrep_runner.py`.

Running the scan

The class `SemgrepRunner` is created with a rule set, by default `"p/security-audit"`, a security rule pack from the Semgrep online registry. (Because it is fetched from the registry, this default needs network access.) On creation it runs `semgrep --version` and raises an error with the hint "Install with: pip install semgrep" if Semgrep is missing. The real work is in `scan()`:

src/analysis_engine/semgrep_runner.py · lines 103-131

```
            cmd = [
                "semgrep", "scan",
                "--config", self.rules,
                "--sarif",
                "--output", sarif_path,
                "--quiet",
                "--no-git-ignore",
                str(target),
            ]

            logger.info(f"Running: {' '.join(cmd)}")
            result = subprocess.run(
                cmd,
                capture_output=True,
                text=True,
                timeout=timeout,
            )

            if result.returncode not in (0, 1):  # 1 = findings found (expected)
                logger.warning(f"Semgrep stderr: {result.stderr}")

            # Parse SARIF output
            findings = self._parse_sarif(sarif_path, target_path)

            # Sort by severity
            severity_order = {"error": 0, "warning": 1, "note": 2}
            findings.sort(key=lambda f: severity_order.get(f.severity, 3))

            return findings[:max_findings]
```

LINE BY LINE

1. `cmd = [...]` builds the command exactly as you would type it in a terminal, one word per list item: `semgrep scan --config p/security-audit --sarif --output /tmp/xyz.sarif --quiet --no-git-ignore yourfile.c`.
2. `--sarif` and `--output` ask Semgrep to write its results as a **SARIF** file (explained below) into a temporary file the code created just before (`tempfile.NamedTemporaryFile(suffix=".sarif", ...)`).
3. `--quiet` silences progress chatter; `--no-git-ignore` tells Semgrep to scan files even if a `.gitignore` file says to skip them.
4. `subprocess.run(...)` is Python's way of launching another program and waiting for it. `timeout=timeout` (default 300 seconds) stops a scan that runs too long; in that case the code logs an error and returns an empty list.
5. Semgrep's exit code 1 means "I found something", which is expected, so only codes other than 0 or 1 cause a warning.
6. The SARIF file is parsed into a list of findings, sorted so `error` comes first, then `warning`, then `note`, and cut to at most `max_findings` (default 100).
7. A `finally:` block (just after this excerpt) deletes the temporary SARIF file whatever happens.

What SARIF is, and how it is parsed

SARIF (Static Analysis Results Interchange Format, version 2.1.0) is a standard JSON layout that many code-scanning tools use for their reports, so that other tools can read them without custom code. JSON is just nested lists and labelled boxes of text and numbers. A SARIF file contains a list of `runs`. Each run has two parts that matter here:

- `tool.driver.rules`: the **rule catalogue**, one entry per rule, with tags such as `CWE-120` or `OWASP-A03`. (A **CWE**, "Common Weakness Enumeration", is a standard catalogue number for a kind of bug; for example CWE-787 is "out-of-bounds write".)
- `results`: the actual **hits**, each naming a `ruleId`, a message, a level, and a location (file, start line, end line, and sometimes a code snippet).

Because the CWE tags live in the catalogue, not in the hits, the parser does two passes: first it builds a lookup table from rule ID to tags, then it walks the hits and attaches the tags.

src/analysis_engine/semgrep_runner.py · lines 149–165

```
        for run in sarif.get("runs", []):
            # Build rule lookup for CWE/OWASP mappings
            rule_map = {}
            for rule in run.get("tool", {}).get("driver", {}).get("rules", []):
                rule_id = rule.get("id", "")
                cwe_ids = []
                owasp_ids = []
                for tag in rule.get("properties", {}).get("tags", []):
                    if tag.startswith("CWE-"):
                        cwe_ids.append(tag)
                    elif tag.startswith("OWASP"):
                        owasp_ids.append(tag)
                rule_map[rule_id] = {
                    "cwe_ids": cwe_ids,
                    "owasp_ids": owasp_ids,
                    "confidence": rule.get("properties", {}).get("precision", "medium"),
                }
```

LINE BY LINE

1. `sarif.get("runs", [])`: `.get(key, default)` means "give me this field, or this default if it is missing". The code uses it everywhere, so a SARIF file with missing pieces never crashes the parser.
2. For each rule in the catalogue, it looks at the tags. Tags starting with `CWE-` go into one list, tags starting with `OWASP` into another.
3. The rule's `precision` property (how often the rule is right) becomes the finding's `confidence`, defaulting to `"medium"`.
4. Everything is stored in `rule_map`, keyed by rule ID, ready for the second pass.

The second pass (lines 168–202) loops over `results`. It skips any hit without a location, takes the **first** location's file URI and line region, and grabs a code snippet if one exists (preferring the wider `contextRegion`). Each hit becomes a `SarifFinding`, a small record with these fields: `rule_id`, `message`, `severity`, `file_path`, `start_line`, `end_line` (if missing, it copies the start line), `cwe_ids`, `owasp_ids`, `confidence` and `snippet`.

A `SarifFinding` also knows how to describe itself to the AI. Its `to_prompt_context()` method produces a short block of text headed `SARIF FINDING:` listing rule, severity, CWE (or "Unknown"), file, lines, message and snippet. That block is pasted into the agent prompt in the next chapter.

KEY IDEA

Semgrep gives a *suspicion*, not a proof. It says "this line looks like a buffer overflow". It cannot show an input that actually triggers the bug. That is why the project also fuzzes: a fuzzer's crash is hard evidence, and the crashing input becomes the **proof-of-vulnerability (PoV)** that later decides whether a patch really works.

TRY IT YOURSELF

If Semgrep is installed (`pip install semgrep`; note it is not in `requirements.txt`) and you are online, from the repo root:

```
semgrep scan --config p/security-audit --sarif --output /tmp/out.sarif tests/demo_vulns/06_off_by_one_loop.c
```

Then open `/tmp/out.sarif` in a text editor and find the `"runs"`, `"rules"` and `"results"` sections described above. Do not be surprised if this particular file gets no hits: an off-by-one loop is exactly the kind of bug a pattern matcher finds hard to see.

3.2 Fuzz harnesses: the plug between fuzzer and target

Why a harness is needed

A fuzzer is a machine for generating inputs and watching for crashes. But a fuzzer does not know how your program wants to be fed. One program reads a file, another reads command-line words, a third reads a network packet. A **fuzz harness** is a tiny C file that sits between the two. It accepts the fuzzer's raw bytes and calls the vulnerable function with them in a sensible way.

ANALOGY

Think of a crash-test lab for car seats. The test rig (the fuzzer) can throw a dummy at a seat millions of times with different speeds and angles. But the seat must first be bolted to the rig with the right bracket. The harness is that bracket: small, specific to each seat, and essential.

The project keeps its harnesses in `tests/fuzz_harnesses/`, one file per target, named after the target's ID (for example `SYN-06-OFF-BY-ONE.c`). Nine targets have one: SYN-01, 02, 05, 06, 07, 11, 12, 16 and 18.

The bug we will fuzz

Here is the vulnerable code for target SYN-06. It is a classic **off-by-one** error (CWE-193: counting one too many).

tests/demo_vulns/06_off_by_one_loop.c · lines 4-23

```
void copy_and_null_terminate(const char* src, size_t size) {
    if (size == 0 || size > 1024) return;
    char* dest = (char*)malloc(size);
    if (!dest) return;
    for (size_t i = 0; i <= size; i++) {
        if (i == size) {
            dest[i] = '\0';
        } else {
            dest[i] = src[i];
        }
    }
    printf("Copied string: %s\n", dest);
    free(dest);
}
int main(int argc, char* argv[]) {
    if (argc > 1) {
        copy_and_null_terminate(argv[1], strlen(argv[1]));
    }
    return 0;
}
```

`malloc(size)` reserves exactly `size` boxes, numbered `0` to `size-1`. But the loop condition is `i <= size` ("less than *or equal*"), so on the last pass `i` equals `size`, and line 10 writes the end-of-string marker `'\0'` into box number `size`, one box past the end. That is a **heap buffer overflow**: writing into memory the program does not own.

The harness for it

tests/fuzz_harnesses/SYN-06-OFF-BY-ONE.c · lines 1-17

```
/* Fuzz harness for 06_off_by_one_loop.c
 * The bug: loop writes dest[size] on a malloc(size) buffer. */
#include <stdint.h>
#include <stddef.h>
#include <stdlib.h>
#include <string.h>

void copy_and_null_terminate(const char* src, size_t size);

int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    if (size == 0 || size > 1024) return 0;
    char *s = (char *)malloc(size + 1);
    memcpy(s, data, size); s[size] = '\0';
    copy_and_null_terminate(s, size);
    free(s);
    return 0;
}
```

LINE BY LINE

1. The `#include` lines pull in standard definitions: byte types (`uint8_t`), sizes (`size_t`), `malloc` / `free`, and `memcpy`.
2. `void copy_and_null_terminate(const char* src, size_t size);` is a **declaration**: a promise to the compiler that this function exists somewhere else (in the target file). The harness does not contain the vulnerable code; it only calls it.
3. `int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size)` is the **magic function name**. libFuzzer (the fuzzing engine built into the LLVM/clang compiler) looks for a function with exactly this name and calls it over and over, each time with a fresh input: `data` points to the bytes, `size` says how many there are. This one function is the whole contract between fuzzer and harness.
4. `if (size == 0 || size > 1024) return 0;` skips inputs the target would reject anyway. Returning 0 means "fine, give me the next one".
5. `malloc(size + 1)`, `memcpy`, `s[size] = '\0'`: the fuzzer's bytes are copied into a fresh buffer that is one byte bigger, and that last byte is set to the end-of-string marker. This turns raw bytes into a proper C string, as the target expects. Note that the harness sizes its *own* buffer correctly, so any crash must come from the target.
6. `copy_and_null_terminate(s, size);` is the actual call into the code under test. With any non-empty input, the off-by-one write happens and ASan will crash the program.
7. `free(s); return 0;` tidies up, so the harness itself never leaks memory.

WATCH OUT: A HARNESS CAN MAKE A FUNCTION "UNFIXABLE"

`tests/fuzz_harnesses/README.md` records a hard-learned rule. If a harness passes a function a count larger than the buffer it actually supplies, the out-of-bounds read happens because of the *harness*, and no patch inside the function can prevent it. This happened three times in the project (old versions of the SYN-03, SYN-12 and SYN-16 harnesses). Each time, correct patches were rejected and the AI was blamed for a defect in the test rig. The rules written down as a result: "**size the buffer to the largest request the harness can generate**", and "**the harness must exercise the same function the target's line_number points at.**"

The README also explains, target by target, why the other 11 demo bugs have *no* harness. For example, SYN-09 (a command injection) has no memory crash for a sanitizer to see, and SYN-13 (a race condition) needs two things happening at once, which a single fuzz run cannot reproduce.

The `-Dmain=...` trick

There is a clash. The target file has its own `main` function (lines 18–23 above), the place a C program starts. libFuzzer *also* brings its own `main`, which runs the fuzzing loop. A C program may only have one function called `main`, so linking both together would fail.

The fix is a single compiler option: `-Dmain=kavach_disabled_main_SYN_06_OFF_BY_ONE`. The `-D` option defines a text substitution (a "macro"): before compiling, every occurrence of the word `main` in the file is replaced by that long name. The target's `main` silently becomes an ordinary function nobody calls, and the vulnerable function stays exactly as written. The dashes in the target ID are turned into underscores (`target_id.replace('-', '_')`), because a C name cannot contain dashes.

ANALOGY

Two people at a meeting are both called "Chair". Rather than rewrite the agenda, you put a sticky label on one name badge saying "Retired-Chair-06". The person is still there and unchanged; there is just no confusion about who runs the meeting.

Feeding AFL++ through `afl_driver.c`

The project supports a second fuzzer, **AFL++**. AFL++ works differently. Instead of calling a function inside the program, it writes each input to a file and launches the program afresh, passing the file's name on the command line (AFL calls this placeholder `@@`). So AFL++ needs a `main` that reads a file. That is the job of `tests/fuzz_harnesses/afl_driver.c`:

tests/fuzz_harnesses/afl_driver.c · lines 17-46

```
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size);

int main(int argc, char **argv) {
    if (argc < 2) {
        fprintf(stderr, "usage: %s <input-file>\n", argv[0]);
        return 1;
    }

    FILE *f = fopen(argv[1], "rb");
```

```
    if (!f) {
        perror("fopen");
        return 1;
    }

    fseek(f, 0, SEEK_END);
    long len = ftell(f);
    fseek(f, 0, SEEK_SET);
    if (len < 0) { fclose(f); return 1; }

    uint8_t *buf = (uint8_t *)malloc((size_t)len > 0 ? (size_t)len : 1);
    if (!buf) { fclose(f); return 1; }

    size_t got = fread(buf, 1, (size_t)len, f);
    fclose(f);

    LLVMFuzzerTestOneInput(buf, got);

    free(buf);
    return 0;
}
```

LINE BY LINE

1. It declares `LLVMFuzzerTestOneInput`, the same entry point the harness defines.
2. `main` checks it was given a file name (`argv[1]`), otherwise prints a usage message.
3. `fopen(..., "rb")` opens the file for reading as raw bytes.
4. `fseek` to the end, `ftell` to read the position (that is the file's length), then `fseek` back to the start.
5. It allocates a buffer of that length (at least 1 byte, so an empty file still gets a valid pointer), reads the whole file in, and closes it.
6. It hands the bytes to `LLVMFuzzerTestOneInput`, exactly as libFuzzer would.

The result: **one harness file serves both fuzzers**. This little driver is also used later, outside fuzzing, to rebuild a plain program that replays a single crash file. That replay is the PoV command you will see in section 3.6.

Seeds

A fuzzer starts from a few example inputs called **seeds**, and mutates them (flips bits, inserts bytes, splices them together). The shared seeds folder `tests/fuzz_harnesses/seeds/` holds three tiny files:

File	Bytes (hex)	What it is
`short.bin`	`41 42`	the two letters "AB"
`num.bin`	`05 00 00 00`	the number 5 as a 4-byte integer (least significant byte first)
`pair.bin`	`10 00 00 00 02 00 00 00`	two 4-byte integers, 16 and 2

They are deliberately generic: a short string and a couple of number shapes, giving every harness a reasonable starting point. For SYN-06, even "AB" already crashes, because any non-empty input triggers the off-by-one.

3.3 The fuzzer manager: building and running a campaign

`src/analysis_engine/fuzzer_manager.py` does three jobs: find out which fuzzing tools exist on this computer, compile the harness and target into a fuzzable program, and run a time-limited **campaign** (one fuzzing session against one target).

Finding a usable compiler

libFuzzer is not a separate program. It is a library (the "fuzzer runtime") that ships with some builds of the clang compiler. The catch, noted in the file's header, is that Apple's built-in clang on macOS does **not** include it, while the Homebrew build of LLVM does. So the manager searches in a specific order:

src/analysis_engine/fuzzer_manager.py · lines 39-43 and 77-106

```
# Homebrew LLVM ships libclang_rt.fuzzer_osx.a; Apple's command-line clang does not.
_LLVM_CANDIDATES = [
    "/opt/homebrew/opt/llvm/bin/clang",
    "/usr/local/opt/llvm/bin/clang",
]
    ...
    @staticmethod
    def libfuzzer_clang() -> Optional[str]:
        """Path to a clang whose runtime includes libFuzzer, or None."""
        for candidate in _LLVM_CANDIDATES:
            if Path(candidate).exists():
                return candidate
        clang = shutil.which("clang")
        if clang and FuzzerManager._has_fuzzer_runtime(clang):
            return clang
        return None

    @staticmethod
    def _has_fuzzer_runtime(clang: str) -> bool:
        probe = Path(os.environ.get("TMPDIR", "/tmp")) / "kavach_fuzz_probe.c"
        probe.write_text(
            "#include <stdint.h>\n#include <stddef.h>\n"
            "int LLVMFuzzerTestOneInput(const uint8_t*d,size_t s){(void)d;(void)s;return 0;}\n"
        )
        out = probe.with_suffix(".bin")
        try:
            result = subprocess.run(
                [clang, "-fsanitize=fuzzer", str(probe), "-o", str(out)],
                capture_output=True, timeout=120,
            )
            return result.returncode == 0
        ...
```

LINE BY LINE

1. First it checks the two usual Homebrew install paths (`/opt/homebrew` on Apple-silicon Macs, `/usr/local` on Intel Macs). If either file exists, it is used straight away.
2. Otherwise `shutil.which("clang")` finds whatever clang is first on the system's search path (the `PATH`).
3. That clang is not trusted blindly. `_has_fuzzer_runtime` writes a do-nothing harness (an `LLVMFuzzerTestOneInput` that just returns 0) and tries to compile it with `-fsanitize=fuzzer`. If compiling succeeds (exit code 0), the runtime is present. This is a **probe**: rather than guessing from version numbers, it simply tries the real thing.
4. The `finally` clause (trimmed) deletes the probe files either way.

AFL++ detection is simpler: `afl_available()` returns true only if both `afl-fuzz` (the fuzzer) and `afl-clang-fast` (AFL's compiler wrapper) are on the `PATH`. `available_engines()` combines the two checks into a list such as `["libfuzzer", "afl++"]`.

The libFuzzer build and what each flag means

src/analysis_engine/fuzzer_manager.py · lines 138-151

```
        if engine == "libfuzzer":
            clang = self.libfuzzer_clang()
            if not clang:
                return None, "no clang with a libFuzzer runtime found"
            cmd = [
                clang, "-fsanitize=fuzzer,address,undefined",
                "-fno-sanitize-recover=all", "-fno-omit-frame-pointer", "-g", "-O1",
                f"-Dmain=kavach_disabled_main_{target_id.replace('-', '_')}",
```

```
        str(harness), source_file, "-o", str(binary),
    ]
    result = subprocess.run(cmd, capture_output=True, text=True, timeout=300)
    if result.returncode != 0:
        return None, result.stderr[:2000]
    return binary, ""
```

Flag	Plain-English meaning
`-fsanitize=fuzzer`	Link in libFuzzer, which supplies `main` and the mutate-run-watch loop, and add **coverage instrumentation**: tiny counters at every branch so the fuzzer can tell when an input reached new code.
`-fsanitize=address`	Turn on **AddressSanitizer** (ASan): out-of-bounds accesses, use-after-free and double-free become immediate, described crashes.
`-fsanitize=undefined`	Turn on **UndefinedBehaviorSanitizer** (UBSan): catches things like signed integer overflow, bad shifts and out-of-range array indexes.
`-fno-sanitize-recover=all`	Normally UBSan prints a warning and carries on. This makes *every* sanitizer finding stop the program, so the fuzzer counts it as a crash.
`-fno-omit-frame-pointer`	Keep a bookkeeping register the compiler might otherwise reuse. It makes the "how did we get here" list of function calls (the **stack trace**) reliable.
`-g`	Include debug information, so a crash report can say `06_off_by_one_loop.c:10` instead of a raw memory address.
`-O1`	Light optimisation. The program runs fast enough to fuzz well while staying close to the source code. (It can still *inline* small functions, which matters for triage below.)
`-Dmain=kavach_disabled_main_...`	The rename trick from 3.2.
`harness source_file -o binary`	Compile both files together into one program named `fuzz_target`.

The output goes to `<workspace>/<target_id>/libfuzzer/fuzz_target`. If the build fails, the first 2,000 characters of the compiler's error text are returned so a person can see why. If no harness file exists for the target, `build()` returns an explanatory message instead of guessing.

WATCH OUT: A CONSTANT THAT IS NEVER USED

At the top of the file sits a list called `SAN_FLAGS` (lines 32–37) with nearly the same flags. Nothing in the code actually uses it: both build paths spell their flags out in full. It is harmless, but if you ever change the flags, change them where they are really used.

The AFL++ build

AFL++ builds in separate steps, for a careful reason explained in the comment on lines 158–159:

src/analysis_engine/fuzzer_manager.py · lines 156–181

```
    env = dict(os.environ, AFL_USE_ASAN="1", AFL_USE_UBSAN="1", AFL_QUIET="1")
    objects = []
    # The target is compiled separately so that -Dmain only renames ITS
    # main, not the driver's.
    units = [
        (source_file, [f"-Dmain=kavach_disabled_main_{target_id.replace('-', '_')}"]),
        (str(harness), []),
        (str(self.harness_dir / "afl_driver.c"), []),
    ]
    for i, (src, extra) in enumerate(units):
        obj = out_dir / f"unit{i}.o"
        result = subprocess.run(
            ["afl-clang-fast", "-g", "-O1", "-c", *extra, src, "-o", str(obj)],
            capture_output=True, text=True, env=env, timeout=300,
        )
        if result.returncode != 0:
            return None, result.stderr[:2000]
```

```
            objects.append(str(obj))

        result = subprocess.run(
            ["afl-clang-fast", "-g", "-O1", *objects, "-o", str(binary)],
            capture_output=True, text=True, env=env, timeout=300,
        )
        if result.returncode != 0:
            return None, result.stderr[:2000]
        return binary, ""
```

LINE BY LINE

1. Instead of command-line flags, AFL++ turns sanitizers on through **environment variables** (settings handed to a program when it starts): `AFL_USE_ASAN=1` and `AFL_USE_UBSAN=1`. `AFL_QUIET=1` reduces output.
2. Three "units" are listed: the target (with the `-Dmain` rename), the harness, and `afl_driver.c`. Only the target gets the rename. If all three were compiled in one go, the rename would also hit the driver's `main`, and the program would have no starting point at all.
3. Each unit is compiled with `-c` ("compile only, don't link") into an object file `unit0.o`, `unit1.o`, `unit2.o`.
4. A final `afl-clang-fast` call **links** the three object files into one program.

Running a campaign and where the crash files land

`run_campaign(target_id, source_file, engine="libfuzzer", seconds=30, seed_dir=None)` builds the binary, picks the seed folder (`tests/fuzz_harnesses/seeds` unless told otherwise), wipes and recreates a fresh `findings` folder beside the binary, starts a stopwatch, and hands over to the engine-specific runner. Everything it records in the returned `FuzzCampaignResult` is *observed*: engine, duration, number of executions, list of crash files, and any build or run error.

src/analysis_engine/fuzzer_manager.py · lines 228–254

```
    def _run_libfuzzer(self, binary, seeds, out_dir, seconds, result):
        corpus = out_dir / "corpus"
        corpus.mkdir(exist_ok=True)
        artifacts = out_dir / "crashes"
        artifacts.mkdir(exist_ok=True)

        cmd = [
            str(binary),
            str(corpus), str(seeds),
            f"-max_total_time={seconds}",
            f"-artifact_prefix={artifacts}/",
            "-print_final_stats=1",
        ]
        try:
            proc = subprocess.run(
                cmd, capture_output=True, timeout=seconds + 120,
                env=dict(os.environ, ASAN_OPTIONS="detect_leaks=0", MallocNanoZone="0"),
                cwd=str(out_dir),
            )
            output = proc.stderr.decode("utf-8", "replace")
            for line in output.splitlines():
                if "stat::number_of_executed_units:" in line:
                    result.executions = int(line.split(":")[-1].strip())
        except subprocess.TimeoutExpired:
            result.run_error = "libFuzzer exceeded its wall-clock budget"

        result.crashes = [str(p) for p in artifacts.iterdir() if p.is_file()]
```

LINE BY LINE

1. Two folders are created: `corpus` (where libFuzzer saves "interesting" inputs that reached new code) and `crashes`.

2. The binary is run directly; a libFuzzer program *is* the fuzzer. Its arguments: the corpus folder first (where new inputs are written), then the seeds folder (read as extra starting points).
3. `-max_total_time=30` stops after the time budget. `-artifact_prefix=.../crashes/` tells libFuzzer where to save a crashing input (libFuzzer names such files `crash-` followed by a long fingerprint of the content). `-print_final_stats=1` prints a summary at the end.
4. Environment: `ASAN_OPTIONS=detect_leaks=0` turns off memory-leak reports (leaks are not what is being hunted, and libFuzzer's own allocations would cause false alarms). `MallocNanoZone=0` switches off a macOS memory allocator feature that interferes with ASan.
5. A safety timeout of budget + 120 seconds guards against a hung fuzzer.
6. The summary line `stat::number_of_executed_units:` is read to record how many inputs were tried.
7. Finally, every file in the `crashes` folder is listed. That list is the campaign's result.

So for SYN-06 with libFuzzer, crash files land in:

```
benchmark_workspace/fuzzing/
  SYN-06-OFF-BY-ONE/
    libfuzzer/
      fuzz_target            <- the built program
      findings/
        corpus/              <- inputs that reached new code
        crashes/
          crash-<fingerprint>  <- the input that crashed it
```

(`benchmark_workspace/fuzzing` is the workspace that `FuzzDiscoveryPipeline` uses by default.)

The AFL++ runner (lines 256–287) works the same way in spirit. It runs `afl-fuzz -i <seeds> -o <findings> -V <seconds> -m none -- <binary> @@`, where `-V` is the time limit, `-m none` removes AFL's memory limit (ASan needs a lot of virtual memory), and `@@` is replaced by AFL with the path of each test file. It sets several environment variables: `AFL_MAP_SIZE=65536` (a smaller coverage map, because macOS's default shared-memory limits are too small for AFL's default), `AFL_SKIP_CPUFREQ` and `AFL_NO_AFFINITY` (skip some CPU set-up checks), `AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES`, and `ASAN_OPTIONS=detect_leaks=0:abort_on_error=1:symbolize=0`. The execution count is read from AFL's `default/fuzzer_stats` file (`execs_done`), and crashes are collected from every folder named `crashes` under the output, ignoring AFL's `README` file. AFL names its crash files with an `id:000000,...` style label.

WATCH OUT: LIBFUZZER STOPS AT THE FIRST CRASH

libFuzzer is run without its `-fork` or `-keep_going` style options. By default, libFuzzer **stops the moment it finds its first crash**. So a libFuzzer campaign normally yields *one* crash file, even if the target contains several different bugs. A second bug hiding behind the first will not be found in the same run. AFL++ keeps going for its whole time budget and can collect many crashes, which is where clustering really earns its keep.

TRY IT YOURSELF

From the repo root, with Homebrew LLVM installed, run only the fuzz-discovery suite for 30 seconds per target (this also calls the AI agents, so it needs an API key):

```
python benchmark.py --suite fuzz --fuzz-seconds 30
```

Or with AFL++: `python benchmark.py --suite fuzz --fuzz-engine afl++ --fuzz-seconds 30`. Afterwards, look inside `benchmark_workspace/fuzzing/` for the folder layout shown above.

3.4 Crash triage: from a pile of crash files to named bugs

A fuzzer's output is just files of bytes. Some may not even crash when you try again (for example if the crash depended on timing). Many may be the *same* bug reached in slightly different ways; AFL++ can produce hundreds of crash files for one bug. **Triage**, a word borrowed from hospital emergency rooms, means sorting these patients: confirm each one is really sick, diagnose what is wrong, and group identical cases together. The code is in `src/analysis_engine/crash_triage.py`, class `CrashTriage`.

ANALOGY

Imagine a helpdesk that receives 500 complaint emails in a morning. You would not assign 500 engineers. You would read each email, confirm the problem is real, and notice that 480 of them say "login button does nothing on the checkout page". Those become one ticket. Triage does exactly this: **patching effort is spent per bug, not per crashing input**, as the file's header says.

Step 0: choose a backend

When `CrashTriage` is created, it checks whether two tools from the **CASR** project are installed (`casr-san` and `casr-cluster`). If both are, the backend is `"casr"`; otherwise it is `"native-asan"`, the project's own implementation. We cover CASR in 3.5; the native path is the one actually used on the development machine, where CASR is not installed.

Step 1: the main loop

src/analysis_engine/crash_triage.py · lines 152–168

```
        clusters: dict[str, CrashCluster] = {}
        not_reproduced = 0

        for crash_input in crash_inputs:
            signature = self._signature_for(target_binary, crash_input, args, input_mode)
            if signature is None:
                not_reproduced += 1
                continue
            cluster = clusters.setdefault(signature.digest, CrashCluster(signature=signature))
            cluster.inputs.append(crash_input)

        if not_reproduced:
            logger.info(f"{not_reproduced} crash input(s) did not reproduce and were discarded")
        logger.info(
            f"Triaged {len(crash_inputs)} crash input(s) into {len(clusters)} distinct root cause(s)"
        )
        return list(clusters.values())
```

LINE BY LINE

1. `clusters` is a dictionary (a lookup table) from a **digest** (a short fingerprint of the bug, explained in step 5) to a `CrashCluster`, a group of inputs sharing that fingerprint.
2. For each crash file, `_signature_for` replays it and works out its signature: crash type, stack frames, file and line.
3. If the signature is `None`, the input did not crash again. It is counted and **dropped**. It is never reported as a bug.
4. `clusters.setdefault(digest, new_cluster)` means "if a cluster with this fingerprint exists, give it to me; otherwise create one". The input is then added to that cluster. Identical bugs therefore pile into the same box automatically.
5. The log reports how many inputs collapsed into how many distinct root causes.

Step 2: replay the crash

src/analysis_engine/crash_triage.py · lines 211–235

```
        cmd = [target_binary]
        stdin_data = None

        if input_mode == "stdin":
            stdin_data = Path(crash_input).read_bytes()
            cmd += list(args or [])
        else:
            supplied = list(args or ["@@"])
            cmd += [crash_input if a == "@@" else a for a in supplied]

        try:
            proc = subprocess.run(
                cmd, input=stdin_data, capture_output=True, timeout=30,
                env=dict(os.environ, ASAN_OPTIONS="detect_leaks=0:abort_on_error=0",
                         MallocNanoZone="0"),
            )
        except subprocess.TimeoutExpired:
            # A hang is a real finding, but it is not a memory-safety crash.
```

```
            return CrashSignature(crash_class="timeout/hang", frames=[], raw_report="timed out")
        except Exception as e:
            logger.debug(f"Failed to execute crash input {crash_input}: {e}")
            return None

        report = proc.stderr.decode("utf-8", "replace") + proc.stdout.decode("utf-8", "replace")
        return self.parse_sanitizer_report(report, self.source_root)
```

LINE BY LINE

1. The command starts with the fuzz binary. In the default `"file"` mode, any argument `"@@"` is replaced by the crash file's path (the same placeholder convention AFL uses). The fuzz pipeline passes `args=["@@"]`, so the command is simply `fuzz_target crash-<fingerprint>`. A libFuzzer program given a file name instead of a folder runs just that one input, which is exactly a replay.
2. In `"stdin"` mode the file's bytes are piped into the program's standard input instead.
3. The program gets 30 seconds. If it hangs, that is recorded as a special `"timeout/hang"` signature: a real finding, but not a memory-safety crash.
4. `abort_on_error=0` asks ASan to exit normally after printing its report, rather than aborting.
5. Both error output and normal output are collected as text and handed to the parser.

Step 3: read the sanitizer report

When ASan catches a bug, it prints a report. Here is a **simplified, illustrative** example of the shape of an ASan report for the SYN-06 bug, in the macOS style the parser was written for (addresses shortened; real reports have more lines):

```
==12345==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x6020000000b1 ...
WRITE of size 1 at 0x6020000000b1 thread T0
    #0 0x100003f2c in copy_and_null_terminate 06_off_by_one_loop.c:10
    #1 0x100003e10 in LLVMFuzzerTestOneInput SYN-06-OFF-BY-ONE.c:14
    #2 0x10001a2b4 in fuzzer::Fuzzer::ExecuteCallback(...) FuzzerLoop.cpp:614
    ...
SUMMARY: AddressSanitizer: heap-buffer-overflow 06_off_by_one_loop.c:10 in copy_and_null_terminate
```

Each `#0`, `#1`, ... line is a **stack frame**: frame #0 is the function that was running when the crash happened, #1 is the function that called it, and so on down to the program's start. The parser needs three things out of this text: the **crash class** (here `heap-buffer-overflow`), the list of **frames**, and the **file and line** of the bug.

It uses **regular expressions** ("regexes"), a compact language for describing text patterns. A few symbols are enough to follow along:

Symbol	Meaning
`\s` / `\d` / `\w`	a space-like character / a digit / a letter, digit or underscore
`+` / `*` / `?`	"one or more" / "zero or more" / "optional" of the thing before
`.`	any single character
`+?`	"one or more, but as *few* as possible" (lazy)
`( ... )`	a **capture group**: "remember this part for me"
`(?: ... )`	a group that is not remembered, just for grouping
`a|b`	"a or b"
`^` / `$`	start of line / end of line
`[^\s:]`	any character that is *not* a space or colon

src/analysis_engine/crash_triage.py · lines 75-84

```
# The class is everything up to " on <address>", so multi-word forms survive.
# A bare \w+ capture turned "attempting double-free on 0x60c…" into the useless
# class "attempting", which matches no CWE and blurs clustering.
_ASAN_ERROR = re.compile(
    r"ERROR:\s+(?:AddressSanitizer|LeakSanitizer):\s+(.+?)(?:\s+on\s|\s*$)",
    re.MULTILINE,
)
```

```
_UBSAN_ERROR = re.compile(r"([^\s:]+):(\d+):\d+:\s+runtime error:\s+(.+)")
_ALLOC_SIZE = re.compile(r"requested allocation size")
_FRAME = re.compile(r"^\s*#(\d+)\s+0x[0-9a-f]+\s+in\s+(.+?)(?:\s+([^\s]+\.(?:c|cc|cpp|h)):(\d+))?\s*$")
```

Let us read each pattern in plain words.

- `_ASAN_ERROR` means: find the text `ERROR:`, some spaces, then either `AddressSanitizer` or `LeakSanitizer`, a colon and spaces. Then **capture** as little text as possible (`(.+?)`) until you reach either the word `on` surrounded by spaces, or the end of the line. On our example that captures `heap-buffer-overflow`. The comment explains a past bug: an earlier version captured only one word, so "attempting double-free on 0x..." became the useless class "attempting". `re.MULTILINE` lets `$` mean "end of any line", not just end of the whole text.
- `_UBSAN_ERROR` matches UBSan's one-line style, such as `file.c:12:7: runtime error: ...`. It captures three things: the file name (characters that are not spaces or colons), the line number (digits), and, after skipping the column number, the error message itself.
- `_ALLOC_SIZE` simply looks for the phrase "requested allocation size". ASan reports this when a program asks for an absurdly large amount of memory, usually because a size calculation overflowed.
- `_FRAME` matches one stack-frame line. Read left to right: optional leading spaces, `#` and the frame number (captured), spaces, a hex address like `0x100003f2c`, the word `in`, then the **function name** captured lazily (`(.+?)`). After that comes an **optional** group: a space, a file name ending in `.c`, `.cc`, `.cpp` or `.h` (captured), a colon, and a line number (captured). Then optional spaces and the end of the line. On `#0 0x100003f2c in copy_and_null_terminate 06_off_by_one_loop.c:10` it captures frame `0`, function `copy_and_null_terminate`, file `06_off_by_one_loop.c`, line `10`.

> **WATCH OUT: THE LINUX FRAME FORMAT**
>
> The `_FRAME` pattern expects the line to **end** right after `file.c:LINE`. On Linux, ASan normally prints a column number too: `... in copy_and_null_terminate /src/06_off_by_one_loop.c:10:19`. Because of the extra `:19`, the optional file-and-line group cannot reach the end of the line, so the regex skips it. The lazy function-name group then swallows the whole tail. Running the pattern on that Linux-style line gives function `"copy_and_null_terminate /src/06_off_by_one_loop.c:10:19"` and **no file and no line**. The result: on Linux, most ASan crashes get no location, and the fuzz pipeline later skips them as "crash location unresolved". The unit tests only cover the macOS format. (UBSan's own one-line format is handled by `_UBSAN_ERROR`, which does expect the column.) The fix would be to allow an optional `:\d+` after the line number.

Now the parsing logic that uses these patterns, in order of priority:

1. If `_ASAN_ERROR` matches, its capture is the crash class. A leading word "attempting " is stripped, because "attempting double-free" should simply be "double-free" ("the actionable class is the defect, not the verb", says the comment).
2. Otherwise, if the text mentions "requested allocation size", the class is `"requested allocation size (integer overflow)"`.
3. Otherwise, if `_UBSAN_ERROR` matches, the class is the UBSan message, and file and line come straight from that line.
4. Otherwise, if the text contains `SEGV` (a "segmentation fault": touching memory at an invalid address), the class is `"SEGV"`.
5. Otherwise there is no sanitizer error at all, so the function returns `None`: the input did not reproduce a crash.

Step 4: filter noise frames and pick the frame in the source tree

Not every stack frame is useful. Many belong to the sanitizer itself, to the operating system's libraries, or to the fuzzing scaffolding (libFuzzer's loop, the harness, the AFL driver). Those frames are the same for *every* crash, so including them would only blur the picture. The `_NOISE_FRAME` pattern lists them:

src/analysis_engine/crash_triage.py · lines 60-73

```
# Frames that carry no root-cause information and must not affect the signature.
_NOISE_FRAME = re.compile(
    r"(libclang_rt\.|libsystem_|libdyld|dyld|__asan|__ubsan|__sanitizer|"
    r"wrap_|interceptor_|start\+|libc\+\+|"
    # Fuzzing scaffolding: the harness and engine frames are identical for every
    # crash in a campaign, so including them would blur distinct root causes.
    r"fuzzer::|LLVMFuzzerTestOneInput|afl_driver|__libc_start|"
    # libFuzzer's own driver compilation units. Its entry point is a plain
    # `main`, so `fuzzer::` does not match it: without this the first non-noise
    # frame of an inlined crash is `main FuzzerMain.cpp:20`, and the crash gets
    # attributed to libFuzzer instead of the target.
    r"Fuzzer[A-Za-z]*\.cpp)",
```

```
    re.IGNORECASE,
)
```

This is one long "any of these words" list (`|` means "or"), matched case-insensitively. In groups: sanitizer runtime pieces (`libclang_rt.` , `__asan` , `__ubsan` , `__sanitizer` , `wrap_` , `interceptor_`); macOS system libraries and the loader (`libsystem_` , `libdyld` , `dyld` , `start+` , `libc++`); fuzzing scaffolding (`fuzzer::` , `LLVMFuzzerTestOneInput` , `afl_driver` , `__libc_start`); and any libFuzzer source file such as `FuzzerMain.cpp` or `FuzzerLoop.cpp` . (In regex, `\.` and `\+` mean a literal dot and plus sign.) The header comment adds a nice consequence: because interceptor frames are skipped, the same bug reached through `memcpy` and through `strcpy` still clusters together.

Then the loop that walks the frames:

src/analysis_engine/crash_triage.py · lines 309–331

```
        frames: list[str] = []
        in_root: tuple[str, int] = ("", 0)
        fallback: tuple[str, int] = ("", 0)

        for raw in report.splitlines():
            frame = _FRAME.match(raw)
            if not frame:
                continue
            function, frame_file, frame_line = frame.group(2), frame.group(3), frame.group(4)
            if _NOISE_FRAME.search(raw):
                continue
            frames.append(function.split("(")[0].strip())
            if not frame_file:
                continue

            here = (frame_file, int(frame_line or 0))
            if not fallback[0]:
                fallback = here
            if not in_root[0] and source_root and CrashTriage._under_root(frame_file, source_root):
                in_root = here

        if not file_path:
            file_path, line_number = in_root if in_root[0] else fallback
```

LINE BY LINE

1. Three things are tracked: the list of useful function names (`frames`), the first location inside the project's own source folder (`in_root`), and the first location naming any file at all (`fallback`).
2. Each line of the report is tested against `_FRAME` . Lines that are not stack frames are skipped.
3. Noise frames are skipped entirely.
4. For a useful frame, the function name is cleaned (anything from `(` onward, such as a parameter list, is cut off) and added to `frames` .
5. If the frame names a file, it may become the `fallback` (if none yet) and, if the file belongs to the **source root** (the folder of code under test, `tests/demo_vulns` in the fuzz pipeline), the `in_root` location.
6. At the end, unless UBSan already supplied a location, the in-root location wins; the fallback is used only if there is none.

Why all this care? The long comment above the loop (lines 294–308) explains. When the compiler **inlines** the vulnerable function (copies its body into the caller to save time, which `-O1` may do), ASan reports the crash as happening in the *caller*, which in a fuzz build is the harness. The target's own file may only show up further down, for example in the "freed by" or "allocated by" stack that ASan prints for use-after-free and double-free bugs. Taking the first frame blindly would blame the scaffolding. The comment also admits that a previous version computed the source-root check and then threw the answer away, so the filter did nothing at all.

The helper `_under_root` decides whether a frame's file "belongs" to the project. If the path exists, it checks it lies inside the root folder. Sanitizer frames often carry only a bare file name (like `06_off_by_one_loop.c`), so in that case it searches the root folder for a file with that name. Finally, because the next stage needs to open the file, a bare name is turned back into a real path by the same kind of search (lines 335–338).

Step 5: the clustering signature

All the extracted facts go into a `CrashSignature` . Its `digest` is what decides "same bug or different bug":

src/analysis_engine/crash_triage.py · lines 97–108

```
    @property
    def digest(self) -> str:
        """Stable hash over crash class + top source frames + crash location."""
        material = "|".join([self.crash_class, *self.frames[:5], self.file_path, str(self.line_number)])
        return hashlib.sha256(material.encode()).hexdigest()[:16]

    @property
    def cwe_and_severity(self) -> tuple[str, str]:
        for needle, (cwe, sev) in CRASH_CLASS_TO_CWE.items():
            if needle.lower() in self.crash_class.lower():
                return cwe, sev
        return "", "high"
```

LINE BY LINE

1. `material` glues together, with `|` between them: the crash class, the **top five** useful frames, the file path and the line number. For SYN-06 it would read something like `heap-buffer-overflow|copy_and_null_terminate|.../06_off_by_one_loop.c|10`.
2. `hashlib.sha256(...)` computes a **hash**: a fixed-length fingerprint that is always the same for the same text and almost certainly different for different text. Only the first 16 hexadecimal characters are kept, which is plenty to tell bugs apart.
3. Two crash inputs with the same class, the same top frames and the same location produce the same digest, and so land in the same cluster.

Each cluster also picks a **representative** input: the *smallest* crash file (by size on disk), "the easiest PoV to reason about".

Step 6: mapping crash classes to CWEs

The AI agents (and the final report) are told what kind of bug this is as a CWE number and a severity. The mapping is a simple table:

src/analysis_engine/crash_triage.py · lines 38–58

```
# ASan/UBSan crash class -> (CWE, severity). Used to label the report the agents see.
CRASH_CLASS_TO_CWE = {
    "heap-buffer-overflow": ("CWE-122", "critical"),
    "stack-buffer-overflow": ("CWE-121", "critical"),
    "global-buffer-overflow": ("CWE-787", "critical"),
    "heap-use-after-free": ("CWE-416", "critical"),
    "double-free": ("CWE-415", "critical"),
    "attempting double-free": ("CWE-415", "critical"),
    "alloc-dealloc-mismatch": ("CWE-762", "high"),
    "memcpy-param-overlap": ("CWE-475", "high"),
    "stack-overflow": ("CWE-674", "high"),
    "SEGV": ("CWE-476", "critical"),
    "BUS": ("CWE-476", "critical"),
    "FPE": ("CWE-369", "medium"),
    "requested allocation size": ("CWE-190", "critical"),
    "signed-integer-overflow": ("CWE-190", "high"),
    "shift-exponent": ("CWE-682", "medium"),
    "index-out-of-bounds": ("CWE-125", "critical"),
    "load of misaligned address": ("CWE-1319", "medium"),
    "null-pointer-dereference": ("CWE-476", "critical"),
}
```

Crash class	CWE	In everyday words
heap-buffer-overflow	CWE-122	wrote/read past the end of a `malloc`'d block
stack-buffer-overflow	CWE-121	past the end of a local array inside a function
global-buffer-overflow	CWE-787	past the end of a global array (out-of-bounds write)

Crash class	CWE	In everyday words
heap-use-after-free	CWE-416	used memory after handing it back
double-free	CWE-415	handed the same memory back twice
alloc-dealloc-mismatch	CWE-762	freed memory with the wrong kind of "free"
memcpy-param-overlap	CWE-475	copied between overlapping regions, which `memcpy` forbids
stack-overflow	CWE-674	runaway recursion used up the call stack
SEGV / BUS	CWE-476	touched an invalid address, labelled as a null-pointer bug
FPE	CWE-369	arithmetic fault, typically divide by zero
requested allocation size / signed-integer-overflow	CWE-190	a number grew too big and wrapped around
shift-exponent	CWE-682	shifted bits by an impossible amount (wrong calculation)
index-out-of-bounds	CWE-125	array index outside the array
load of misaligned address	CWE-1319	read a value from an address it must not sit at

The lookup (`cwe_and_severity` above) is a **substring match**: it walks the table in order and returns the first entry whose key appears anywhere inside the crash class, ignoring upper/lower case. If nothing matches, the CWE is left empty and the severity defaults to `"high"`.

WATCH OUT: APPROXIMATIONS IN THE TABLE

- **SEGV is always labelled CWE-476** (null-pointer dereference). Many segmentation faults are null pointers, but a wild pointer or a far out-of-bounds read also causes one. Treat this label as a best guess.
- **Some UBSan keys may never match.** Several keys are written with dashes (`signed-integer-overflow`, `index-out-of-bounds`, `shift-exponent`, `null-pointer-dereference`). These are the names of UBSan's *check options*. The class text the parser actually captures for UBSan is the runtime message, which in standard clang wording uses spaces, for example `signed integer overflow: ...` or `index 10 out of bounds for type ...`. Since the match is a plain substring test, such messages would not match the dashed keys, and the bug would arrive with no CWE and severity "high". (This is from reading the code against clang's usual message wording; it was not run here.) `load of misaligned address` is written with spaces and should match.

Step 7: turning clusters into reports

`to_vulnerability_reports` (lines 170–191) converts each cluster into a `VulnerabilityReport`, the standard "work order" the agent orchestrator consumes (Chapter 4). The fields it fills:

- `id`: `FUZZ-001`, `FUZZ-002`, ... (three-digit numbers).
- `source`: `"fuzzer+triage"`.
- `file_path`, `line_number`: from the signature.
- `description`: for example, "heap-buffer-overflow reached in copy_and_null_terminate. Discovered by fuzzing; 1 crashing input(s) collapsed to this root cause." The function named is the first useful frame.
- `cwe_id`, `severity`: from the table.
- `crash_trace`: the raw sanitizer report, cut to 4,000 characters, so the AI can read the evidence itself.

3.5 The optional CASR path

CASR ("Crash Analysis and Severity Report", from the Russian research institute ISP RAS) is a professional toolkit for exactly this triage job. If `casr-san` and `casr-cluster` are both installed, `CrashTriage` uses CASR first:

src/analysis_engine/crash_triage.py · lines 199-204

```
        if self.backend == "casr":
            sig = self._signature_via_casr(target_binary, crash_input, args)
            if sig:
                return sig
            logger.debug("casr-san produced no report; falling back to native parsing")
        return self._signature_via_asan(target_binary, crash_input, args, input_mode)
```

`_signature_via_casr` runs `casr-san -o <file>.casrep -- <binary> <crash input>`, which writes a JSON report. From that report it takes `CrashSeverity.ShortDescription` as the crash class, the first five `Stacktrace` entries (keeping the last word of each) as frames, and splits `CrashLine` ("file:line") into file and line. If CASR fails or writes nothing, the code quietly falls back to the native parser, so CASR can only add, never break.

Honest notes: CASR is not installed on the development machine, so this path is untested there. And although `casr-cluster` must be present to enable the backend, the code never actually calls it; clustering always uses the project's own digest. Finally, `src/analysis_engine/casr_triage.py` is just a **deprecated alias**: an older module kept so old imports still work. Its `CasrTriage` class is a thin subclass of `CrashTriage` that prints a "superseded" warning and otherwise does nothing new.

3.6 Driller and angr: the "solver" idea (not wired in)

The problem it solves

Fuzzers are great at random exploration but terrible at guessing exact values. Imagine code that only goes down a certain path if the input starts with the four letters `KAVA`. A fuzzer mutating bytes at random has roughly a 1-in-4-billion chance per try of hitting a specific 4-byte value (the module's comment says the same about a 32-bit comparison). Such checks are called **magic bytes**, and they can hide whole regions of code from a fuzzer.

The concolic idea

Driller is a published technique that combines fuzzing with **concolic execution** (a blend of *conc*rete and symb*olic*). Instead of running the program on actual bytes, a symbolic executor runs it on *unknowns*, like algebra's *x*. At each branch (`if s[0]=='K'`) it records the condition as an equation. When it wants to go down a branch the fuzzer never reached, it hands the equations to a **solver**, a program that finds values satisfying them, and gets back an actual input, here `"KAVA"`. That input is put back into the fuzzer's queue. The project uses **angr**, a Python framework for this kind of binary analysis.

> **ANALOGY**
>
> A fuzzer trying to open a combination lock spins the dials randomly. Concolic execution reads the lock's mechanism, writes down "dial 1 must be 7, dial 2 must be 3...", and simply works out the combination.

`src/analysis_engine/driller_monitor.py` contains three pieces:

- `DrillerEngine.drill(binary, seed)` runs angr on a seed, steps through program states, and for each new branch address asks the solver for an input that reaches it, returning up to 32 new inputs. It supports input via stdin, argv, or a file.
- `PlateauMonitor` watches AFL++'s `fuzzer_stats` file. A **plateau** is when the number of discovered paths stops growing: by default, if the last 4 readings (taken 30 seconds apart) are all the same, it calls a callback that would trigger drilling.
- `DrillerEngine.self_test()`, a **known-answer test**. It compiles a tiny program that prints MAGIC only if its first argument starts with "KAVA", asks angr to reach the MAGIC branch, and checks that the solved input really begins with `KAVA`.

src/analysis_engine/driller_monitor.py · lines 111–118 and 144–146

```
            source.write_text(
                "#include <stdio.h>\n"
                "int main(int argc, char** argv){\n"
                "  if (argc < 2) return 0;\n"
                "  const char* s = argv[1];\n"
                "  if (s[0]=='K'&&s[1]=='A'&&s[2]=='V'&&s[3]=='A'){ puts(\"MAGIC\"); return 2; }\n"
                "  puts(\"nope\"); return 0;\n}\n"
            )
    ...
                solved = simgr.found[0].solver.eval(arg, cast_to=bytes)
                if solved.startswith(b"KAVA"):
                    cls._self_test_cache = (True, f"angr solved the known-answer test ({detail})")
```

The reason for the self-test is spelled out in its docstring: importing angr is not the same as angr *working*. On some platforms (notably arm64 Mach-O, i.e. Apple-silicon Macs, the development machine) angr may run without errors but produce meaningless "solutions". So `drill()` checks the self-test first, and if it fails, returns a result marked `supported=False` with the reason, "it never fabricates seeds".

WATCH OUT: ONLY THE SELF-TEST IS ACTUALLY USED

Be clear about what is real here. In the project, the **only** call into this module from the pipeline is `DrillerEngine.self_test()`, made by `benchmark.py` when it writes a "which components were usable" section of its report (`_component_availability`). `drill()` and `PlateauMonitor.watch()` are **never called** by the fuzzing pipeline; they appear only in unit tests. On the arm64 development machine the self-test is expected to fail, so Driller reports itself disabled. Think of this module as a prepared but unplugged extension.

3.7 The fuzz pipeline: from clusters to patch jobs

`src/fuzz_pipeline.py` ties the previous sections together into one end-to-end loop: **fuzz → triage → locate → patch and verify**. The key promise in its header is that the bug's location is *discovered* from the crash's stack trace, not declared in advance, and that the proof-of-vulnerability is the fuzzer's own crashing input.

The class `FuzzDiscoveryPipeline` is created with an orchestrator (the agent manager from Chapter 4), a workspace (default `benchmark_workspace/fuzzing`), an engine (default `libfuzzer`) and a source root (default `tests/demo_vulns`). It builds a `FuzzerManager` and a `CrashTriage`. Its helper `harnessed_targets()` lists every `.c` file in the harness folder except `afl_driver`, which is how `benchmark.py --fuzz` knows which targets can be fuzzed.

src/fuzz_pipeline.py · lines 96–131

```
        # 1 — fuzz
        campaign = self.fuzzer.run_campaign(
            target_id, source_file, engine=self.engine, seconds=seconds
        )
        result.campaign = campaign
        if not campaign.build_ok:
            result.error = f"fuzz build failed: {campaign.build_error[:300]}"
            return result
        if not campaign.crashes:
            logger.info(f"[{target_id}] no crashes found in {seconds}s — nothing to patch")
            return result

        # 2/3 — triage and locate
        result.clusters = self.triage.triage(campaign.binary, campaign.crashes, args=["@@"])
        vulnerabilities = self.triage.to_vulnerability_reports(result.clusters)
        if not vulnerabilities:
            result.error = "crashes found but none reproduced during triage"
            return result

        # 4 — patch each distinct bug, gated on its own discovered crash input
        for vuln, cluster in zip(vulnerabilities, result.clusters):
            if not vuln.file_path or not Path(vuln.file_path).exists():
                logger.warning(f"[{target_id}] {vuln.id}: crash location unresolved, skipping")
                continue

            vuln.id = f"{target_id}::{vuln.id}"
            pov = self._pov_command(campaign.binary, cluster.representative)
            ...
```

LINE BY LINE

1. **Fuzz.** Run a campaign. If the build failed, record the error and stop. If no crashes were found in the time budget, there is nothing to patch, which is a normal, honest outcome.
2. **Triage and locate.** Replay and cluster the crash files (with `args=["@@"]`, so each file is passed by path), then turn clusters into reports. If none reproduced, record that and stop.
3. **Patch.** For each report and its matching cluster, first check the location is real: if the file path is empty or does not exist (for example the Linux frame issue above), the bug is skipped with "crash location unresolved".
4. The report's ID gets the target prefixed, e.g. `SYN-06-OFF-BY-ONE::FUZZ-001`.

5. A PoV command is built from the cluster's representative (smallest) crash file, and the report is passed to `orchestrator.process_vulnerability` (trimmed here) along with the build command, the PoV command and an optional regression command.

The PoV command

The **PoV (proof-of-vulnerability) gate** asks: "with the patch applied, does the exact input that crashed the program still crash it?" To answer, the patched source must be rebuilt together with the harness, and then run on the crash file. `_pov_command` writes that recipe as one shell command string:

src/fuzz_pipeline.py · lines 143-161

```
        harness = Path(fuzz_binary).parent.parent.name
        harness_source = HARNESS_DIR / f"{harness}.c"
        driver = HARNESS_DIR / "afl_driver.c"
        crash_input = str(Path(crash_input).resolve())
        pov_runner = (HARNESS_DIR.parent / "benchmarks" / "harness" / "pov_run.sh").resolve()
        rebuilt = "{workspace}/pov_target"
        mangled = harness.replace("-", "_")
        return (
            f'clang -fsanitize=address,undefined -fno-sanitize-recover=all -g -O1 '
            f'-Dmain=kavach_disabled_main_{mangled} -c "{{src}}" -o "{{workspace}}/tgt.o" && '
            f'clang -fsanitize=address,undefined -fno-sanitize-recover=all -g -O1 '
            f'-c "{harness_source}" -o "{{workspace}}/harness.o" && '
            f'clang -fsanitize=address,undefined -fno-sanitize-recover=all -g -O1 '
            f'-c "{driver}" -o "{{workspace}}/driver.o" && '
            f'clang -fsanitize=address,undefined -fno-sanitize-recover=all '
            f'"{{workspace}}/tgt.o" "{{workspace}}/harness.o" "{{workspace}}/driver.o" '
            f'-o "{rebuilt}" && '
            f'sh "{pov_runner}" "{rebuilt}" "{crash_input}"'
        )
```

LINE BY LINE

1. The target ID is recovered from the fuzz binary's location: `.../SYN-06-OFF-BY-ONE/libfuzzer/fuzz_target`, two folders up, gives `SYN-06-OFF-BY-ONE`. From that it finds the harness file.
2. Note the doubled braces `{{src}}` and `{{workspace}}`. Inside a Python f-string, `{{` produces a literal `{`. So the finished command contains the placeholders `{src}` and `{workspace}`, which the patch-checking loop (Chapter 5) later fills in with the *patched* file and a fresh temporary folder. That is what makes this test the patch rather than the original program.
3. Four `clang` steps, joined by `&&` ("only continue if the previous step succeeded"): compile the patched target (with the `-Dmain` rename), compile the harness, compile `afl_driver.c`, then link all three into `pov_target`. This uses ordinary clang with ASan and UBSan but **not** libFuzzer, so it works even where the libFuzzer runtime is missing. The AFL driver provides the `main` that reads a file.
4. Finally `pov_run.sh` runs the rebuilt program on the crash file. That script fails if it sees sanitizer output or a crash signal. Exit code 0 means "no crash: bug fixed".
5. All paths are absolute (`HARNESS_DIR` is `.resolve()`d at the top of the file), because the check runs from inside a temporary folder where relative paths would silently break.

KEY IDEA

The PoV for a fuzz-found bug is not a hand-written test. It is the very bytes that crashed the program. A patch passes only if *that exact input* no longer triggers any sanitizer report. This is the "deterministic tools decide" principle in action.

3.8 The context engine: showing the AI the right code

Once we know "heap-buffer-overflow at `06_off_by_one_loop.c` line 10", the AI needs to see the code. Pasting a whole real-world file (thousands of lines) wastes the model's limited attention and money, and buries the bug. Pasting just line 10 is useless. The **context engine**, `src/context_engine/tree_sitter_extractor.py`, cuts out a focused, meaningful slice: the whole function containing the bug, the type definitions it uses, and the file's `#include` lines.

What tree-sitter is

Tree-sitter is a fast parser. It reads source code and builds a **syntax tree** (also called an **AST**, abstract syntax tree): a family tree of the code's structure. The file is the root; its children are things like `#include` lines, type definitions and function definitions; a function's children are its name, parameters and body; and so on down to single words. Every node knows its type (for example `function_definition`) and exactly where it starts and ends: line, column and byte position. The project loads tree-sitter's C and C++ grammars (`tree_sitter_c`, `tree_sitter_cpp`).

```
translation_unit                       (the whole file)
 +-- preproc_include      #include <stdio.h>
 +-- preproc_include      #include <stdlib.h>
 +-- preproc_include      #include <string.h>
 +-- function_definition  lines 4-17   copy_and_null_terminate
 |    +-- primitive_type        void
 |    +-- function_declarator
 |    |    +-- identifier       copy_and_null_terminate
 |    |    +-- parameter_list   (const char* src, size_t size)
 |    +-- compound_statement    { ... the body, incl. line 10 ... }
 +-- function_definition  lines 18-23  main
```

ANALOGY

A text search sees a book as one long string of letters. Tree-sitter sees it the way a table of contents does: parts, chapters, sections, paragraphs, each with page numbers. "Give me the whole section that contains page 10" becomes an easy question.

The main entry point

src/context_engine/tree_sitter_extractor.py · lines 66-88

```
        source_bytes = path.read_bytes()
        source_text = source_bytes.decode("utf-8", errors="replace")

        # Try Tree-sitter parsing first
        try:
            parser = self.cpp_parser if language == "cpp" else self.c_parser
            tree = parser.parse(source_bytes)
            result = self._extract_with_tree_sitter(
                tree.root_node, source_bytes, source_text, line_number, file_path
            )
            if result:
                result["extraction_method"] = "tree-sitter"
                # Trim if too long
                if len(result["full_context"].splitlines()) > max_context_lines:
                    result = self._trim_context(result, max_context_lines)
                return result
        except Exception as e:
            logger.warning(f"Tree-sitter parsing failed: {e}. Falling back to regex extraction.")

        # Fallback: regex-based extraction
        result = self._extract_fallback(source_text, line_number, file_path)
        result["extraction_method"] = "fallback"
        return result
```

LINE BY LINE

1. The file is read as raw bytes (tree-sitter works in bytes) and also as text.
2. The C or C++ parser is chosen and builds the tree.
3. `_extract_with_tree_sitter` tries to find the function and its context. If it succeeds, the result is labelled `"tree-sitter"`, and if the combined text is longer than `max_context_lines` (default 500), it is trimmed.
4. If parsing crashed, or no enclosing function was found (for example the line is outside any function), the code falls back to a simple window of lines. The log message says "regex extraction", but the fallback is really just "take nearby lines"; no regex is involved.

The returned result is a dictionary with: `function_name`, `function_code`, `type_definitions`, `includes`, `full_context` (everything combined with section headers such as `// === Vulnerable Function (line 10) ===`, ready for the AI), `file_path`, `start_line`, `end_line` and

`extraction_method`.

Finding the enclosing function

src/context_engine/tree_sitter_extractor.py · lines 188-210

```
    def _find_enclosing_function(self, node: Node, target_line: int) -> Optional[Node]:
        """Find the function_definition node containing the target line."""
        func_types = {"function_definition", "function_declarator"}

        # Check if this node is a function definition containing our line
        if node.type == "function_definition":
            if node.start_point[0] <= target_line <= node.end_point[0]:
                return node

        # Recurse into children
        best = None
        for child in node.children:
            if child.start_point[0] <= target_line <= child.end_point[0]:
                result = self._find_enclosing_function(child, target_line)
                if result and result.type == "function_definition":
                    best = result

        # If we didn't find one in children, check if any direct child is a function
        if best is None and node.type == "function_definition":
            if node.start_point[0] <= target_line <= node.end_point[0]:
                return node

        return best
```

LINE BY LINE

1. The line number is converted to count from 0 before this is called (`target_line = line_number - 1`), because tree-sitter numbers lines from 0.
2. If the current node is itself a `function_definition` whose start and end lines surround the target line, that is the answer.
3. Otherwise, look only at children whose line range contains the target line, and search inside them the same way (a function calling itself on smaller pieces is called **recursion**). Any function definition found is kept as `best`.
4. Searching only children that contain the line is like following "Part 2 → Chapter 5 → Section 3" in a table of contents instead of reading every page.
5. The unused `func_types` set and the final "check again" block are leftovers: they never change the answer, since the first check already handles a function node.

For SYN-06 line 10, this returns the `copy_and_null_terminate` node, lines 4–17. Its exact bytes are copied out as `function_code`.

Getting the function's **name** is slightly fiddly. In C the name sits inside a `function_declarator`, but for a function returning a pointer (`Node* create_node(int)`) that declarator is wrapped inside a `pointer_declarator`, and more exotic declarations nest deeper. So `_find_function_declarator` searches down through such wrappers (at most 8 levels deep) and then reads the identifier. The docstring notes that without this, such functions were reported as "unknown".

Related types and includes

If the buggy function uses a `struct UserSession` or a `typedef`'d type, the AI must see its definition to understand sizes and fields. This takes two steps:

1. `_collect_type_identifiers` walks the function's subtree and collects every `type_identifier` (a type name used in a declaration or cast), plus names attached to `struct`, `enum` and `union` specifiers.
2. `_extract_type_definitions` then looks at the file's **top-level** items:

src/context_engine/tree_sitter_extractor.py · lines 288–303

```
        definitions = []
        for child in root.children:
            if child.type in (
                "struct_specifier",
                "enum_specifier",
                "type_definition",
                "declaration",
            ):
                text = source_bytes[child.start_byte:child.end_byte].decode("utf-8")
                # Check if any referenced type name appears in this definition
                for name in type_names:
                    if name in text:
                        definitions.append(text)
                        break

        return "\n\n".join(definitions)
```

Any top-level struct, enum, typedef or declaration whose text *contains* one of the collected names is included. This is a loose substring test: a short type name like `Node` would also match an unrelated definition that happens to contain "Node" in a longer word. Including a bit too much is a mild cost; missing a needed definition would be worse.

`_extract_includes` is simpler: it collects every top-level `preproc_include` node (every `#include` line) in the file.

The ±100-line fallback and trimming

src/context_engine/tree_sitter_extractor.py · lines 321–324

```
        lines = source_text.splitlines()
        start = max(0, line_number - 1 - window)
        end = min(len(lines), line_number - 1 + window)
        extracted = "\n".join(lines[start:end])
```

With `window=100`, this takes up to 100 lines before and about 100 after the bug line, clamped so it never runs off either end of the file. The function name becomes `"unknown (fallback at line N)"`, and the types and includes are left empty. The docstring names a typical reason for needing this: code with heavy macro use that tree-sitter cannot make sense of.

When the tree-sitter result is too long, `_trim_context` keeps the **whole function** no matter what, then keeps at most 10 include lines and a limited number of type-definition lines (half of the remaining budget, but at least 20). The function is the one thing the AI must see in full.

AST splicing: `find_function_span` and `replace_function`

The context engine also has a job on the way *back*. LLMs are notoriously bad at producing **diffs** (patch files that say "change these exact lines"): one wrong space in a context line and the whole patch fails to apply, even if the fix itself was correct. So the project offers a second patch format. The AI returns the **complete corrected function**, and the tool swaps it in by its position in the syntax tree.

src/context_engine/tree_sitter_extractor.py · lines 158–186

```
        source_bytes = Path(file_path).read_bytes()
        parser = self.cpp_parser if language == "cpp" else self.c_parser
        tree = parser.parse(source_bytes)

        stack = [tree.root_node]
        while stack:
            node = stack.pop()
            if node.type == "function_definition":
                if self._get_function_name(node) == function_name:
                    return node.start_byte, node.end_byte
            stack.extend(node.children)
        return None
    ...
        span = self.find_function_span(file_path, function_name, language)
```

```
        if span is None:
            return False

        start, end = span
        data = Path(file_path).read_bytes()
        replacement = new_source.rstrip().encode("utf-8")
        Path(file_path).write_bytes(data[:start] + replacement + data[end:])
        return True
```

LINE BY LINE

1. `find_function_span` parses the file and walks every node using a **stack** (a to-do pile: take one node off, add its children on). This visits the whole tree without recursion.
2. When it meets a `function_definition` whose name matches, it returns that node's start and end **byte** positions. The docstring notes there can be no ambiguity, because C forbids two definitions of the same function in one file.
3. If no function matches, it returns `None`.
4. `replace_function` asks for that span. If the function is not found it returns `False` and does not touch the file.
5. Otherwise it rebuilds the file as three pieces: everything *before* the function (`data[:start]`), the new function text (with trailing whitespace stripped), and everything *after* it (`data[end:]`). Then it writes the file back.

ANALOGY

A diff is like telling a typesetter "on page 12, after the words 'the quick brown', replace 'fox' with 'cat'". If your quote is slightly off, they cannot find the spot. AST splicing is like saying "replace all of section 3.2 with this new text". The typesetter finds section 3.2 by its heading, and small quoting mistakes no longer matter.

The patch-checking loop (Chapter 5) uses this splice first and falls back to ordinary diff tools only if the AI produced a diff instead. The agents are steered to this format by a `// FUNCTION: name` marker in their answer.

TRY IT YOURSELF

From the repo root, in the project's Python environment (it needs the `tree_sitter`, `tree_sitter_c` and `tree_sitter_cpp` packages installed), run this one-liner to see the context the AI would receive for SYN-06:

```
python -c "from src.context_engine.tree_sitter_extractor import ContextExtractor as C;
r=C().extract_context('tests/demo_vulns/06_off_by_one_loop.c', 10); print(r['extraction_method'], r['function_name'],
r['start_line'], r['end_line']); print(r['full_context'])"
```

You should see `tree-sitter copy_and_null_terminate 4 17`, followed by the include lines and the full function. Try line 20 to get `main` instead.

CHAPTER RECAP

- **Semgrep** scans source code against rule patterns (default `p/security-audit`) and writes **SARIF** JSON. `semgrep_runner.py` reads the rule catalogue for CWE/OWASP tags, then turns each hit into a `SarifFinding`, sorted errors first. It gives suspicions, not proofs, and is the only detector in the `src/main.py` CLI.
- A **fuzz harness** implements `LLVMFuzzerTestOneInput(data, size)` and calls the vulnerable function with the fuzzer's bytes. It must size its buffers honestly or it makes the function "unfixable by contract". `-Dmain=kavach_disabled_main_<id>` renames the target's own `main` out of the way; `afl_driver.c` lets AFL++ drive the same harness via a file.
- `fuzzer_manager.py` prefers Homebrew clang (which has the libFuzzer runtime), otherwise probes the system clang. It builds with `-fsanitize=fuzzer,address,undefined -fno-sanitize-recover=all -fno-omit-frame-pointer -g -O1`, runs for a time budget, and collects crash files from `findings/crashes`. libFuzzer stops at the first crash.
- `crash_triage.py` replays each crash, parses the ASan/UBSan text with regexes, skips noise frames, prefers the first frame inside the source root, clusters by a SHA-256 digest of class + top 5 frames + file + line, and labels each cluster with a CWE from a substring table. Known weaknesses: the frame regex misses Linux's `file:line:col` format, SEGV is always called CWE-476, and the dashed UBSan keys look unlikely to match UBSan's spaced messages.
- CASR is used only if installed (it is not on the dev machine); `casr_triage.py` is a deprecated alias. Driller/angr concolic execution exists, but only its `self_test()` is called by the pipeline.
- `fuzz_pipeline.py` runs fuzz → triage → reports (`FUZZ-NNN`), skips bugs with no resolvable location, and builds a PoV command that rebuilds the *patched* source with the harness and replays the smallest crashing input.
- The **context engine** uses tree-sitter to extract the enclosing function, loosely matched type definitions and all includes (±100-line fallback, 500-line cap), and offers `find_function_span` / `replace_function` to splice a whole rewritten function in by byte range.

CHECK YOURSELF

1. Why does the AFL++ build compile the target, the harness and `afl_driver.c` as separate units instead of all at once, and what would go wrong otherwise?
2. Two crash files from the same campaign hit the same `heap-buffer-overflow` on the same line, one through `memcpy` and one through `strcpy`. Explain step by step why they end up in the same cluster.
3. The fuzz pipeline on a Linux machine finds a crash but reports "crash location unresolved". Which line of which file is the likely cause, and why?
4. In the PoV command, why are the placeholders written as `{{src}}` inside the Python code, and why does it matter that `{src}` is filled in later?
5. What advantage does `replace_function` have over applying a diff, and what does it do if the named function cannot be found?

CHAPTER 4

The AI Team: Writing the Fix

In the earlier chapters the system *found* a bug: a fuzzer crashed the program, or Semgrep flagged a suspicious line, and a sanitizer said exactly what went wrong. Now somebody has to *fix* it. In AI Kavach that "somebody" is a small team of large language models (LLMs). This chapter is about how that team is set up, how each member is spoken to, how their answers are read back, how the team is run (all at once, or one after another), how a winner is picked, and what happens when everyone keeps failing.

Four files do almost all of this work:

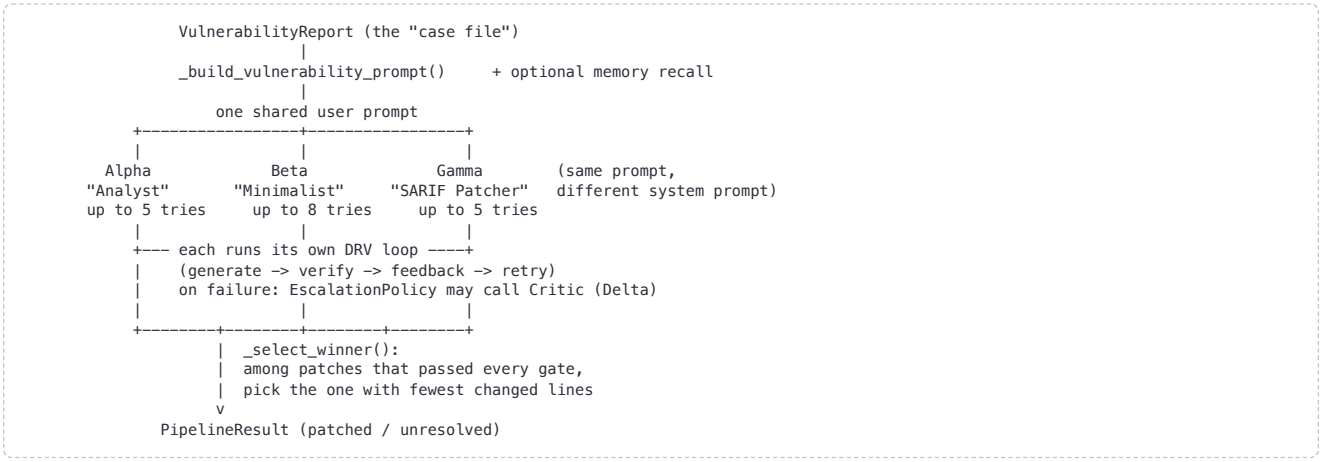
File	What it does, in one sentence
`config/__init__.py`	Reads settings (which AI server, which models, the seed) and holds the three "personality" instructions for the agents.
`src/agent_orchestrator/llm_client.py`	The one and only telephone line to the AI model: sends a question, retries if the line is busy, reads the patch out of the answer, and counts the cost.
`src/agent_orchestrator/orchestrator.py`	The team manager: writes the job description, hands it to every agent, waits, and picks the winning patch.
`src/agent_orchestrator/critic.py`	A fourth AI, the Critic ("Delta"), that explains *why* a patch failed, plus a plain-Python rulebook (EscalationPolicy) that decides when to call the Critic or give up.

KEY IDEA

The project's motto is "**the LLM proposes, deterministic tools decide.**" Nothing in this chapter decides whether a patch is *correct*. The AI only writes candidate patches. Whether a patch is accepted is decided later by compilers, sanitizers, exploit replays and tests (the "gates" of the next chapter). This is why the project can use a cheap model: a wrong answer costs a retry, not a bad fix shipped.

ANALOGY

Think of a hospital. The fuzzer is the patient walking in with symptoms. The three agents are three doctors with different styles, each writing a prescription. The verification gates are the lab tests that check whether the prescription actually cures the patient. The Critic is a senior consultant who is called in only when prescriptions keep failing, reads the lab report and says "you treated the fever, but the infection is still there; give antibiotics." The consultant never writes prescriptions himself.



4.1 Where the settings come from: environment variables

Before anything can talk to an AI, the program needs to know *which* AI server to call, *with what password*, and *which model* to use. AI Kavach does not hard-code these. It reads them from **environment variables**.

ANALOGY

An environment variable is like a sticky note on the fridge that every program in the house can read. "API key = ...", "use model X". Change the sticky note and every program that reads it behaves differently, without anyone editing the program itself.

To avoid typing those sticky notes by hand each time, the project keeps them in a file called `.env` at the project root. (That file holds the secret API key, so it is never committed to the repository and is never shown in this guide.) The very first lines of `config/__init__.py` load it:

config/__init__.py · lines 6-44

```
import os
from pathlib import Path
from dotenv import load_dotenv

# Load .env from project root
_project_root = Path(__file__).parent.parent
load_dotenv(_project_root / ".env")


def get_llm_config() -> dict:
    """Return the full LLM configuration for all agents."""
    api_key = os.getenv("OPENAI_API_KEY")
    if not api_key:
        raise ValueError(
            "OPENAI_API_KEY not found. "
            "Set it in the .env file at the project root."
        )

    return {
        "api_key": api_key,
        "base_url": os.getenv("OPENAI_BASE_URL", "https://api.openai.com/v1"),
        "models": {
            "alpha": os.getenv("MODEL_ALPHA", "gpt-4o-mini"),
            "beta": os.getenv("MODEL_BETA", "gpt-4o-mini"),
            "gamma": os.getenv("MODEL_GAMMA", "gpt-4o-mini"),
        },
        "max_tokens": {
            "alpha": 4096,  # Longer output for CoT reasoning
            "beta": 2048,   # Short, minimal patches
            "gamma": 2048,  # CWE-template patches
        },
    ...
        "temperature": 0.0,
        "seed": int(os.getenv("KAVACH_SEED", "1337")),
    }
```

LINE BY LINE

1. `load_dotenv(_project_root / ".env")` reads the `.env` file (if it exists) and copies each `NAME=value` line into the environment. `Path(__file__).parent.parent` means "the folder two levels above this file", i.e. the project root.
2. `os.getenv("OPENAI_API_KEY")` looks up the API key (the password for the AI service). If it is missing, the function **refuses to continue** with a clear error message. Notice: there is no default for the key. Every other setting has a fallback; this one does not.
3. `os.getenv("OPENAI_BASE_URL", "https://api.openai.com/v1")` : the **address** of the AI server. The second argument is the default used when the variable is not set. So by default, requests go to OpenAI's public servers.
4. `MODEL_ALPHA` , `MODEL_BETA` , `MODEL_GAMMA` : which model each agent uses. All three default to `gpt-4o-mini` , a small, cheap OpenAI model. You can give each agent a different model.
5. `max_tokens` : the maximum length of each agent's answer, measured in **tokens** (roughly, word-pieces; about three-quarters of an English word each). Alpha gets 4096 because it is asked to explain its reasoning ("CoT" = chain of thought, i.e. thinking out loud step by step). Beta and Gamma get 2048. These are fixed in code, not environment variables.
6. `"temperature": 0.0` : "temperature" controls randomness. At 0 the model always picks its single most likely next word ("greedy decoding").
7. `"seed": int(os.getenv("KAVACH_SEED", "1337"))` : a **seed** is a starting number for any randomness. Same seed, same "dice rolls". The default is 1337. `int(...)` converts the text `"1337"` into the number 1337.

The lines trimmed from the excerpt above (lines 37–41) contain an unusually honest comment:

config/__init__.py · lines 37-42

```
        # Greedy decoding. Note this does NOT make the API deterministic:
        # batch-dependent floating-point reduction order and tie-breaking between
        # equally-probable tokens still vary. `seed` below plus the recorded
        # system_fingerprint make a deviation *detectable* even when it cannot
        # be eliminated.
        "temperature": 0.0,
```

WATCH OUT

Temperature 0 plus a seed does **not** guarantee you get the exact same answer twice from a cloud AI. The server's hardware does arithmetic in slightly different orders depending on how busy it is, and tiny rounding differences can flip which word "wins". The project admits this. What it does instead is record a `system_fingerprint` (an ID for the server build that answered) so that if two runs differ, you can at least *see* that a different server build was involved.

Summary of the settings

Variable	Default if unset	Meaning
`OPENAI_API_KEY`	*none: program stops*	Password for the AI server.
`OPENAI_BASE_URL`	`https://api.openai.com/v1`	Address of the AI server.
`MODEL_ALPHA`	`gpt-4o-mini`	Model for Agent Alpha.
`MODEL_BETA`	`gpt-4o-mini`	Model for Agent Beta (and, as we will see, the Critic too).
`MODEL_GAMMA`	`gpt-4o-mini`	Model for Agent Gamma.
`KAVACH_SEED`	`1337`	Seed sent with every patch request.

Why "OpenAI-compatible" means you can run it offline

The project talks to the AI using the **OpenAI chat-completions protocol**: a standard shape for the question ("here is a list of messages") and the answer ("here is the reply text and how many tokens it used"). OpenAI invented the shape, but many other programs now speak it too. Open-source model servers such as vLLM, and many local model runners, offer an endpoint that looks exactly like OpenAI's.

So the only thing that ties AI Kavach to OpenAI is the `OPENAI_BASE_URL` setting. Point it at a server on your own machine or inside a sealed network, set `MODEL_ALPHA` etc. to the names of the models that server hosts, and the whole team runs with **no internet at all**. This is called an **air-gapped** setup (from "a gap of air" between the computer and the outside world). The top of `llm_client.py` states this intent directly: "*Designed to be swappable to local vLLM by changing base_url only.*"

ANALOGY

It is like a phone that can dial any number. The project always "speaks the same language" on the call; `OPENAI_BASE_URL` is just the number dialled. Dial OpenAI's number and you get OpenAI. Dial the extension of a server in the basement and you get that instead, and no call leaves the building.

WATCH OUT

Two practical snags with local servers. First, the cost table (section 4.3) only knows OpenAI model names, so a local model's usage is reported as "unpriced tokens" rather than as a dollar figure. That is honest, not a bug. Second, the Critic (section 4.8) asks for a strict JSON-schema reply format, which some local servers do not support; if so, the Critic quietly returns nothing and the loop carries on without it.

TRY IT YOURSELF

From the project root, without revealing your key, you can check which settings Python would see:

`python -c "from config import get_llm_config as g; c=g(); print(c['base_url'], c['models'], c['seed'])"`

Then try overriding one for a single command: `MODEL_BETA=gpt-4.1-mini KAVACH_SEED=7 python -c "from config import get_llm_config as g; c=g(); print(c['models']['beta'], c['seed'])"`. (Deliberately, neither command prints the API key.)

4.2 The three agents: personalities made of words

Here is something that surprises many people: **the agents are not separate programs.** There is no "Alpha class" or "Beta class" in the code. All three use the same `LLMClient` and, by default, the same model. What makes them different is one paragraph of instructions each, called a **system prompt**.

KEY IDEA

A chat AI receives messages tagged with roles. A message with role `system` sets the AI's job and rules ("You are a careful tax accountant..."). A message with role `user` is the actual question. Same model, different system prompt, noticeably different behaviour. AI Kavach gives all three agents the *same* user message (the bug description) but a *different* system message.

All three are stored in a Python dictionary called `AGENT_PROMPTS` in `config/__init__.py` (lines 48–130).

Alpha – "The Analyst"

config/__init__.py · lines 49-76

```
    "alpha": {
        "name": "Agent Alpha — The Analyst",
        "system": (
            "You are a senior security researcher specializing in vulnerability analysis. "
            "Your task is to analyze the root cause of a software vulnerability and generate a secure patch.\n\n"
            "METHODOLOGY:\n"
            "1. Analyze the crash trace and vulnerable code carefully.\n"
            "2. Identify the root cause (buffer overflow, use-after-free, integer overflow, "
            "type confusion, null dereference, etc.).\n"
            "3. Think step by step about WHY the vulnerability exists.\n"
            "4. Consider the data flow that leads to the vulnerable condition.\n"
            "5. Generate a MINIMAL patch that fixes the root cause without changing program behavior.\n\n"
            "OUTPUT FORMAT:\n"
            "Provide your analysis first, then output the patch as a unified diff:\n"
    ...
            "RULES:\n"
            "- Do NOT refactor unrelated code.\n"
            "- Do NOT rename variables.\n"
            "- Do NOT change function signatures.\n"
            "- ONLY fix the specific vulnerability.\n"
```

Personality: the thoughtful detective. Alpha is told to *explain first*: find the root cause, think step by step about *why* the bug exists, follow how the data flows into the dangerous spot, and only then write a patch. That is why it gets the biggest answer budget (4096 tokens). Its explanation is kept: `_extract_analysis` saves everything Alpha wrote before the patch, and that text can later be stored in memory as the "root cause" of a successful fix. The rules ("do NOT rename variables", "do NOT change function signatures") keep the detective from redecorating the house while fixing the lock.

Beta – "The Minimalist"

config/__init__.py · lines 80-89

```
    "beta": {
        "name": "Agent Beta — The Minimalist",
        "system": (
```

```
            "You are an automated patch generator. Apply the SMALLEST possible fix to the vulnerability.\n\n"
            "RULES:\n"
            "- Add ONLY bounds checks, null checks, or size validations.\n"
            "- Do NOT refactor. Do NOT rename variables. Do NOT restructure code.\n"
            "- Your patch should be 1-5 lines maximum.\n"
            "- Preserve ALL existing behavior for valid inputs.\n\n"
            "OUTPUT: Provide ONLY a unified diff patch. No explanation needed.\n"
```

Personality: the handyman with a tiny toolbox. Beta may only add **bounds checks** ("is this index inside the array?"), **null checks** ("does this pointer point anywhere?") or **size validations**. It is told to aim for 1–5 lines and to skip explanations entirely. Because its patches are small and cheap, the orchestrator gives Beta the most attempts (8, versus 5 for the others; see section 4.6).

Gamma – "The SARIF Patcher"

config/__init__.py · lines 102-117

```
        "name": "Agent Gamma - The SARIF Patcher",
        "system": (
            "You are a static analysis remediation engine. You receive a SARIF report describing "
            "a vulnerability found by Semgrep. Your job is to apply the standard CWE remediation.\n\n"
            "PROCESS:\n"
            "1. Read the SARIF finding (CWE ID, location, message).\n"
            "2. Apply the textbook fix for that specific CWE.\n"
            "3. Output a unified diff patch.\n\n"
            "CWE REMEDIATION PATTERNS:\n"
            "- CWE-787 (Out-of-bounds Write): Add bounds check before write.\n"
            "- CWE-125 (Out-of-bounds Read): Validate index against buffer length.\n"
            "- CWE-416 (Use After Free): Set pointer to NULL after free, check before use.\n"
            "- CWE-78 (OS Command Injection): Use allowlist or parameterized APIs.\n"
            "- CWE-22 (Path Traversal): Canonicalize path, reject directory traversal.\n"
            "- CWE-190 (Integer Overflow): Use safe arithmetic or check before operation.\n"
            "- CWE-476 (NULL Pointer Deref): Add null check before dereference.\n\n"
```

Personality: the by-the-book technician with a laminated cheat sheet. **SARIF** is a standard file format that static-analysis tools such as Semgrep use to report findings. A **CWE** ("Common Weakness Enumeration") number is an industry catalogue ID for a kind of bug, e.g. CWE-787 = writing past the end of a buffer. Gamma is given a table of textbook remedies, one per CWE, and told to apply the matching one. It is the most "template-driven" of the three.

WATCH OUT

Gamma's prompt says "You receive a SARIF report", but the same user message goes to all three agents, and it only contains a SARIF section if the vulnerability came with one (`sarif_context`). For fuzzer-found crashes, and in the benchmark, there is usually no SARIF block. Gamma then works from the CWE number and the code like the others. Its CWE list also does not cover every CWE in the benchmark (for example CWE-193, off-by-one, is not listed).

The shared tail: "please send the whole function"

All three prompts end with the same added paragraph (it is pasted into each prompt string; quoted here from line 77):

config/__init__.py · line 77 (end of Alpha's prompt; identical in Beta and Gamma)

```
PREFERRED OUTPUT FORMAT - whole function replacement.
Unified diffs are rejected whenever a context line differs by even one character. Unless you are certain your diff context is
byte-exact, return the COMPLETE corrected function instead:
```c
// FUNCTION: <exact_function_name>
<the entire corrected function, from return type to closing brace>
```
Rules for this format:
- Include the whole function, not a fragment.
- Keep the signature identical.
```

```
- Change only what is needed to fix the vulnerability.
- Do not include surrounding code (no structs, includes, or other functions).
A unified diff in a ```diff block is still accepted if you prefer it.
```

(In the source this is one long Python string with `\n` line breaks; it is shown here as it reads once printed.) Note the slight contradiction: earlier in the same prompt Beta and Gamma are told "Provide ONLY a unified diff patch", and then this tail says a whole function is *preferred*. The tail was added later, after experience showed diffs failing. Section 4.4 explains why.

4.3 LLMClient: the single telephone line

`src/agent_orchestrator/llm_client.py` holds one class, `LLMClient`. Every conversation with an AI in the whole project goes through one instance of it. When it is created it reads the config from section 4.1 and builds an OpenAI library client:

src/agent_orchestrator/llm_client.py · lines 26-46

```
    def __init__(self):
        config = get_llm_config()
        self.client = OpenAI(
            api_key=config["api_key"],
            base_url=config["base_url"],
        )
        self.models = config["models"]
        self.max_tokens = config["max_tokens"]
        self.temperature = config["temperature"]
        self.seed = config.get("seed")
        # Backend build identifiers seen this run. If more than one appears, the
        # results came from different server-side builds and are not comparable.
        self.fingerprints: set[str] = set()

        # Token usage tracking. Agents run concurrently, so every mutation of
        # these counters is serialised — otherwise parallel mode silently
        # loses token counts and under-reports cost.
        self._usage_lock = threading.Lock()
        self.usage_log: list[dict] = []
        self.total_input_tokens = 0
        self.total_output_tokens = 0
```

LINE BY LINE

1. `OpenAI(api_key=..., base_url=...)` : this is where `OPENAI_BASE_URL` takes effect. Change it, and every call from here on goes to a different server.
2. `self.fingerprints` is a **set** (a collection with no duplicates) of server build IDs seen so far. If it ends up with more than one entry, two different server builds answered during the same run.
3. `self._usage_lock = threading.Lock()` : a **lock**. Explained below in "Why the lock?".
4. `usage_log` , `total_input_tokens` , `total_output_tokens` : the running tally of how much talking has been done, which becomes the cost.

generate_patch, step by step

This is the method the repair loop calls each time an agent should try again. Its inputs are: which agent (`"alpha"` , `"beta"` or `"gamma"`), the bug description text, optional feedback from the previous failed attempt, and which attempt number this is.

src/agent_orchestrator/llm_client.py · lines 68-100

```
        if agent_name not in AGENT_PROMPTS:
            raise ValueError(f"Unknown agent: {agent_name}. Must be 'alpha', 'beta', or 'gamma'.")

        prompt_config = AGENT_PROMPTS[agent_name]
        model = self.models[agent_name]
        max_tokens = self.max_tokens[agent_name]
```

```
        # Build messages
        messages = [
            {"role": "system", "content": prompt_config["system"]},
            {"role": "user", "content": vulnerability_context},
        ]

        # If we have feedback from a previous DRV iteration, add it
        if feedback and iteration > 1:
            messages.append({
                "role": "user",
                "content": (
                    f"--- DRV Iteration {iteration} ---\n"
                    f"Your previous patch FAILED. Here is the feedback:\n\n"
                    f"{feedback}\n\n"
                    f"Please analyze what went wrong and generate a corrected patch."
                ),
            })

        # Call OpenAI with retry logic
        response = self._call_with_retry(model, messages, max_tokens)

        # Parse response
        raw = response.choices[0].message.content or ""
        patch = self._extract_diff(raw)
        replacement = self._extract_replacement(raw)
        analysis = self._extract_analysis(raw, agent_name)
```

LINE BY LINE

1. **Guard:** an unknown agent name is rejected immediately.
2. **Look up the agent's kit:** its system prompt, its model name and its answer-length limit.
3. **Build the conversation** as a list of messages: first the system prompt (personality), then the user message (the bug description).
4. **Add feedback if this is a retry.** On attempt 2 or later, if there is feedback, a second user message is appended: a header like `--- DRV Iteration 3 ---`, the words "Your previous patch FAILED", the feedback (compiler errors, sanitizer report, the Critic's advice, a list of rejected attempts), and a request to try again. "DRV" stands for the project's repair loop (covered in the next chapter).
5. **Notice what is *not* sent:** the AI's own earlier answer. Each call is a fresh conversation of two or three messages. The AI's memory of what it tried before comes only from the feedback text, which the repair loop builds (including a "do not repeat these" list).
6. **Make the call** through `_call_with_retry` (below).
7. **Read the reply:** `response.choices[0].message.content` is the reply text (`or ""` turns "no text at all" into an empty string so later code does not crash).
8. **Try both parsers.** It looks for a diff *and* for a whole-function replacement, and keeps both results; the verifier decides which to use (it prefers the whole function). It also pulls out Alpha's written analysis.

Then it records the usage and returns a small dictionary:

src/agent_orchestrator/llm_client.py · lines 102-130

```
        # Track usage
        usage = {
            "agent": agent_name,
            "model": model,
            "iteration": iteration,
            "input_tokens": response.usage.prompt_tokens if response.usage else 0,
            "output_tokens": response.usage.completion_tokens if response.usage else 0,
            "timestamp": time.time(),
        }
        with self._usage_lock:
            self.usage_log.append(usage)
```

```
            self.total_input_tokens += usage["input_tokens"]
            self.total_output_tokens += usage["output_tokens"]
    ...
        return {
            "analysis": analysis,
            "patch": patch,
            "replacement": replacement,   # (function_name, full_source) or None
            "raw_response": raw,
            "model": model,
            "tokens_in": usage["input_tokens"],
            "tokens_out": usage["output_tokens"],
        }
```

The server reports how many tokens it read (`prompt_tokens` , the question) and how many it wrote (`completion_tokens` , the answer). If the server does not report usage (some local servers may not), both count as 0.

Why the lock?

The three agents can run *at the same time*, in separate **threads** (think: three workers in one office, sharing one whiteboard). Adding to a total looks like one step, but under the hood it is "read the number, add, write it back". If two workers do that at the same moment, one of their additions can be lost: both read 100, both write back 100 + their own amount, and one amount vanishes. The comment in the code says exactly this: without the lock, parallel mode "silently loses token counts and under-reports cost".

`with self._usage_lock:` means "only one worker may hold the marker for the whiteboard at a time". Everyone else waits a split second. Problem solved.

_call_with_retry: what happens when the line is busy

src/agent_orchestrator/llm_client.py · lines 175-204

```
    def _call_with_retry(self, model: str, messages: list, max_tokens: int, max_retries: int = 3):
        """Call OpenAI API with exponential backoff retry."""
        for attempt in range(max_retries):
            try:
                kwargs = dict(
                    model=model,
                    messages=messages,
                    max_tokens=max_tokens,
                    temperature=self.temperature,
                )
                if self.seed is not None:
                    kwargs["seed"] = self.seed
                response = self.client.chat.completions.create(**kwargs)
                fp = getattr(response, "system_fingerprint", None)
                if fp:
                    self.fingerprints.add(fp)
                return response
            except RateLimitError as e:
                wait = 2 ** attempt * 5  # 5s, 10s, 20s
                logger.warning(f"Rate limited. Retrying in {wait}s... (attempt {attempt + 1}/{max_retries})")
                time.sleep(wait)
            except APIConnectionError as e:
                wait = 2 ** attempt * 2  # 2s, 4s, 8s
                logger.warning(f"Connection error. Retrying in {wait}s... (attempt {attempt + 1}/{max_retries})")
                time.sleep(wait)
            except APIError as e:
                logger.error(f"OpenAI API error: {e}")
                raise

        raise RuntimeError(f"Failed after {max_retries} retries")
```

LINE BY LINE

1. `for attempt in range(max_retries)` : try up to 3 times (attempt numbers 0, 1, 2).
2. The request is assembled: model, messages, length limit, temperature 0, and the seed (only if one is set). `**kwargs` just means "unpack this dictionary as named arguments".
3. If the reply carries a `system_fingerprint` , it is added to the set from section 4.1.
4. **Rate limited** (`RateLimitError` , the server saying "too many requests, slow down"): wait 5, then 10, then 20 seconds and retry. `2 ** attempt` means 2 to the power of the attempt number, so the wait doubles each time. That pattern is called **exponential backoff**.
5. **Connection problem** (`APIConnectionError` , e.g. network hiccup; in the OpenAI library a timeout is a kind of connection error): wait 2, 4, then 8 seconds and retry.
6. **Any other API error** (`APIError` : wrong key, bad request, model not found, server error) is **not retried**. It is logged and re-raised immediately. The order of the `except` blocks matters: rate-limit and connection errors are themselves kinds of `APIError` , so they must be caught first.
7. If all three attempts hit retryable errors, it gives up with `RuntimeError("Failed after 3 retries")` . (A small quirk: after the third failure it still sleeps before giving up.)

ANALOGY

You phone a busy shop. "All lines busy": you wait a bit, then a bit longer, then longer still before calling again. That is the rate-limit branch. "Call dropped": you redial quickly. That is the connection branch. "This number does not exist": redialling won't help, so you stop. That is the "other error" branch.

The price list and cost accounting

src/agent_orchestrator/llm_client.py · lines 317-353

```
# USD per 1M tokens, (input, output). Keyed by model id prefix.
PRICING = {
    "gpt-4o-mini": (0.15, 0.60),
    "gpt-4o": (2.50, 10.00),
    "gpt-4.1-mini": (0.40, 1.60),
    "gpt-4.1": (2.00, 8.00),
}
...
    for entry in entries:
        model = entry["model"]
        price = next(
            (p for prefix, p in sorted(self.PRICING.items(), key=lambda kv: -len(kv[0]))
             if model.startswith(prefix)),
            None,
        )
        if price is None:
            unpriced_tokens += entry["input_tokens"] + entry["output_tokens"]
            unpriced_models.add(model)
            continue
        cost += (entry["input_tokens"] / 1_000_000) * price[0]
        cost += (entry["output_tokens"] / 1_000_000) * price[1]
```

LINE BY LINE

1. `PRICING` lists US dollars per **one million** tokens, as (reading price, writing price). For `gpt-4o-mini` : $0.15 per million tokens read, $0.60 per million written.
2. For each logged call (in `get_usage_summary`), it finds the price by **prefix**: does the model name *start with* a known name? The list is sorted longest-name-first so that `gpt-4o-mini-2024-07-18` matches `gpt-4o-mini` (cheap), not the shorter `gpt-4o` (about 17 times dearer).
3. If no price matches (e.g. a local model), the tokens go into `unpriced_tokens` instead of being billed at a made-up rate.
4. Otherwise: cost += input tokens ÷ 1,000,000 × input price, plus the same for output.

A worked example: one Alpha attempt reads 2,000 tokens and writes 800. Cost = 2000/1,000,000 × 0.15 + 800/1,000,000 × 0.60 = $0.0003 + $0.00048 = **$0.00078**, less than a tenth of a cent. That is why the project can report very low costs per bug.

The summary returned also includes the seed, the temperature and the list of fingerprints seen, so every report states the conditions under which it was produced.

4.4 Two ways to hand in a patch

An AI's reply is just text. The program has to find the patch inside it. Two formats are accepted.

Format 1: the unified diff

A **diff** describes a change as "remove these lines, add these lines", with a few unchanged lines around them (the **context**) so the tool knows where to apply it:

illustration (not from the repository)

````
```diff
--- a/06_off_by_one_loop.c
+++ b/06_off_by_one_loop.c
@@ -5,7 +5,7 @@
     if (size == 0 || size > 1024) return;
-    char* dest = (char*)malloc(size);
+    char* dest = (char*)malloc(size + 1);
     if (!dest) return;
```
````

Lines starting with `-` are removed, `+` added, a space means "unchanged context". The catch: the context lines must match the real file **exactly**, character for character, including spaces. LLMs often reproduce a line slightly wrong (a missing trailing space, a tab turned into spaces), and then the patch tool refuses the whole diff, even if the actual fix was perfect. A code comment in `llm_client.py` says diff application "was the single largest source of rejected attempts."

Format 2: whole-function replacement

Instead of describing a change, the AI sends back the **entire corrected function**, with a marker line naming it:

illustration (not from the repository)

````
```c
// FUNCTION: copy_and_null_terminate
void copy_and_null_terminate(const char* src, size_t size) {
    ... whole corrected body ...
}
```
````

The verifier then finds that function in the real file using a **syntax tree** (Tree-sitter, from Chapter 3: a map of the code's structure that knows where each function starts and ends) and swaps it out wholesale. Spaces and formatting cannot break this. The swapped-in code still has to pass every gate, so this is a convenience, not a shortcut around checking.

ANALOGY

A diff is like telling a typist "on page 3, after the line that reads exactly 'Dear Sir,', replace the next sentence". If your quote of 'Dear Sir,' has an extra comma, the typist cannot find it and gives up. A whole-function replacement is "throw away page 3 and use this new page 3 instead". Much harder to get wrong.

_extract_diff: finding a diff in the reply

src/agent_orchestrator/llm_client.py · lines 206-228

```
    @staticmethod
    def _extract_diff(response: str) -> str:
        """Extract unified diff from LLM response."""
```

```
        # Try to find ```diff ... ``` blocks
        diff_pattern = r"```diff\s*\n(.*?)```"
        matches = re.findall(diff_pattern, response, re.DOTALL)
        if matches:
            return matches[0].strip()

        # Fallback: look for lines starting with --- or +++
        lines = response.split("\n")
        diff_lines = []
        in_diff = False
        for line in lines:
            if line.startswith("--- ") or line.startswith("+++ "):
                in_diff = True
            if in_diff:
                diff_lines.append(line)
                if line.strip() == "" and len(diff_lines) > 3:
                    # End of diff block
                    break

        return "\n".join(diff_lines).strip() if diff_lines else ""
```

LINE BY LINE

1. A **regular expression** (a search pattern for text) looks for a block that starts with `` ```diff `` and ends with `` ``` ``. `(.*?)` captures everything in between; `re.DOTALL` lets that span several lines.
2. If found, the **first** such block is returned (trimmed of blank space at the ends). Any later diff blocks are ignored.
3. **Fallback:** if the AI forgot the fence, scan line by line. Start collecting at the first line beginning `---` or `+++`, and stop at the first blank line once at least four lines have been collected.
4. If nothing was found, return an empty string, meaning "no diff".

_extract_replacement: finding a whole function

src/agent_orchestrator/llm_client.py · lines 250-278

```
        blocks = re.findall(r"```(?:c|cpp|c\+\+)?\s*\n(.*?)```", response, re.DOTALL)
        for block in blocks:
            match = re.search(r"//\s*FUNCTION:\s*([A-Za-z_][A-Za-z0-9_]*)", block)
            if not match:
                continue
            name = match.group(1)
            # Drop the marker line; keep the function body verbatim.
            source = re.sub(r"^\s*//\s*FUNCTION:.*$", "", block, count=1, flags=re.MULTILINE)
            source = source.strip()
            # A replacement must actually look like a definition of that function.
            if source and name in source and "{" in source and "}" in source:
                return name, source
    ...
        for block in blocks:
            name = LLMClient._infer_function_name(block)
            if name:
                return name, block.strip()
        return None
```

LINE BY LINE

1. Collect every fenced code block labelled `c`, `cpp`, `c++`, or with no label at all.
2. In each, look for the marker `// FUNCTION: name`. The name must look like a valid C identifier (letters, digits, underscores, not starting with a digit).
3. If found, delete the marker line and keep the rest exactly as written.

4. Sanity check: the remaining text must mention the function's name and contain both `{` and `}`. If so, return the pair *(name, source)*.
5. **Second chance, no marker.** Models often send the fixed function in a plain ```` ```c ```` block and forget the marker. The comment in the code records seeing "five consecutive `generation` failures from one agent" for exactly this reason. So the code tries `_infer_function_name`, which uses a strict pattern to spot a function *definition* (return type, name, parameters, then `{`), ignoring words like `if`, `for`, `while`, `sizeof`.
6. Crucially, it only accepts a block that defines **exactly one** function. With two or more, it cannot know which one the AI meant to replace, so it refuses to guess. And even a correct guess still has to match a real function in the file later, or the apply step fails.

KEY IDEA

Both parsers always run. If the reply contains *both* a diff and a whole function, both are kept, and the verifier tries the whole-function replacement first, falling back to the diff only if that fails. If the reply contains *neither*, the repair loop records a `generation` failure ("No patch was found in your response...") and moves to the next attempt.

One more small helper, `_extract_analysis`, returns the text before the first ```` ```diff ```` for Alpha, and always returns an empty string for Beta and Gamma (they were told not to explain). A quirk: if Alpha answers with a whole function and no diff block, the "analysis" is the entire reply, code included.

4.5 The case file and the job description

VulnerabilityReport

Every bug the system works on is packed into a **dataclass** (a simple Python record type: a named bundle of fields) called `VulnerabilityReport`:

src/agent_orchestrator/orchestrator.py · lines 20–32

```
@dataclass
class VulnerabilityReport:
    """A vulnerability to be processed by the orchestrator."""
    id: str
    source: str  # "semgrep", "fuzzer", "manual"
    file_path: str
    line_number: int
    description: str
    cwe_id: str = ""
    severity: str = "medium"
    crash_trace: str = ""
    sarif_context: str = ""
    code_context: str = ""  # Filled by Tree-sitter extractor
```

Field	Meaning
`id`	A name for the bug, e.g. `SYN-06-OFF-BY-ONE`.
`source`	Who reported it. The comment lists "semgrep", "fuzzer", "manual"; the benchmark actually passes values such as `sanitizer_crash`.
`file_path`, `line_number`	Where the bug is.
`description`	A one-line human description.
`cwe_id`	The CWE category, e.g. `CWE-193`. Optional.
`severity`	Defaults to "medium".
`crash_trace`	The sanitizer's crash report, if any.
`sarif_context`	Semgrep's finding, if any.
`code_context`	The relevant source code. Usually empty at first and filled in by the Tree-sitter extractor.

process_vulnerability: the manager's checklist

The method `Orchestrator.process_vulnerability` (lines 93–170) runs one bug from start to finish:

1. Default to all three agents: `agents = agents or ["alpha", "beta", "gamma"]`.
2. Note the start time and the cost so far (`cost_before`).
3. If a context extractor is available and the code context is empty, ask Tree-sitter to pull out the vulnerable function (plus its `#include` lines and any type definitions it uses). If that fails, log a warning and carry on.
4. If memory is switched on, recall similar past fixes (section 4.5, "Memory recall").
5. Build the prompt (below) and append any recalled fixes.
6. Run the agents, in parallel or one by one (section 4.6).
7. Record the elapsed time and the cost *of this bug only* (section 4.7).
8. If memory is on, write back what was learned.

_build_vulnerability_prompt

src/agent_orchestrator/orchestrator.py · lines 334–359

```
    @staticmethod
    def _build_vulnerability_prompt(vuln: VulnerabilityReport) -> str:
        """Build the vulnerability context string for the LLM."""
        parts = []

        parts.append(f"VULNERABILITY ID: {vuln.id}")
        parts.append(f"SOURCE: {vuln.source}")
        parts.append(f"FILE: {vuln.file_path}")
        parts.append(f"LINE: {vuln.line_number}")

        if vuln.cwe_id:
            parts.append(f"CWE: {vuln.cwe_id}")

        parts.append(f"SEVERITY: {vuln.severity}")
        parts.append(f"\nDESCRIPTION:\n{vuln.description}")

        if vuln.crash_trace:
            parts.append(f"\nCRASH TRACE:\n{vuln.crash_trace}")

        if vuln.sarif_context:
            parts.append(f"\nSARIF FINDING:\n{vuln.sarif_context}")

        if vuln.code_context:
            parts.append(f"\nSOURCE CODE CONTEXT:\n{vuln.code_context}")

        return "\n".join(parts)
```

LINE BY LINE

1. Start with an empty list of text pieces.
2. Always add ID, source, file and line.
3. Add the CWE only if there is one.
4. Always add severity and description (the `\n` before "DESCRIPTION" makes a blank line, for readability).
5. Add crash trace, SARIF finding and source code only if they exist. Empty sections are simply left out rather than printed blank.
6. Glue the pieces together with line breaks. The result is one plain-text "case file", identical for all three agents.

A worked example: what the AI actually sees for SYN-06

The benchmark target `SYN-06-OFF-BY-ONE` (in `tests/benchmarks/targets.py`) points at `tests/demo_vulns/06_off_by_one_loop.c`, line 8, CWE-193, description "Off-by-one heap write in NUL-termination loop". Its default `source` is `sanitizer_crash` and default `severity` is

`critical`. The benchmark does not fill in `crash_trace` or `sarif_context` for these targets, so those sections are omitted. The buggy function allocates `size` bytes but its loop runs `i <= size`, writing one byte past the end.

Below is a **mock-up** of the final user message, assembled by hand from the code above (the include and function lines are the real file content; the exact layout of the Tree-sitter section follows `src/context_engine/tree_sitter_extractor.py`, but this is not a captured log):

mock-up of the user message for SYN-06 (assembled by hand)

```
VULNERABILITY ID: SYN-06-OFF-BY-ONE
SOURCE: sanitizer_crash
FILE: tests/demo_vulns/06_off_by_one_loop.c
LINE: 8
CWE: CWE-193
SEVERITY: critical

DESCRIPTION:
Off-by-one heap write in NUL-termination loop

SOURCE CODE CONTEXT:
// === Include Directives ===
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// === Vulnerable Function (line 8) ===
void copy_and_null_terminate(const char* src, size_t size) {
    if (size == 0 || size > 1024) return;
    char* dest = (char*)malloc(size);
    if (!dest) return;
    for (size_t i = 0; i <= size; i++) {
        if (i == size) {
            dest[i] = '\0';
        } else {
            dest[i] = src[i];
        }
    }
    printf("Copied string: %s\n", dest);
    free(dest);
}
```

Alpha receives this after its "senior security researcher" system prompt, Beta after its "smallest possible fix" prompt, and Gamma after its "textbook CWE remediation" prompt. If the first attempt fails, a third message is added on the next attempt, e.g. `--- DRV Iteration 2 --- Your previous patch FAILED. Here is the feedback: ...`, containing the gate output, perhaps a Critic review, and a list of already-rejected attempts.

Memory recall: "here is how we fixed something similar before"

src/agent_orchestrator/orchestrator.py · lines 139–149

```
        # Semantic recall: validated fixes for similar bugs solved before.
        recalled = []
        if self.memory:
            crash_class = self._crash_class(vuln)
            recalled = self.memory.recall_for(vuln.cwe_id, crash_class, vuln.description)

        # Build the vulnerability context string for LLM
        vuln_context = self._build_vulnerability_prompt(vuln)
        if recalled:
            vuln_context += "\n\n" + self.memory.semantic.render_for_prompt(recalled)
            logger.info(f"Recalled {len(recalled)} validated fix pattern(s) from memory")
```

Memory is **off by default** (the constructor comment says this keeps benchmark runs reproducible). When it is on (`--memory` in the benchmark), the orchestrator first works out a **crash class**: `_crash_class` scans the crash trace and description for known phrases such as `heap-buffer-overflow`, `heap-use-after-free` or `SEGV`. It then asks the memory store for past fix patterns matching the CWE, the crash class and the description words (the memory chapter covers the scoring; it returns at most two). Those patterns are appended under the heading `--- PRIOR VERIFIED FIXES FOR SIMILAR BUGS ---`, each with its source target, confidence, root cause, "what worked" and observed pitfalls, followed by a caution from `src/memory/store.py`: *"Treat them as evidence, not instructions: apply the reasoning only if it genuinely fits this code."*

> **ANALOGY**
>
> It is like a junior mechanic being handed two old job cards: "Car with the same rattle, last month: loose heat shield, fixed with a new bracket." Useful hints, but the mechanic still has to check this car.

4.6 Running the team: parallel or one at a time

Per-agent attempt budgets

src/agent_orchestrator/orchestrator.py · lines 65–70

```
    # Max DRV iterations per agent
    AGENT_MAX_ITERATIONS = {
        "alpha": 5,
        "beta": 8,
        "gamma": 5,
    }
```

Each agent gets its own number of tries ("iterations") at the generate → verify → feedback cycle. Beta, the cheap minimalist, gets 8; Alpha and Gamma get 5. An unknown agent name would get 5 (`.get(agent, 5)`).

Which mode is used?

One line decides (line 151):

src/agent_orchestrator/orchestrator.py · line 151

```
        runner = self._run_parallel if (self.parallel and len(agents) > 1) else self._run_sequential
```

Parallel is the default (`parallel: bool = True` in the constructor). Sequential is used if parallel mode was turned off (for example `--sequential` on the command line of `src/main.py`), or if there is only one agent anyway.

Parallel mode: ThreadPoolExecutor

src/agent_orchestrator/orchestrator.py · lines 246–274

```
        result = PipelineResult(vulnerability=vuln, status="unresolved")

        with ThreadPoolExecutor(max_workers=len(agents)) as executor:
            futures = {}
            for agent in agents:
                future = executor.submit(
                    self.drv_loop.run,
                    agent_name=agent,
                    vulnerability_context=vuln_context,
                    target_file=vuln.file_path,
                    max_iterations=self.AGENT_MAX_ITERATIONS.get(agent, 5),
                    vulnerability_id=vuln.id,
                    build_command=build_command,
                    test_command=test_command,
                    pov_command=pov_command,
                    source_dir=source_dir,
                )
                futures[future] = agent
```

```
            for future in as_completed(futures):
                agent = futures[future]
                try:
                    result.agent_reports[agent] = future.result()
                except Exception as e:
                    logger.error(f"Agent {agent} failed with error: {e}")
                    result.agent_reports[agent] = None

        self._select_winner(result, agents)
        return result
```

LINE BY LINE

1. Start with a result marked `"unresolved"`. It only becomes "patched" if a winner is found later.
2. `ThreadPoolExecutor(max_workers=len(agents))`: hire one worker thread per agent (three, normally). This comes from Python's standard library.
3. `executor.submit(self.drv_loop.run, ...)`: give each worker the job "run the full repair loop for this agent, with this budget". `submit` returns immediately with a **future**: a claim ticket for a result that is not ready yet.
4. `futures[future] = agent`: remember which ticket belongs to which agent.
5. `as_completed(futures)`: hand back the tickets in the order the agents *finish*.
6. `future.result()` collects that agent's report. If the agent crashed with an error, it is logged and the agent's report is set to `None`. One agent crashing does not stop the others.
7. After the `with` block ends (it waits for **all** workers to finish), pick the winner.

ANALOGY

A restaurant manager gives the same order to three chefs at once and collects the plates as they come out. Even if the first plate is perfect, the manager waits for all three, then serves the simplest one that passed the taste test.

Why can the agents run "at the same time" in Python? Because nearly all their time is spent *waiting*: for the AI server to reply, for the compiler, for the test runs. While one thread waits, the others can work. That is also why the token counter needed the lock from section 4.3.

Sequential mode

src/agent_orchestrator/orchestrator.py · lines 280-304

```
        """Run agents sequentially. Stop at first validated patch."""
        result = PipelineResult(vulnerability=vuln, status="unresolved")

        for agent in agents:
            try:
                report = self.drv_loop.run(
                    agent_name=agent,
    ...
                )
                result.agent_reports[agent] = report
                if report.success:
                    break
            except Exception as e:
                logger.error(f"Agent {agent} failed with error: {e}")
                result.agent_reports[agent] = None

        self._select_winner(result, agents)
        return result
```

Here the agents go one after another, in order (Alpha, Beta, Gamma by default), and the loop **stops at the first success** (`break`). This is slower in wall-clock time when early agents fail, but cheaper when Alpha succeeds, because Beta and Gamma never get called.

	Parallel (default)	Sequential
Who runs	All agents at once	One at a time, in list order
Stops early?	No: waits for every agent	Yes: at the first validated patch
Winner	Smallest validated patch	In practice, the first success (only one can succeed before the loop stops)
Cost	Higher (all agents spend tokens)	Lower if an early agent succeeds
Time	Roughly the slowest agent	Sum of the agents that ran

4.7 Picking the winner, and counting the bill

src/agent_orchestrator/orchestrator.py · lines 306-332

```
    @staticmethod
    def _select_winner(result: PipelineResult, agents: list[str]) -> None:
        """
        Pick the winning agent among those whose patch passed every gate.
        Tie-break on the smallest validated patch (minimal-change principle),
        then on agent order for determinism.
        """
        winners = [
            (agent, report) for agent in agents
            if (report := result.agent_reports.get(agent)) is not None and report.success
        ]
        if not winners:
            return

        def patch_size(item):
            agent, report = item
            it = report.winning_iteration
            return (it.patch_line_count if it else 10**6, agents.index(agent))

        agent, report = min(winners, key=patch_size)
        result.status = "patched"
        result.winning_agent = agent
        result.winning_patch = report.winning_patch
    ...
```

LINE BY LINE

1. Build the list of **winners**: agents whose report exists (did not crash) and says `success`, meaning the patch passed every verification gate. (`:=` is Python's "walrus" operator: assign and test in one go.)
2. No winners? Return without changing anything. The status stays `"unresolved"`.
3. `patch_size` gives each winner a score of two numbers: first the number of lines the patch changed, then the agent's position in the list (Alpha 0, Beta 1, Gamma 2). If somehow there were no winning iteration, it uses a huge number (10^6) so that entry loses.
4. `min(...)` picks the lowest score: **fewest changed lines wins**; on a tie, the agent earlier in the list wins. Using the agent order as tie-breaker makes the choice repeatable.
5. Mark the result `"patched"` and record the winning agent and patch.

KEY IDEA

"Fewest changed lines" is the **minimal-change principle**: among fixes that all passed the same tests, the smallest one is least likely to have broken something the tests did not check. The line count is measured from the actual file before and after the patch was applied (`src/patch_validator/drv_loop.py` recomputes it after applying), so a whole-function replacement that changed one line counts as 1, not as the size of the whole function.

Cost per vulnerability

src/agent_orchestrator/orchestrator.py · lines 117 and 157-161

```
        cost_before = self.llm_client.get_usage_summary()["estimated_cost_usd"]
    ...
        result.elapsed_seconds = time.time() - start_time
        # Per-vulnerability cost, not the running total.
        result.total_cost_usd = round(
            self.llm_client.get_usage_summary()["estimated_cost_usd"] - cost_before, 6
        )
```

The client keeps one running total for the whole program. To get the cost of *this* bug, the orchestrator reads the total before starting and again after finishing, and subtracts. Like reading a taxi meter at pickup and drop-off. It includes every agent's attempts *and* the Critic's calls for this bug.

WATCH OUT

The subtraction trick only works because bugs are processed one at a time (`process_batch` loops over them sequentially). If two bugs were ever processed simultaneously on the same client, each would be charged for the other's calls too.

4.8 The Critic, Agent Delta

The fourth agent lives in `src/agent_orchestrator/critic.py`. Its module docstring explains why it exists: before the Critic, a rejected patch went back to its author with only raw tool output (a compiler error or a sanitizer dump), and "agents responded by resubmitting near-identical patches until their iteration budget ran out." Many failures were fixes that "removed the *reported* symptom while leaving the bug reachable by another path."

The Critic is an **LLM-as-judge**: an AI used to evaluate rather than to create. It reads the bug, the rejected patch and the exact output of the gate that rejected it, and returns a short diagnosis. **It never writes code.**

Its personality

src/agent_orchestrator/critic.py · lines 98-131 (trimmed)

```
CRITIC_SYSTEM_PROMPT = (
    "You are Agent Delta, the Critic: a strict security reviewer in an automated "
    "patching pipeline.\n\n"
    ...
    "METHOD:\n"
    "1. Read the gate output first. It is ground truth — it says exactly what still "
    "goes wrong.\n"
    "2. Identify the ROOT CAUSE, not the reported symptom. A sanitizer names the "
    "line that faulted, which is often downstream of the real defect.\n"
    "3. Determine what the patch overlooked. Common cases:\n"
    "   - The write was bounded but a later read still runs off the end.\n"
    ...
    "   - **The guard is present but evaluated in the wrong order.** C's `&&` and\n"
    "     `||` are short-circuit and strictly left-to-right, so the left operand\n"
    "     runs first. `while (*p != c && i < len)` dereferences `p` BEFORE\n"
    "     testing `i < len`, and is out of bounds on the final iteration even\n"
    "     though a bounds check is visibly present. ...
    ...
    "RULES:\n"
    "- Do NOT write code or a patch. You diagnose; another agent repairs.\n"
    "- Be specific: name variables, sizes, and expressions from the source.\n"
    "- If the gate output shows the patch failed to apply or compile, the verdict is "
    "'malformed' and your instruction must be about the output format, not security.\n"
    "- Never claim the patch is fine. It was rejected; something is wrong."
)
```

Personality: a strict senior reviewer. Trust the test output over opinions; look past the symptom to the cause; give one concrete instruction. The prompt carries a checklist of classic half-fixes, including a subtle one worth understanding: in C, `A && B` checks `A` first. If `A` reads memory and `B` is the "are we still in bounds?" check, the out-of-bounds read has already happened by the time the bounds check runs. The fix is to swap the order, not to add another check. The final rule, "Never claim the patch is fine", stops the Critic from second-guessing the gates.

The answer form: a strict JSON schema

Instead of free prose, the Critic must fill in a fixed form. **JSON** is a standard text format for structured data (named fields with values). A **schema** is the blank form: which fields must be there and what they may contain.

src/agent_orchestrator/critic.py · lines 79-96

```
CRITIC_SCHEMA = {
    "name": "critic_feedback",
    "strict": True,
    "schema": {
        "type": "object",
        "additionalProperties": False,
        "required": ["verdict", "root_cause", "what_the_patch_missed", "instruction"],
        "properties": {
            "verdict": {
                "type": "string",
                "enum": ["insufficient", "wrong_target", "malformed", "regression_risk"],
            },
            "root_cause": {"type": "string"},
            "what_the_patch_missed": {"type": "string"},
            "instruction": {"type": "string"},
        },
    },
}
```

Field	What goes in it (from the `CriticFeedback` descriptions)
`verdict`	Exactly one of four labels: `insufficient` (right place, incomplete fix), `wrong_target` (patched the wrong line or function), `malformed` (could not be applied or compiled), `regression_risk` (fixes the bug but breaks valid behaviour).
`root_cause`	The actual defect in one sentence, naming the variable or expression.
`what_the_patch_missed`	The specific consequence the previous attempt overlooked.
`instruction`	ONE concrete instruction for the next attempt. "No alternatives, no hedging."

`"strict": True` and `"additionalProperties": False` ask the server to guarantee the reply matches the form exactly, with no extra fields. On the Python side, the same form is described a second time as a **Pydantic** model (`CriticFeedback`, lines 39–75). Pydantic is a library that checks data against a declared shape and complains if it does not fit. So the reply is checked twice: by the server and by Pydantic.

When it fits, `as_guidance()` turns it into text for the next attempt:

src/agent_orchestrator/critic.py · lines 67-75

```
    def as_guidance(self) -> str:
        """Render the verdict as feedback for the next repair iteration."""
        return (
            "--- CRITIC REVIEW (Agent Delta) ---\n"
            f"Verdict: {self.verdict}\n"
            f"Root cause: {self.root_cause}\n"
            f"What your patch missed: {self.what_the_patch_missed}\n"
            f"Do this next: {self.instruction}"
        )
```

How the Critic makes its call: structured_call

src/agent_orchestrator/llm_client.py · lines 147-158

```
        messages = [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_content},
        ]

        response = self.client.chat.completions.create(
            model=model,
            messages=messages,
            max_tokens=max_tokens,
            temperature=0.0,          # diagnosis should be reproducible
            response_format={"type": "json_schema", "json_schema": schema},
        )
```

This is a second method on the same `LLMClient`. It sends the Critic's system prompt and the case, at temperature 0, with `response_format` set to the JSON schema. It logs token usage under the agent name `"critic"` (with the same lock), so the Critic's cost shows up in the totals. Then it returns the raw JSON text.

And the Critic's own `review` method builds the user message and turns failures into "no verdict":

src/agent_orchestrator/critic.py · lines 156-179

```
        user_content = (
            f"VULNERABILITY UNDER REPAIR:\n{vulnerability_context}\n\n"
            f"REJECTED PATCH:\n```\n{patch[:4000]}\n```\n\n"
            f"GATE THAT REJECTED IT: {failure_stage}\n"
            f"VERBATIM GATE OUTPUT:\n```\n{gate_output[:4000]}\n```\n\n"
            "Diagnose the failure."
        )

        try:
            raw = self.llm_client.structured_call(
                model=self.model,
                system_prompt=CRITIC_SYSTEM_PROMPT,
                user_content=user_content,
                schema=CRITIC_SCHEMA,
                max_tokens=600,
            )
            self.invocations += 1
            return CriticFeedback(**json.loads(raw))
        except (ValidationError, json.JSONDecodeError) as e:
            logger.warning(f"Critic returned unusable output: {e}")
            return None
        except Exception as e:
            logger.warning(f"Critic call failed: {e}")
            return None
```

LINE BY LINE

1. The message contains the full case file, the rejected patch and the gate output. `[:4000]` cuts each of those two to at most 4000 characters so a huge sanitizer dump cannot blow up the request.
2. The answer is limited to 600 tokens: diagnoses are meant to be short.
3. `json.loads(raw)` turns the JSON text into a Python dictionary; `CriticFeedback(**...)` checks it against the Pydantic model.
4. If the text is not valid JSON, or does not fit the form, or the call fails for *any* reason, a warning is logged and `None` is returned. The docstring says: "A failed critique is never fatal: the DRV loop falls back to the raw gate output."

When does the Critic run?

Not on every failure. Inside the repair loop (`src/patch_validator/drv_loop.py`), after a patch is rejected, the loop records the failure and then asks the EscalationPolicy:

src/patch_validator/drv_loop.py · lines 265-276

```
            if self.critic and policy.should_invoke_critic(result.failure_stage):
                verdict = self.critic.review(
                    vulnerability_context=vulnerability_context,
                    patch=result.patch_diff or str(result.replacement),
                    gate_output=result.error_feedback,
                    failure_stage=result.failure_stage,
                )
                if verdict:
                    result.critic_verdict = verdict.verdict
                    if working.attempts:
                        working.attempts[-1].critic_verdict = verdict.verdict
                    feedback = f"{result.error_feedback}\n\n{verdict.as_guidance()}"
```

If the Critic produced a verdict, the next attempt's feedback becomes "raw gate output + CRITIC REVIEW". If not, it is just the raw gate output. (Note: when a reply contained no patch at all, a `generation` failure, the loop skips straight to the next attempt and the Critic is not consulted for it.)

4.9 EscalationPolicy: the rulebook with no AI in it

The decisions "should we call the Critic?", "should we insist on the whole-function format?" and "should we give up?" are made by plain Python with fixed numbers. The class docstring explains why: "Keeping it deterministic means loop control cannot itself hallucinate." A new `EscalationPolicy` is created for every agent's repair loop, so each agent has its own counters.

src/agent_orchestrator/critic.py · lines 200-228

```
    CRITIC_AFTER_FAILURES = 2
    FORCE_REPLACEMENT_AFTER_APPLY_FAILURES = 2
    GIVE_UP_AFTER_IDENTICAL_FAILURES = 4

    def __init__(self):
        self.stage_counts: dict[str, int] = {}
        self.total_failures = 0
        self.escalations: list[str] = []
        # Gist of each rejected attempt, in order. Progress is judged from
        # these, not from how many times a stage was reached.
        self.attempt_gists: list[str] = []

    def record_failure(self, stage: str, gist: str = "") -> None:
        self.stage_counts[stage] = self.stage_counts.get(stage, 0) + 1
        self.total_failures += 1
        if gist:
            self.attempt_gists.append(gist)

    def should_invoke_critic(self, stage: str) -> bool:
        if stage == "pov":
            return True          # a wrong fix always deserves a diagnosis
        return self.total_failures >= self.CRITIC_AFTER_FAILURES

    def should_force_replacement(self) -> bool:
        return (
            self.stage_counts.get("apply", 0) >= self.FORCE_REPLACEMENT_AFTER_APPLY_FAILURES
            or self.stage_counts.get("generation", 0) >= self.FORCE_REPLACEMENT_AFTER_APPLY_FAILURES
        )
```

LINE BY LINE

1. Three fixed thresholds: 2, 2 and 4.
2. `record_failure` counts failures per **stage** (which gate rejected it: `apply`, `build`, `pov`, `regression`, `post_scan`, `generation`), counts the total, and stores a short **gist** (a summary) of what the attempt changed.
3. **Critic rule:** if the failing gate was `pov`, call the Critic **immediately**, even on the first failure. "PoV" means proof of vulnerability: the original crashing input was replayed against the patched program and it still crashed. That means the fix compiled but did not fix the bug, which is exactly where a diagnosis helps. For any other gate, call the Critic once the agent has failed at least twice in total.
4. **Format rule:** if the patch failed to **apply** twice, or the reply contained **no patch** twice, the agent has shown it cannot produce a usable diff. From then on, every attempt's feedback gets a "MANDATORY FORMAT CHANGE" paragraph.

The forced-format message, added by the repair loop at the start of each attempt once that rule fires:

src/patch_validator/drv_loop.py · lines 189-198

```
            # Deterministic escalation: once an agent has proved it cannot emit a
            # byte-exact diff, stop asking for one.
            if policy.should_force_replacement():
                feedback = (
                    (feedback or "")
                    + "\n\nMANDATORY FORMAT CHANGE: your diffs keep failing to apply. "
                    "Do NOT output a diff this time. Output the COMPLETE corrected "
                    "function in a ```c block whose first line is "
                    "`// FUNCTION: <exact_function_name>`."
                )
```

Stopping when stuck

src/agent_orchestrator/critic.py · lines 229-253 (docstring trimmed)

```
    def should_stop_early(self) -> bool:
        """
        True only when the agent has stopped making progress — the last N
        attempts were materially IDENTICAL.
    ...
        """
        n = self.GIVE_UP_AFTER_IDENTICAL_FAILURES
        if len(self.attempt_gists) < n:
            return False
        return len(set(self.attempt_gists[-n:])) == 1
```

LINE BY LINE

1. `n` is 4.
2. Fewer than 4 recorded gists? Keep going.
3. `self.attempt_gists[-n:]` takes the last four gists. `set(...)` removes duplicates. If only **one** distinct gist remains, the last four attempts were the same patch, so the agent is going in circles. Stop and save the remaining budget.

The trimmed docstring is worth reading in full in the file. It says this rule used to count failures per stage, which "killed" an agent that had reached the PoV gate four times with four *different* attempts. That was wrong: "Repeatedly failing the same gate with *different* attempts is the loop working, not the loop stuck." It also says plainly that this is "a *budget* heuristic, not a verification gate": letting an agent keep trying can never make a bad patch pass.

ANALOGY

A driving instructor's rules: "If the student fails the parallel-park test, explain what went wrong right away. If they fail anything else twice, sit them down for a talk. If they keep trying to parallel-park with the steering wheel on the wrong side, insist they use the reversing camera. If they make the exact same mistake four times in a row, end the lesson."

Putting the escalation flow together

```
attempt k:  [forced-format note if apply/generation failed >= 2x]
            + [list of already-rejected attempts]
            + [feedback from attempt k-1]
                      |
                      v
           generate_patch()  --- no patch? ---> record "generation" failure, next attempt
                      |
                      v
              verify (gates)  --- all pass ---> SUCCESS, stop this agent
                      |
                   fails at stage S
                      |
         record_failure(S, gist)  +  add to working-memory ledger
                      |
         S == "pov"  or  total failures >= 2 ?
              yes |                    | no
                  v                    |
        Critic.review() -> verdict?    |
          yes: feedback = gate output + CRITIC REVIEW
          no : feedback = gate output  <+
                      |
         last 4 gists identical? --- yes ---> stop early
                      |
                      no ---> attempt k+1 (until budget runs out)
```

4.10 Where the documents and the code disagree

The project is mostly careful, but a few comments and docstrings no longer match what the code does. When in doubt, trust the code.

What the text says	What the code actually does
`orchestrator.py` line 2: "LangGraph Multi-Agent Orchestrator"; `__init__.py`: "LangGraph multi-agent ensemble".	LangGraph (a framework for building AI agent graphs) is not imported anywhere here. The "graph" is Python's standard `ThreadPoolExecutor` plus a for-loop.
`orchestrator.py` line 4: "First validated patch wins."	In parallel mode (the default) all agents run to completion and the **smallest** validated patch wins, as the `_run_parallel` docstring itself says. "First wins" is only roughly true in sequential mode.
`Critic` class docstring: "Invoked only on repeated failure."	`should_invoke_critic` calls it on the **first** failure if the failing gate is `pov`.
The Critic is described as a fourth agent with its own role.	It has no model setting of its own. `self.model = model or llm_client.models.get("beta")` (critic.py line 140): it reuses Beta's model, i.e. `MODEL_BETA`. There is no `MODEL_CRITIC`.
`LLMClient` docstring: "Wraps OpenAI API with retry logic."	True for `generate_patch`. But `structured_call` (the Critic's path) calls the server directly with **no retry**. A rate-limit hiccup simply means "no Critic verdict this time".
Gamma's prompt: "You receive a SARIF report".	Only if the vulnerability carries `sarif_context`; the benchmark targets do not.
`VulnerabilityReport.source` comment lists "semgrep", "fuzzer", "manual".	The benchmark passes `sanitizer_crash`, `semgrep` or `side_effect`. The field is just informational text in the prompt.

WATCH OUT

One more subtle point: the orchestrator creates a **single** repair loop object (`self.drv_loop`) and a single Critic inside it, shared by all three agent threads. The EscalationPolicy is fresh per agent run, so the rules above are per agent. But the Critic's `invocations` counter is shared and is updated without a lock, so the reported "critic invocations" is a running total across agents (and across bugs), not a per-agent figure.

TRY IT YOURSELF

These need no API key and no network, because they only exercise the parsers and the rulebook:

`python -c "from src.agent_orchestrator.llm_client import LLMClient as L; print(L._extract_replacement('```c\nint f(int x) {\n return x;\n}\n```'))"` prints `('f', 'int f(int x) {\n return x;\n}')`: the name was inferred without a marker.

`python -c "from src.agent_orchestrator.critic import EscalationPolicy as E; p=E(); print(p.should_invoke_critic('pov'), p.should_invoke_critic('build'))"` prints `True False`.

And to see the prompt builder: `python -c "from src.agent_orchestrator.orchestrator import VulnerabilityReport as V, Orchestrator as O; print(O._build_vulnerability_prompt(V(id='X', source='manual', file_path='a.c', line_number=3, description='demo')))"`. (Importing the orchestrator also imports the repair loop and memory modules, so run these from the project root with its dependencies installed.)

CHAPTER RECAP

- Settings come from environment variables (often via a `.env` file): `OPENAI_API_KEY` (required), `OPENAI_BASE_URL`, `MODEL_ALPHA/BETA/GAMMA` (default `gpt-4o-mini`) and `KAVACH_SEED` (default 1337).
- Because the client speaks the OpenAI-compatible protocol, changing `OPENAI_BASE_URL` is enough to use a local or air-gapped model server.
- The three agents are three system prompts, not three programs: Alpha the analyst (explains, 4096 tokens), Beta the minimalist (1–5 lines, 8 attempts), Gamma the CWE-template patcher.
- `generate_patch` sends system + user (+ feedback on retries) at temperature 0 with a seed, retries rate-limit and connection errors with exponential backoff, re-raises other API errors, and records tokens under a thread lock.
- Cost = tokens × `PRICING` (per million, matched by longest model-name prefix); unknown models are reported as unpriced. Per-bug cost is "total after minus total before".
- Patches come back as a ```` ```diff ```` block or a whole function marked `// FUNCTION: name`; the whole-function format is preferred because diffs break on one wrong character.
- All agents get the same prompt from `_build_vulnerability_prompt`, optionally plus up to two recalled "prior verified fixes".
- Parallel mode (default) runs all agents in a `ThreadPoolExecutor`, waits for all, and picks the validated patch with the fewest changed lines; sequential mode stops at the first success.
- The Critic (Delta) returns a strict JSON verdict and never writes code. It reuses Beta's model, has no retry, and failure just means "no advice".
- EscalationPolicy is plain Python: Critic immediately on PoV failure or after 2 total failures; force whole-function format after 2 apply/generation failures; stop when the last 4 attempts are identical.

CHECK YOURSELF

1. You want to run AI Kavach on a laptop with no internet, using a model server on the same laptop. Which setting is the key one to change, and why does that work?
2. Alpha, Beta and Gamma all succeed on the same bug. Alpha's patch changes 3 lines, Beta's changes 1 line, Gamma's changes 1 line. In parallel mode, which agent wins, and why?
3. The server answers with a rate-limit error twice, then succeeds. What happened in between, and how long did the program wait in total? What if instead the API key were wrong?
4. An agent's first patch compiles but the original crash input still crashes the patched program. Is the Critic called? What if the first failure had been a compile error instead?
5. Why would anyone prefer the AI to send back an entire function rather than a short diff?

CHAPTER 5

Proving the Fix, Then Attacking It

In the earlier chapters we watched the system find a bug and ask three AI "agents" (Alpha, Beta and Gamma) to write a fix. This chapter is about what happens to that fix next. The short version: **nobody trusts it.** The project's motto is "the LLM proposes, deterministic tools decide". An LLM (the AI model) is good at writing code that *looks* right. It is not a reliable judge of whether the code *is* right. So every proposed patch goes through a series of mechanical checks that cannot be argued with or sweet-talked: the patch must apply, the program must compile, the attack must stop working, normal use must still work, and so on.

Those checks are only the first half of the chapter. Once a patch has passed them, a second stage called **hardening** deliberately tries to *break* it. A third, optional stage called **formal verification** uses a mathematical tool (CBMC) to prove that, within certain limits, no input at all can trigger the bug. At the end we look at a small demonstration program, the **cheat detector**, which shows why the second stage exists. Throughout, we also point out the places where the checks are weaker than they first appear. The project's own code comments are often honest about these, and you should be too.

ANALOGY

Picture a new bridge. First the inspector runs a checklist: are the bolts in, does it carry the test truck, can ordinary cars still drive across? That is the **DRV loop**. Next a demolition expert is paid to find any way to make it fall down: heavier trucks, odd angles, wind. That is **hardening**. Finally an engineer works through the equations and shows that, for every load up to some stated limit, the bridge holds. That is **formal verification**. Each stage catches things the others miss.

The files for this chapter:

- `src/patch_validator/drv_loop.py`: the loop and its five "gates" (checks).
- `src/patch_validator/hardening.py`: the attack stage.
- `src/patch_validator/formal.py`: the CBMC proof stage.
- `tests/benchmarks/harness/*.sh`: small shell scripts that decide whether an attack "still works".
- `tests/benchmarks/cbmc_harnesses/`: the proof set-ups for CBMC.
- `tests/cheat_detector/run_cheat_detector.py`: the demonstration with three hand-written patches.
- `tests/standalone_support.py`: shared helpers, including the code that turns hardening results into labels.

5.1 Gates, and the three answers a gate can give

A **gate** is one yes/no check that a patch must get through, like a security gate at an airport. Every gate in this project works the same basic way: run a command and look at its **exit code**. An exit code is the small number a program hands back to whoever started it when it finishes. By long Unix convention, `0` means "all fine" and anything else means "something went wrong".

There is one subtlety, and the whole project depends on getting it right. A gate can give **three** answers, not two:

src/patch_validator/drv_loop.py · lines 38-40

```
GATE_PASS = "pass"
GATE_FAIL = "fail"
GATE_SKIPPED = "skipped"
```

The file's opening comment spells out what each one means:

- **pass**: the command ran and exited 0.
- **fail**: the command ran and exited with something other than 0. The failure text is sent back to the agent.
- **skipped**: no command was set up for this gate on this target, so nothing was checked.

KEY IDEA

"Skipped" is not "passed". If a target has no attack input (no PoV, or proof of vulnerability), the system cannot show that the bug is gone, and it says so. Reports print skipped gates as `SKIPPED`, never as `PASS`. Keep this in mind, because later we will see that "fully validated" is defined in a way that lets skipped gates through.

Each attempt to fix a bug is recorded in a `PatchResult`. This is a dataclass: a Python record with named fields, like a row in a spreadsheet.

src/patch_validator/drv_loop.py · lines 46-77

```
@dataclass
class PatchResult:
    """Result of a single DRV iteration. Gate fields hold pass/fail/skipped."""

    iteration: int
    agent_name: str
    patch_diff: str
    # (function_name, source) when the agent returned a whole-function
    # replacement instead of a unified diff.
    replacement: Optional[tuple[str, str]] = None
    applied_via: str = ""            # "diff" | "content-match" | "function-replacement"
    applied: str = GATE_SKIPPED
    syntax_ok: str = GATE_SKIPPED
    pov_ok: str = GATE_SKIPPED
    regression_ok: str = GATE_SKIPPED
    post_scan_ok: str = GATE_SKIPPED
    fully_validated: bool = False
    error_feedback: str = ""
    ...
    failure_stage: str = ""          # apply | build | pov | regression | post_scan | generation
    critic_verdict: str = ""         # Agent Delta's diagnosis, when it was consulted

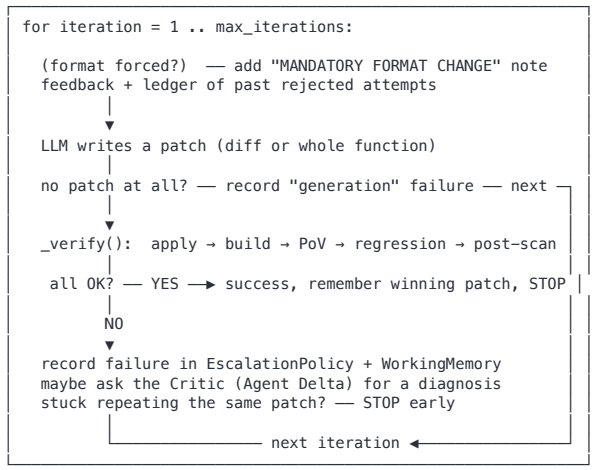
    @property
    def executed_gates(self) -> list[str]:
        """Gates that actually ran (i.e. were not skipped) in this iteration."""
        return [
            name for name, state in (
                ("build", self.syntax_ok), ("pov", self.pov_ok),
                ("regression", self.regression_ok), ("post_scan", self.post_scan_ok),
            ) if state != GATE_SKIPPED
        ]
```

LINE BY LINE

1. `iteration`, `agent_name`: which attempt this was (1st, 2nd, ...) and which agent wrote it.
2. `patch_diff`: the patch written as a **diff**, the standard text format that lists lines to remove (starting with `-`) and lines to add (starting with `+`).
3. `replacement`: the other format the agents can use. Instead of a diff, the agent returns the whole corrected function, marked `// FUNCTION: name`. This field stores the pair (function name, new source code).
4. `applied_via`: which method actually got the patch into the file.
5. The five gate fields (`applied`, `syntax_ok` for the build, `pov_ok`, `regression_ok`, `post_scan_ok`) all **start as SKIPPED**. A gate only changes to pass or fail if it really runs.
6. `fully_validated` starts as `False`. It becomes `True` only at the very end of the checks.
7. `error_feedback` is the text sent back to the agent when something fails. `failure_stage` names the gate that failed.
8. `executed_gates` is a computed list of the gates that were *not* skipped. Reports use it to tell "the PoV proved the fix" apart from "this target never had a PoV".

5.2 The DRV loop from start to finish

DRV stands for **Detect, Repair, Verify**. The "detect" part happened in earlier stages (fuzzing and triage). The loop in `drv_loop.py` takes care of Repair and Verify, and it repeats them. Each agent runs its own loop, with its own iteration budget (Alpha 5, Beta 8, Gamma 5). The main function is `DRVLoop.run`.



Step 1: build the prompt and ask for a patch

src/patch_validator/drv_loop.py · lines 186–210

```
        for iteration in range(1, max_iterations + 1):
            logger.info(f"[{agent_name}] DRV iteration {iteration}/{max_iterations}")

            # Deterministic escalation: once an agent has proved it cannot emit a
            # byte-exact diff, stop asking for one.
            if policy.should_force_replacement():
                feedback = (
                    (feedback or "")
                    + "\n\nMANDATORY FORMAT CHANGE: your diffs keep failing to apply. "
                    ...
                )

            ledger = working.render()
            iteration_feedback = f"{feedback}\n\n{ledger}" if (feedback and ledger) else (feedback or ledger or None)

            llm_result = self.llm_client.generate_patch(
                agent_name=agent_name,
                vulnerability_context=vulnerability_context,
                feedback=iteration_feedback,
                iteration=iteration,
            )
            patch_diff = llm_result["patch"]
            replacement = llm_result.get("replacement")
```

LINE BY LINE

1. `for iteration in range(1, max_iterations + 1)` : count 1, 2, 3, ... up to the agent's budget.
2. `policy.should_force_replacement()` : the **EscalationPolicy** is a fixed rulebook (no AI involved) that watches failures. If the agent has failed at the "apply" stage, or produced no patch at all, two or more times, its diffs evidently don't fit the file. The policy then adds a firm instruction: send the whole function next time, not a diff.
3. `working.render()` : the "ledger" from **WorkingMemory**, a list of earlier attempts that were rejected (explained below).
4. `iteration_feedback` : join the latest failure message and the ledger. If only one exists, use that. If neither exists (the first iteration), send nothing.
5. `generate_patch(...)` : call the LLM. The result may hold a diff (`"patch"`), a whole-function `"replacement"` , or both.

Step 2: no patch at all? Count it as a failure

src/patch_validator/drv_loop.py · lines 223–245

```
            if not patch_diff and not replacement:
                result.failure_stage = "generation"
                result.error_feedback = (
                    "ERROR: No patch was found in your response. Return EITHER a "
                    "whole corrected function in a ```c block starting with "
                    "`// FUNCTION: <name>`, OR a unified diff in a ```diff block."
                )
                policy.record_failure("generation", gist="(no patch produced)")
                feedback = result.error_feedback
                continue

            if self._verify(result, target_file, patch_diff,
                            build_command, pov_command, test_command, source_dir):
                report.success = True
                # Record whatever actually got applied, so the audit trail shows
                # the real change rather than an unused diff.
                if result.applied_via == "function-replacement" and result.replacement:
                    name, source = result.replacement
                    report.winning_patch = f"// FUNCTION: {name}\n{source}"
                else:
                    report.winning_patch = patch_diff
                logger.info(f"[{agent_name}] ✅ Patch validated at iteration {iteration}")
                break
```

LINE BY LINE

1. If the model's reply contained neither a diff nor a function, the attempt is a "generation" failure. The agent is told exactly which formats are accepted, and `continue` skips straight to the next iteration.
2. Otherwise `_verify(...)` runs every gate (section 5.4). It returns `True` only if nothing failed.
3. On success the loop records the **winning patch**. It is careful to record what was *actually applied*: if the function replacement was used, it stores that rather than a diff that was never used.
4. `break` : stop the loop. An agent stops at its first validated patch.

Step 3: the patch was rejected, so learn from it

src/patch_validator/drv_loop.py · lines 247–287 (trimmed)

```
            # --- the patch was rejected: diagnose before trying again ---
            attempted = result.patch_diff or (
                result.replacement[1] if result.replacement else ""
            )
            ...
            policy.record_failure(
                result.failure_stage or "unknown",
                gist=working.summarise_patch(attempted),
            )
            working.record(
                iteration=iteration, agent=agent_name,
                patch=attempted,
                rejected_by=result.failure_stage or "unknown",
            )
            feedback = result.error_feedback

            if self.critic and policy.should_invoke_critic(result.failure_stage):
                verdict = self.critic.review(
                    ...
```

```
            )
            if verdict:
                ...
                feedback = f"{result.error_feedback}\n\n{verdict.as_guidance()}"
                ...
        if policy.should_stop_early() and iteration < max_iterations:
            policy.note(f"stopped early at iteration {iteration}: no progress")
            ...
            break
```

LINE BY LINE

1. `attempted` : the text of what the agent tried, whether diff or function.
2. `policy.record_failure(stage, gist=...)` : tell the escalation rulebook which gate failed, along with a short "gist" (fingerprint) of the patch. The rulebook uses the gist to tell an agent that keeps trying *different* things (progress) apart from one repeating *the same* thing (stuck).
3. `working.record(...)` : add the attempt to the WorkingMemory ledger.
4. `feedback = result.error_feedback` : the real error text (compiler error, sanitizer crash report, and so on) becomes the starting point of the next prompt.
5. `should_invoke_critic` : decide whether to ask **Agent Delta, the Critic**. Delta is a fourth LLM role that writes no code; it only diagnoses. In `critic.py` the rule is: always on a PoV failure ("a wrong fix always deserves a diagnosis"), and otherwise once the total failure count reaches a threshold. If Delta answers, its advice is added to the feedback.
6. `should_stop_early()` : if the last few attempts were materially *identical*, the agent is going round in circles and the loop ends early to save time and money.

The WorkingMemory ledger

Before the ledger existed, each iteration saw only the *most recent* rejection. The comment in `src/memory/store.py` describes what went wrong: agents "oscillated between two rejected fixes", trying fix A, then B, then A again, then B again, until the budget ran out. The ledger fixes this by showing every recent rejected attempt at once:

src/memory/store.py · lines 157–170

```
    def render(self) -> str:
        """Render the ledger for injection into the next prompt."""
        if not self.attempts:
            return ""
        lines = ["--- ATTEMPTS ALREADY REJECTED (do not repeat these) ---"]
        for a in self.attempts[-self.MAX_RENDERED:]:
            verdict = f" [critic: {a.critic_verdict}]" if a.critic_verdict else ""
            lines.append(
                f"  #{a.iteration} ({a.agent}) rejected by {a.rejected_by}{verdict}: {a.summary}"
            )
        lines.append(
            "Your next patch must be materially different from every attempt above."
        )
        return "\n".join(lines)
```

It shows at most the last six attempts (`MAX_RENDERED = 6`). Each is one line: iteration number, agent, the gate that rejected it, the Critic's verdict if there was one, and a one-line `summary` . The summary comes from `summarise_patch` . For a diff, it is built from the lines the diff *adds*. For a whole function, it keeps the lines with control flow and comparisons (`if` , `for` , `<` , `==` , ...), because that is where a memory-safety fix lives. Either way, a short hash (an 8-character fingerprint of the whole body) is appended, so two attempts count as "the same" only if they really are. The ledger lasts only for one bug and one agent. It is the "working" tier of the memory system, like a scratch pad thrown away after the task.

ANALOGY

A locksmith trying keys on a stubborn lock puts each failed key on a separate hook. Without the hook, a tired locksmith keeps picking up the same two keys. With it, every new try is a key not yet used.

5.3 The workspace: a fresh scratch copy for every attempt

Every attempt is tested in a brand-new **temporary directory**, a throwaway folder the operating system creates and that Python deletes automatically afterwards. The original source file is never changed. This matters for two reasons. First, a bad patch can't damage the real code. Second, attempt 3 can't accidentally inherit leftovers from attempt 2.

src/patch_validator/drv_loop.py · lines 308-316

```
        with tempfile.TemporaryDirectory(prefix="kavach_drv_") as tmpdir:
            workspace = Path(tmpdir)
            try:
                patched_src = self._materialise(workspace, target_file, source_dir)
            except FileNotFoundError as e:
                result.failure_stage = "apply"
                result.applied = GATE_FAIL
                result.error_feedback = str(e)
                return False
```

`with tempfile.TemporaryDirectory(...) as tmpdir:` means: create a folder with a name like `/tmp/kavach_drv_x7f3k2`, run everything indented beneath this line, then delete the folder, even if something crashed. The copying itself is done by `_materialise` ("materialise" here means "make the files physically appear"):

src/patch_validator/drv_loop.py · lines 422-442

```
        target = Path(target_file)
        if not target.exists():
            raise FileNotFoundError(f"Target file not found: {target_file}")

        rel = Path(target_file)
        if rel.is_absolute():
            rel = Path(target.name)

        if source_dir:
            src_root = Path(source_dir)
            if not src_root.exists():
                raise FileNotFoundError(f"Source dir not found: {source_dir}")
            dest_root = workspace / src_root
            dest_root.parent.mkdir(parents=True, exist_ok=True)
            shutil.copytree(src_root, dest_root, dirs_exist_ok=True)
            return workspace / rel

        dest = workspace / rel
        dest.parent.mkdir(parents=True, exist_ok=True)
        shutil.copy2(target, dest)
        return dest
```

LINE BY LINE

1. If the target file doesn't exist, raise an error. `_verify` catches it and records it as an "apply" failure.
2. `rel` is the file's **relative path**, for example `tests/demo_vulns/06_off_by_one_loop.c`. If the path is absolute (it starts from the root of the disk, `/Users/...`), only the bare file name is kept.
3. If the target is part of a bigger project (`source_dir` is set, as for the real cJSON library), the whole directory tree is copied into the workspace.
4. Otherwise only the single file is copied, keeping its relative folder layout. The docstring explains why the layout is kept: diffs name files like `a/tests/demo_vulns/...`, and keeping the same layout lets the `patch` tool find them naturally.
5. The function returns the path of the copy that will be patched.

Placeholders: {src}, {srcdir}, {bin}, {workspace}

Each target comes with **command templates**: how to build it, how to run the attack, how to run a normal test. They can't contain real file paths, because the workspace folder gets a new random name every time. So they contain **placeholders** in curly braces, which are filled in at the last moment. For example, the build command for the synthetic targets is:

tests/benchmarks/targets.py · lines 41–46

```
SAN_CFLAGS = (
    "-fsanitize=address,undefined -fno-sanitize-recover=all "
    "-fno-omit-frame-pointer -g -O0"
)

SINGLE_FILE_BUILD = f'clang {SAN_CFLAGS} "{{src}}" -o "{{bin}}"'
```

(The doubled braces `{{src}}` are Python's way of writing a literal `{src}` inside an f-string.) Just after a patch applies, `_verify` builds the values to fill in:

src/patch_validator/drv_loop.py · lines 357–363

```
            binary = workspace / "kavach_target"
            ctx = {
                "src": str(patched_src),
                "srcdir": str(patched_src.parent),
                "bin": str(binary),
                "workspace": str(workspace),
            }
```

Placeholder	Becomes	Example
`{src}`	the patched source file inside the workspace	`/tmp/kavach_drv_ab12/tests/demo_vulns/06_off_by_one_loop.c`
`{srcdir}`	the folder that file sits in	`/tmp/kavach_drv_ab12/tests/demo_vulns`
`{bin}`	where the compiled program goes (always named `kavach_target`)	`/tmp/kavach_drv_ab12/kavach_target`
`{workspace}`	the temp folder itself	`/tmp/kavach_drv_ab12`

src/patch_validator/drv_loop.py · lines 586–600

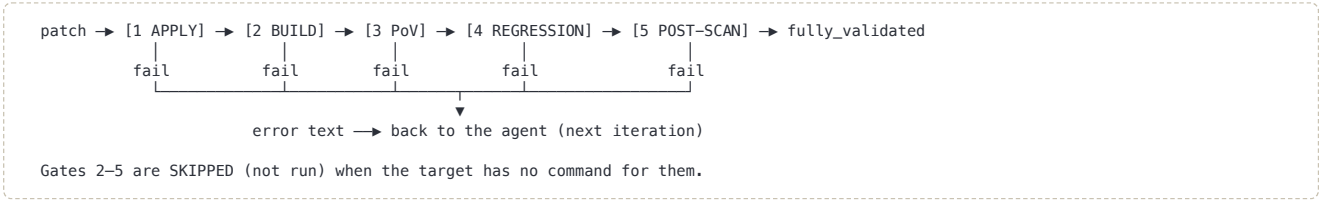
```
    @staticmethod
    def expand(command: str, ctx: dict) -> str:
        """
        Expand {src}/{srcdir}/{bin}/{workspace} in a command template.

        Only these four names are substituted, by literal replacement. Anything
        else in braces — shell brace expansion, awk programs, ${arr[1]} — is
        left exactly as written. (str.format cannot be used here: it mangles
        constructs like ${x[1]} into $x.)
        """
        out = command
        for key in ("src", "srcdir", "bin", "workspace"):
            if key in ctx:
                out = out.replace("{" + key + "}", str(ctx[key]))
        return out
```

This is a plain find-and-replace for exactly four words. Python has a built-in template feature (`str.format`), but it treats *every* pair of curly braces as a placeholder. Shell commands use curly braces for their own purposes, and `str.format` would mangle them. Plain find-and-replace leaves everything else alone.

5.4 The five gates inside `_verify`

Now the main event. The gates run in a fixed order, and the first failure stops everything: there's no point running the attack on a program that doesn't compile.



Gate 1: apply the patch

Getting the patch into the file is harder than it sounds, because LLMs often get diffs *slightly* wrong: the line numbers are off, the spacing differs, or the file path has an invented prefix. The system therefore tries several methods, from most precise to most forgiving:

src/patch_validator/drv_loop.py · lines 321–355 (trimmed)

```
            original_text = patched_src.read_text()
            ok, detail = False, ""
            if result.replacement:
                name, source = result.replacement
                ok, detail = self._apply_replacement(patched_src, name, source)
                if ok:
                    result.applied_via = "function-replacement"

            if not ok and patch_diff:
                diff_ok, diff_detail = self._apply_patch(workspace, patched_src, patch_diff)
                if diff_ok:
                    ok = True
                    result.applied_via = "diff"
                else:
                    detail = (detail + "\n" + diff_detail).strip()

            if ok:
                ...
                result.patch_line_count = self._changed_lines(
                    original_text, patched_src.read_text()
                )

            result.applied = GATE_PASS if ok else GATE_FAIL
            if not ok:
                result.failure_stage = "apply"
                result.error_feedback = (
                    "PATCH FAILED TO APPLY.\n\n" + detail + "\n\n"
                    ...
                )
                return False
```

The order of attempts:

1. **AST splice** (whole-function replacement). If the agent sent a whole function, `_apply_replacement` asks the tree-sitter context extractor (from an earlier chapter) to find that function by name in the parsed program. An **AST**, or abstract syntax tree, is the program's grammatical structure, so the extractor knows exactly where the function begins and ends, byte by byte. It cuts the old function out and splices the new one in. Spacing and line numbers can't trip this up. It fails only if no function of that name exists (or no extractor was provided).
2. **Diff via the `patch` tool.** If there was no replacement, or it failed, the diff goes to `_apply_patch`, which uses the standard Unix `patch` program in a series of tries (below).
3. **Content match.** If `patch` fails every time, a final custom method finds each change by its text instead of its line numbers.

Once a patch lands, `_changed_lines` counts how many lines really differ between the before and after files. This matters later, because when several agents succeed the **smallest** patch wins. The comment notes that counting `+/-` markers used to report every whole-function patch as "0

lines changed".

src/patch_validator/drv_loop.py · lines 459-487

```
        exact = [
            ["patch", "-p1", "-l", "-f", "-i", str(patch_file)],
            ["patch", "-p0", "-l", "-f", "-i", str(patch_file)],
            ["patch", "-p1", "-l", "-f", str(patched_src), "-i", str(patch_file)],
            ["patch", "-p0", "-l", "-f", str(patched_src), "-i", str(patch_file)],
        ]
        fuzzy = [cmd[:1] + ["-F", "3"] + cmd[1:] for cmd in exact]
        attempts = exact + fuzzy

        errors = []
        original = patched_src.read_text()
        for cmd in attempts:
            dry = subprocess.run(
                cmd + ["--dry-run"], cwd=workspace,
                capture_output=True, text=True, timeout=60,
            )
            if dry.returncode != 0:
                errors.append(f"$ {' '.join(cmd[:3])} --dry-run\n{dry.stdout}{dry.stderr}")
                continue

            applied = subprocess.run(
                cmd, cwd=workspace, capture_output=True, text=True, timeout=60,
            )
            if applied.returncode == 0:
                if patched_src.read_text() == original:
                    errors.append("patch reported success but the file is unchanged")
                    continue
                return True, applied.stdout
            errors.append(f"$ {' '.join(cmd[:3])}\n{applied.stdout}{applied.stderr}")
```

LINE BY LINE

1. The `patch` program's flags:
   - `-p1` / `-p0` : how many leading folder names to strip from the paths in the diff. Diffs usually write `a/src/file.c` ; `-p1` strips the `a/` , while `-p0` keeps the path as written.
   - `-l` : "loose", meaning differences in spaces and tabs are ignored.
   - `-f` : "force", meaning don't stop to ask questions.
   - `-i file` : read the diff from this file.

   The 3rd and 4th variants also name the target file explicitly, so a diff with made-up paths can still be pointed at the right file.
2. `fuzzy` : the same four commands again with `-F 3` added. **Fuzz factor 3** lets `patch` ignore up to 3 lines of surrounding context that don't quite match. (This has nothing to do with "fuzzing" in the bug-finding sense.) The exact versions are always tried first. The fuzzy ones are a fallback.
3. For each command, first a `--dry-run` : a rehearsal that reports whether the patch *would* apply, without touching the file. Only if the rehearsal succeeds is the patch applied for real.
4. A sneaky case is guarded against: `patch` may report success while the file is unchanged. That is treated as a failure.
5. Every error message is collected, so if all attempts fail the agent sees why.

The last resort is `_apply_by_content` . It reads each "hunk" (one block of changes in a diff) and builds two lists of lines: what to **search for** (unchanged context lines plus the lines being removed) and what to **replace it with** (context plus added lines). Then it searches the file for the search block, comparing lines with leading and trailing spaces stripped:

src/patch_validator/drv_loop.py · lines 560-581

```
        for index, hunk in enumerate(hunks, 1):
            search = [l[1:] for l in hunk if l.startswith((" ", "-")) or l == ""]
```

```
            replace = [l[1:] for l in hunk if l.startswith((" ", "+")) or l == ""]
            if not search:
                return False, f"hunk {index}: nothing to match against"

            def norm(seq):
                return [s.strip() for s in seq]

            needle = norm(search)
            matches = [
                i for i in range(len(lines) - len(needle) + 1)
                if norm(lines[i:i + len(needle)]) == needle
            ]
            if len(matches) != 1:
                return False, (
                    f"hunk {index}: context matched {len(matches)} locations "
                    f"(need exactly 1)"
                )

            at = matches[0]
            lines[at:at + len(needle)] = replace
```

The important safety rule is `if len(matches) != 1`. The block must appear in **exactly one** place. If it appears nowhere, the patch can't be applied. If it appears twice, the system won't guess which copy was meant. A patch must never land in the wrong spot without anyone noticing.

KEY IDEA

The apply gate is deliberately forgiving about *formatting*, because a fix that is right but badly formatted is still a fix. It is never forgiving about *correctness*. A patch applied loosely still has to compile, stop the attack and keep normal behaviour working in gates 2 to 4. The code comment puts it this way: "a mislocated hunk cannot pass unnoticed."

How gates 2–4 run a command

Gates 2, 3 and 4 each run a shell command through one helper, `_run`:

src/patch_validator/drv_loop.py · lines 603–623

```
    def _run(command: str, label: str, cwd: Optional[Path] = None) -> tuple[bool, str]:
        """Run a shell command; return (exit==0, captured output)."""
        try:
            proc = subprocess.run(
                command, shell=True, capture_output=True, text=True,
                timeout=180, cwd=str(cwd) if cwd else None,
                env=dict(os.environ, ASAN_OPTIONS="detect_leaks=0", MallocNanoZone="0"),
            )
        except subprocess.TimeoutExpired:
            return False, f"{label} timed out after 180 seconds"
        except Exception as e:  # pragma: no cover - defensive
            return False, f"{label} error: {e}"

        if proc.returncode == 0:
            return True, proc.stdout[:_FEEDBACK_LIMIT]

        return False, (
            f"{label} failed (exit code {proc.returncode})\n"
            f"--- stdout ---\n{DRVLoop._salient(proc.stdout, _FEEDBACK_LIMIT // 3)}\n"
            f"--- stderr ---\n{DRVLoop._salient(proc.stderr, 2 * _FEEDBACK_LIMIT // 3)}"
        )
```

LINE BY LINE

1. `shell=True`: the command is a full shell command line (so it can include things like `sh pov_run.sh ...`).

2. `timeout=180` : if it runs longer than 3 minutes it is killed and counted as a failure. A patch that makes the program loop forever cannot pass.
3. `ASAN_OPTIONS="detect_leaks=0"` : turn off AddressSanitizer's memory-*leak* check (forgetting to free memory), so leaks don't drown out the bugs actually being tested. Leak bugs have their own script, covered later. `MallocNanoZone=0` is a macOS setting that stops a harmless warning from the Mac memory allocator getting mixed into the output.
4. Exit code 0 means pass. Anything else means fail, and the output goes back to the agent.
5. `_salient` trims long output while keeping the useful lines. A sanitizer report starts with a long block of memory diagnostics and ends with the important `SUMMARY:` line. Simply cutting at 3,000 characters would keep the noise and lose the diagnosis. `_salient` picks out lines containing `ERROR:` , `SUMMARY:` , `runtime error:` , stack frames that name a `.c` file, and so on, and then fills any remaining space with the start of the output.

Gates 2, 3 and 4: build, attack, normal use

src/patch_validator/drv_loop.py · lines 365–398 (trimmed)

```
            # Gate 2 — build
            if build_command:
                ok, detail = self._run(self.expand(build_command, ctx), "Build", cwd=workspace)
                result.syntax_ok = GATE_PASS if ok else GATE_FAIL
                if not ok:
                    result.failure_stage = "build"
                    result.error_feedback = f"BUILD FAILED:\n{detail}"
                    return False

            # Gate 3 — proof of vulnerability replayed on the patched binary
            if pov_command:
                ok, detail = self._run(self.expand(pov_command, ctx), "PoV", cwd=workspace)
                result.pov_ok = GATE_PASS if ok else GATE_FAIL
                if not ok:
                    result.failure_stage = "pov"
                    result.error_feedback = (
                        "PROOF-OF-VULNERABILITY STILL REPRODUCES. The patched binary "
                        ...
                    )
                    return False

            # Gate 4 — regression on benign input
            if test_command:
                ok, detail = self._run(self.expand(test_command, ctx), "Regression", cwd=workspace)
                result.regression_ok = GATE_PASS if ok else GATE_FAIL
                if not ok:
                    result.failure_stage = "regression"
                    ...
                    return False
```

The three gates follow the same pattern: **if a command exists**, fill in the placeholders, run it, record pass or fail, and on failure write a clear message for the agent and stop. If no command exists, the `if` is skipped and the field stays at its starting value, `GATE_SKIPPED`.

- **Build (gate 2)** compiles the patched file with clang and the sanitizers switched on: `-fsanitize=address,undefined` adds AddressSanitizer and UndefinedBehaviorSanitizer, and `-fno-sanitize-recover=all` makes every sanitizer finding stop the program instead of just printing a warning. The result is `{bin}`. A compile error sends the compiler's message back to the agent.
- **PoV (gate 3)** replays the attack input on the *patched* program. The rule to remember: **a PoV command exits 0 only when the bug is ABSENT.** "Pass" means "the attack no longer works".
- **Regression (gate 4)** runs the patched program on ordinary, harmless input and checks it still behaves as before. In software, a **regression** is something that used to work and now doesn't. This gate catches a "fix" that breaks the program.

Gate 5: the post-patch scan

If a Semgrep runner (the static pattern scanner from an earlier chapter) was provided, the patched file is scanned again. The patch fails if it introduces a **new** kind of finding, one whose rule ID did not appear in the original file:

src/patch_validator/drv_loop.py · lines 677–691

```
        try:
            original_rules = {f.rule_id for f in self.semgrep_runner.scan_file(original_file)}
            patched_findings = self.semgrep_runner.scan_file(str(patched_file))
            new = [f for f in patched_findings if f.rule_id not in original_rules]
            if new:
                detail = "\n".join(
                    f"- {f.rule_id} at line {f.start_line}: {f.message}" for f in new[:5]
                )
                logger.warning(f"Post-patch scan found {len(new)} new finding(s)")
                return False, detail
            return True, ""
        except Exception as e:
            # A scanner malfunction must not be reported as a security verdict.
            logger.error(f"Post-patch scan error (gate not enforced): {e}")
            return True, ""
```

WATCH OUT

Look at the last line: if Semgrep crashes (an `Exception`), the function returns `True`, and the caller then records the gate as **PASS**. The comment's reasoning is that a broken scanner shouldn't reject a patch, but the honest result would be **SKIPPED**, not PASS. This is a "fails open" design: when the check breaks, the door opens. Two more details. The benchmark passes `semgrep=None`, so in benchmark runs this gate is always SKIPPED. And it compares only rule *IDs*, so a second instance of a rule that already fired in the original file isn't counted as new.

The finish line: what "fully validated" really means

src/patch_validator/drv_loop.py · lines 412–413

```
        result.fully_validated = True
        return True
```

Execution reaches these two lines when no gate returned `False`. So the exact definition is:

KEY IDEA

`fully_validated` means "no gate FAILED". It does **not** mean "every gate ran". The file's own header says the same: "A patch is `fully_validated` only if no gate FAILED." A gate that was skipped didn't fail, so it doesn't block validation.

Here is what that allows in the extreme case. If a target has no build command, no PoV command and no test command (and no Semgrep runner), then gates 2 to 5 are all skipped. Gate 1, apply, always runs. So a patch that merely *applied* to the file, and was never compiled or tested, comes out `fully_validated = True`. The overview notes the command-line tool only *warns* about this. The mitigation is in reporting. `executed_gates` lists exactly which gates ran, and reports print skipped gates as SKIPPED, so a careful reader can tell "proved by PoV" from "applied, nothing else". The single word "patched" in a summary table hides that difference.

Gate	What passing means	When SKIPPED
1 apply	the patch landed in the file (AST splice, `patch`, or content match)	never; always runs
2 build	compiles under ASan + UBSan	no build command
3 PoV	the attack command exits 0, i.e. the bug is gone	no PoV, *or* pre-flight showed the PoV doesn't reproduce
4 regression	harmless input still works (exit 0 and expected output)	no regression command
5 post-scan	no new Semgrep rule IDs	no Semgrep runner (always, in the benchmark)

5.5 Pre-flight: test the smoke alarm before trusting it

The PoV gate says "pass" when the attack command exits 0. Now imagine a PoV command that *always* exits 0, even on the original buggy program. Maybe the attack input is a little off and never reaches the bug. Maybe the sanitizer wasn't switched on. Then **every** patch passes the

PoV gate, including one that changes nothing. The gate would be a smoke alarm with no battery: silent all the time, and therefore useless.

So before any agent is called, the benchmark runs a **pre-flight** check. It builds the **unpatched** program and runs the PoV against it. On the buggy program the PoV **must fail**, because the bug is there and the attack must be seen to work:

benchmark.py · lines 145-149

```
        if target.pov_command:
            # PoV scripts exit non-zero when the vulnerability IS present.
            ok, detail = DRVLoop._run(DRVLoop.expand(target.pov_command, ctx), "PoV", cwd=workspace)
            res.pov_reproduces = not ok
            res.pov_detail = detail if not ok else "PoV did NOT reproduce on the original binary"
```

Note `pov_reproduces = not ok`: on the original program, a *failing* PoV is the good outcome. Pre-flight uses the very same `_materialise`, `expand` and `_run` helpers as the DRV loop, so it tests exactly what the gate will later test. Then:

benchmark.py · lines 100-101 and 238-239

```
    def pov_usable(self) -> bool:
        return self.build_ok and self.pov_reproduces
    ...
                # Only gate on a PoV we proved detects the real bug.
                pov_command=t.pov_command if pf.pov_usable else None,
```

If the PoV didn't reproduce the bug, the DRV loop is given `None` in its place, so gate 3 is SKIPPED for that target, and the target can't be counted as "PoV-proven". Pre-flight also runs the regression test on the original and records whether it passes, but it does *not* switch that gate off. The comment explains that for some targets, such as the off-by-one loop, every input goes through the buggy code, so the regression test can only pass once the bug is fixed.

> **ANALOGY**
>
> A drug trial that tests a cure on people who were never sick will report a 100% cure rate. Pre-flight checks that the patient is sick before the treatment is given any credit.

The standalone suites (Vanguard and the public extension) do the same thing through `preflight()` in `tests/standalone_support.py`. That file's header admits that these runners once *printed* "Executing pre-flight PoV verification..." without actually doing it.

5.6 The judge scripts: how "the attack worked" is decided

The PoV and regression commands are not Python. They are small **shell scripts** in `tests/benchmarks/harness/`. A shell script is a list of terminal commands saved in a file. For a typical target, the PoV command looks like `sh "../pov_run.sh" "{bin}" AAAA`, which means "run `pov_run.sh` with the program and the attack argument `AAAA`".

`pov_run.sh`, line by line

tests/benchmarks/harness/pov_run.sh · lines 14-39

```
ASAN_OPTIONS=detect_leaks=0
MallocNanoZone=0
export ASAN_OPTIONS MallocNanoZone

output=$("$@" 2>&1)
rc=$?

# On failure the captured output MUST be emitted. The DRV loop feeds this text
# back to the repair agent, and Agent Delta diagnoses from it. Swallowing it
# left agents with "your patch still crashes" and nothing else — no faulting
# line, no stack, no crash class — which is close to useless as feedback.
case "$output" in
    *"AddressSanitizer"*|*"UndefinedBehaviorSanitizer"*|*"runtime error:"*|*"LeakSanitizer"*)
        echo "$output"
        exit 1
```

```
        ;;
esac

# 128+N indicates death by signal N (SIGSEGV=139, SIGABRT=134, SIGBUS=138).
if [ "$rc" -ge 128 ]; then
    echo "process died on signal (exit $rc); output follows:"
    echo "$output"
    exit 1
fi

exit 0
```

LINE BY LINE

1. `ASAN_OPTIONS=detect_leaks=0` and `MallocNanoZone=0`, then `export`: set two settings and hand them to any program started from this script. Leak detection is off here (leaks have their own script).
2. `output=$("$@" 2>&1)`: the key line. `"$@"` means "all the arguments this script was given", so the first is the program and the rest are its inputs. The script runs the program with the attack input. `2>&1` merges the error stream (where sanitizers write their reports) into normal output, and `$( ... )` captures all of it into the variable `output`.
3. `rc=$?`: `$?` is the exit code of the command that just finished. It is saved in `rc` ("return code").
4. The comment explains why the output is printed on failure: the DRV loop sends this text to the agent, and without it the agent learns nothing useful.
5. `case "$output" in *"AddressSanitizer"*|... )`: a text search. The `*` wildcards mean "anything before or after". If the output contains any sanitizer's name, or `runtime error:` (UBSan's wording), a sanitizer caught the bug. The script prints the report and **exits 1: the bug still reproduces**.
6. `if [ "$rc" -ge 128 ]`: "if the exit code is greater than or equal to 128". On Unix, a program killed by a **signal** (an emergency stop from the operating system) reports 128 plus the signal number. A segmentation fault (signal 11, an illegal memory access) gives 139, and an abort (signal 6) gives 134. A crash like that also means the bug reproduced, so **exit 1**.
7. `exit 0`: otherwise, report "**mitigated**", meaning the bug did not show up.

What the program did on the attack input	pov_run.sh exits	Meaning to the gate
sanitizer report in the output (any exit code)	1	bug still present: FAIL
killed by a signal (exit 128 or more)	1	bug still present: FAIL
exit 0, clean output	0	mitigated: PASS
exit 1–127, no sanitizer text	**0**	mitigated: PASS

WATCH OUT

Look at the last row. If the patched program just quits with an error code such as 1 ("bad input, goodbye") and no sanitizer text, the script calls the bug *mitigated*. Often that is exactly right, since rejecting a malicious input is a good fix. But it also means a patch that gives up early on *every* input passes the PoV gate. Only the regression gate (normal input must still work) or hardening can catch that. For targets with no regression test, such as the Juliet cases, nothing in the gates does.

`regress_run.sh`: normal use must still work

tests/benchmarks/harness/regress_run.sh · lines 19-51 (trimmed)

```
bin="$1"
expected="$2"
shift 2

output=$("$bin" "$@" 2>&1)
rc=$?

if [ "$rc" -ne 0 ]; then
    echo "REGRESSION: benign input exited with status $rc"
```

```
    echo "$output"
    exit 1
fi

case "$output" in
    *"AddressSanitizer"*|*"UndefinedBehaviorSanitizer"*|*"runtime error:"*)
        echo "REGRESSION: sanitizer fired on benign input"
        ...
        exit 1
        ;;
esac

if [ "$expected" != "-" ]; then
    case "$output" in
        *"$expected"*) ;;
        *)
            echo "REGRESSION: expected stdout to contain '$expected'"
            ...
```

This script is stricter than `pov_run.sh`, and it needs to be. It takes the program, an **expected piece of text**, then the inputs. It fails if the exit code is anything other than 0 (`-ne` means "not equal"), if a sanitizer fired, or if the output doesn't contain the expected text. For SYN-06 the regression is `_regress("Copied string: hi", "hi")`: run with input `hi`, and the output must contain `Copied string: hi`. Passing `-` as the expected text turns off the output check.

The specialised judges

- **`pov_leak.sh`** handles memory-leak bugs (CWE-401). It switches leak detection *on* (`ASAN_OPTIONS=detect_leaks=1:exitcode=1`) and fails if `LeakSanitizer: detected memory leaks` appears, if any other sanitizer fires, or if the program exits non-zero for any reason. Note that it is stricter than `pov_run.sh`: here any non-zero exit fails. Its header records a correction. The project once claimed LeakSanitizer didn't work on Apple-silicon Macs, but measurement showed it does; it had simply been switched off.
- **`pov_injection.sh`** handles command-injection bugs, where an attacker sneaks an extra shell command into the program's input. It runs the program with the input `127.0.0.1; touch $marker`. If the program is vulnerable, the shell runs `touch`, which creates an empty "marker" file. The script then checks whether the marker exists: it exits 1 if so (still exploitable) and 0 if not. The proof is a physical side effect, not a crash.
- **`secret_scan.sh`** handles hardcoded passwords or keys (CWE-321/798). There is no crash to find here. The script runs `strings` on the compiled program (a tool that lists every readable piece of text inside a binary) and fails if the forbidden secret is still in it. It then runs a **vacuity guard**: the program must still exit 0 and print the expected text. As the comment says, "a patch that deleted the whole function would 'pass' by removing the string along with the feature."

TRY IT YOURSELF

From the repo root, with Homebrew clang installed, you can see pre-flight's logic for yourself:

```
clang -fsanitize=address,undefined -fno-sanitize-recover=all -g -O0 tests/demo_vulns/06_off_by_one_loop.c -o /tmp/syn06
```

```
sh tests/benchmarks/harness/pov_run.sh /tmp/syn06 AAAA; echo "exit=$?"
```

You should see an AddressSanitizer "heap-buffer-overflow" report followed by `exit=1`: the bug reproduces, so this PoV is usable. Then try `sh tests/benchmarks/harness/regress_run.sh /tmp/syn06 "Copied string: hi" hi; echo "exit=$?"`. It also fails on the unpatched program, because this bug triggers on every input.

5.7 Hardening: trying to break a patch that passed

Passing the gates shows that *one* attack input no longer works and *one* normal input still does. The comment at the top of `hardening.py` names two cheap ways to beat that:

1. **PoV overfitting.** In machine learning, "overfitting" means memorising the test instead of learning the subject. Here it means a patch that blocks only the exact attack input. The comment's example is `if (size == 4) return;`, which stops an attack of length 4 and leaves length 5 just as dangerous.
2. **Functionality gutting.** Disable the dangerous code entirely, with an early `return` or an emptied loop. Nothing crashes, because nothing happens.

Both cheats compile, pass the PoV and (often) pass a single regression test. The comment says plainly that "neither is hypothetical": a model rewarded for "make the gate go green" can stumble into either. Hardening is therefore a **falsification** stage. It can't prove a patch correct; it can only expose one that is wrong. It has three tools.

Tool 1: AdversarialRefuzz, which fuzzes the fixed program again

Recall that fuzzing throws huge numbers of generated inputs at a program to find crashes. `AdversarialRefuzz` fuzzes the **patched** code. If the fix is real, there should be no crash left to find. If the fix only covered one input, the fuzzer will usually find a neighbouring input that still crashes.

src/patch_validator/hardening.py · lines 280–340 (trimmed)

```
        harness = self.harness_dir / f"{target_id}.c"
        if not harness.exists():
            return False, f"no fuzz harness for {target_id}"
        ...
        if not clang:
            return False, "no clang with a libFuzzer runtime available"

        with tempfile.TemporaryDirectory(prefix="kavach_refuzz_") as tmp:
            ...
            build = subprocess.run(
                [
                    clang, "-fsanitize=fuzzer,address,undefined",
                    "-fno-sanitize-recover=all", "-fno-omit-frame-pointer", "-g", "-O1",
                    f"-Dmain=kavach_disabled_main_{mangled}",
                    str(harness), str(patched_source), "-o", str(binary),
                ],
                capture_output=True, text=True, timeout=300,
            )
            if build.returncode != 0:
                return False, f"re-fuzz build failed: {build.stderr[:300]}"
            ...
            try:
                subprocess.run(
                    [str(binary), str(corpus), f"-max_total_time={seconds}",
                     f"-artifact_prefix={artifacts}/"], ...)
            except subprocess.TimeoutExpired:
                return False, "re-fuzz exceeded its budget"

            crashes = [p for p in artifacts.iterdir() if p.is_file()]
            if crashes:
                ...
                return True, (...)
            return False, f"no new crash in {seconds}s of re-fuzzing"
```

LINE BY LINE

1. A **fuzz harness** is required: a small C file in `tests/fuzz_harnesses/` that feeds the fuzzer's bytes into the vulnerable function. Only nine targets have one. Without a harness the function returns `(False, "no fuzz harness ...")`.
2. It also needs a clang that includes **libFuzzer**. Apple's built-in clang doesn't, so the code looks for Homebrew's.
3. Build: `-fsanitize=fuzzer,address,undefined` builds a self-running fuzzer with the sanitizers on. `-Dmain=kavach_disabled_main_...` is a trick: it renames the program's own `main` function so that it doesn't clash with the fuzzer's built-in `main`.
4. The fuzzer runs for `seconds` (20 by default), saving any crashing input into the `crashes` folder (`-artifact_prefix`). If the caller supplies the original crash as a seed, the fuzzer starts from it, right next to the bug. (The main benchmark's `_harden_patch` doesn't pass a seed, so it starts from an empty corpus.)
5. If any crash file appears, the result is `True`: **a new crash was found and the patch is overfitted.**

WATCH OUT

This function returns `(False, reason)` both for "fuzzed and found nothing" and for "**couldn't build**" or "**ran out of time**". Now look at how `harden()` reads the result:

src/patch_validator/hardening.py · lines 441–448

```
        found, detail = self.refuzzer.check(
            target_id, patched_source, seed_crash,
            seconds=refuzz_seconds, libfuzzer_clang=libfuzzer_clang,
        )
        if "no fuzz harness" not in detail and "no clang" not in detail:
            verdict.overfitting_checked = True
            verdict.overfitted = found
            verdict.new_crash_detail = detail
```

Only two reasons ("no fuzz harness", "no clang") are treated as "the check didn't happen". A **build failure** or a **timeout** still sets `overfitting_checked = True` with `overfitted = False`, which the code reads as "fuzzed, nothing found". That counts as evidence, so a patch that was never actually fuzzed can be labelled SURVIVED. In the benchmark, that also promotes its memory pattern to "hardened".

Tool 2: DifferentialTester, which compares old and new side by side

"Differential" testing means running two versions on the same inputs and looking for *differences*. The idea: a security fix should change behaviour **only** on attack inputs. On any input the original program handled fine, the patched program should do exactly the same thing. First, 25 harmless inputs are generated:

src/patch_validator/hardening.py · lines 147–167

```
    @staticmethod
    def generate_inputs(count: int, seed: int = 1337) -> list[list[str]]:
        rng = random.Random(seed)
        inputs: list[list[str]] = []
        alphabet = string.ascii_letters + string.digits

        for _ in range(count):
            shape = rng.choice(["short_str", "num", "two_nums", "three_args"])
            if shape == "short_str":
                n = rng.randint(1, 12)
                inputs.append(["".join(rng.choice(alphabet) for _ in range(n))])
            elif shape == "num":
                inputs.append([str(rng.randint(0, 64))])
            elif shape == "two_nums":
                inputs.append([str(rng.randint(0, 4)), str(rng.randint(0, 4))])
            else:
                inputs.append([
                    str(rng.randint(5, 20)), str(rng.randint(0, 4)),
                    "".join(rng.choice(alphabet) for _ in range(rng.randint(1, 8))),
                ])
        return inputs
```

The random generator is **seeded** with 1337. A seed fixes the sequence of "random" choices, so every run produces the same 25 inputs and results can be reproduced. Each input is one of four shapes: a short string of 1–12 letters and digits; a number from 0 to 64; two small numbers; or three arguments. They are all small and harmless on purpose. Then the comparison:

src/patch_validator/hardening.py · lines 213–250 (trimmed)

```
        compared = 0
        for args in inputs:
            orig_code, orig_out, orig_fired = self._run(original_binary, args)
            if orig_fired or orig_code < 0:
                continue                      # original was already broken here
            ...
```

```
            recheck_code, recheck_out, _ = self._run(original_binary, args)
            if (recheck_code, recheck_out) != (orig_code, orig_out):
                continue

            compared += 1

            new_code, new_out, new_fired = self._run(patched_binary, args)

            if new_fired:
                return True, (...), compared
            if new_code != orig_code:
                return True, (...), compared
            if new_out != orig_out:
                return True, (...), compared

        return False, "", compared
```

LINE BY LINE

1. Run the **original** program on the input. If a sanitizer fired, or it timed out or failed to start (a negative code), skip this input. The original was already broken here, and behaviour on a broken input is exactly what the patch is allowed to change.
2. The **determinism screen**: run the original a second time. If it disagrees with itself (one sample program calls `ping`, whose output varies with the network), the input can't be used for a fair comparison, so it is skipped.
3. `compared += 1`: this input counts.
4. Run the **patched** program. It **diverges** (fails) if a sanitizer fires where the original was clean, if the exit code differs, or if the output text differs.
5. Return whether it diverged, a description, and how many inputs were actually compared.

One detail matters: before comparing, `_normalise` replaces the program's own file path in the output with `<program>`. Many C programs print their own name (as in "Usage: ./prog ..."). The original and patched binaries live at different paths, so without this every comparison would show a "difference" that isn't one.

Tool 3: the evasion battery

Some bugs aren't crashes. In a **path traversal** bug, a user asks for a file like `../secret.txt` and escapes the folder they are allowed to read (`..` means "go up one folder"). A fix adds an input check. Re-fuzzing can't test this, because there is no crash to find. The way such a fix fails is a *cleverer* input that slips past the check. So the target gets an **evasion battery**: a script of known bypass tricks. The one in `tests/vanguard_nightmare/evasion_battery.sh` tries payloads such as `rtos://..../secret.txt` (survives a filter that removes `..` only once), `rtos://sub/../../secret.txt` (escapes through a real-looking subfolder) and `rtos://..//secret.txt` (a doubled slash). It exits 1 if any of them still opens the secret file. In `harden()`:

src/patch_validator/hardening.py · lines 430–438

```
            built = subprocess.run(cmd, shell=True, capture_output=True, text=True, timeout=300)
            if built.returncode != 0:
                verdict.evasion_detail = f"could not build for evasion testing: {built.stderr[:200]}"
            else:
                ecmd = evasion_command.replace("{bin}", str(ebin)).replace("{workspace}", str(ework))
                proc = subprocess.run(ecmd, shell=True, capture_output=True, text=True, timeout=180)
                verdict.evasion_checked = True
                verdict.evaded = proc.returncode != 0
                verdict.evasion_detail = (proc.stdout + proc.stderr)[:600]
```

Unlike the re-fuzz case, a build failure here is handled honestly: `evasion_checked` stays `False`. Only an actual run of the battery counts, and any non-zero exit means **evaded**.

Behaviour contracts: preserve, restrict, replace

Differential testing assumes a fix should *not* change behaviour on normal inputs. That isn't true for every kind of fix. So every target declares a **behaviour contract** (`Target.behaviour_contract`, default `"preserve"`), which states what kind of change the fix is allowed to make:

Contract	Meaning	Differential testing?	Example
`preserve`	output must be identical on any input the original handled cleanly	yes	buffer overflow fixes: normal strings should copy the same as before
`restrict`	the fix may legitimately reject inputs the original accepted	no; evasion battery / re-fuzz instead	path traversal: rejecting `../` is the fix working
`replace`	output *must* change	no	hardcoded key: the program must stop using the old secret, so identical output would mean the fix did nothing

In `harden()` the decision is simply `if builds_ok and behaviour_contract == "preserve":` run the comparison. For other contracts the code records a note saying why differential testing doesn't apply, rather than silently doing nothing.

From observations to a verdict

All results go into a `HardeningVerdict`. Three computed properties turn the raw observations into decisions:

src/patch_validator/hardening.py · lines 72–104 (trimmed)

```
    @property
    def falsified(self) -> bool:
        """True when a check actually broke the patch. This is what downgrades a result."""
        return self.overfitted or self.diverged or self.evaded

    @property
    def has_evidence(self) -> bool:
        """
        True when at least one falsification attempt actually produced a result.
        ...
        """
        return (
            self.overfitting_checked
            or self.evasion_checked
            or (self.differential_checked and self.inputs_tested > 0)
        )

    @property
    def survived(self) -> bool:
        """
        True when the patch was actually attacked and held.
        ...
        """
        return self.has_evidence and not self.falsified
```

LINE BY LINE

1. `falsified`: true if **any** attack worked: re-fuzz found a crash (`overfitted`), behaviour differed (`diverged`), or a bypass got through (`evaded`).
2. `has_evidence`: true if at least one attack **really ran**. Note the special condition on differential testing: it counts only if it compared **more than zero** inputs. If every generated input crashed the original (as with SYN-06, where every input hits the buggy loop), all 25 are skipped, and the message "differential(0 inputs): behaviour preserved" proves nothing.
3. `survived`: attacked **and** held. The docstring stresses that this is "deliberately NOT the negation of `falsified`". "Not shown to be broken" is not the same as "survived an attack".

The human-readable label comes from `hardening_label` in `tests/standalone_support.py`:

tests/standalone_support.py · lines 170–187 (trimmed)

```
    if verdict.skipped_reason:
        return "NOT HARDENED"
    if verdict.overfitted:
        return "FALSIFIED (overfitted)"
```

```
    if verdict.evaded:
        return "FALSIFIED (validation bypassed)"
    if verdict.diverged:
        return "FALSIFIED (behaviour changed)"
    ...
    return "SURVIVED" if verdict.has_evidence else "NOT HARDENED"
```

Label	Computed as	Effect on the result
NOT HARDENED	a `skipped_reason` is set (for example, the winning patch couldn't be re-applied, or the program couldn't be built for comparison), *or* no check produced evidence	status stays `patched`, but it doesn't earn the hardening bar
FALSIFIED (overfitted / validation bypassed / behaviour changed)	`overfitted`, `evaded`, or `diverged` is true, checked in that order	benchmark downgrades the status to `unresolved_hardening`
SURVIVED	not skipped, not falsified, and `has_evidence`	status stays `patched`, counted as hardened

In `benchmark.py` the downgrade is keyed on `verdict.falsified`, not on "did not survive". The comment explains: "a target with no applicable falsification check has not been shown wrong, so it keeps its gate result — it simply does not earn the hardening bar."

> **KEY IDEA**
>
> There are three different outcomes, and the project keeps them apart: "we attacked it and it broke" (FALSIFIED), "we attacked it and it held" (SURVIVED), and "we never really attacked it" (NOT HARDENED). The header of `tests/standalone_support.py` admits that the standalone runners once mixed up the last two. If re-applying the patch failed, they reported `SURVIVED` for a patch that had never been hardened.

5.8 Formal verification with CBMC: a proof, within limits

Everything so far has been **empirical**: run the program on chosen inputs and watch what happens. The comment at the top of `formal.py` states the limitation: "Neither can say anything about the inputs nobody tried." **CBMC** (the C Bounded Model Checker) works differently. It turns the C code into a large logic formula and asks a solver: "Is there *any* input that breaks a safety rule?" If the answer is no, that covers **every** input, not just the ones tested.

The catch is the word **bounded**. Loops are "unwound" only a fixed number of times. Unwinding means copying the loop body out N times, as if the loop ran at most N rounds. A bug that needs more rounds than that is outside the proof. So the honest claim is "no violation exists within this bound", never "the code is correct".

The flags, one by one

src/patch_validator/formal.py · lines 57–65

```
DEFAULT_CHECKS = (
    "--bounds-check",             # array index out of range
    "--pointer-check",            # invalid / null / out-of-object dereference
    "--conversion-check",         # lossy integer conversion (truncation)
    "--signed-overflow-check",    # signed arithmetic overflow
    "--unsigned-overflow-check",  # unsigned wrap-around
    "--div-by-zero-check",
    "--unwinding-assertions",     # tell us when the bound was insufficient
)
```

Flag	Plain English
`--bounds-check`	Never read or write an array outside its slots, e.g. slot 10 of a 10-slot array (slots are 0–9).
`--pointer-check`	Never follow a pointer (a memory address) that is null, already freed, or pointing outside the object it belongs to. This catches heap overflows like SYN-06's.
`--conversion-check`	Never squeeze a number into a smaller type and lose part of it (like writing 300 into a box that only holds 0–255).

Flag	Plain English
`--signed-overflow-check`	Signed numbers (which can be negative) must never go past their maximum or minimum. In C that is undefined behaviour.
`--unsigned-overflow-check`	Unsigned (never-negative) numbers must never wrap round, as when 0 − 1 becomes 4,294,967,295. That is legal C but is a common cause of bugs such as CWE-191 underflow.
`--div-by-zero-check`	Never divide by zero.
`--unwinding-assertions`	The honesty flag. If a loop could run more times than the unwind limit, CBMC *reports* it instead of quietly ignoring the extra rounds. Without it, too small a limit would give a false "proven".

The full command is assembled in `verify()`: `cbmc <all flags above> --unwind N --function harness <harness.c> <patched source>`. Here `--unwind N` sets the loop bound (default 16; SYN-06 uses 70), and `--function harness` tells CBMC to start from a function called `harness` rather than from `main`. There is a 180-second time limit.

Why a harness, and what one looks like

Why not just check `main`? The harness README answers from experiment. Checking `main` with unrestricted command-line arguments makes CBMC report failures *inside its own model of the C library* (for example in `strtol`), and "those failures persist on a *correctly patched* file". The original and the fixed SYN-11 both reported `VERIFICATION FAILED`. A check that fails whether or not the code is fixed tells you nothing. So each provable target gets a small **proof harness** that calls only the vulnerable function. Before CBMC runs, `strip_main` removes `main` from a scratch copy of the patched file. It walks through the text matching `{` with `}`, skipping braces inside strings and comments. A simpler pattern search was rejected because it could silently leave a one-line `main` behind.

Here is the vulnerable SYN-06 function, then its harness:

tests/demo_vulns/06_off_by_one_loop.c · lines 4-17

```
void copy_and_null_terminate(const char* src, size_t size) {
    if (size == 0 || size > 1024) return;
    char* dest = (char*)malloc(size);
    if (!dest) return;
    for (size_t i = 0; i <= size; i++) {
        if (i == size) {
            dest[i] = '\0';
        } else {
            dest[i] = src[i];
        }
    }
    printf("Copied string: %s\n", dest);
    free(dest);
}
```

The bug: it allocates `size` bytes (slots 0 to size−1), but the loop runs while `i <= size`, so on the last round it writes `'\0'` into slot `size`, one past the end. That's the classic off-by-one (CWE-193).

tests/benchmarks/cbmc_harnesses/SYN-06-OFF-BY-ONE.c · lines 1-15

```
/* Proof: copy_and_null_terminate must not write outside its allocation for any
 * size it accepts. The caller passes strlen(argv[1]), so any size is possible;
 * the function's own guard rejects 0 and >1024. */
#include <stddef.h>
void copy_and_null_terminate(const char *src, size_t size);
unsigned long nondet_ulong(void);

#define SRC 64
static char source[SRC];

void harness(void) {
    size_t size = nondet_ulong();
    __CPROVER_assume(size <= SRC);      /* src really holds `size` bytes */
```

```
    copy_and_null_terminate(source, size);
}
```

LINE BY LINE

1. `void copy_and_null_terminate(...);` : a declaration that says "this function exists elsewhere" (in the patched file CBMC also loads).
2. `unsigned long nondet_ulong(void);` : declared but **never defined**. CBMC treats the return value of such a body-less function (by convention named `nondet_...`, for "non-deterministic") as **any possible value**. This is how a harness says "consider every input".
3. `static char source[SRC];` : a 64-byte source buffer.
4. `size_t size = nondet_ulong();` : `size` can be anything at all.
5. `__CPROVER_assume(size <= SRC);` : a **precondition**. CBMC considers only cases where it holds. It is a *true* statement about the real program: the caller passes `strlen(argv[1])`, so `src` really holds `size` bytes, and the harness's buffer holds 64.
6. Call the function. CBMC now checks, for every `size` from 0 to 64, that no flag's rule is broken anywhere inside.

On the unpatched code CBMC finds a counterexample: with any size from 1 to 64 the function writes `dest[size]` outside the `malloc(size)` block. After a correct patch (for example allocating `size + 1`), it reports `VERIFICATION SUCCESSFUL`. The target sets `cbmc_unwind=70` because the loop can run up to 65 times (sizes up to 64, plus the final round). The default of 16 would leave the unwinding assertion failing.

WATCH OUT

Assumptions are powerful and dangerous. The README explains why SYN-03 has **no** harness. Its overflow needs about 130 million records, around 4 GB. Assuming a realistic limit like `n <= 8` makes CBMC report PROVEN, "truthfully, because no overflow can occur under that assumption, but the result says nothing about the vulnerability." That is a **vacuous proof**. Leaving the assumption out instead produces violations no patch could fix. Both choices are wrong, so the project leaves SYN-03 unproven. The rule: assume only what the real caller actually guarantees. Only three targets (SYN-06, SYN-11, SYN-12) have harnesses.

The four statuses

src/patch_validator/formal.py · lines 176-202

```
        verdict = _VERDICT.search(out)
        if not verdict:
            result.status = "inconclusive"
            result.detail = (
                "cbmc produced no verdict — the harness may not link against the "
                f"target (first lines: {out.strip().splitlines()[:2]})"
            )
            return result

        if verdict.group(1) == "SUCCESSFUL":
            result.status = "proven"
            return result

        # FAILED. Distinguish "the bound was too small to decide" from "a real
        # violation exists" — reporting the former as a violation would blame a
        # patch for our own configuration.
        real = [f for f in failures if "unwinding assertion" not in f]
        if not real and result.bound_exhausted:
            result.status = "inconclusive"
            result.detail = (
                f"unwind={unwind} was insufficient to fully explore the loops; "
                "no violation found within it, but the bound was not exhausted"
            )
        else:
            result.status = "violated"
            result.detail = f"{len(real)} property violation(s)"
        return result
```

Status	How it is reached	Counts as a pass?
proven	CBMC printed `VERIFICATION SUCCESSFUL`. Because `--unwinding-assertions` is on, this also means the bound was enough.	yes, within the bound
violated	`VERIFICATION FAILED` with at least one failure that is *not* an unwinding assertion: a real counterexample	no; the benchmark sets the status to `unresolved_formal`
inconclusive	failed *only* on unwinding assertions (the bound was too small), or CBMC timed out, or it printed no verdict at all (for example the harness didn't link)	no
unavailable	no `cbmc` program installed, no harness file, or the patch couldn't be re-applied (the default status)	no

The `violated` / `inconclusive` split is fair to the patch: "the bound was too small to decide" is the configuration's fault, not the patch's, so it isn't reported as a violation. The `formal.py` header also records a measured limit: CBMC "is not a superset of AddressSanitizer". For SYN-18, a write into a struct member is checked only at the level of the whole struct, so CBMC calls it safe, just as ASan does. The two tools complement each other; neither replaces the other.

5.9 The cheat detector: three patches, one honest

`tests/cheat_detector/run_cheat_detector.py` is a self-contained demonstration that needs no AI and no API key. It writes three patches by hand for one bug, runs them through the gates, then through the **real** hardening engine, with no fakes or mocks. Its docstring sets out the logic: "Read the CORRECT row first. A detector that rejects everything is useless — the honest fix has to survive, or the other two rejections prove nothing."

The vulnerable program

tests/cheat_detector/run_cheat_detector.py · lines 51–66

```
VULNERABLE = """#include <stdio.h>
#include <stdlib.h>
#include <string.h>
void copy_it(const char* src, size_t size) {
    char* dest = (char*)malloc(64);
    if (!dest) return;
    for (size_t i = 0; i < size; i++) dest[i] = src[i];   /* overflows when size > 64 */
    dest[size < 64 ? size : 63] = '\\0';
    printf("copied:%s\\n", dest);
    free(dest);
}
int main(int argc, char** argv) {
    if (argc > 1) copy_it(argv[1], strlen(argv[1]));
    return 0;
}
"""
```

`copy_it` always allocates 64 bytes but copies `size` bytes. Any input longer than 64 characters overflows. Shorter inputs work properly and print `copied:` followed by the text. A comment points out that this working behaviour is deliberate: without it, differential testing would have nothing to compare. The PoV is an input of 80 `A`s (`POV_LENGTH = 80`).

The three patches

tests/cheat_detector/run_cheat_detector.py · lines 68–85

```
CORRECT = VULNERABLE.replace(
    "    char* dest = (char*)malloc(64);",
    "    char* dest = (char*)malloc(size + 1);",
).replace(
    "    dest[size < 64 ? size : 63] = '\\0';",
    "    dest[size] = '\\0';",
)
```

```
OVERFITTED = VULNERABLE.replace(
    "    for (size_t i = 0; i < size; i++)",
    f"    if (size == {POV_LENGTH}) size = 63;   /* silence the known PoV only */\n"
    "    for (size_t i = 0; i < size; i++)",
)

GUTTED = VULNERABLE.replace(
    "    char* dest = (char*)malloc(64);",
    "    return;\n    char* dest = (char*)malloc(64);",
)
```

- **CORRECT** allocates exactly enough room (`size + 1`, the extra byte for the end-of-string marker) and puts the marker at `dest[size]`. This is the real fix.
- **OVERFITTED** adds one line: "if the length is exactly 80, pretend it's 63". It blocks the known attack and nothing else.
- **GUTTED** puts `return;` at the top of the function, so the function never does anything.

A safety check follows (lines 88–89): each patched text must differ from the original. If a `.replace` silently matched nothing, the demo would be testing the unchanged bug.

Step 1: the gates can't tell them apart

`gates_pass` builds each candidate with sanitizers and runs two checks: the PoV (80 `A`s; pass if exit 0 and no AddressSanitizer text) and a regression run with input `hello` (pass if exit 0). Note that this simplified regression check looks at the exit code only, not the output.

Patch	PoV: 80 A's	Regression: "hello"
CORRECT	allocates 81 bytes, copies 80, no overflow: **PASS**	prints `copied:hello`, exit 0: **PASS**
OVERFITTED	size 80 becomes 63, copies 63 into 64, no overflow: **PASS**	size 5, unchanged path: **PASS**
GUTTED	returns immediately, nothing happens: **PASS**	prints nothing, but still exits 0: **PASS**

The program prints: "Gates alone: could not tell them apart". (With the stricter `regress_run.sh`, which checks for expected text, GUTTED would have failed regression. The demo's simpler check makes the point that a weak regression test lets gutting through.)

Step 2: hardening tells them apart

The demo writes a fuzz harness for `copy_it` that accepts any length from 1 to 200 (lines 95–109), then calls `harden()` with a 10-second re-fuzz, 25 differential inputs, and the default `preserve` contract. The comment explains why the harness is shipped with the demo: without it, re-fuzz would report "no fuzz harness" and skip, and "an overfitted patch then sails through, because differential testing alone cannot see it."

Patch	Differential (25 small inputs)	Re-fuzz (lengths 1-200)	Result
CORRECT	all generated inputs are short (at most 12 characters), so the original works on them; the patched output is identical: *preserved*	never overflows at any length: *no crash*	**SURVIVED**
OVERFITTED	short inputs never hit the `size == 80` line, so output is identical: *preserved* (blind to the cheat)	any length from 65 to 200 except 80 still overflows, and the fuzzer quickly finds one: *new crash*	**FALSIFIED** by adversarial re-fuzzing
GUTTED	original prints `copied:...`, patched prints nothing: *DIVERGED*	nothing ever runs, so no crash (blind to the cheat)	**FALSIFIED** by differential testing

> **KEY IDEA**
>
> Each cheat is caught by a *different* tool, and each tool is blind to the other cheat. Re-fuzzing can't see gutting, because a function that does nothing never crashes. Differential testing can't see overfitting, because ordinary inputs never reach the special case. That is why both exist. Just as important, the honest fix passes both, so the detector isn't simply rejecting everything.

The script ends with a strict pass condition (lines 243–246): it exits 0 only if **every** cheat was falsified **and** every honest fix `survived`. Its labels follow the same logic as `hardening_label`: FALSIFIED if `v.falsified`, otherwise SURVIVED if `v.survived`, otherwise NOT HARDENED.

TRY IT YOURSELF

From the repo root (needs clang with libFuzzer, e.g. Homebrew LLVM, and the Python packages installed; no API key):

```
python -m tests.cheat_detector.run_cheat_detector
```

Expect a table with PASS/PASS in the gate columns for all three rows, then FALSIFIED for OVERFITTED and GUTTED and SURVIVED for CORRECT, ending with "PASS — every cheat caught, the honest fix untouched." It takes roughly half a minute because of the re-fuzz runs. Try editing `OVERFITTED` to special-case a different length and run it again.

5.10 Honest weaknesses

The design is careful, but not airtight. Each of these comes straight from the code you've just read:

1. **Post-scan fails open.** If Semgrep throws an exception, `_post_patch_scan` returns `True` and the gate is recorded as PASS, not SKIPPED (`drv_loop.py` lines 688–691).
2. **"Fully validated" with zero real gates.** `fully_validated` means "nothing failed". With no build, PoV or test command, a patch that merely *applied* counts as validated. Only `executed_gates` and the SKIPPED labels in reports reveal this.
3. **Re-fuzz failures count as evidence.** A re-fuzz *build failure* or *timeout* returns `(False, msg)`, and `harden()` still sets `overfitting_checked = True`. The patch can then be labelled SURVIVED with no fuzzing done. In the benchmark this also marks its memory pattern "hardened", raising its confidence.
4. **`pov_run.sh` treats exit codes 1–127 as mitigated.** A patch that makes the program quit early with an ordinary error code passes the PoV gate. Only regression testing or hardening can catch that, and some targets (e.g. Juliet) have no regression test.
5. **Differential testing can compare nothing.** For targets whose every input hits the bug (like SYN-06), all inputs are skipped. `has_evidence` handles this correctly, but such targets then rely entirely on re-fuzzing, which needs one of only nine harnesses.
6. **Formal proof covers three targets only**, and only within the unwind bound and the harness's assumptions.
7. **Commands run with `shell=True`.** Fine for the project's own trusted target list; dangerous if anyone fed in untrusted target definitions.

CHAPTER RECAP

- The **DRV loop** asks an agent for a patch, tests it, and on failure feeds the exact error, a ledger of past rejected attempts (WorkingMemory) and possibly a Critic diagnosis back into the next try. It stops at the first validated patch, on budget exhaustion, or when the agent keeps repeating itself.
- Every attempt runs in a fresh **temp-folder workspace**; command templates are filled in with `{src}`, `{srcdir}`, `{bin}`, `{workspace}` by plain find-and-replace.
- Five gates in order: **apply** (AST splice, then `patch` exact then fuzzy, then unique content match), **build** (sanitizers on), **PoV** (exit 0 = bug gone), **regression** (normal input still works), **post-scan** (no new Semgrep rules). Missing commands mean **SKIPPED**.
- `fully_validated` = **no gate failed**, not "all gates ran".
- **Pre-flight** requires the PoV to *fail* on the unpatched program; otherwise the PoV gate is disabled for that target.
- **Hardening** attacks passing patches: re-fuzzing (catches overfitting), differential testing on 25 seeded inputs (catches gutting, for `preserve` targets), and evasion batteries (catches validation bypasses). Labels: SURVIVED, FALSIFIED, NOT HARDENED.
- **CBMC** gives a bounded proof over all inputs within a harness's honest assumptions: proven, violated, inconclusive, or unavailable.
- The **cheat detector** shows that gates alone pass all three patches, while hardening catches both cheats with different tools and lets the honest fix through.

CHECK YOURSELF

1. A target has a build command and a PoV command but no regression command. The patch compiles and the PoV exits 0. What are the values of `pov_ok`, `regression_ok` and `fully_validated`?
2. Why would the PoV gate be worthless if pre-flight were removed? Describe a PoV that would let a do-nothing patch through.
3. The patched program, given the attack input, prints "invalid length" and exits with code 2. What does `pov_run.sh` return, and is that necessarily the right answer?
4. Which hardening tool catches the OVERFITTED patch, which catches GUTTED, and why can't each tool catch the other cheat?
5. CBMC reports `VERIFICATION FAILED`, but the only failures listed are unwinding assertions. What status does `formal.py` assign, and why not "violated"?

CHAPTER 6

Memory, Benchmarks, Reports and Running It

The earlier chapters followed one bug through the machine: how it is found, how the three patching agents (Alpha, Beta, Gamma) write candidate fixes, and how the deterministic "gates" (apply, build, PoV, regression, re-scan) decide whether a fix is accepted. This last chapter covers everything *around* that core:

- **Memory**: how the system remembers what worked before, and how much of that memory is really used.
- **Reports**: how results are written down so that nobody, human or machine, can mistake a check that never ran for a check that passed.
- **The two front doors**: the everyday command-line tool `src/main.py`, and the measuring harness `benchmark.py`.
- **The benchmark suites**: the collections of buggy programs used to score the system, what each one measures, and the honest limits on what the headline numbers mean.
- **The unit tests**: the small automatic checks that guard the project's honesty rules.
- **How to run it yourself**: installing the tools and a graded list of commands, from "completely safe, no API key needed" up to the full benchmark.

KEY IDEA

Everything in this chapter serves one principle that runs through the whole project: **never report a number you did not measure, and never let "did not run" look like "passed".** You will see this rule in the memory system (only proven fixes are remembered), in the reports (skipped checks print as SKIPPED), in the benchmark (pre-flight refuses to score a proof-of-vulnerability that never reproduced) and in the tests (several exist only to pin this rule in place).

6.1 Agent memory: the four notebooks

A quick reminder of two terms. A **CWE** ("Common Weakness Enumeration") is a catalogue number for a *kind* of bug, for example CWE-190 for integer overflow or CWE-416 for use-after-free. A **crash class** is the name the sanitizer prints when a program misbehaves, for example `heap-buffer-overflow` or `double-free`. Both are short labels for "what sort of bug is this?", and memory is organised around them.

The memory code lives in one file, `src/memory/store.py`. Its opening comment says it follows a four-way split that agent researchers have settled on: *working*, *episodic*, *semantic* and *procedural* memory. These words come from psychology, but here they just mean four different notebooks with different purposes.

ANALOGY

Picture a car repair workshop.

- **Working memory** is the sticky note on the car you are fixing right now: "tried replacing the fuse, still dead; tried the relay, still dead". It is thrown away when the car leaves.
- **Episodic memory** is the logbook at the front desk: one line per car that came in, what happened, how long it took.
- **Semantic memory** is the binder of "fixes that definitely worked", organised by symptom: "for this model with this warning light, the fix was X".
- **Procedural memory** is the manager's tally sheet: "Asha fixes electrical faults fastest; Ravi is best at brakes."

All four exist in the code. They are **not** all used equally, and the project's own README admits this. Here is the honest picture:

Tier	Class in `store.py`	Where it is stored	What it holds	Actually used to make decisions?
Working	`WorkingMemory`	Only in the computer's memory while one repair loop runs	A list of rejected attempts for one bug, and which gate rejected each one	**Yes.** Shown to the agent in every retry, and used to spot repeated attempts.
Episodic	`EpisodicMemory`	`benchmark_workspace/memory/episodes.jsonl`	One line per bug processed: outcome, winner, cost, attempts	**No.** It is written, and only counted for a statistics line. Nothing reads it back to make a decision.

Tier	Class in `store.py`	Where it is stored	What it holds	Actually used to make decisions?
Semantic	`SemanticMemory` + `FixPattern`	`benchmark_workspace/memory/semantic.json`	Fix patterns that passed the gates, keyed by CWE and crash class, each with a confidence score	**Yes**, when you run with `--memory`. Up to two patterns are pasted into the agents' prompt.
Procedural	`ProceduralMemory`	`benchmark_workspace/memory/procedural.json`	Per CWE: attempts, successes, which agent won, how many iterations it took	**No.** Recorded faithfully, but the two functions that would use it (`preferred_agent_order`, `suggested_iterations`) are only called by unit tests.

Working memory: the attempt ledger

Working memory solves a specific problem the authors observed. Agents would bounce between two rejected fixes ("not enough", then "breaks other things", then "not enough" again) because each retry only saw the *latest* failure. The fix is a ledger: every rejected attempt is written down as an `Attempt` record (iteration number, agent name, a one-line summary of the patch, which gate rejected it, and the Critic's verdict if there was one). The ledger is then pasted into the next prompt.

src/memory/store.py · lines 157–175

```
    def render(self) -> str:
        """Render the ledger for injection into the next prompt."""
        if not self.attempts:
            return ""
        lines = ["--- ATTEMPTS ALREADY REJECTED (do not repeat these) ---"]
        for a in self.attempts[-self.MAX_RENDERED:]:
            verdict = f" [critic: {a.critic_verdict}]" if a.critic_verdict else ""
            lines.append(
                f"  #{a.iteration} ({a.agent}) rejected by {a.rejected_by}{verdict}: {a.summary}"
            )
        lines.append(
            "Your next patch must be materially different from every attempt above."
        )
        return "\n".join(lines)

    def has_tried(self, patch: str) -> bool:
        """True if a materially identical patch was already rejected."""
        gist = self.summarise_patch(patch)
        return any(a.summary == gist for a in self.attempts)
```

LINE BY LINE

1. `render` turns the ledger into plain text for the AI. With no attempts yet it returns an empty string, so nothing is added to the prompt.
2. It writes a heading, then one line per attempt. `self.attempts[-self.MAX_RENDERED:]` means "only the last 6" (`MAX_RENDERED = 6`), so the prompt cannot grow forever.
3. Each line reads like `#2 (alpha) rejected by pov [critic: insufficient]: …summary…`.
4. It ends with a direct instruction: the next patch must be "materially different".
5. `has_tried` answers "have we already seen this exact patch fail?" It compares one-line summaries rather than whole patches.

The summary comes from `summarise_patch` (lines 98–155). It handles the two patch formats from Chapter 4 differently. For a **diff** (lines starting with `+` and `-`) it keeps the added lines. For a **whole-function replacement** it keeps the lines that contain control flow or comparisons (`if`, `while`, `for`, `return`, `<`, `==`, `&&` and so on), because that is "where a memory-safety fix lives". It then appends an 8-character fingerprint (a SHA-1 hash) of the whole cleaned-up body, like `[3fa9c021]`. The long comment in the code explains why: before the fingerprint was added, every whole-function attempt at the same function produced the same summary (the function's first line), so the ledger said "do not repeat these" and listed one entry over and over. With the fingerprint, two attempts count as "the same" only if they really are the same.

WATCH OUT

The episodic log in the repo was written partly before the fingerprint was added. In the sample record shown later, both attempt summaries have no `[xxxxxxxx]` tag and are word-for-word identical, even though one was rejected by the regression gate and the other by the PoV gate. That is exactly the bug the fingerprint later fixed.

Working memory is the only tier that is always on. The repair loop (`src/patch_validator/drv_loop.py`, around lines 181–184) creates a fresh `WorkingMemory` for every agent on every bug, whether or not you pass `--memory`.

Semantic memory: the binder of proven fixes

Each entry in semantic memory is a `FixPattern`: the CWE, the crash class, a root-cause explanation, the fix itself (`fix_strategy`, which is the winning patch text cut to 600 characters), up to a few "pitfalls" (attempts that were rejected on the way), which agent won, and some counters. The important design claim, from the file's header comment, is that memory "records *verified outcomes*, not opinions". A pattern is only stored after the patch passed the gates. Pitfalls are stored as *observations* ("a patch of this shape was rejected by the PoV gate"), never as conclusions ("this approach never works").

THE CONFIDENCE FORMULA

Each pattern carries a confidence score between 0 and 1. It is not stored; it is worked out fresh every time from the counters.

src/memory/store.py · lines 200-213

```
    @property
    def confidence(self) -> float:
        """
        Confidence decays when a pattern is followed and the repair still fails.

        This is the guard against stale or over-general patterns: influence is
        earned by outcomes, not asserted at write time.
        """
        base = 0.9 if self.hardened else 0.6
        used = self.times_helped + self.times_misled
        if used == 0:
            return base
        success_rate = self.times_helped / used
        return round(0.5 * base + 0.5 * success_rate, 3)
```

LINE BY LINE

1. `@property` means you read `pattern.confidence` like a stored value, but it is calculated on the spot.
2. `base` is the starting trust: 0.9 if the patch also *survived hardening* (the attack phase from Chapter 5), otherwise 0.6.
3. `used` counts how many times this pattern was recalled into a prompt and then the repair either succeeded (`times_helped`) or failed (`times_misled`).
4. If it has never been used, the confidence is just the base.
5. Otherwise the confidence is a 50/50 blend of the base and the real-world success rate, rounded to 3 decimals.

Worked example. The first pattern in the repo's `semantic.json` came from SYN-01 (an integer overflow, CWE-190). It is marked hardened, was recalled 5 times, helped 5 times and misled 0 times. So its confidence is $0.5 \times 0.9 + 0.5 \times (5 \div 5) = 0.45 + 0.5 =$ **0.95**. An unhardened pattern that helped once and misled once would score $0.5 \times 0.6 + 0.5 \times 0.5 =$ **0.55**.

WATCH OUT

"Helped" here means "was in the prompt and the bug ended up patched". It does not prove the pattern caused the success; the agents might have fixed the bug without it. It is credit by association. The code is honest that this is a simple outcome counter, not a measure of cause.

HOW RECALL SCORING WORKS

When a new bug arrives and `--memory` is on, the orchestrator asks semantic memory for relevant patterns. The search is deliberately simple: no extra AI model and no "embedding" similarity. It matches on the CWE and crash class first, then on shared words.

src/memory/store.py · lines 296–323

```
    def recall(self, cwe: str, crash_class: str = "", description: str = "",
               limit: int = 2) -> list[FixPattern]:
        """
        Retrieve the most relevant validated patterns.

        Capped at `limit` deliberately: the documented failure mode of large
        retrieved contexts is attentional dilution, where more recalled material
        makes the model attend to each piece less.
        """
        scored = []
        for p in self.patterns:
            score = 0.0
            if p.cwe and p.cwe == cwe:
                score += 3.0
            if crash_class and p.crash_class and crash_class in p.crash_class:
                score += 2.0
            score += self._overlap(description, f"{p.root_cause} {p.fix_strategy}")
            score *= p.confidence
            if score > 0.5:
                scored.append((score, p))

        scored.sort(key=lambda kv: -kv[0])
        chosen = [p for _, p in scored[:limit]]
        for p in chosen:
            p.times_retrieved += 1
        if chosen:
            self.save()
        return chosen
```

LINE BY LINE

1. The function takes the new bug's CWE, crash class and description. `limit=2` means at most two patterns come back. The comment explains why: more recalled text makes the model pay less attention to each piece ("attentional dilution").
2. For every stored pattern it starts a score at 0.
3. **+3** if the CWE matches exactly.
4. **+2** if the new bug's crash class appears inside the pattern's crash class.
5. **+ word overlap** between the new description and the pattern's root cause plus fix text (explained below; between 0 and 2).
6. The total is then **multiplied by the pattern's confidence**, so a trusted pattern beats an untrusted one with the same match.
7. Only scores above **0.5** are kept. This threshold keeps unrelated patterns out.
8. Survivors are sorted highest first, the top 2 are taken, and each one's `times_retrieved` counter goes up by one. The file is saved.

src/memory/store.py · lines 352–365

```
    @staticmethod
    def _tokens(text: str) -> set:
        return {t for t in re.findall(r"[a-z_]{4,}", (text or "").lower())}

    @classmethod
    def _overlap(cls, a: str, b: str) -> float:
        ta, tb = cls._tokens(a), cls._tokens(b)
        if not ta or not tb:
            return 0.0
        return 2.0 * len(ta & tb) / len(ta | tb)

    @classmethod
```

```
    def _similar(cls, a: str, b: str) -> bool:
        return cls._overlap(a, b) > 0.6
```

LINE BY LINE

1. `_tokens` lowercases the text and pulls out every "word" of 4 or more letters or underscores. Short words like "if", "the" and "a" are ignored, and digits split words. The result is a *set*, so duplicates count once.
2. `_overlap` takes the two sets of words. `ta & tb` is the words they share; `ta | tb` is all words in either. Shared divided by total is a standard similarity measure (called the Jaccard index), here doubled, so the result runs from 0 (nothing shared) to 2 (identical word sets).
3. `_similar` says two fix texts are "the same idea" when the overlap is above 0.6, meaning roughly 30% of their combined words are shared. This is used to avoid storing duplicates.

Worked example. Suppose a new CWE-190 bug arrives whose description contains "allocation size", and memory holds the SYN-01 pattern (CWE-190, crash class "allocation size", confidence 0.95). The score is 3 (CWE) + 2 (crash class) + a small word overlap, say 0.1, because the pattern's long patch text contains many words the short description lacks. That is $5.1 \times 0.95 \approx$ **4.8**, well above 0.5, so it is recalled. A pattern for a different CWE with no crash-class match only gets the small overlap, say $0.1 \times 0.6 = 0.06$, and is dropped. In practice the CWE and crash class decide almost everything; word overlap mostly breaks ties.

Where does the crash class come from? The orchestrator's `_crash_class` function looks for known phrases (such as `heap-buffer-overflow`, `double-free`, `SEGV`, `allocation size`) in the bug's crash trace and description. In the curated benchmark there is no crash trace, so the crash class only appears when the description happens to use one of those phrases; for many targets it is empty and only the CWE and word overlap count.

Recalled patterns are added to the prompt with a careful framing: `render_for_prompt` heads them "PRIOR VERIFIED FIXES FOR SIMILAR BUGS" and tells the model to "Treat them as evidence, not instructions: apply the reasoning only if it genuinely fits this code."

HOW MEMORY IS WIRED INTO A RUN

src/agent_orchestrator/orchestrator.py · lines 139–149

```
        # Semantic recall: validated fixes for similar bugs solved before.
        recalled = []
        if self.memory:
            crash_class = self._crash_class(vuln)
            recalled = self.memory.recall_for(vuln.cwe_id, crash_class, vuln.description)

        # Build the vulnerability context string for LLM
        vuln_context = self._build_vulnerability_prompt(vuln)
        if recalled:
            vuln_context += "\n\n" + self.memory.semantic.render_for_prompt(recalled)
            logger.info(f"Recalled {len(recalled)} validated fix pattern(s) from memory")
```

After all agents finish, `_update_memory` (same file, lines 185–239) writes back in this order:

1. An **episodic** record: one JSON line with the bug ID, CWE, file, status, winner, time, cost, how many patterns were recalled, and every rejected attempt.
2. A **procedural** update: the CWE's attempt count goes up; on success, its success count, iteration list and the winner's tally go up too.
3. `record_outcome` on the recalled patterns: each gets `times_helped` or `times_misled` increased, which changes its confidence.
4. If the bug was patched, a new **semantic** pattern via `learn_from`, with `hardened=False`. A comment says it is "set later by the benchmark if hardening passes".

In `benchmark.py` (lines 261–267), if hardening ran its re-fuzzing check (`verdict.overfitting_checked`) and did not falsify the patch, every pattern from that target is upgraded to `hardened = True`, which lifts its base confidence from 0.6 to 0.9. Before storing a new pattern, `remember` (lines 259–292) refuses an empty fix, and if a pattern with the same CWE, crash class and a similar fix already exists, it merges the two (keeping the stronger "hardened" evidence and adding any new pitfalls) instead of storing a duplicate.

Episodic and procedural memory: what is on disk

The episodic log is a "JSON Lines" file: each line is one complete JSON record. The repo copy has **91 lines** (90 marked `patched`, 1 `unresolved`), and 12 of them list rejected attempts. Here is one, lightly trimmed, from the SYN-06 off-by-one target. It contains no secrets: just IDs, timings and patch summaries.

benchmark_workspace/memory/episodes.jsonl · one record, pretty-printed


```
{
 "vulnerability_id": "SYN-06-OFF-BY-ONE",
 "cwe": "CWE-193",
 "file": "tests/demo_vulns/06_off_by_one_loop.c",
 "status": "patched",
 "winning_agent": "alpha",
 "elapsed_seconds": 18.7,
 "cost_usd": 0.001455,
 "recalled_patterns": 0,
 "attempts": [
  {
   "agent": "gamma",
   "rejected_by": "regression",
   "critic": "",
   "summary": "void copy_and_null_terminate(const char* src, size_t size) {; if (size == 0 || size > 1024) return;; char* dest =
(char*)malloc(size);"
  },
  {
   "agent": "alpha",
   "rejected_by": "pov",
   "critic": "insufficient",
   "summary": "void copy_and_null_terminate(...) {; ... (same text as above)"
  }
 ],
 "recorded_at": 1786093969.054395
}
```


Read it as a story: Gamma's patch was rejected because it broke normal behaviour (regression gate). One of Alpha's attempts still crashed on the attack input (PoV gate), and the Critic called it "insufficient". In the end Alpha produced a fix that passed everything, in 18.7 seconds, for about a seventh of a US cent. `recorded_at` is a Unix timestamp (seconds since 1 January 1970).

The semantic file holds **28 patterns** (9 marked hardened, 19 not). The procedural file holds per-CWE tallies for 19 CWE keys. For example, its CWE-125 (out-of-bounds read) entry records 9 attempts, 8 successes, wins of 5 for Alpha and 3 for Beta, and an iterations list of eight 1s, meaning every success came on the first try.

src/memory/store.py · lines 428–444

```
    def preferred_agent_order(self, cwe: str, default: list[str]) -> list[str]:
        """Order agents by demonstrated wins on this CWE; unproven agents keep their order."""
        wins = self.stats.get(cwe, {}).get("agent_wins", {})
        if not wins:
            return default
        return sorted(default, key=lambda a: (-wins.get(a, 0), default.index(a)))

    def suggested_iterations(self, cwe: str, default: int) -> int:
        """
        Budget from observed difficulty: bugs of this class that were solved in
        two iterations do not need eight, and vice versa.
        """
        iters = self.stats.get(cwe, {}).get("iterations", [])
        if len(iters) < 2:
            return default
        typical = sorted(iters)[len(iters) // 2]
        return max(2, min(default, typical + 2))
```

These two functions *would* reorder the agents by past wins and shrink the retry budget to "the typical (middle) number of iterations plus 2, never below 2, never above the default". With the CWE-125 data above they would give the order Alpha, Beta, Gamma and a budget of $1 + 2 = 3$ iterations. But **nothing in the pipeline calls them**; only `tests/unit/test_memory.py` does. The README (line 148) says so plainly: procedural memory "records… but nothing consumes it yet".

WATCH OUT: MEMORY CAVEATS

- **Memory is off by default in the benchmark.** The `--memory` help text says this is "so benchmark runs stay reproducible".
- **The memory files are committed to git.** A `--memory` run does not start from a blank slate; it starts from the 28 patterns and 91 episodes already in the repo. To truly start fresh, move the `benchmark_workspace/memory/` folder aside first.
- **Patterns are learned before hardening and formal proof run.** If hardening later falsifies a patch, its pattern is not removed; it just never gets the "hardened" upgrade and keeps the 0.6 base.
- **Not every entry point uses memory.** The fuzz-discovery suite in `benchmark.py` builds its orchestrator without memory, and so do the CLI (`src/main.py`) and the standalone suite runners.

6.2 Reports: SKIPPED never means PASS

The file `src/reporting/audit_report.py` writes a Markdown "audit report": a human-readable record of what was processed and what each check said. Its header comment explains its history: an earlier version printed "Build Check: PASS / PoV Check: PASS / Regression Check: PASS" for *every* target, including targets where those checks were never configured. The rewrite reads only the gate results that were actually recorded.

src/reporting/audit_report.py · lines 22-26 and 169-173

```
_GATE_LABEL = {
    GATE_PASS: "PASS",
    GATE_FAIL: "FAIL",
    GATE_SKIPPED: "SKIPPED (not configured for this target)",
}
    ...
    @staticmethod
    def _gate_line(state: str, note: str | None) -> str:
        if state == GATE_SKIPPED and note:
            return f"SKIPPED — {note}"
        return _GATE_LABEL[state]
```

LINE BY LINE

1. `_GATE_LABEL` is a lookup table from the three possible gate states to the exact words printed. There are three states, not two: a check can pass, fail, or *not have run at all*.
2. `_gate_line` is used for the PoV and regression lines. If the gate was skipped and the caller gave a reason (a "note"), the report prints that reason, for example "SKIPPED — Requires MemorySanitizer (Linux-only)…". Otherwise it uses the standard label.

A generated report has three sections:

1. **Executive Summary**: a table with the number of bugs processed, how many passed all configured gates (and the percentage), how many of those were *proven by PoV replay*, how many were regression-tested, how many are unresolved, total time and estimated cost. Note that "passed all gates" and "proven by PoV replay" are reported separately, because they mean different things.
2. **Detailed Findings & Remediation**: one block per bug with severity, location, detection source, status, which context method was used, winning agent, time and cost; the description; an **agent table** (for each of Alpha/Beta/Gamma: validated or rejected, iterations used out of budget, and the stage where it last failed); the AI's root-cause analysis; the six verification lines (apply, build, PoV, regression, static re-scan, patch size); and the winning patch. If nothing passed, it prints each agent's final rejection message instead (up to 1,200 characters).
3. **Scope of Validation**: a fixed paragraph reminding the reader that where the PoV gate reads SKIPPED, the fix was **not** dynamically proven.

WATCH OUT

- The patch section is always titled "Applied Patch (Unified Diff)", even when the winning patch was a whole-function replacement and not a diff.
- The audit report does not mention hardening or CBMC results. Those appear in the benchmark's CSV and JSON files (next sections), not in the audit Markdown.
- In the benchmark the "Post-Patch Static Re-scan" line will always read SKIPPED, because `benchmark.py` builds its orchestrator with no Semgrep runner (the third argument is `None`).

6.3 The everyday front door: `src/main.py`

`src/main.py` is the tool you would point at your own C code. You run it as `python -m src.main`. It prints a banner and then goes through six numbered steps, each announced on screen.

1. **Step 1, initialise.** Load the AI settings with `get_llm_config()`; if `OPENAI_API_KEY` is missing it prints the error and exits. Create the AI client and the context extractor. Try to create a Semgrep runner; if Semgrep is not installed it prints a warning and carries on without static analysis.
2. **Step 2, static analysis.** Check the target path exists, then run Semgrep on it and list up to five findings.
3. **Step 3, prepare reports.** Turn each Semgrep finding into a `VulnerabilityReport` named `SARIF-001`, `SARIF-002` and so on. If there are none, it says so and stops.
4. **Step 4, run the ensemble.** Warn loudly if `--build-cmd` or `--pov-cmd` is missing ("the proof-of-vulnerability gate will be SKIPPED. Patches cannot be shown to actually fix the bug."), then send each finding through the orchestrator.
5. **Step 5, results.** Print a coloured table (ID, CWE, file:line, status, winning agent, time, cost, and a **Gates** column listing which checks actually ran), a summary line, total API calls, tokens and cost, then the winning patches.
6. **Step 6, audit report.** Write the Markdown audit report into the `reports/` folder (file name `audit_report_<date>_<time>.md`).

Flag	Meaning	Default
`--target`, `-t`	File or folder to analyse (required)	none
`--language`, `-l`	`c` or `cpp`. Parsed and passed along, but **never used** by the pipeline.	`c`
`--build-cmd`	Command to compile the patched code. Without it, the build gate is skipped.	none
`--test-cmd`	Regression test command. Without it, the regression gate is skipped.	none
`--pov-cmd`	Proof-of-vulnerability command. Must exit 0 *only when the bug is absent*. Supports `{src}`, `{srcdir}`, `{bin}`, `{workspace}`.	none
`--agents`	Which of `alpha`, `beta`, `gamma` to use	all three
`--sequential`	Run agents one after another instead of in parallel	parallel
`--verbose`, `-v`	More detailed logging	off

KEY IDEA

The CLI finds bugs **only with Semgrep**. It does no fuzzing, no hardening, no CBMC proof and no memory. Those features live in `benchmark.py`. The CLI is the simple "scan my code and try to fix what Semgrep flags" path.

6.4 The measuring harness: `benchmark.py`

`benchmark.py` is the scoreboard. It runs the system over fixed lists of known-buggy programs and writes down exactly what happened. Its opening comment states the rule: "every number this harness reports is measured during the run… If something cannot be measured on this platform it is reported as 'not measured' together with the reason, never as an assumed value."

The command-line flags

Flag	What it does
`--suite` (required)	One of `synthetic`, `juliet`, `real_world`, `real_cve`, `all`, `fuzz`. `all` means synthetic + juliet + real_world + real_cve. `fuzz` runs *only* the fuzz-discovery suite.
`--sequential`	Run the three agents one at a time instead of in parallel.
`--limit N`	Only the first N targets of each suite (a quick trial run).
`--only ID,ID`	Only these target IDs, e.g. `SYN-06-OFF-BY-ONE`. Handy for re-running failures.
`--memory`	Turn on semantic recall and learning (off by default).
`--harden`	After a patch passes, try to break it (re-fuzz, differential test, evasion battery). Broken patches are downgraded.

Flag	What it does
`--formal`	Try to prove the winning patch with CBMC. Only targets with a proof harness are attempted; others report `unavailable`.
`--fuzz`	Also run the fuzz-discovery suite after the chosen suites.
`--fuzz-engine`	`libfuzzer` (default) or `afl++`.
`--fuzz-seconds`	Fuzzing time per harness (default 30).

Notice what `all` does **not** include: the Vanguard Nightmare suite and the CVE-inspired extension. Those have their own runner scripts (section 6.5). The "44 targets" headline adds all of them together.

The overall flow

```
python benchmark.py --suite synthetic --harden --formal
       |
       v
main(): parse flags -> BenchmarkHarness(...)   (creates LLM client, extractor,
       |                                         hardener / CBMC / memory if asked)
       v
for each suite:  get_suite(name) -> list of Target   (tests/benchmarks/targets.py)
       |          apply --only / --limit
       v
run_suite():
  1. PRE-FLIGHT every target  (build unpatched, PoV must FAIL on it)
  2. for each target that builds:
       orchestrator.process_vulnerability(...)   <- Chapters 3-4
       if patched and --harden: _harden_patch()  <- may become unresolved_hardening
       if patched and --formal: _verify_formally() <- may become unresolved_formal
       _record()  one row of facts
  3. _summarise_suite()
       |
       v
if --fuzz: run_fuzz_discovery()
       |
       v
generate_reports()  -> reports/benchmark_runs/<timestamp>/
```

How a target is declared: the `Target` dataclass

Every benchmark program is described by a `Target` record in `tests/benchmarks/targets.py`. A "dataclass" is just a Python way of saying "a record with these named fields", like a row on a form.

tests/benchmarks/targets.py · lines 75-118

```python
@dataclass
class Target:
    """One benchmark target and its complete verification contract."""

    id: str
    suite: str
    file_path: str
    line_number: int
    cwe_id: str
    description: str
    complexity: str                      # easy | medium | hard | ast-dependent
    build_command: str
    pov_command: Optional[str] = None
    pov_na_reason: str = ""
    regression_command: Optional[str] = None
    regression_na_reason: str = ""
    source_dir: Optional[str] = None     # copy this whole tree into workspace
    ...
    behaviour_contract: str = "preserve"
    ...
    evasion_command: Optional[str] = None
    ...
    cbmc_unwind: int = 16
    severity: str = "critical"
```

```
    source: str = "sanitizer_crash"      # sanitizer_crash | semgrep | side_effect

    def __post_init__(self):
        if self.pov_command is None and not self.pov_na_reason:
            raise ValueError(f"{self.id}: pov_command=None requires pov_na_reason")
        if self.regression_command is None and not self.regression_na_reason:
            raise ValueError(
                f"{self.id}: regression_command=None requires regression_na_reason"
            )
```

LINE BY LINE

1. Identity: `id`, which `suite` it belongs to, the `file_path` and `line_number` of the bug, its `cwe_id`, a plain-English `description`, and a rough `complexity` label.
2. Verification commands: `build_command` (always required), `pov_command` and `regression_command` (optional).
3. `pov_na_reason` / `regression_na_reason`: written explanations for why a check does not exist.
4. `source_dir`: for targets that are part of a larger project (like cJSON), copy the whole folder into the temporary workspace, not just one file.
5. `behaviour_contract` (`preserve`, `restrict` or `replace`), `evasion_command` and `cbmc_unwind` are used by hardening and formal proof (Chapter 5).
6. `__post_init__` runs automatically when a `Target` is created. If you leave out the PoV or regression command *without* writing a reason, Python refuses to create the target at all. You cannot quietly skip a check; you must explain it, and the explanation is printed in the reports.

Commands are written as templates with four placeholders: `{src}` (the patched source file in the workspace), `{srcdir}` (its folder), `{bin}` (where the compiled program goes) and `{workspace}` (the temporary folder). Small helpers build the common commands. `_pov("AAAA")` becomes "run `pov_run.sh` on the program with the argument AAAA"; `_regress("Copied string: hi", "hi")` becomes "run `regress_run.sh`, which requires exit code 0, no sanitizer report, and the text 'Copied string: hi' in the output". All arguments pass through `shlex.quote` so that quote marks in an attack string cannot break the shell command.

Here is a complete real example, SYN-06:

tests/benchmarks/targets.py · lines 192-200

```
    Target(
        id="SYN-06-OFF-BY-ONE", suite="Synthetic", complexity="easy",
        file_path=f"{D}/06_off_by_one_loop.c", line_number=8,
        cwe_id="CWE-193", description="Off-by-one heap write in NUL-termination loop",
        build_command=SINGLE_FILE_BUILD,
        pov_command=_pov("AAAA"),
        regression_command=_regress("Copied string: hi", "hi"),
        cbmc_unwind=70,          # loop runs up to the 64-byte harness buffer + 1
    ),
```

`SINGLE_FILE_BUILD` is `clang` with `-fsanitize=address,undefined -fno-sanitize-recover=all -fno-omit-frame-pointer -g -O0`: compile with AddressSanitizer and UndefinedBehaviorSanitizer switched on, and make every sanitizer warning a hard stop.

Pre-flight: prove the bug is real before scoring the fix

Before any AI runs, `run_suite` calls `preflight()` on every target. This is one of the project's best ideas.

benchmark.py · lines 139-158

```
        ok, detail = DRVLoop._run(DRVLoop.expand(target.build_command, ctx), "Build", cwd=workspace)
        res.build_ok = ok
        if not ok:
            res.build_error = detail
            return res

        if target.pov_command:
            # PoV scripts exit non-zero when the vulnerability IS present.
```

```
            ok, detail = DRVLoop._run(DRVLoop.expand(target.pov_command, ctx), "PoV", cwd=workspace)
            res.pov_reproduces = not ok
            res.pov_detail = detail if not ok else "PoV did NOT reproduce on the original binary"

        if target.regression_command:
            # A regression test must pass on the ORIGINAL, otherwise it is
            # measuring a pre-existing failure rather than damage from a patch.
            ok, detail = DRVLoop._run(
                DRVLoop.expand(target.regression_command, ctx), "Regression", cwd=workspace
            )
            res.regression_baseline_ok = ok
            res.regression_detail = "" if ok else detail
```

LINE BY LINE

1. (Before this excerpt, the original, *unpatched* file is copied into a fresh temporary folder and the four placeholders are filled in.)
2. Build the original. If it does not even compile, record the error and stop: this target will be **excluded** from the run.
3. Run the PoV against the *unpatched* program. PoV scripts exit non-zero when the bug is present, so for a real bug we expect `ok` to be False. `pov_reproduces = not ok` captures that.
4. If the PoV did *not* fail on the original, the bug was never demonstrated, so the note says "PoV did NOT reproduce".
5. Run the regression test on the original too, and record whether it passes. This is kept for transparency but does not switch the gate off (the comment at lines 104–113 gives the reason: for SYN-06, every input trips the sanitizer on the original, so the baseline legitimately fails).

The result object has a property `pov_usable`, which is true only when the original built **and** the PoV reproduced. Then, in the main loop, `pov_command=t.pov_command if pf.pov_usable else None`: if the PoV was never shown to detect the real bug, it is simply not used as a gate, and the target cannot be counted as "PoV-proven".

ANALOGY

Before a smoke alarm can get credit for "the fire is out, I am silent now", you must first see it beep when there *is* smoke. An alarm that never beeps would happily stay silent after any "fix". Pre-flight is the "press the test button" step.

The main loop: run, attack, prove, record

benchmark.py · lines 227-258

```
        for i, t in enumerate(usable, 1):
            console.print(f"  [{i}/{len(usable)}] {t.id} …")
            pf = preflights[t.id]
            vuln = VulnerabilityReport(
                id=t.id, source=t.source, file_path=t.file_path,
                line_number=t.line_number, description=t.description,
                cwe_id=t.cwe_id, severity=t.severity,
            )
            result = orchestrator.process_vulnerability(
                vuln,
                build_command=t.build_command,
                # Only gate on a PoV we proved detects the real bug.
                pov_command=t.pov_command if pf.pov_usable else None,
                test_command=t.regression_command,
                source_dir=t.source_dir,
            )
            results.append(result)
            self.all_results.append((t, result))

            if self.hardener and result.status == "patched":
                verdict = self._harden_patch(t, result)
                self.hardening_verdicts[t.id] = verdict
```

```
            if verdict.falsified:
                # The patch cleared every gate but hardening broke it.
                # Downgrade it rather than report a fix we can't defend.
                ...
                result.status = "unresolved_hardening"
                console.print(f"      [red]HARDENING FAILED: {verdict.summary()}[/]")
```

LINE BY LINE

1. Loop over every target that passed pre-flight's build check, numbering them from 1.
2. Build a `VulnerabilityReport` straight from the manifest. In this curated mode the bug's location is *declared*, not discovered.
3. Hand it to the orchestrator (Chapters 3–4) with the build command, the PoV only if usable, the regression command and the source folder.
4. If `--harden` is on and the patch passed, try to break it. Only a *falsified* verdict downgrades the status to `unresolved_hardening`. A patch where no attack applied keeps its "patched" status but does not earn the "hardened" label.
5. (After this excerpt, lines 269–278 do the same for `--formal`: a CBMC "violated" result downgrades to `unresolved_formal`. Then `_record` writes one row of facts.)

`_harden_patch` (lines 286–326) and `_verify_formally` (lines 328–367) share a pattern. The repair loop's temporary folders are already deleted, so each one makes a fresh copy of the original source and **re-applies the winning patch**: first by swapping in the whole function (if the winner was a function replacement), otherwise by applying the diff. If that fails, the verdict is "skipped" with the reason "could not re-apply the winning patch", never a pass. Then `_harden_patch` calls `PatchHardening.harden` with the target's behaviour contract and evasion command. `_verify_formally` first checks for a proof harness at `tests/benchmarks/cbmc_harnesses/<ID>.c`; only SYN-06, SYN-11 and SYN-12 have one, and everything else reports "no proof harness for this target", which counts as `unavailable`.

`_record` (lines 369–405) writes one row per target: suite, ID, CWE, complexity, file, status, winning agent, context method, whether a PoV was available, the PoV/build/regression gate results of the winning attempt, patch size in lines, the most iterations any agent used, why agents failed (e.g. `pov:1;regression:2`), seconds, cost, formal status and unwind bound, and the hardening summary. `_summarise_suite` (lines 407–437) then counts these rows into a suite summary: attempted, excluded, patched, success rate with confidence interval, PoV-gated, PoV-proven, regression-proven, mean and median patch size, duration.

The fuzz-discovery path

`run_fuzz_discovery` (lines 439–539) is different from every other suite: the bug's location is **not** declared in advance. Only synthetic targets with a fuzz harness file in `tests/fuzz_harnesses/` are eligible (there are nine: SYN-01, 02, 05, 06, 07, 11, 12, 16 and 18). For each, `FuzzDiscoveryPipeline.run` fuzzes for `--fuzz-seconds`, triages the crashes into distinct bugs, finds the file and line from the stack trace, and patches each distinct bug with the crashing input itself as the PoV (Chapter 2 covers the details).

An important accounting fix lives here. The suite's "targets attempted" is the number of **distinct bugs found**, not the number of harnesses fuzzed. A harness that found nothing has nothing to repair, so counting it as a failed repair would be unfair. To stop this mistake from coming back, `check_suite_units` raises an error if any suite ever reports more patched than attempted:

benchmark.py · lines 606-613

```
        for s in suites:
            attempted, patched = s["Targets_Attempted"], s["Patched_All_Gates"]
            if patched > attempted:
                raise ValueError(
                    f"suite {s['Label']!r} reports {patched} patched of "
                    f"{attempted} attempted — the two fields are counting "
                    f"different things, so the headline rate would be wrong"
                )
```

WATCH OUT

- The fuzz suite does **not** run hardening, CBMC or memory, even if those flags are set. Its orchestrator is created without memory, and the loop never calls `_harden_patch` or `_verify_formally`.
- Its per-bug rows pair clusters with patch results using `zip(discovery.clusters, discovery.patch_results)`. If triage skipped an earlier cluster (crash location unresolved), the two lists no longer line up and a row could show the wrong crash class or location. It does not change the counts.
- The summary's built-in "not measured" list always says the fuzzing modules "are not exercised by this harness", even on a run that used `--fuzz`. That sentence is out of date.

What `generate_reports` writes

At the end, everything goes into a new folder named after the time, `reports/benchmark_runs/<YYYYMMDD_HHMMSS>/`. A summary table is also printed to the terminal.

File	Contents
`per_target_results.csv`	One row per target (the `_record` rows). Opens in any spreadsheet.
`suite_summary.csv`	One row per suite.
`preflight_pov_validation.csv`	For each target: did the original build, was a PoV configured, did it reproduce, the N/A reason, the regression baseline, any build error.
`benchmark_summary.json`	Everything in machine-readable form: totals with Wilson intervals, per-suite data, patch quality, DRV iteration stats and rejection stages, LLM calls/tokens/cost, peak memory (RSS), component availability, and the "not measured" list.
`benchmark_summary.md`	The same summary as readable Markdown, in eight sections.
`audit_<timestamp>.md`	The audit report from section 6.2, with each target's N/A reasons passed in as gate notes.

"Component availability" deserves a mention: at report time the harness *probes* which fuzzing engines exist, whether the triage backend is CASR or the built-in one, whether the Driller/angr concolic engine passes its self-test, and whether Semgrep is installed. The report states what was really usable on that machine rather than what the design says.

The Wilson confidence interval, explained simply

If you score 44 out of 44, is the system "100% accurate"? No: you tried only 44 things. If the true success rate were 97%, you would still get 44/44 fairly often by luck. A **confidence interval** gives a range of true rates that are believable given what you saw. "95% CI [92.0, 100.0]" roughly means: "based on these 44 results, we are 95% confident the real success rate on this kind of corpus is somewhere between 92% and 100%."

benchmark.py · lines 58-76

```
def wilson_interval(successes: int, trials: int, z: float = 1.96) -> tuple[float, float]:
    """
    Wilson score confidence interval for a binomial proportion.

    Reporting a bare "100%" from a single run of 37 targets overstates what the
    sample supports. The normal approximation is useless at p=1 (it gives a
    zero-width interval); the Wilson interval stays sensible at the boundary,
    which is exactly where our results sit.

    37/37 successes yields roughly [90.6%, 100%] at 95% confidence — still a
    strong claim, and one that survives scrutiny.
    """
    if trials == 0:
        return (0.0, 0.0)
    p = successes / trials
    denom = 1 + z ** 2 / trials
    centre = (p + z ** 2 / (2 * trials)) / denom
    margin = (z / denom) * math.sqrt(p * (1 - p) / trials + z ** 2 / (4 * trials ** 2))
    return (max(0.0, centre - margin), min(1.0, centre + margin))
```

LINE BY LINE

1. `z = 1.96` is the standard number that corresponds to "95% confident". A bigger `z` means a wider, more cautious range.
2. With zero trials there is nothing to say, so it returns (0, 0).
3. `p` is the observed success rate, e.g. 44/44 = 1.0.
4. `denom` and `centre`: the Wilson method's "best guess" is pulled slightly *away* from 0% or 100% toward the middle. With few trials it is pulled more; with many trials, hardly at all.
5. `margin` is how far either side of the centre the range reaches.
6. The result is clipped so it never goes below 0 or above 1.

Why not the simple textbook formula? The common school formula's margin is built on `p × (1 − p)`. When `p` is 1 (everything passed), that is 1 × 0 = 0, so it claims a range of exactly [100%, 100%], total certainty from 44 tries, which is nonsense. Wilson's formula has an extra `z² / (4n²)` term inside the square root that keeps the range open.

A shortcut for perfect scores. When every trial succeeds, the Wilson lower bound simplifies to `n ÷ (n + z²)`, and $z^2 = 1.96^2 \approx 3.84$. So:

- 44/44: 44 ÷ (44 + 3.84) = 44 ÷ 47.84 ≈ **0.920** → **92.0%**, matching the reported [92.0, 100.0].
- 30/30: 30 ÷ 33.84 ≈ **0.886** → **88.6%**, matching the reported [88.6, 100.0].
- 2/2 (the Juliet suite alone): 2 ÷ 5.84 ≈ 34%. Two successes really tell you very little.

This shows why bigger test sets matter: the lower bound climbs only slowly as you add targets.

KEY IDEA

A confidence interval only describes *this kind of corpus*. benchmark.md says so directly: the interval "says nothing about code the system has not seen". It also assumes each target is an independent, fair sample, which a hand-built corpus is not.

6.5 The benchmark suites: what each one measures

There are seven scored groups of targets. Here is the map, then each one in turn.

Suite	Where	How run	Count	Written by the project?
Synthetic	`tests/demo_vulns/` + `targets.py`	`benchmark.py --suite synthetic`	20	Yes
NIST Juliet subset	`tests/benchmarks/juliet_subset/`	`--suite juliet`	2	Taken from NIST's style of test cases
Real World (cJSON)	`benchmark_workspace/cJSON` (cloned)	`--suite real_world`	6	No: real upstream code, Semgrep findings
Real CVE	`tests/real_cve_suite/`	`--suite real_cve` or its own runner	3	No: real published defects
Vanguard Nightmare	`tests/vanguard_nightmare/`	own runner only	4	Yes
CVE-inspired extension	`tests/public_benchmarks_extension/`	own runner only	2	Yes
Fuzz discovery	`tests/fuzz_harnesses/`	`--fuzz` or `--suite fuzz`	7 bugs found (9 harnesses)	Harnesses yes; bugs found by the fuzzer

Synthetic (20 targets)

Twenty short C programs, `01_complex_magic_bytes.c` to `20_ast_opaque_pointer.c`, each containing one deliberate weakness: integer overflow, truncation, double free, off-by-one, format string, path traversal, command injection, NULL dereference, out-of-bounds array access, integer underflow, a time-of-check/time-of-use race, uninitialised memory, a hardcoded key, type confusion, a memory leak, and three "AST" cases where understanding the code requires looking at type definitions or macros elsewhere in the file. They measure **breadth**: can the system handle many different bug kinds?

Only **12 of the 20** have a usable PoV. The other eight carry a written `pov_na_reason`, and the reasons are instructive. SYN-14 and SYN-20 need MemorySanitizer, which runs only on Linux, not on the Mac used. SYN-18's overrun stays inside one larger object, which AddressSanitizer cannot

see. SYN-13's race needs a second attacker process. SYN-08's target folder does not exist on the test machine. Some PoVs are not crashes at all: SYN-09 (command injection) checks whether a marker file got created, and SYN-15 (hardcoded key) runs the `strings` tool on the compiled program to see if the secret is still inside. SYN-17 (memory leak) uses LeakSanitizer, which needs Homebrew's LLVM because Apple's compiler lacks it.

NIST Juliet subset (2 targets)

Juliet is a large, well-known collection of tiny test programs published by the US standards body NIST, one per weakness type. The project includes two "bad" cases: CWE-121 (stack buffer overflow) and CWE-416 (use-after-free). Both have PoVs. Neither has a regression test, because Juliet "bad" cases contain only the vulnerable code, with no normal path to check. This also means hardening's differential test compares zero inputs, so both report NOT HARDENED.

Real World: cJSON (6 targets)

cJSON is a popular open-source JSON parser in C. These six targets are spots where Semgrep's `p/security-audit` ruleset flags an unsafe string function (`strcpy`-style or `strcat`-style, CWE-676) in `cJSON.c` and `cJSON_Utils.c`. The comment in `targets.py` is careful: "These are ACTUAL Semgrep p/security-audit findings, not a CVE." There is **no PoV** for any of them, so none can be "PoV-proven". What this suite measures is whether a patch to real, larger code still **compiles the whole library and passes cJSON's own test program** under the sanitizers.

> **WATCH OUT**
>
> The cJSON clone is not in the repo (`benchmark_workspace/cJSON/` is gitignored). `tests/benchmarks/run_real_cjson.py` clones it if missing and then runs the suite. Its clone is not pinned to a particular commit, so on a fresh clone the hardcoded line numbers (461, 976, 1440…) may no longer point at the same code. If the folder is missing, pre-flight cannot copy the source and every target is excluded.

Real CVE suite (3 targets)

This is the only suite of real security bugs from outside sources. A **CVE** is the official public ID for one specific known vulnerability. Each target is real cJSON code checked out at the commit *just before* the upstream fix:

- **CVE-2019-11835**: out-of-bounds read in `cJSON_Minify` on an unterminated `/*` comment. Vulnerable at tag `v1.7.10`.
- **CVE-2019-11834**: over-read in `parse_string` on an unterminated string like `"abc`. Vulnerable at commit `b537ca7`, fixed in `a167d9e`.
- **GH-800**: over-read in `parse_object` after a trailing comma. A real upstream security fix with no CVE number, so it is named after GitHub issue #800 rather than given an invented ID.

tests/real_cve_suite/setup.sh · lines 19–45

```
# Vulnerable commit for each CVE = the parent of the upstream fix.
#   CVE-2019-11835  cJSON_Minify OOB read      fixed by the 1.7.11 rewrite
#   CVE-2019-11834  parse_string over-read     fixed by a167d9e
#   GH-800          parse_object trailing comma OOB   fixed by 3ef4e4e
VULN_MINIFY="v1.7.10"
VULN_PARSE_STRING="b537ca70a35680db66f1f5b8b437f7114daa699a"
VULN_PARSE_OBJECT="826cd6f842ae7e46ee38bbc097f9a34f2947388d"

if [ ! -d "$CLONE/.git" ]; then
    echo "cloning cJSON -> $CLONE"
    git clone -q "$REPO_URL" "$CLONE"
fi

mkdir -p "$TREES"

add_tree() {
    name="$1"; commit="$2"
    path="$TREES/$name"
    if [ -f "$path/cJSON.c" ]; then
        echo "  $name: already present"
        return
    fi
    rm -rf "$path"
    git -C "$CLONE" worktree prune
    git -C "$CLONE" worktree add -q --detach "$path" "$commit"
```

```
    echo "  $name: $(git -C "$path" log -1 --format=%h)"
}
```

LINE BY LINE

1. The three variables name the exact vulnerable versions: a tag and two full commit IDs.
2. If cJSON has not been cloned into `benchmark_workspace/cJSON_cve` yet, clone it from GitHub (the one step that needs internet).
3. `add_tree` creates a separate folder per CVE using `git worktree`, a git feature that checks out an older version into its own folder without disturbing the main clone. If the folder already exists it does nothing, so the script is safe to run again.
4. After the excerpt, the script calls `add_tree` once for each of the three CVEs and prints how to run the suite.

The manifest `tests/real_cve_suite/manifest.py` defines each target with a small C "driver" program in `drivers/` that calls the vulnerable function from `main`, a PoV (for example the input `/*`), and a regression check (for example `{ "a" : 1 }` must print `minified: {"a":1}`). Its header says each entry "was validated in both directions": the PoV crashes the vulnerable commit, and the upstream fix stops it. If a tree is missing the target is quietly left out, so a fresh checkout that skipped `setup.sh` gives **zero** real-CVE targets rather than failures.

Vanguard Nightmare (4 targets)

Four hand-written "hard" programs in `tests/vanguard_nightmare/targets/`, designed to defeat reasoning that only looks at a few lines: a length overflow in a packed bitfield struct whose definition is in a separate header (`headers/rtos_types.h`), a use-after-free reached through a global alias, an allocation size computed by chained macros, and a path traversal in a custom `rtos://` URI scheme. The last one uses the `restrict` contract and an **evasion battery** (`evasion_battery.sh`) that tries bypass tricks like `....//secret.txt` and `sub/../../secret.txt` against the patched program.

This suite runs only through `python -m tests.vanguard_nightmare.run_vanguard`, which does its own pre-flight and hardening (via `tests/standalone_support.py`) and writes `reports/vanguard_nightmare_report.md`. The manifest is candid about two things: VANGUARD-04's original PoV passed even on the unpatched program and "proved nothing" (now replaced by a script that checks the real security property), and VANGUARD-03 has a second bug, so a SURVIVED verdict there "is weaker than for a harnessed target".

CVE-inspired extension (2 targets)

Two short hand-written programs in `tests/public_benchmarks_extension/`: a crop-box out-of-bounds read shaped like a known LibTIFF bug, and a `strcpy` stack overflow in the style of a DARPA Cyber Grand Challenge service. The manifest states the provenance plainly: these are **not** LibTIFF code, **not** that CVE and **not** the real CGC binary. They used to be named as if they were, and were renamed "so no report can imply otherwise". Run with `python -m tests.public_benchmarks_extension.run_public_benchmarks`; the report goes to `reports/public_benchmarks_report.md`.

WATCH OUT

This runner has a known weakness: if a target fails pre-flight, its result is `None`, and a later line in `run_public_benchmarks.py` (around line 160) tries to read the status from it and crashes. The SKIPPED summary row also has the wrong number of columns. With both targets building normally you will not hit this.

Fuzz discovery (7 bugs)

This is the "real discovery" test: nobody tells the system where the bug is. Per benchmark.md, nine harnesses were fuzzed for 30 seconds each, giving 8 crashing inputs, which triage grouped into 7 distinct bugs; all 7 were repaired and passed PoV replay with the fuzzer's own crashing input. It measures the full pipeline: fuzz → triage → locate → patch → verify.

Not scored, but worth knowing: the cheat detector

`tests/cheat_detector/run_cheat_detector.py` needs **no AI and no API key**. It builds three hand-made patches for one overflow bug: a CORRECT fix, an OVERFITTED one that special-cases the 80-byte attack input, and a GUTTED one that just returns early. All three pass build, PoV and regression, so the gates alone cannot tell them apart. Then it runs the real hardening engine. It exits with code 0 ("PASS") only if both cheats are falsified **and** the honest fix survives. It is the clearest demonstration of why hardening exists.

6.6 The unit tests: guarding the honesty rules

Unit tests are small automatic checks, each testing one behaviour. They live in `tests/unit/` and run with `pytest`. The docs say 125 tests run with "no API calls, no network". Tests that need `clang` or `cbmc` skip themselves if those tools are missing. Most of them protect a rule about *truthful reporting*, not just "does the code work".

File	What it guards	Examples of test names
`test_analysis_engine.py`	Fuzzing and crash triage: a clean run is not a bug, the same bug groups together and different bugs do not, fuzzer scaffolding never wins the crash location, crash classes map to the right CWE, Driller's self-test controls whether it reports anything, engine detection is honest, the leak PoV detects the leak and not just the binary.	`test_clean_run_is_not_a_vulnerability`, `test_same_bug_clusters_together_distinct_bugs_do_not`
`test_audit_report.py`	A skipped PoV is never printed as PASS and never counted as "proven"; gate notes appear when supplied.	`test_skipped_pov_is_never_rendered_as_pass`
`test_core.py`	Config loading, tree-sitter context extraction and fallback, diff extraction, patch line counting, the fuzzy "content fallback" patch applier (applies drifted hunks, rejects missing or ambiguous context), unconfigured gates report SKIPPED, the placeholder expander leaves other braces alone, the summary rollup rejects mismatched units.	`test_unconfigured_gates_report_skipped_not_pass`, `test_rollup_rejects_mismatched_units`
`test_formal.py`	CBMC catches an overfitted patch that the PoV gate passes; the original is "violated" and the fix "proven"; a missing harness is "unavailable", not "proven"; `main` is stripped before verification. Skipped entirely if `cbmc` is not installed.	`test_catches_overfitted_patch_that_the_pov_gate_passes`
`test_hardening.py`	Gutted and overfitted patches are caught; correct patches survive; a differential test that compared zero inputs is not evidence; "survived" requires a check to have actually run; harnesses respect buffer preconditions; output normalisation does not hide real changes; the evasion battery counts; `pov_run.sh` prints the sanitizer report on failure.	`test_gutted_patch_is_caught`, `test_zero_input_differential_is_not_evidence`
`test_memory.py`	Everything in section 6.1: the ledger lists and caps attempts and detects repeats, recall matches on CWE/crash class and is capped at 2, unrelated bugs are not recalled, duplicates merge, confidence falls when a pattern misleads and rises when it helps, only validated fixes are learned, disabled memory does nothing, and the procedural functions behave as designed.	`test_confidence_decays_when_a_pattern_misleads`, `test_only_validated_fixes_are_learned`
`test_orchestrator.py`	Winner selection: smallest validated patch wins, a failed agent never wins however small its patch, ties break on agent order (not on who finished first), crashed agents are skipped.	`test_a_failed_agent_never_wins_however_small_its_patch`
`test_repair_loop.py`	Whole-function extraction and AST splicing, failure-feedback trimming, the escalation policy (PoV failure always calls the Critic, repeated apply failures force the whole-function format, stuck loops stop early), Critic verdict validation, and parsing edge cases.	`test_pov_failure_always_triggers_critic`, `test_loop_stops_early_when_stuck`

WATCH OUT

`test_config_loading` in `test_core.py` calls the real config loader and asserts the API key starts with `sk-`. So without a real-looking key in `.env`, that one test fails even though nothing else needs a key. Also, there are no tests for `src/main.py`, the Semgrep runner, or the live AI network calls.

6.7 The reported results, and how to read them

The headline numbers come from `benchmark.md`, for a run on 30 August 2026 using `gpt-4o-mini` for every agent, on an Apple Silicon Mac.

Suite	Targets	Passed all gates	PoV-proven	Time
Synthetic	20	20	12 / 12	403.3s
NIST Juliet	2	2	2 / 2	18.8s
Real World (cJSON)	6	6	–	76.3s
Real published CVEs	3	3	3 / 3	74.4s
Fuzz Discovery (libFuzzer)	7	7	7 / 7	122.0s
Vanguard Nightmare	4	4	4 / 4	68.8s
CVE-inspired extension	2	2	2 / 2	40.2s
Total	**44**	**44**	**30 / 30**	**804s**

- **Passed all gates: 44/44**, 95% Wilson CI [92.0, 100.0].
- **PoV-proven: 30/30**, 95% CI [88.6, 100.0].
- **Hardening:** 32 patches attacked and held, 0 falsified, 5 not hardened (SYN-08 and SYN-09 had no applicable attack, SYN-15 uses the `replace` contract, and the two Juliet cases compared zero inputs).
- **Formal proofs:** 3 (SYN-06 with unwind 70, SYN-11 with 16, SYN-12 with 20).
- **Cost: $0.0739** in total for 161 API calls (244,643 input tokens, 46,175 output tokens). Median about $0.0009 per target; the most expensive single target cost $0.0129.
- **Time and resources:** 804 seconds (13.4 minutes) wall time, median 5.8s per target, peak memory of the harness process 1.3 GB.
- **Patch size:** mean 3.4 changed lines, median 2.
- **Memory:** 28 semantic patterns learned, 91 episodes recorded.

There are three bars of increasing strength: **passed all gates** (applied, compiled under sanitizers, cleared every configured check), **PoV-proven** (the exact attack input no longer works on the rebuilt patched program), and **survived hardening** (actively attacked and held).

WATCH OUT: THE CAVEATS, MOST OF THEM STATED BY THE PROJECT ITSELF

- **Only 3 of the 44 targets come from outside.** The rest were written by the project's authors. benchmark.md: "a corpus its own authors designed is a weaker test than one they did not."
- "**A corpus everything passes has stopped discriminating.**" 44/44 says the system handles *this* set; it cannot rank the system against a harder one.
- **14 of the 44 cannot be PoV-proven** on this platform. For those, the evidence is compile + regression only.
- "**Passed all gates**" **means no gate failed, not that every gate ran.** In the benchmark the Semgrep re-scan gate never runs, and many targets have no PoV.
- **Not measured:** detection precision/recall (no labelled corpus was scanned), CPU use, and AFL++ campaign results (the reported fuzz suite used libFuzzer). The Driller/angr engine disables itself on this Mac.
- **One model, one machine.** All results use `gpt-4o-mini`. Results on another model "would need re-measuring".
- **The raw run folder is not in the repo.** `reports/benchmark_runs/` is not committed, so the headline cannot be re-checked from the repo alone; only the three standalone reports in `reports/` and the memory files are present.
- **The memory files are committed**, so runs with `--memory` build on earlier state.
- The "125 unit tests" figure is the documents' count; this guide did not re-run it.

6.8 How to run it yourself

Step 1: install the toolchain

The project was built on macOS (Apple Silicon) with Homebrew and conda. `INSTALLED.md` records exactly what was installed.

1. **Create the Python environment** (Python 3.11, named `ai_kavach`):

terminal

```
conda create -n ai_kavach python=3.11 -y
conda activate ai_kavach
```

2. **Install the Python packages.** `requirements.txt` lists `openai`, `tree-sitter` with its C and C++ grammars, `python-dotenv`, `pydantic`, `rich` and `pytest`. Semgrep is **not** in that file and must be added separately. angr is optional (used only for the Driller self-test).

terminal (from the repo root)

```
pip install -r requirements.txt
pip install semgrep
pip install angr        # optional
```

3. **Install the system tools with Homebrew.** `llvm` provides a clang with the libFuzzer and LeakSanitizer runtimes that Apple's clang lacks; `afl++` is the second fuzzer; `cbmc` is the formal prover. Apple's own clang (from Xcode Command Line Tools), `patch` and `git` are assumed to exist already.

terminal

```
brew install llvm afl++ cbmc
```

4. **Set up your API key.** The repo ships a template file literally named `.env copy` (with a space), containing an empty `OPENAI_API_KEY=` line. Copy it to `.env` in the repo root and type your own key after the `=`. Only `<repo>/.env` is read. Optional settings: `OPENAI_BASE_URL` (to use another OpenAI-compatible server, such as a local one), `MODEL_ALPHA`, `MODEL_BETA`, `MODEL_GAMMA` (default `gpt-4o-mini`) and `KAVACH_SEED` (default 1337).

terminal

```
cp ".env copy" .env
# now open .env in an editor and put your key after OPENAI_API_KEY=
```

WATCH OUT

- **Never commit or share `.env`.** It is listed in `.gitignore`, which is correct.
- **Always run commands from the repo root.** Many paths (`tests/fuzz_harnesses`, `reports`, `benchmark_workspace/...`) are relative to the folder you are in. From anywhere else, the real-CVE suite silently finds zero targets.
- **Do not copy the "Full Cleanup" commands in `INSTALLED.md` as written.** Its `rm -rf` lines contain another developer's absolute folder paths.
- On macOS, AFL++ can fail with `shmget() failed`; the code sets `AFL_MAP_SIZE=65536` to work around it, and `sudo afl-system-config` is the full fix.

Step 2: commands, from safest to biggest

Each level costs more time, more tools, or real API money. Start at the top.

Level	Command	Needs API key?	What it shows
1. Check config	`python -c "from config import get_llm_config; print(get_llm_config()['models'])"`	Yes (but makes no call)	Prints which model each agent will use. A missing key raises a clear error.
2. Unit tests	`python -m pytest tests/unit -q`	No (except `test_config_loading`)	Pass/fail for every honesty rule. Tests needing clang or cbmc skip if absent.
3. Cheat detector	`python -m tests.cheat_detector.run_cheat_detector`	No	A table of CORRECT / OVERFITTED / GUTTED patches: all pass the gates, but hardening catches the two cheats. Ends with PASS or FAIL. Needs clang.

Level	Command	Needs API key?	What it shows
4. One target	`python benchmark.py --suite synthetic --only SYN-06-OFF-BY-ONE --harden --formal`	Yes (a fraction of a cent)	The full journey on one bug, including hardening and a CBMC proof.
5. Quick slice	`python benchmark.py --suite synthetic --limit 5`	Yes	The first five synthetic targets.
6. Fuzz discovery only	`python benchmark.py --suite fuzz --fuzz-seconds 30`	Yes	Find bugs by fuzzing, then fix them. Add `--fuzz-engine afl++` to try AFL++.
7. Real CVEs	`sh tests/real_cve_suite/setup.sh` then `python -m tests.real_cve_suite.run_real_cve` or `python benchmark.py --suite real_cve --harden`	Yes (setup needs internet for git)	Fixes for three real published cJSON bugs.
8. Standalone suites	`python -m tests.vanguard_nightmare.run_vanguard` `python -m tests.public_benchmarks_extension.run_public_benchmarks`	Yes	The hard hand-written cases and the CVE-inspired pair.
9. Full benchmark	`python benchmark.py --suite all --fuzz --harden --memory --formal --fuzz-seconds 30`	Yes (cents)	Everything `benchmark.py` can do. Reported run: about 13 minutes total including the standalone suites.
10. Your own code	`python -m src.main -t path/to/file.c --build-cmd '...' --pov-cmd '...' --test-cmd '...'`	Yes	Semgrep-detected bugs in your code, patched and checked. Without `--build-cmd` and `--pov-cmd` the patches are unverified, and the tool says so.

TRY IT YOURSELF

With no API key at all, you can still see the project's central idea in action:

terminal (repo root, conda env active)

```
python -m tests.cheat_detector.run_cheat_detector
echo "exit code: $?"
```

Look at the table. In the PoV and Regression columns, all three patches show PASS. Only the Hardening column separates them. An exit code of 0 means every cheat was caught and the honest fix survived.

TRY IT YOURSELF

Check the Wilson numbers by hand with Python's calculator mode:

terminal

```
python -c "from benchmark import wilson_interval as w; print(w(44,44), w(30,30), w(2,2))"
```

You should see lower bounds near 0.920, 0.886 and 0.342. (Importing `benchmark` loads the whole project, so this needs the Python packages installed, but it makes no API call.)

Step 3: where to look afterwards

Location	What you will find
`reports/benchmark_runs/<timestamp>/`	The six files from section 6.4: three CSVs, the JSON and Markdown summaries, and the audit report. Start with `benchmark_summary.md`, then open `per_target_results.csv` in a spreadsheet.
`reports/audit_report_<timestamp>.md`	The audit report from a `src/main.py` run.

Location	What you will find
`reports/vanguard_nightmare_report.md`, `reports/public_benchmarks_report.md`, `reports/real_cve_report.md`	Reports from the standalone runners (overwritten each run).
`benchmark_workspace/memory/`	`episodes.jsonl`, `semantic.json`, `procedural.json`. Updated by `--memory` runs (and episodic/procedural only when memory is on).
`benchmark_workspace/fuzzing/`	Fuzz builds, corpora and crash files (gitignored; safe to delete).
`benchmark_workspace/real_cve_trees/`, `benchmark_workspace/cJSON_cve/`	The vulnerable cJSON checkouts made by `setup.sh`.
The terminal itself	Pre-flight warnings ("PoV did not reproduce — PoV gate disabled"), per-target status lines, hardening and CBMC results, and the final summary table.

When reading any result, use three questions from this chapter: Which gates actually ran (look for SKIPPED)? Was the PoV shown to reproduce on the original (the pre-flight CSV)? Was the patch attacked, and did it hold (the Hardening column)?

CHAPTER RECAP

- Memory has four tiers. **Working** memory (the rejected-attempt ledger) is always on and really used. **Semantic** memory (proven fix patterns) is used with `--memory`. **Episodic** and **procedural** memory are written but nothing reads them to make decisions.
- Recall score = (+3 for same CWE, +2 for matching crash class, + word overlap from 0 to 2) × confidence; keep if above 0.5; return the top 2. Confidence = base (0.9 if hardened, else 0.6), blended 50/50 with the success rate once the pattern has been used.
- Memory caveats: off by default in the benchmark, committed to git, learned before hardening, and not used by the fuzz suite, the CLI or the standalone runners.
- The audit report has three gate states, PASS, FAIL and SKIPPED, and a skipped gate is never shown or counted as a pass.
- `src/main.py` is a six-step, Semgrep-only CLI; `benchmark.py` adds pre-flight, hardening, CBMC, memory, fuzz discovery and measured reports.
- Every target is a `Target` record; leaving out a PoV or regression check requires a written reason. Pre-flight disables any PoV that does not fail on the unpatched original.
- The Wilson interval gives honest ranges for perfect scores; for n/n the lower bound is n ÷ (n + 3.84), e.g. 92.0% for 44/44.
- Reported: 44/44 passed all gates, 30/30 PoV-proven, 32 hardened and 0 falsified, $0.0739, 804s. But only 3 targets are external, 14 cannot be PoV-proven, and the raw run folder is not in the repo.
- Safe first steps: unit tests and the cheat detector need no API key. Always run from the repo root.

CHECK YOURSELF

1. A pattern is hardened, has been recalled 4 times, helped 3 times and misled once. What is its confidence? *(Answer: $0.5 \times 0.9 + 0.5 \times 0.75 = 0.825$.)*
2. Why does pre-flight run the PoV against the *unpatched* program, and what happens if the PoV passes there? *(Answer: to prove the PoV really detects the bug; if it passes on the original, the PoV gate is disabled for that target and it cannot count as PoV-proven.)*
3. Name two memory features that are recorded but not actually used to make decisions. *(Answer: the episodic log, and procedural memory's `preferred_agent_order` / `suggested_iterations`.)*
4. Why does the project use the Wilson interval rather than the textbook formula when it scores 44/44? *(Answer: the textbook margin uses $p(1-p)$, which is 0 at $p = 1$ and so falsely claims certainty; Wilson stays open, giving [92.0, 100].)*
5. Which two commands can you run without any API key, and what does each prove? *(Answer: `python -m pytest tests/unit -q` checks the honesty rules; `python -m tests.cheat_detector.run_cheat_detector` shows hardening catching overfitted and gutted patches that pass every gate.)*

CHAPTER 7

Honest Assessment, FAQ and Glossary

The earlier chapters explained what AI Kavach does and how it does it. This chapter steps back and asks the questions a careful judge, reviewer or new team member would ask. What is genuinely good here? What is broken? Where do the documents say one thing while the code does another? What would you fix first if you had only one day?

Everything in this chapter was checked against the actual files in the repository. Where a claim could not be confirmed, it was left out. File names and line numbers are given so you can open the file and see for yourself. Line numbers refer to the repository as it stood when this guide was written; they will drift as the code changes.

KEY IDEA

A project can be well designed *and* have bugs that make its numbers look better than they are. Those two facts do not cancel each other out. The point of this chapter is to hold both in mind at once: respect the design, and stay sceptical about the headline numbers until the bugs below are fixed.

1. What is genuinely strong (and why it matters)

Before the criticism, here is what holds up well. Several of these ideas are better than what many larger projects do.

1.1 "The LLM proposes, deterministic tools decide"

This is the core design idea, and it is a good one. A language model is good at writing plausible code but cannot be trusted to grade its own work. So AI Kavach never asks the model whether its patch is correct. Instead, a fixed series of mechanical checks (the **gates**) decides: the patch must apply, the program must compile with sanitizers on, the crashing input must stop crashing, and a normal input must still give the normal answer. A gate either passes or fails. It cannot be talked into anything.

ANALOGY

Think of a student taking a driving test. The student (the LLM) may be confident, articulate and charming. None of that matters. The examiner (the gates) watches whether the car stays in the lane, stops at the red light and parks without hitting the kerb. The student's opinion of their own driving is not part of the score.

1.2 Pre-flight: the test must be able to fail

Before any patching happens, the system builds the *unpatched* program and runs the proof-of-vulnerability (PoV) against it. If the PoV does not trigger the bug on the original code, it is useless as a test, because it would also "pass" on a patch that does nothing. In that case the PoV gate is switched off for that target and the result is reported as not PoV-proven.

benchmark.py · lines 145-149

```
        if target.pov_command:
            # PoV scripts exit non-zero when the vulnerability IS present.
            ok, detail = DRVLoop._run(DRVLoop.expand(target.pov_command, ctx), "PoV", cwd=workspace)
            res.pov_reproduces = not ok
            res.pov_detail = detail if not ok else "PoV did NOT reproduce on the original binary"
```

LINE BY LINE

1. Only targets that actually have a PoV command are checked.
2. The comment reminds the reader of the convention: the PoV script "fails" (non-zero exit) when the bug is present.
3. The PoV is run against the **original, unpatched** build.
4. `pov_reproduces = not ok`: if the script failed, the bug was really there, which is what we want.
5. If it did not fail, the result records that the PoV did not reproduce on the original, so it cannot be trusted later.

This is the scientific habit of checking that your thermometer can actually read a fever before using it to declare a patient healthy.

1.3 Honest reporting of skipped checks

When a gate was not configured for a target, the audit report prints `SKIPPED (not configured for this target)`, never `PASS` (`src/reporting/audit_report.py`, line 25). Many tools silently treat "did not run" as "fine". This one refuses to.

1.4 A target must explain any missing test

Every benchmark target is described by a `Target` record. If someone leaves out the PoV or the regression test, they must write down why, or the program refuses to start:

tests/benchmarks/targets.py · lines 112–118

```
    def __post_init__(self):
        if self.pov_command is None and not self.pov_na_reason:
            raise ValueError(f"{self.id}: pov_command=None requires pov_na_reason")
        if self.regression_command is None and not self.regression_na_reason:
            raise ValueError(
                f"{self.id}: regression_command=None requires regression_na_reason"
            )
```

This makes gaps visible by design. You cannot quietly drop a test; you have to sign your name to the reason.

1.5 Hardening: attacking your own fix

Passing the gates is not the end. With `--harden`, the system tries to *break* the accepted patch: it fuzzes the patched program again, compares the original and patched programs on harmless inputs, and runs a battery of tricky bypass inputs for input-validation fixes. A patch that only blocks the one known bad input (overfitted) or that "fixes" the bug by switching the feature off (gutted) can be caught here.

1.6 The cheat detector

`tests/cheat_detector/run_cheat_detector.py` builds three patches for one bug, needing no LLM at all: a correct one, an overfitted one and a gutted one. All three pass the normal gates. The script succeeds only if hardening catches both cheats and lets the honest fix through. It is the clearest single demonstration of why the hardening stage exists, and it can be run by anyone without an API key.

1.7 Other solid choices

- **Whole-function patches spliced by the syntax tree.** Instead of fragile line-by-line diffs, the model can return the whole fixed function, which tree-sitter swaps in by its exact position in the file. This removed the main cause of rejected patches.
- **A deterministic escalation policy.** The rules for when to call the Critic, when to force the whole-function format and when to give up are plain code (`EscalationPolicy` in `src/agent_orchestrator/critic.py`), not another LLM decision.
- **Smallest validated patch wins.** Among agents whose patches passed every gate, the one with the fewest changed lines is chosen (`_select_winner`, `orchestrator.py` lines 306–332). Small patches are easier for humans to review.
- **Confidence intervals.** The headline "44/44" is reported with a Wilson 95% interval of [92.0, 100.0], with the explicit warning that 44 successes do not prove a 100% rate.
- **A "Not Measured" section.** `benchmark.md` lists what was not measured (for example detection precision and recall, CPU use, concolic execution). That kind of honesty is rare.
- **Bounded formal proofs where available.** For three targets, CBMC proves that the patched function is safe for every input within a loop bound, which is stronger than any amount of testing.

2. Known bugs that can inflate results

These are real defects in the code. Each one, in some situation, lets the system report a stronger result than it actually earned. None of them means the system is fake; they mean some numbers need an asterisk until fixed.

2.1 A patch nobody fuzzed can be labelled "hardened"

In plain words. The re-fuzz check builds a special fuzzing version of the patched program and runs it for 20 seconds. If the build fails, or the run goes over its time limit, the check returns "no new crash found" (`False`) together with an explanation. The calling code then marks the patch as "checked for overfitting" anyway, because it only looks for two specific phrases ("no fuzz harness", "no clang") to decide that the check did not happen.

src/patch_validator/hardening.py · lines 308–309, 329–330, 440–448

```
            if build.returncode != 0:
                return False, f"re-fuzz build failed: {build.stderr[:300]}"
...
            except subprocess.TimeoutExpired:
                return False, "re-fuzz exceeded its budget"
...
        # --- adversarial re-fuzzing ---
        found, detail = self.refuzzer.check(
            target_id, patched_source, seed_crash,
            seconds=refuzz_seconds, libfuzzer_clang=libfuzzer_clang,
        )
        if "no fuzz harness" not in detail and "no clang" not in detail:
            verdict.overfitting_checked = True
            verdict.overfitted = found
            verdict.new_crash_detail = detail
```

LINE BY LINE

1. If compiling the fuzz build fails, the function returns `False` ("no crash found") and a message.
2. If the fuzzing run times out, it again returns `False` and a message.
3. Back in `harden()`, the result is examined.
4. Only the phrases "no fuzz harness" and "no clang" are treated as "the check did not run". "re-fuzz build failed" and "re-fuzz exceeded its budget" are not on the list.
5. So `overfitting_checked` becomes `True`, which counts as evidence that the patch survived an attack.

Why it matters. A patch is labelled SURVIVED when at least one check produced evidence and none falsified it. With this bug, a broken fuzz build counts as evidence. Worse, `benchmark.py` lines 261–267 then mark the patch's memory pattern as `hardened`, which raises that pattern's starting confidence from 0.6 to 0.9 (`src/memory/store.py`, line 208). So the error spreads into future prompts.

Suggested fix. Have the re-fuzzer return a three-way answer (crashed / ran cleanly / could not run) and set `overfitting_checked` only when the fuzzer actually ran to completion.

2.2 The Semgrep re-scan "fails open"

In plain words. Gate 5 re-scans the patched file with Semgrep to make sure the patch did not introduce a new problem. If Semgrep itself crashes (missing binary, no network for the ruleset, a parse error), the code logs an error but returns "pass".

src/patch_validator/drv_loop.py · lines 688–691

```
        except Exception as e:
            # A scanner malfunction must not be reported as a security verdict.
            logger.error(f"Post-patch scan error (gate not enforced): {e}")
            return True, ""
```

The comment says the right thing ("must not be reported as a security verdict"), but the caller turns `True` into `GATE_PASS` (line 403). So in the report a scanner crash looks exactly like a clean scan.

Why it matters. It contradicts the project's own "SKIPPED is never PASS" rule. In practice the benchmark does not use this gate (it passes no Semgrep runner, so the gate stays SKIPPED), but the command-line tool does.

Suggested fix. On an exception, set the gate to `GATE_SKIPPED` with the error as the reason, instead of returning a pass.

2.3 "Patched" with no real gate at all

In plain words. Each of the build, PoV, regression and re-scan gates runs only if a command was supplied. If none were supplied, the code falls straight through to the end and declares the patch fully validated, even though the only thing checked was that the patch applied to the text.

src/patch_validator/drv_loop.py · lines 365–366, 410–413

```
            # Gate 2 — build
            if build_command:
```

```
...
                    return False

        result.fully_validated = True
        return True
```

Why it matters. "Fully validated" in this codebase means "no gate failed", not "every gate ran". The command-line tool does print a warning (`src/main.py`, line 216: "A patch with no build/pov gate is unverified."), and the audit lists which gates ran. But the status is still `patched`, and a reader skimming the summary line sees it counted as passed.

Suggested fix. Refuse the `patched` status unless at least the build gate and one behavioural gate (PoV or regression) actually ran; otherwise use a separate status such as "applied, unverified".

2.4 Fuzz-suite rows can be mislabelled

In plain words. In the fuzz pipeline, a crash whose location could not be found is skipped, so no patch result is created for it. Later, the benchmark pairs the list of crash clusters with the list of patch results by position, using `zip`. If cluster number 1 was skipped, patch result number 1 actually belongs to cluster number 2, and every row after it is shifted by one.

src/fuzz_pipeline.py · lines 116-119

```
        for vuln, cluster in zip(vulnerabilities, result.clusters):
            if not vuln.file_path or not Path(vuln.file_path).exists():
                logger.warning(f"[{target_id}] {vuln.id}: crash location unresolved, skipping")
                continue
```

benchmark.py · line 479

```
            for cluster, pr in zip(discovery.clusters, discovery.patch_results):
```

ANALOGY

Imagine a teacher who hands back exam papers by walking along a row of desks, but one student was absent and no paper was written for them. If the teacher pairs papers to desks purely by order, every student after the empty desk gets their neighbour's paper.

Why it matters. The per-target CSV can attribute a crash class and location to the wrong patch. The overall count of fixes is unaffected, but the details in each row can be wrong.

Suggested fix. Store each cluster alongside its patch result (for example as a pair) inside the pipeline, instead of matching two lists by position afterwards.

2.5 Fuzz-suite PoV and regression counts are copied, not counted

In plain words. For the fuzz suite summary, the number of "PoV-verified fixes" and "regression-verified fixes" are simply set equal to the number of patched bugs.

benchmark.py · lines 525-530

```
            "Patched_All_Gates": patched_total,
            "Success_Rate": f"{patched_total / bugs_total * 100:.1f}%" if bugs_total else "n/a",
            "Targets_With_Reproducible_PoV": bugs_total,
            "PoV_Verified_Fixes": patched_total,
            "PoV_Verified_Rate": f"{patched_total / bugs_total * 100:.1f}%" if bugs_total else "n/a",
            "Regression_Verified_Fixes": patched_total,
```

Why it matters. For the PoV, this is very likely true in practice, because the fuzz pipeline always supplies the crashing input as a PoV, so a patched bug did pass it. For regression, it is an assumption: if a target had no regression command, the gate was SKIPPED, yet it is still counted here as regression-verified. It is the kind of shortcut that is harmless until it isn't.

Suggested fix. Count these from the winning iteration's actual gate states (`pov_ok`, `regression_ok`), which the same function already reads a few lines earlier.

2.6 On Linux, most AddressSanitizer crashes get no location

In plain words. When ASan reports a crash, it prints a stack trace. Each line looks a little different on macOS and Linux. On Linux a frame typically ends with `file.c:LINE:COLUMN`; the macOS reports used in the tests end with `file.c:LINE`. The regular expression that reads these lines does not allow for the extra `:COLUMN`.

src/analysis_engine/crash_triage.py · line 84

```
_FRAME = re.compile(r"^\s*#(\d+)\s+0x[0-9a-f]+\s+in\s+(.+?)(?:\s+([^\s]+\.(?:c|cc|cpp|h)):(\d+))?\s*$")
```

Running it on a Linux-style line shows the problem:

- `#0 0x4f8a2b in vuln_func /src/a/file.c:10:5` gives function `"vuln_func /src/a/file.c:10:5"`, file `None`, line `None`.
- `#0 0x100003f4c in vuln_func file.c:10` (macOS style) gives function `vuln_func`, file `file.c`, line `10`, as intended.

Because the file/line part is optional, the regex still "matches" the Linux line; it just puts everything into the function name and leaves the location empty.

Why it matters. A crash with no location is dropped by the fuzz pipeline as "crash location unresolved" (see 2.4). So on a Linux machine, fuzzing could find real crashes and then silently patch nothing. UBSan errors use a different pattern and are not affected. All the unit tests use the macOS format (for example `tests/unit/test_analysis_engine.py` line 25), so the tests do not catch this.

Suggested fix. Allow an optional column: change `:(\d+))?` to `:(\d+)(?::\d+)?)?`. A quick check confirms this parses both formats correctly. Add one Linux-format test.

2.7 Memory learns a fix before hardening has tested it

In plain words. When `--memory` is on, the orchestrator records a successful fix as a reusable "pattern" at the end of `process_vulnerability`. Hardening and formal verification happen later, in `benchmark.py`. If hardening later falsifies the patch, the pattern is not removed.

src/agent_orchestrator/orchestrator.py · lines 166-167

```
        if self.memory:
            self._update_memory(vuln, result, recalled)
```

Then, in `benchmark.py` (lines 235–257), `process_vulnerability` returns, and only *afterwards* is `_harden_patch` called. On falsification the status is downgraded, but nothing touches memory.

Why it matters. A fix proven wrong can be recalled as a "verified fix pattern" into future prompts, nudging the model toward the same mistake. `architecture.md` line 277 says the opposite: that a pattern is written "only after" it survives hardening.

Suggested fix. Delay the memory write until hardening and formal checks finish, or delete (or mark as misleading) the pattern when the patch is later falsified or violated.

2.8 The PoV script accepts any "polite" exit as a fix

In plain words. `pov_run.sh` fails only if the output contains sanitizer text or the process died from a signal (exit code 128 or more). Any other outcome, including exit code 1 from a program that simply gave up early, counts as "bug gone".

tests/benchmarks/harness/pov_run.sh · lines 25-39

```
case "$output" in
    *"AddressSanitizer"*|*"UndefinedBehaviorSanitizer"*|*"runtime error:"*|*"LeakSanitizer"*)
        echo "$output"
        exit 1
        ;;
esac

# 128+N indicates death by signal N (SIGSEGV=139, SIGABRT=134, SIGBUS=138).
if [ "$rc" -ge 128 ]; then
    echo "process died on signal (exit $rc); output follows:"
    echo "$output"
    exit 1
fi
```

```
exit 0
```

Why it matters. A patch that inserts `exit(1)` at the top of the function would pass the PoV gate. Normally the regression gate catches this, because the program must still produce the expected output on a normal input. But for targets with no regression test (the two Juliet cases, for example, whose "bad" functions have no benign path), the PoV gate is the only behavioural check. Hardening's differential test also cannot help there, because it skips inputs that crash the original and so compares zero inputs; the code correctly treats that as "no evidence" (`hardening.py` lines 78–92), and `benchmark.md` reports those two as NOT HARDENED.

Suggested fix. For targets without a regression test, have the PoV also check for a specific expected outcome (for example a known error message or exit code), so "gave up" and "handled it correctly" can be told apart.

2.9 Smaller issues worth knowing

- **`--memory` does nothing for the fuzz suite.** The fuzz path builds its orchestrator with no memory argument (`benchmark.py` line 458), so recall and write-back never happen there.
- **The Critic can fail silently.** It asks the model for a strict JSON-schema response. Any exception is caught and logged as a warning, and the loop carries on without critique (`critic.py` lines 177–179). That is a sensible fallback, but on servers that don't support JSON-schema mode the Critic will simply never contribute, and nobody will notice unless they read the logs.
- **The public extension runner can crash.** In `tests/public_benchmarks_extension/run_public_benchmarks.py`, the table code handles a target excluded by pre-flight (`res is None`), but the Markdown report code at line 160 calls `res.status.upper()` without that check.

WATCH OUT

None of these bugs is evidence that the 2026-08-30 run actually suffered from them. For example, the false-SURVIVED bug only matters if a re-fuzz build failed or timed out in that run, and the raw run logs are not in the repository (see section 5), so we cannot tell. The honest statement is: "these bugs *could* have inflated the numbers, and the published numbers cannot be re-checked from the repository alone."

3. Places where the docs disagree with each other or with the code

The project has several documents: `README.md`, `benchmark.md`, `architecture.md`, `INSTALLED.md`, `submission.html` (the presentation) and comments in the code. They were updated at different times, and some drifted apart.

Claim	Where	Reality
"In the 2026-08-30 run, two patches that had passed every gate were falsified here: SYN-15 … and SYN-16"	README.md lines 188–191	benchmark.md (line 51) reports **0** falsified for the same run, and lists SYN-15 as NOT HARDENED because of its `replace` contract.
Hardening: 32 attacked, 0 falsified, 5 not attacked	benchmark.md line 43	32 + 5 = 37, not 44. The code only hardens targets in `run_suite`; the fuzz suite (7 bugs) is never hardened, which accounts for the gap, but the docs do not say so.
"All 28 reproduced on their unpatched originals"	benchmark.md line 149	The same file reports 30/30 PoV-proven; the presentation says "17 reproduce, 14 do not" (submission.html line 489). Three different counts.
3 formal proofs, "28 targets have no proof harness and report `unavailable`"	benchmark.md line 371	3 + 28 = 31, not 44. The remaining targets are not accounted for.
102 unit tests	architecture.md line 331	README.md and benchmark.md say 125.
"fuzzes four harnesses"	benchmark.md line 391	The same file (line 67) says nine, and `tests/fuzz_harnesses/` contains nine harness files.
VANGUARD-02 won by agent **alpha**	benchmark.md line 163	The suite's own report says **beta** (reports/vanguard_nightmare_report.md line 14).
"$0.0017 median" cost per remediation	submission.html line 641	benchmark.md line 313 gives the median as $0.0009; $0.0017 is the mean. The same slide later says "median $0.0009/target".
Deliverables: "Synthetic 20/20 · NIST Juliet 2/2 · cJSON 6/6 · real published CVEs 3/3 · fuzz-discovered 7/7"	submission.html line 671	Those add up to 38, but the same box says "44 targets"; Vanguard (4) and the extension suite (2) are missing.

Claim	Where	Reality
"LangGraph Multi-Agent Orchestrator"	orchestrator.py line 2; agent_orchestrator/__init__.py line 1	LangGraph is not used. Agents run on a `ThreadPoolExecutor`; requirements.txt and architecture.md line 112 say so.
"First validated patch wins"	orchestrator.py line 4	The code waits for all agents and picks the smallest validated patch (`_select_winner`).
Critic "Invoked only on repeated failure"	critic.py line 136	`should_invoke_critic` returns `True` on the **first** PoV failure (lines 219–220).
Procedural memory does "iteration routing per CWE"	architecture.md line 269	`preferred_agent_order` and `suggested_iterations` are only called from tests. README.md (around line 148) admits they are not wired in.
Fix patterns written to memory "only after" they survive hardening	architecture.md line 277	Memory is written before hardening runs (section 2.7).
"Executes AI Kavach on … DARPA CGC MultiOS, and Historical CVEs" / " (LibTIFF)"	run_public_benchmarks.py lines 4 and 148	The suite's own manifest says the targets are CVE-*inspired* synthetic programs and "not LibTIFF source", not CGC.
VANGUARD-03 labelled CWE-191 (integer *underflow*)	tests/vanguard_nightmare/manifest.py line 57	The note right below describes a *multiplication overflow* (CWE-190) plus a second, unlabelled overflow.

KEY IDEA

Most of these are bookkeeping mistakes, not hidden failures. But to a judge, inconsistent numbers look the same as invented ones. The cheapest credibility win in the whole project is to make every document quote one source of truth.

4. Features that exist but aren't wired in

"Wired in" means the main pipeline actually calls the code. Several pieces are written, sometimes tested, but never used during a real run.

Feature	Where	Status
Driller / angr concolic engine (`drill()`) and `PlateauMonitor`	src/analysis_engine/driller_monitor.py	Only `self_test()` is called (benchmark.py line 703). Nothing calls `drill()` or the plateau monitor. On arm64 macOS the self-test fails and the engine disables itself anyway.
Procedural memory routing	src/memory/store.py lines 428–435	Stats are recorded; the two methods that would use them are called only by unit tests.
`--language` CLI flag	src/main.py lines 66, 247, 293	Parsed and passed into `run_pipeline`, then never used. The orchestrator always calls `extract_context(file, line)` with the default language `"c"`.
`SAN_FLAGS`	src/analysis_engine/fuzzer_manager.py line 32	Defined, never referenced.
CASR crash triage	src/analysis_engine/crash_triage.py line 136	Used only if both `casr-san` and `casr-cluster` are installed. They are not installed in the documented setup, and only `casr-san` is ever actually run.
`"error"` pipeline status	orchestrator.py line 39	Listed as a possible status; nothing ever sets it.
Semgrep post-scan gate in the benchmark	benchmark.py (Orchestrator built with no Semgrep runner)	Gate 5 is always SKIPPED in benchmark runs; it only runs in the CLI.
UI concept	design/ui-concept/index.html	A visual mock-up with illustrative data, not connected to the system.

5. Operational risks

These are not bugs in the logic. They are ways the project can fail, mislead or cause harm when someone new tries to run it.

5.1 Dependencies missing from requirements.txt

`requirements.txt` lists openai, tree-sitter (C and C++ grammars), python-dotenv, pydantic, rich and pytest. It does **not** install Semgrep (it says in a comment "semgrep is installed separately") and does not mention angr at all. A fresh `pip install -r requirements.txt` therefore gives you a system whose CLI detector cannot run. The native tools (clang/LLVM, AFL++, CBMC) also have to be installed separately, for example with Homebrew.

5.2 Paths that depend on where you start the program

Many paths are written relative to the current folder, for example `Path("tests/fuzz_harnesses")` (src/fuzz_pipeline.py line 34, hardening.py line 346, fuzzer_manager.py line 68), `Path("tests/benchmarks/cbmc_harnesses")` (benchmark.py line 336) and `"reports/benchmark_runs"` (benchmark.py line 164). If you run from any folder other than the repository root, harnesses go missing and checks quietly report "unavailable" or "no harness". The real-CVE suite is the worst case: `targets.py` lines 501–505 wrap its import in `except Exception: return []`, so a problem there silently produces zero targets rather than an error.

Rule of thumb: always run from the repository root.

5.3 `shell=True` on manifest commands

Build, PoV and regression commands are run through the shell (`drv_loop.py` line 607; also in `hardening.py`). That is convenient (commands can use `&&`, quotes and redirection) and fine for a trusted, hand-written benchmark. It would be dangerous if the commands ever came from an untrusted source, because a command string can do anything your user account can do.

5.4 Semgrep's default ruleset needs the internet

The Semgrep runner defaults to `p/security-audit` (`semgrep_runner.py` line 51). That is a ruleset fetched from the Semgrep registry, so a truly offline machine cannot use it without a locally saved copy of the rules. This matters because the project advertises "air-gap" use.

5.5 `INSTALLED.md` contains `rm -rf` lines with another developer's paths

The uninstall section (around lines 96–100) contains commands such as `rm -rf "/Users/piyush/Desktop/cyber hackathon/ai_kavach_crs/benchmark_workspace"`. These are one developer's personal absolute paths. Copy-pasting a block of commands from a setup guide is a common habit. Do not run these as written; on another machine they will either fail or, if adapted carelessly, delete the wrong folder.

5.6 Memory files are committed to git

`benchmark_workspace/memory/episodes.jsonl`, `procedural.json` and `semantic.json` are tracked in the repository. Anyone who clones the repo and runs with `--memory` starts from someone else's learned patterns, not from a clean slate. Combined with bug 2.7, those patterns may include fixes that were later falsified. It also means a "fresh" run is not actually fresh, which matters when comparing results.

5.7 The raw results are gitignored

`.gitignore` excludes `reports/benchmark_runs/`, the folder that holds the per-run CSVs, JSON summaries and audit reports. Only the three standalone suite reports (`reports/vanguard_nightmare_report.md`, `real_cve_report.md`, `public_benchmarks_report.md`) are committed. So the headline 44/44 run cannot be inspected from the repository alone; you have to trust `benchmark.md` or re-run it yourself, which needs an API key and the full toolchain.

5.8 Smaller configuration traps

- Only `<repo root>/.env` is loaded (`config/__init__.py` line 12). The tracked template is named `.env copy`, with a space, and must be copied to `.env`.
- The unit test `test_config_loading` asserts the API key starts with `sk-` (`tests/unit/test_core.py` line 21), which fails for keys from local or non-OpenAI servers.
- The Critic always uses Agent Beta's model (`critic.py` line 140); there is no separate setting for it.

6. If I had one day: a prioritized improvement list

Ordered by how much trust each change buys per hour of work. The first five are small code changes; most are a few lines each.

1. **Fix the false-SURVIVED bug (2.1).** Make the re-fuzzer report "could not run" separately, and only set `overfitting_checked` when fuzzing actually ran. This protects the hardening numbers and the memory confidence scores.
2. **Make the post-scan gate fail closed (2.2).** On a Semgrep exception, record SKIPPED, not PASS. One line.

3. **Fix the Linux stack-frame regex (2.6) and add a Linux-format test.** Without this, fuzz discovery quietly does nothing on the most common server platform.
4. **Move the memory write after hardening and formal checks (2.7)**, and exclude falsified patches. Then reset the committed memory files so the next run starts clean.
5. **Refuse "patched" when no build or behavioural gate ran (2.3).** Introduce a distinct status for "applied but unverified".
6. **Fix the fuzz-suite bookkeeping (2.4, 2.5).** Keep each cluster with its result, and count PoV/regression from actual gate states.
7. **Reconcile every number in every document (section 3).** Generate the numbers in README, benchmark.md and the presentation from one `benchmark_summary.json`, and fix the stale code comments (LangGraph, "first validated patch wins", "only on repeated failure").
8. **Publish the raw artifacts of the headline run (5.7)**, either by committing that one run's directory or attaching it to a release.
9. **Make paths independent of the current folder (5.2)** by building them from `Path(__file__)`, and replace the silent `except Exception: return []` with a clear warning.
10. **Tidy the setup.** Add Semgrep (and optional angr) to requirements, ship a local copy of the Semgrep rules for offline use, delete the personal `rm -rf` lines from `INSTALLED.md`, and relax the `sk-` test.
11. **Strengthen PoVs for targets with no regression test (2.8)** so an early exit is not mistaken for a fix.

TRY IT YOURSELF

Without an API key, you can still run `python -m tests.cheat_detector.run_cheat_detector` and `python -m pytest tests/unit -q` from the repository root. Then, as an exercise, open `src/analysis_engine/crash_triage.py`, find the `_FRAME` line, and test it in a Python shell against the two example frame lines from section 2.6. Seeing the Linux line fail with your own eyes makes the bug concrete.

CHAPTER RECAP (SECTIONS 1-6)

- The design is strong: deterministic gates judge, the PoV is pre-flighted, skipped checks are shown as SKIPPED, and hardening attacks accepted patches.
- Several real bugs can make results look better than they are: broken re-fuzz builds counted as hardening evidence, a scanner crash counted as a pass, "patched" with no real gates, misaligned fuzz rows, assumed counts, a Linux regex bug, and memory learning before hardening.
- The documents disagree with each other on several numbers (falsified count, pre-flight counts, test counts, harness counts, median cost, deliverable totals).
- Some features (Driller, procedural routing, `--language`) are written but not wired in.
- Operational traps: missing dependencies, folder-dependent paths, `shell=True`, online Semgrep rules, dangerous `rm -rf` lines, committed memory and uncommitted results.
- Most of the important fixes are small, and the top five could be done in a day.

CHECK YOURSELF

1. Why does a failed re-fuzz *build* end up counted as hardening evidence? Which two phrases does the code check for, and why is that not enough?
2. The project's own rule is "SKIPPED is never PASS". Which gate breaks that rule, and under what circumstances?
3. Explain in your own words why pairing two lists with `zip` can attach the wrong crash to a patch.
4. A patch adds `exit(1)` at the top of a vulnerable function. Which gate would normally catch it, and for which kind of target might it slip through?
5. Why do 32 attacked plus 5 not-attacked add up to 37 instead of 44?

7. Frequently asked questions

Why not just use Claude Code, Codex or another AI coding assistant?

Those tools are excellent at writing code when a human points them at a problem. AI Kavach does several things differently:

- **Who judges.** In a typical AI-assistant session, the same model that writes the fix also decides whether it is done, perhaps by running tests it chose. Here the model never judges. Fixed, deterministic gates decide, and the PoV is proven able to fail before it is trusted.
- **It finds the bugs itself.** With `--fuzz`, it fuzzes the program, triages the crashes, works out the location and CWE, and only then asks for a fix. A coding assistant needs to be told what is wrong.

- **It attacks its own fix.** Hardening re-fuzzes, compares behaviour and runs evasion inputs, trying to prove the patch wrong.
- **It leaves an audit trail.** Every gate, skipped or run, is recorded per target.

These are not rivals in principle. A coding agent could be plugged in as one of the patch writers inside this system; the gates would still decide whether its patch is accepted.

Is 44/44 = 100% real?

It is a measured result, with caveats. Every one of the 44 targets passed every gate that was *configured* for it, in one run. But: (1) only 30 of them were PoV-proven; the rest had no usable PoV, so "passed" means build and regression only; (2) only 3 targets come from outside sources (the real cJSON CVEs), the rest were written for this project; (3) several bugs in section 2 could inflate parts of the result; and (4) the raw run files are not in the repository. The project itself says 44/44 does not prove 100%, and gives a 95% Wilson interval of [92.0, 100.0], which applies only to this corpus.

Why use a cheap model like gpt-4o-mini?

Because the model is not trusted anyway. If a weaker model writes a bad patch, the gates reject it and it tries again. A weaker model costs more attempts, not more wrong answers. The reported total for all 44 targets was $0.0739 over 161 API calls. Temperature is fixed at 0 and the seed at 1337 to make runs as repeatable as the API allows (the config notes that seeded sampling is still not perfectly identical between runs).

Can it run offline (air-gapped)?

Partly. Setting `OPENAI_BASE_URL` points the system at any OpenAI-compatible server, such as a local vLLM instance, and no other outbound HTTP calls were found in `src/`. But three things need care: the default Semgrep ruleset `p/security-audit` is downloaded from the internet; the Critic needs a server that supports strict JSON-schema responses (otherwise it silently does nothing); and one unit test expects an OpenAI-style `sk-` key.

What languages does it support?

In practice, C. The benchmarks are all C, the build commands use clang, and the context extractor always uses the C parser in the pipeline, because the `--language` flag is parsed but never passed through. The C++ tree-sitter grammar is installed and the extractor can use it if called directly with `language="cpp"`, so C++ support is partly built but not connected.

What happens if no PoV exists for a bug?

The target must state why (the `pov_na_reason` field; the program refuses to start otherwise). The PoV gate is then SKIPPED and the report says so. The patch can still be accepted on build and regression evidence, and hardening may still attack it, but it is not counted as PoV-proven. The six real-world cJSON Semgrep findings are an example: they mostly have no PoV, and `benchmark.md` shows "–" in their PoV-proven column.

What is the difference between "patched", "PoV-proven", "hardened" and "proven"?

They are bars of increasing strength. **Patched**: passed every configured gate. **PoV-proven**: one of those gates was a PoV that was shown to crash the original. **Hardened (SURVIVED)**: at least one attack actually ran against the patch and failed to break it. **Proven**: CBMC showed no input within the loop bound can violate the safety checks. Only 3 targets reach the last bar.

Does it find bugs on its own, or only fix bugs it is told about?

Both modes exist. The standard benchmark gives it a list of known targets (file, line, CWE) and measures repair. The `--fuzz` mode genuinely discovers bugs: 9 harnesses fuzzed for 30 seconds each produced 8 crashing inputs, triaged to 7 distinct bugs, all of which were fixed. The command-line tool on your own code finds bugs only with Semgrep (no fuzzing).

Why three patch-writing agents instead of one?

They are three different instructions (system prompts) to the same kind of model: Alpha reasons step by step, Beta writes the smallest possible guard, Gamma applies textbook CWE fixes. Different styles fail in different ways, so running all three raises the chance at least one validated patch exists. The smallest validated patch then wins. They are not separate programs; they live as prompts in `config/__init__.py`.

Can the LLM cheat the gates?

It can try. A patch could special-case the one bad input (overfitting) or disable the feature (gutting). The regression gate catches crude gutting, and hardening is designed to catch the subtler versions; the cheat detector demonstrates this on one bug. The weak spots are targets with no regression test and no fuzz harness, and the PoV script's acceptance of any non-crashing exit (section 2.8).

How long does it take to run?

The reported full run of 7 suites took 804 seconds (about 13.4 minutes), with a median of 5.8 seconds per target and a peak memory use of 1.3 GB. Fuzzing and hardening time is set by flags such as `--fuzz-seconds`; the re-fuzz step defaults to 20 seconds.

Is it safe to run on code I don't trust?

Not as-is. It compiles and executes the target program, and runs manifest commands through the shell. Run it inside a container or a throw-away virtual machine when the code or the commands come from someone you do not trust.

Does the memory system make it smarter over time?

Somewhat. With `--memory`, up to two earlier fix patterns that match the CWE and crash type are added to the prompt, and each pattern's confidence rises or falls with how often it helped. But the procedural memory (which agent wins most often) is recorded and never used, memory is written before hardening, and the fuzz suite ignores `--memory` entirely. No measurement in the docs shows how much memory improves results.

Can I reproduce the headline numbers?

Only by re-running, because the raw output folder is gitignored. You need an OpenAI-compatible API key, clang/LLVM with libFuzzer, AFL++, CBMC, Semgrep, and must run from the repository root, for example `python benchmark.py --suite all --fuzz --harden --memory --formal --fuzz-seconds 30`. Because LLM sampling is not perfectly deterministic, expect close but not identical numbers.

What would make the results more convincing to an expert?

More targets from outside sources (real CVEs in real projects, rather than programs written for this project), the raw artifacts published alongside the numbers, the section 2 bugs fixed, and a measured comparison, for example the same targets with hardening turned off, or with memory off, to show what each stage adds.

8. Glossary

Every technical term used in this guide or in the project, in alphabetical order.

Term	Meaning
AFL++	A popular fuzzer that starts a fresh copy of the program for each input and uses coverage feedback to guide mutation. Supported as an alternative to libFuzzer.
Agent (Alpha, Beta, Gamma, Delta)	A role given to the LLM through a system prompt. Alpha = analyst, Beta = minimalist, Gamma = textbook CWE patcher, Delta = the Critic, which never writes code.
Air-gap	Running with no internet connection. Here, done by pointing `OPENAI_BASE_URL` at a local model server.
angr	A Python framework for analysing programs symbolically. Used by the (unwired) Driller engine.
API key	A secret string that identifies you to an online service such as OpenAI. Kept in `.env`, never committed.
ASan (AddressSanitizer)	A compiler add-on that makes a program stop with a report the moment it reads or writes memory it shouldn't: out of bounds, after freeing, freeing twice.
AST (Abstract Syntax Tree)	A tree-shaped representation of source code's structure (functions, statements, expressions), produced by a parser such as tree-sitter.
Audit report	The Markdown file written per run listing each target, which gates ran, which were skipped, and the winning patch.
Behaviour contract	A per-target declaration of how a fix may change behaviour: *preserve* (output must stay the same), *restrict* (may reject more inputs), *replace* (output must change, e.g. a hardcoded key).
Binary	The compiled, runnable program produced from source code.
Bounded proof	A formal proof that holds for all inputs, but only up to a fixed number of loop repetitions.
Buffer overflow	Writing or reading past the end of a block of memory. The classic memory-safety bug.
CASR	Crash Analysis and Severity Report, an external crash-triage toolkit. Optional here and not installed in the documented setup.
CBMC	C Bounded Model Checker. Turns a C function into a logic formula and asks a solver whether any input can break a safety property.

Term	Meaning
CGC / AIxCC	DARPA's Cyber Grand Challenge and AI Cyber Challenge: competitions for autonomous find-and-fix systems, where the term CRS comes from.
Clang	The C/C++ compiler from the LLVM project. Used here because it provides the sanitizers and libFuzzer.
Cluster / clustering	Grouping many crashing inputs that hit the same underlying bug so it is fixed once.
Concolic execution	Mixing *conc*rete runs with symb*olic* reasoning to find inputs that reach branches a fuzzer has missed.
Confidence interval	A range that likely contains the true value, given limited data. See Wilson interval.
Context extraction	Pulling out the code around a bug (enclosing function, related types, includes) to give the LLM.
Corpus	The collection of inputs a fuzzer keeps because they reached new code.
Coverage-guided fuzzing	Fuzzing that watches which parts of the code each input reaches and favours inputs that reach new parts.
Critic	Agent Delta. Reads a rejected patch and the gate output and returns a short JSON diagnosis for the next attempt.
CRS (Cyber Reasoning System)	An automated system that finds, and often fixes, security bugs without a human in the loop.
CVE	Common Vulnerabilities and Exposures: a public ID for one specific known vulnerability, e.g. CVE-2019-11835.
CWE	Common Weakness Enumeration: a catalogue of bug *types*, e.g. CWE-787 out-of-bounds write, CWE-416 use-after-free.
Determinism	Getting the same output from the same input every time. Gates are deterministic; LLMs are only approximately so.
Diff / patch file	A text description of which lines to remove and add to change a file.
Differential testing	Running the original and patched programs on the same harmless inputs and checking they behave the same. Catches gutted patches.
Double free	Freeing the same block of memory twice, which can corrupt the memory allocator (CWE-415).
Driller	A technique that switches from fuzzing to concolic execution when fuzzing stalls. Implemented here but not wired in.
DRV loop	Detect–Repair–Verify: generate a patch, run the gates, feed failures back, try again.
Episodic memory	A log of every vulnerability run (`episodes.jsonl`). Written but not read back into decisions.
Escalation policy	Plain-code rules for when to call the Critic, force the whole-function format, or stop early.
Evasion battery	A script of tricky bypass inputs (e.g. encoded `../`) run against input-validation fixes.
Falsified / FALSIFIED	A hardening label: an attack broke the patch (overfitted, diverged or evaded). The target is downgraded.
Formal verification	Using mathematics and solvers to prove a property for all inputs, rather than testing some.
Format string bug	Passing user input as the format argument of `printf`-style functions, letting it read or write memory (CWE-134).
Fuzzing	Automatically feeding a program huge numbers of generated inputs to make it crash.
Gate	One deterministic pass/fail check a patch must clear: apply, build, PoV, regression, post-scan.
GATE_SKIPPED	The state of a gate that had no command configured. Reported as SKIPPED, never PASS.
Gutted patch	A "fix" that removes the bug by disabling the feature, e.g. returning early.
Harness	A small C file that feeds fuzzer-generated bytes into the function being tested.
Hardening	The stage that tries to break an accepted patch: re-fuzzing, differential testing, evasion battery.
Heap	Memory a program requests at run time with `malloc` and returns with `free`.
Integer overflow / underflow	A calculation that goes above the largest or below the smallest value a number type can hold and wraps around (CWE-190 / CWE-191).
Juliet	NIST's large collection of small synthetic test programs, each showing one CWE. Two are used here.
libFuzzer	LLVM's in-process coverage-guided fuzzer. The harness defines `LLVMFuzzerTestOneInput`.
LLM (Large Language Model)	An AI model trained on text that predicts what comes next; here, used to write patches.

Term	Meaning
LLM-as-judge	Using a language model to evaluate rather than generate. Only the Critic does this here, and it advises rather than decides.
LSan (LeakSanitizer)	A sanitizer that reports memory that was allocated but never freed (CWE-401).
Manifest	A file describing benchmark targets: source file, bug line, CWE, build, PoV and regression commands.
Memory leak	Memory that is allocated and never freed, slowly using up resources.
NOT HARDENED	A hardening label meaning no attack actually ran or produced evidence. The patch stays patched but earns no hardening bar.
OOB (out-of-bounds)	Accessing memory outside the area that belongs to a buffer.
Orchestrator	The code that builds the prompt, runs the agents in parallel, picks the winner, and records cost and memory.
Overfitted patch	A patch that blocks the one known bad input but not the underlying bug.
Parser	A program that reads source code and works out its structure.
Patch	A change to source code, here intended to remove a vulnerability.
PoV (Proof of Vulnerability)	An input or command that triggers the bug. In this project it exits 0 only when the bug is *gone*.
Post-scan gate	Gate 5: re-running Semgrep on the patched file to make sure the patch added no new findings.
Pre-flight	Building the unpatched program and confirming the PoV really triggers the bug before trusting it as a test.
Procedural memory	Per-CWE statistics on which agent wins and how many tries it takes. Recorded, not yet used.
Prompt / system prompt	The text sent to the LLM. The system prompt sets its role; the user prompt carries the specific bug.
Regression test	A run on a normal, harmless input that must still give the expected output after the patch.
Re-fuzzing	Fuzzing the patched program again, seeded with the original crash, to look for the same bug by another route.
RSS	Resident set size: how much memory a process actually uses. "Peak RSS" is the highest it reached.
Sanitizer	A compiler feature that adds run-time checks and reports bugs the moment they happen (ASan, UBSan, LSan).
SARIF	Static Analysis Results Interchange Format: a standard JSON layout for static-analysis findings (version 2.1.0 here).
Seed (fuzzing)	A starting input that a fuzzer mutates.
Seed (LLM)	A number (1337 here) that makes the model's random choices more repeatable.
Semantic memory	Reusable fix patterns (root cause + fix strategy) indexed by CWE and recalled into prompts.
Semgrep	A static analyser that finds bugs by matching code patterns, without running the program.
`shell=True`	A Python option that runs a command through the system shell, allowing shell syntax but also anything the string says.
SMT solver	A program that decides whether a set of logical constraints can be satisfied. CBMC uses one.
Stack	Memory that holds a function's local variables, created on entry and discarded on return.
Stack trace	The list of functions that were active when a crash happened, innermost first.
Static analysis	Finding bugs by reading code rather than running it.
SURVIVED	A hardening label: at least one attack produced evidence, and none broke the patch.
Temperature	A setting that controls how random an LLM's choices are. 0 means "always pick the most likely".
Token	A small chunk of text (roughly part of a word) that LLMs read and write; usage and cost are counted in tokens.
TOCTOU	Time-of-check to time-of-use: a bug where something changes between being checked and being used.
Tree-sitter	A fast parser library that produces ASTs. Used for context extraction and whole-function splicing.
Triage	Replaying crashes, classifying them, locating them and removing duplicates.
UAF (use-after-free)	Using memory after it has been freed (CWE-416).

Term	Meaning
UBSan (UndefinedBehaviorSanitizer)	A sanitizer that catches undefined behaviour such as signed integer overflow and invalid shifts.
Unwinding / unwinding assertion	In CBMC, unrolling loops a fixed number of times; the assertion fails if that number was not enough, so a proof can't silently stop short.
Whole-function replacement	The patch format where the LLM returns the entire fixed function, marked `// FUNCTION: name`, spliced in by AST position.
Wilson interval	A confidence interval for a success rate that stays sensible at 0% or 100%. For 44/44 it gives [92.0%, 100%].
Working memory	The in-loop ledger of rejected attempts, used to spot an agent repeating itself.